



Tabla de contenidos

UNIDAD I INTRODUCCIÓN Y USO DE AUTÓMATAS EN LOS COMPILADORES	2
1.1 SISTEMAS FORMALES Y TEORÍA DE CONJUNTOS.	2
1.2 CONCEPTOS Y ELEMENTOS PARA EL DISEÑO DE LENGUAJES.	7
1.3 SISTEMAS DE TRADUCCIÓN Y COMPILACIÓN.	18
1.4 FASES Y PASOS DEL COMPILADOR.	36
1.5 TRADUCTOR SIMPLE ORIENTADO A LA SINTAXIS Y EL ANÁLISIS SEMÁNTICO.	43
UNIDAD II USO DE GRAMÁTICAS	US60
2.1 CONCEPTOS DE EXPRESIONES REGULARES Y AUTÓMATAS FINITOS.	60
2.2 TRANSFORMACIÓN DE EXPRESIONES REGULARES A AUTÓMATAS FINITOS NO DETERMINISTAS CON ÉPSILON (ϵ) TRANSICIONES.	69
2.3 TRANSFORMACIÓN DE AUTÓMATAS FINITOS NO DETERMINISTAS CON ÉPSILON (ϵ) TRANSICIONES A AUTÓMATAS FINITOS NO DETERMINISTAS SIN ÉPSILON (ϵ) TRANSICIONES.	74
2.4 TRANSFORMACIÓN DE AUTÓMATAS FINITOS NO DETERMINISTAS SIN ÉPSILON (ϵ) TRANSICIONES A AUTÓMATAS FINITOS DETERMINISTAS.	78
UNIDAD III IMPLEMENTACIÓN DE UN ANALIZADOR LÉXICO	89
3.1 ELEMENTOS DE DISEÑO DE ANALIZADORES LÉXICOS.....	89
3.2 DISEÑO O DESCRIPCIÓN DE UNA APLICACIÓN QUE REQUIERA EL USO DE UN LENGUAJE DE PROGRAMACIÓN.	99
3.3 DISEÑO DE UN LENGUAJE SIMPLE DE PROGRAMACIÓN.....	102
3.4 DEFINICIÓN DE LÉXICO MEDIANTE EXPRESIONES REGULARES.....	107
3.5 IMPLEMENTACIÓN DE UN ANALIZADOR LÉXICO MEDIANTE UNA HERRAMIENTA AUTOMATIZADA.	112
UNIDAD IV IMPLEMENTACIÓN DE UN ANALIZADOR SINTÁCTICO	115
4.1 GRAMÁTICAS LIBRES DE CONTEXTO.	115
4.2 CONCEPTOS BÁSICOS RELACIONADOS CON GRAMÁTICAS.....	115
4.1.2 ÁRBOLES DE DERIVACIÓN Y DERIVACIÓN POR LA DERECHA E IZQUIERDA.....	124
4.1.3 RECURSIVIDAD POR LA DERECHA E IZQUIERDA.	126
4.2 OBTENER LA FORMA POSITIVA DE UNA GRAMÁTICA.	139
4.3 HACER ADMISIBLE Y ARBORESCENTE UNA GRAMÁTICA.....	148
4.4 OBTENER LA FORMA NORMAL DE CHOMSKY DE UNA GRAMÁTICA LIBRE DE CONTEXTO.	153
4.5 OBTENER LA FORMA NORMAL DE GREIBACH DE UNA GRAMÁTICA LIBRE DE CONTEXTO Y GENERAR SU AUTÓMATA DE PILA (AP).....	156
UNIDAD V TRADUCCIÓN ORIENTADA POR LA SINTAXIS.....	170
5.1 ANÁLISIS SINTÁCTICO DESCENDENTE PREDICTIVO Y GRAMÁTICAS LL.....	173
5.2 HERRAMIENTA AUTOMATIZADA PARA EL DESARROLLO DE UN ANALIZADOR SINTÁCTICO.	189
5.3 DIFERENTES NOTACIONES PARA ESPECIFICAR LA ESTRUCTURA O SINTAXIS DE UN LENGUAJE DE PROGRAMACIÓN Y LA TABLA DE SÍMBOLOS.	199
5.4 PROCEDIMIENTOS PARA LA GENERACIÓN DE LA TABLA DE ANÁLISIS SINTÁCTICO (PRIMERO, SIGUIENTE Y OTROS).	210



UNIDAD I

1.1 Sistemas formales y teoría de conjuntos.

Para poder definir el término de sistema formal, se tiene que definir primero cada uno de los subterminos que lo conforman.

*“**SISTEMA**, es un conjunto de elementos independientes pero interrelacionados (recursos) que interactúan, directa o indirectamente, que forman un todo indivisible y organizado (nombre del sistema), que afecta y se ve afectado por su medio ambiente y realiza actividades, para el cumplimiento de su propósito u objetivo, valiéndose de mecanismos de retroalimentación que le permiten efectuar su auto control para continuar su existencia”.*

Al aplicar este concepto, bajo cualquier campo de investigación o disciplina, cualquier hecho, suceso o fenómeno de la naturaleza a ser estudiado como sistema, permitirá precisar y delimitar sus elementos con mayor objetividad y comprenderlo mejor.

Fuente: Silva [2010], página 19.

Un ejemplo de un sistema real es el sistema respiratorio en el cuerpo de un ser humano, que está compuesto por distintos órganos, entre ellos pulmones, nariz, tráquea, entre otros, cuyo fin común es la asimilación del oxígeno para la alimentación de las células del cuerpo interactuando con la complejidad del cerebro humano para formar acciones involuntarias pero vitales. También interactúa con los sistemas circulatorio, digestivo, cardiovascular, sistema óseo, etc.

El concepto de formal se define como el cumplimiento de las condiciones necesarias o con los requisitos establecidos para llevarse a cabo, un ejemplo de esto puede ser la formalidad en cuanto a vestimenta que es requerida en ocasiones específicas, por ejemplo, una boda o el trabajo.

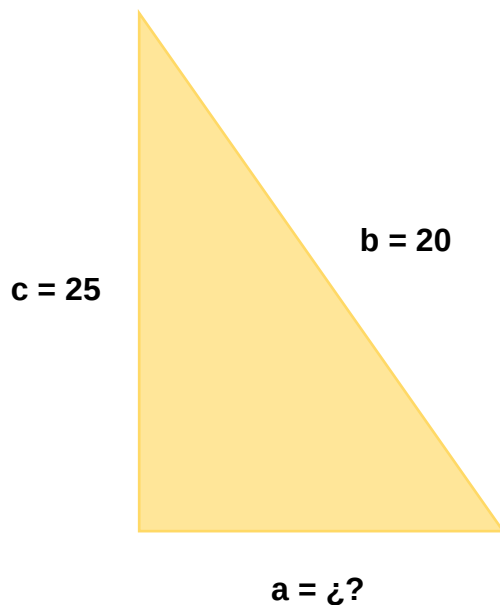
Por lo tanto, se define a un sistema formal como el conjunto de los elementos destinados a cumplir cierta tarea o tareas siguiendo condiciones o, los requisitos establecidos. Aplicado a la teoría de lenguajes y compiladores, esto puede verse desde un punto de vista matemático y axiomático.

Los **Sistemas Formales** poseen propiedades, las cuales son:

- **Coherencia:** El sistema formal es coherente si cada teorema o cadena integrada por símbolos yuxtapuestos al ser interpretados corresponden a una decisión verdadera.
- **Completitud:** El sistema formal es completo si cada proposición verdadera puede ser representada mediante un teorema. Es incompleto si alguna verdad no puede expresarse.
- **Decidibilidad:** Un sistema formal es decidible (de decisión), si existe un algoritmo (conjunto de instrucciones o reglas bien definidas, ordenadas y finitas, que permite realizar una actividad mediante pasos sucesivos que no generen dudas a quien deba realizar dicha actividad) que diga en tiempo finito, si una cadena cualquiera es un teorema o no lo es.

Aquí colocamos como ejemplo el conocido teorema de Pitágoras donde se reúnen los tres elementos ya descritos: Coherencia, completitud y decidibilidad. La intención de describir procedimientos de manera más exacta y precisa es lo que ha motivado la formalización.

Teorema de Pitágoras como ejemplo de que reúne los elementos de un sistema formal.



Teorema de Pitágoras: El cuadrado de la hipotenusa (b) es igual a la suma de los cuadrados de los dos catetos (a y c)

$$a^2 + b^2 = c^2$$

Proposiciones:

- 1) $a^2 + 20^2 = 25^2$ Se sustituyen valores
- 2) $a^2 + 400 = 625$ Se resta $c-b$
- 3) $625 - 400 = 225$
- 4) $\sqrt{225} = 15$



En este sistema tenemos la coherencia, ya que el teorema se cumple como verdadero.

Tenemos la completitud ya que cada proposición es verdadera y la decidibilidad es gracias a que se puede realizar mediante un conjunto de pasos sucesivos de sus proposiciones.

La existencia de una escritura o una notación	La capacidad de abstraer formas o esquemas lingüísticos	La capacidad de proceder sin tomar en cuenta el resultado
Una primera condición para poder formalizar es la existencia de un medio que permita fijar símbolos de una manera que se les pueda distinguir explícitamente y que se les pueda presentar en un determinado orden. A todo esto, se le puede considerar una escritura; esta condición es compartida con cualquier lengua	La descripción de los procedimientos debe ser esquemática. Esto quiere decir que los símbolos se toman como esquemas a interpretar en diferentes casos. Esto significa, en definitiva, establecer reglas generales de aplicación general para los diferentes casos a los que les cabe un procedimiento. Aquí aparece la idea de abstracción formal.	En la formalización se requiere del significado que tengan los símbolos. En un sistema formal se tienen reglas de manipulación para los símbolos las cuales se aplican con dependencia del significado que estos símbolos tengan. Las reglas indican como obtener estructuras de símbolos a partir de otras.

Los sistemas formales se aplican en diferentes campos, un ejemplo es la medicina.

Teoría de conjuntos.

Un conjunto es un grupo de elementos u objetos especificados en tal forma que se puede afirmar con certeza si cualquier objeto dado pertenece o no a la agrupación. Para denotar a los conjuntos, se usan letras mayúsculas.

Cuando un elemento x pertenece a un conjunto A se expresa de forma simbólica como: $x \in A$. En caso de que un elemento y no pertenezca a este mismo conjunto se utiliza la notación: $y \notin A$

Existen cuatro formas de enunciar a los conjuntos:

- 1) Por extensión o enumeración: los elementos son encerrados entre llaves y separados por comas. Es decir, el conjunto se describe listando todos sus elementos entre llaves. Ejemplo: Sean los conjuntos

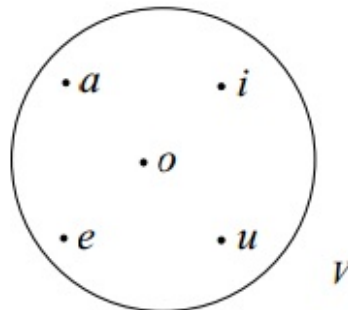
$$A = \{1, 3, 5, 7, 9\} \text{ y } B = \{10, 11, 12\}$$

- 2) Por comprensión: los elementos se determinan a través de una condición que se establece entre llaves. En este caso se emplea el símbolo $|$ que significa "tal que". En forma simbólica es:

$$A = \{x | P(x)\} = (x_1, x_2, x_3 \dots x_n)$$

- 3) Diagramas de Venn: son regiones cerradas que sirven para visualizar el contenido de un conjunto o las relaciones entre conjuntos.

Por diagrama de Venn:



En donde las vocales representan al universo o la totalidad de los elementos del conjunto definido como $V = \{a, e, i, o, u\}$. En caso de estar en conjuntos separados las vocales, por ejemplo, en el conjunto A y B , se sobrepondrían las partes que conforman, o incluso podrían no tener relación.

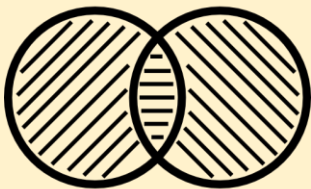
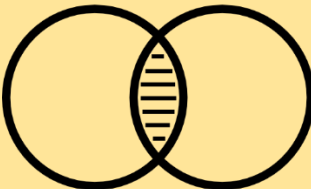
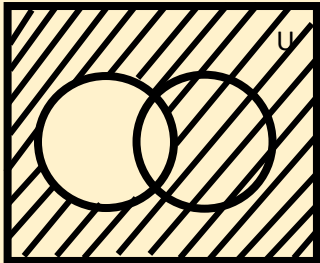
- 4) Por descripción verbal: Es un enunciado que describe la característica que es común para los elementos. Un ejemplo sencillo de aplicación de lo anterior, son las cinco vocales.

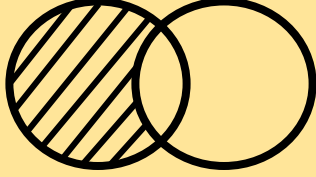
De forma verbal, se define simplemente como tal: "El conjunto de las letras vocales"

Por extensión: $V = \{a, e, i, o, u\}$

Por comprensión: $V = \{x | x \text{ sea una vocal} \}$

La teoría de conjuntos es indispensable para las matemáticas discretas, siendo estas dos indispensables para la teoría de la computación. Basándonos en este ejemplo podemos definir la existencia de operaciones con conjuntos. Los cuales se especifican de manera general en la siguiente tabla.

OPERACIONES CON CONJUNTOS		
OPERACION	DEFINICION	DIAFRAMA DE VENN
Unión	La unión de los conjuntos A y B es el conjunto de todos los elementos de A con todos los elementos de B sin repetir ninguno y se denota como $A \cup B$. Esto es: $A \cup B = \{x \mid x \in A \vee x \in B\}$	
Intersección	La intersección de los conjuntos A y B es el conjunto de los elementos de A que también pertenecen a B y se denota como $A \cap B$. Esto es: $A \cap B = \{x \mid x \in A \wedge x \in B\}$	
Complemento	La intersección de los conjuntos A y B es el conjunto de los elementos de A que también pertenecen a B y se denota como $A \cap B$. Esto es: $A - B = \{x \mid x \in A \wedge x \notin B\}$	

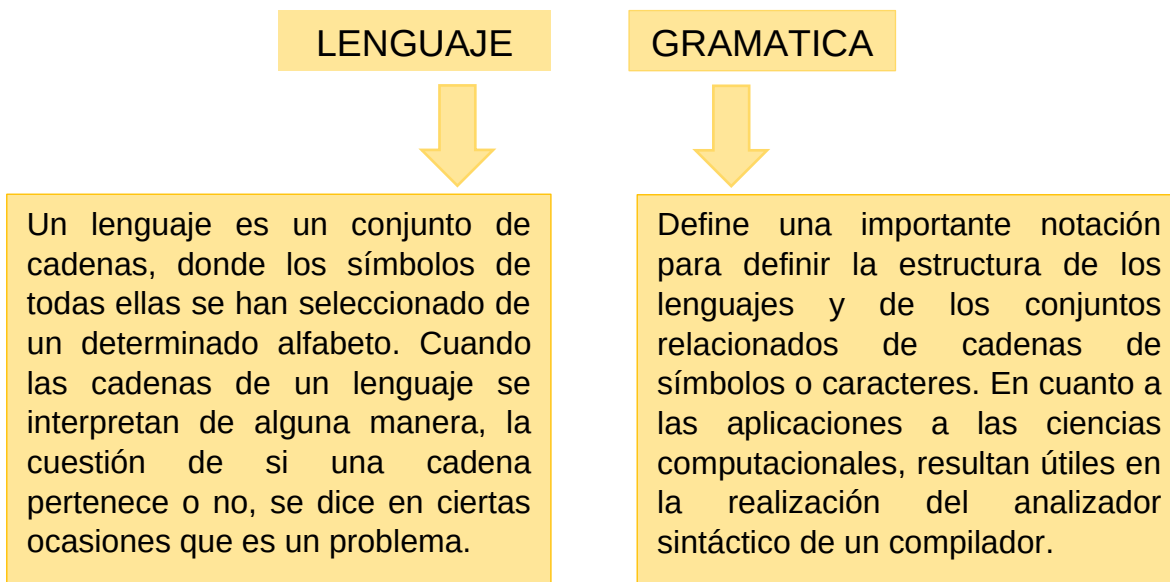
Diferencia	La diferencia de los conjuntos A y B (en ese orden) es el conjunto de los elementos que pertenecen a A y no pertenecen a B y se denota como $A - B$. Esto es: $A - B = \{x \mid x \in A \text{ y } x \notin B\}$	
-------------------	---	---

“Puede emplearse un número indeterminado de conjuntos, no solo uno o dos.”

1.2 Conceptos y elementos para el diseño de lenguajes.

Un lenguaje es cualquier sistema de comunicación estructurado, para el que existen reglas de uso y ciertos principios combinatorios formales. Existen contextos naturales y artificiales; del mismo modo los lenguajes se dividen en tres principales que son: lenguaje humano, lenguaje animal y lenguaje formal.

Los lenguajes formales son construcciones artificiales humanas, que se usan en matemáticas y otras áreas formales, incluyendo lenguajes de programación. Estas construcciones tienen estructuras internas que comparten con el lenguaje humano natural, por lo que pueden ser en parte analizados con los mismos conceptos que éste.





Otras definiciones nos dicen que es un lenguaje cuyos símbolos y reglas para unir esos símbolos están formalmente especificados. Al conjunto de esos símbolos se le llama vocabulario del lenguaje, y al conjunto de reglas se le conoce como gramática formal o sintaxis.

Podemos considerar que cualquier lenguaje cae dentro de la clasificación de cualquiera de los siguientes grupos:

Lenguajes Humanos (Naturales):

Este grupo se creó desde el momento en que el hombre tuvo la necesidad de compartir o de dar a conocer sus ideas, por lo que no se tiene un origen preciso de su aparición. También los podemos considerar como lenguajes naturales; en la actualidad podemos decir que inglés, francés, español, alemán, italiano etc., pertenecen a este grupo.

Lenguajes Artificiales:

Este grupo ha sido creado por el mismo ser humano, con el objetivo de poder entablar una comunicación directa con diversos objetos, de entre los cuales encontramos a las computadoras u ordenadores. Dentro de esta clasificación de lenguajes, tenemos dos divisiones:

Los lenguajes de alto nivel:

Los podemos definir, como un lenguaje simbólico en el que una instrucción de programa fuente da lugar a varias instrucciones de lenguaje máquina, debido a que es compilado o interpretado.

“Chomsky Clasifica las gramáticas como no restringidas, sensibles al contexto, independientes de él, y regulares.”

El lenguaje máquina, es en otras palabras, el lenguaje específico de las computadoras, en el que las instrucciones se expresan en código binario directamente asimilable por la máquina.

En estos apuntes principalmente hablaremos del segundo grupo, el de los Lenguajes Artificiales, conocidos también como Lenguajes de Programación de Alto Nivel o bien Simbólicos; esto se debe a que no son directamente utilizados por las computadoras, sino que primero pasan por un proceso de traducción, un proceso de ensamble, hasta llegar al punto en que son comprendidos por las computadoras.



Podemos definir Lenguajes de Programación como un conjunto de reglas y elementos gramaticales, de los que un usuario programador dispone para escribir un conjunto de instrucciones o indicaciones para la realización de una labor específica en la resolución de un problema.

Como se ha visto hasta este momento, los lenguajes se pueden clasificar por alguna de las siguientes maneras:

- **Niveles con respecto al hardware.**
- **Generaciones.**
- **Modelos de Interpretación.**
- **Aplicación.**
- **Tipos.**
- **Paradigmas.**

Es por lo que tenemos como ejemplo los siguientes lenguajes:

- Lenguajes Específicos conocidos como Metalenguajes.
- Lenguajes de Presentación y Formateo de Datos como el lenguaje de marcación de hipertextos HTML.
- Lenguajes de Definición de Datos como DDI dentro de SQL, XML, etc.
- Lenguajes de Propósito General como C, Pascal, Visual Basic, etc.
- Lenguajes Orientados a Objetos como C++ y Java.
- Lenguajes Orientados a la Inteligencia Artificial como Lisp.

Es muy importante tener una panorámica muy amplia de los lenguajes de programación que ya existen en el mundo de la computación desde la creación de las computadoras son más de 2500. Con esta introducción sobre las características que guardan todos los lenguajes de programación que se han diseñado en la historia de los lenguajes de computadora que existan hasta nuestros días estaremos en posibilidad de aplicar algunos elementos importantes a considerar en el diseño de los lenguajes de programación.

ELEMENTOS PARA EL DISEÑO DE LENGUAJES

Algunas de las características básicas a considerar en el diseño de los lenguajes de programación son:



- **Abstracción.** El lenguaje debe evitar forzar a los programadores a tener que enunciar algo más de una vez. El lenguaje debe permitir al programador la identificación de patrones repetitivos y automatizar tareas mecánicas, tediosas o susceptibles de cometer errores. Ejemplos de técnicas de abstracción son los procedimientos y funciones, la generalidad, los lenguajes de patrones de diseño, etc.
- **Concisión notacional.** El lenguaje proporciona un marco conceptual para pensar algoritmos y expresar dichos algoritmos con el nivel de detalle adecuado. El lenguaje debe ser una ayuda al programador (incluso antes de comenzar a codificar) proporcionando un conjunto de conceptos de forma clara, simple y unificada. La sintaxis debe ser legible por el programador (o por otras personas que vayan a utilizar esos programas). Deben buscarse soluciones de compromiso entre lenguajes demasiado crípticos (por ejemplo, C) y lenguajes con código demasiado largo o dilatado con exceso. (Cobol, XSLT).
- **Eficiencia.** El programador debe poder expresar algoritmos suficientemente eficientes o el lenguaje debe incorporar técnicas de optimización de los programas escritos en él.
- **Entorno.** Aunque el entorno no forma parte del lenguaje, muchos lenguajes débiles técnicamente son ampliamente utilizados debido a que disponen de un entorno de desarrollo potente o agradable. De la misma forma, la disposición de documentación, ejemplos de programas e incluso programadores pueden ser factores clave de la popularidad de un lenguaje de programación.
- **Expresividad.** El programador debe poder expresar sus intenciones. En ocasiones, demasiada expresividad puede implicar falta de seguridad. De hecho, algunos sistemas limitan la expresividad para mejorar la fiabilidad de los programas (por ejemplo, la aritmética de punteros no es permitida en algunos lenguajes).
- **Extensibilidad.** El lenguaje debe facilitar mecanismos para que el programador pueda aumentar la capacidad expresiva del lenguaje añadiendo nuevas construcciones. En Haskell, por ejemplo, el programador puede definir sus propias estructuras de control.
- **Librerías e interacción con el exterior.** La inclusión de un conjunto de librerías que facilita el rápido desarrollo de aplicaciones es una componente esencial de la popularidad de los lenguajes. Si no se dispone de tales librerías, es necesario contemplar mecanismos de enlace con otros lenguajes que facilitan la incorporación de librerías externas.



- **Ortogonalidad.** Ofrece la posibilidad de combinar características de todas las formas posibles (sin excepciones). La falta de ortogonalidad puede suponer la enumeración de situaciones excepcionales o la aparición de incoherencias. Un ejemplo de falta de ortogonalidad es la limitación que impone Pascal para que una función devuelva determinados tipos de valores.
- **Portabilidad.** El lenguaje debe facilitar la creación de programas que funcionen en el mayor número de entornos computacionales. Este requisito es una garantía de supervivencia de los programas escritos en el lenguaje y, por tanto, del propio lenguaje. Para conseguir la portabilidad, es necesario limitar las características dependientes de una arquitectura concreta.
- **Seguridad.** La fiabilidad de los productos software es cada vez más importante. Lo ideal es que los programas incorrectos no pertenezcan al lenguaje y sean rechazados por el compilador. Por ejemplo, los sistemas con chequeo de tipos establecen restricciones a los posibles programas que pueden escribirse en un lenguaje para evitar que en tiempo de ejecución se produzcan errores. Existen lenguajes como Charity que garantizan la terminación de sus programas.

PREVENCIÓN Y DETECCIÓN DE ERRORES

Tener una serie de defensas tal que, si un error no es detectado por el programador que desarrolla el código, este error probablemente sea detectado por otro programador. Los errores deben ser detectados por el compilador, si un mecanismo no es capaz de detectar un error es necesario implementar otro que lo detecte, pero nunca ignorarlo.

Prevención de errores

- El programador comete errores. Hay que prevenir los errores
- El programador es su fuente. El programador no sabe lo que hace y el compilador ha de limitar sus acciones (EUCLID, PASCAL). Hacer imposible cierto tipo de errores.

Ejecutar datos → control de flujo limitado.

Errores en el uso de datos → código fuente fuertemente "tipeado".

Apuntadores erróneos → Gestión de memoria implícita (LISP, PROLOG, ML)



Hay que facilitar su detección, identificación y corrección. Tener que declarar antes de utilizar. Evitar coerciones inductoras de errores.

- float a int por su pérdida de precisión.
- Comprobaciones en tiempo de ejecución. Índice de “array” (arreglos) fuera de límites. Control sobre los apuntadores a NULL.

Prevención y tolerancia de fallos.

Hay dos formas de aumentar la fiabilidad de un sistema:

Prevención de fallos: Se trata de evitar que se introduzcan fallos en el sistema antes de que entre en funcionamiento.

Prevención de fallos.

Se realiza en dos etapas:

- Evitación de fallos: Se trata de impedir que se introduzcan fallos durante la construcción del sistema.
- Eliminación de fallos: Consiste en encontrar y eliminar los fallos que se producen en el sistema una vez construido.

Tolerancia de fallos: Se trata de conseguir que el sistema continúe funcionando, aunque produzcan fallos. En ambos casos el objetivo es desarrollar sistemas con modos de fallo bien definidos.

Detección de errores o Por el entorno de ejecución o hardware (p.ej. instrucción ilegal) o Núcleo o sistema operativo (p.ej. puntero nulo) o Por el software de aplicación o Duplicación (redundancia con dos versiones) o Comprobaciones de tiempo o Inversión de funciones o Códigos detectores de error o Validación de estado o Validación estructural.

EFICACIA.

Los lenguajes de programación contienen un repertorio de tipos básicos o primitivos junto con una serie de operaciones sobre dichos tipos. Los tipos básicos más habituales son:

- Números enteros: Normalmente se representan mediante un número de bytes fijo, limitando su rango. Algunos lenguajes, como Haskell, incluyen una representación ilimitada.



- Caracteres: Como en el caso anterior, suelen representarse mediante un número fijo de bytes. En algunos lenguajes, los caracteres se identifican con los enteros.
- Números reales representados en punto flotante. La representación también suele realizarse mediante un número fijo de bytes, limitando la precisión de los programas.
- Booleanos: Con los valores verdadero o falso.
- Referencias. Algunos lenguajes incluyen un tipo básico que se utiliza para hacer referencia a otros elementos. Estas referencias pueden implementarse mediante direcciones de memoria.

Cada tipo básico contiene un conjunto de operaciones primitivas. Por ejemplo, los enteros y flotantes incluyen operaciones aritméticas, los caracteres, operaciones de conversión y los booleanos (operaciones lógicas).

Los lenguajes con chequeo estático de tipos permiten comprobar en tiempo de compilación que en tiempo de ejecución no se van a producir errores de tipos. El chequeo estático de tipos aumenta la seguridad de los programas, al detectar errores antes de la ejecución. Otra ventaja es la eficiencia, ya que en la fase de ejecución no es necesario realizar comprobaciones de tipo.

Otros lenguajes, como LISP, BASIC, Perl, Prolog, etc. no incluyen chequeo estático de tipos. Las ventajas de no incluirlo son una mayor flexibilidad (es posible construir más programas) y sencillez para el programador. El programador no se preocupa de incluir declaraciones de tipos y los programas dan menos errores de tipo al compilar (aunque pueden darlos al ejecutarse).

Algunos lenguajes, como Haskell o ML, incluyen además un sistema de inferencia de tipos. El programador no tiene obligación de declarar el tipo de las expresiones, ya que el sistema es capaz de inferirlo. En caso de que el programador lo hubiese declarado, el sistema puede comprobar que el tipo declarado coincide con el tipo inferido.

*LENGUAJES DE PROGRAMACION: DISEÑO E IMPLEMENTACION del autor PRATT
(ISBN 9789701700464). (Cita completa más adelante)*



COMPATIBILIDAD

La elección de un lenguaje de programación depende de sus conocimientos del lenguaje y del ámbito de la aplicación que está generando. Las aplicaciones de pequeño tamaño se suelen crear utilizando un único lenguaje, y es frecuente implementar aplicaciones grandes utilizando varios lenguajes.

Por ejemplo, si está ampliando una aplicación con servicios Web de XML existentes, podría utilizar un lenguaje de secuencias que no requiera apenas tareas de programación. Para aplicaciones cliente-servidor, utilizaría probablemente el lenguaje del que tiene más conocimientos para toda la aplicación.

Para nuevas aplicaciones empresariales, donde un equipo grande de programadores crea componentes y servicios para implementarlos en varios sitios remotos, la mejor opción sería utilizar varios lenguajes dependiendo de los conocimientos de los programadores y de las expectativas de mantenimiento a largo plazo.

Los lenguajes de programación de la plataforma .NET, incluidos Visual Basic .NET, Visual C# y las Extensiones administradas de C++, y otros lenguajes de distintos fabricantes, utilizan los servicios y características de .NET Framework a través de un conjunto común de clases unificadas. Las clases unificadas de .NET proporcionan un método coherente de acceso a la funcionalidad de la plataforma. Si aprende a utilizar la biblioteca de clases, observará que todas las tareas siguen la misma arquitectura uniforme. Ya no necesitará aprender ni administrar distintas arquitecturas API para escribir las aplicaciones.

En la mayoría de las situaciones, puede utilizar de manera eficiente todos los lenguajes de programación de Microsoft. Sin embargo, cada lenguaje de programación tiene sus puntos fuertes, y es recomendable comprender las características únicas para cada uno de ellos. Las secciones siguientes le ayudarán a seleccionar el lenguaje de programación que mejor se ajuste a su aplicación. Algunos de los lenguajes Una noción importante de estos lenguajes es la herencia que facilita la reutilización de código.

- Simula. Desarrollado por O. J. Dahl y K. Nygaard entre 1962 y 1967 como un dialecto de ALGOL para simulación de eventos discretos. Fue el primer lenguaje que desarrolla el concepto de clases y subclases.
- Smalltalk. Desarrollado por A. C. Kay en 1971 como un sistema integral orientado a objetos que facilitaba el desarrollo de aplicaciones interactivas.



- C++. Creado en 1985 por B. Stroustrup añadiendo capacidades de orientación a objetos al lenguaje C. El objetivo era disponer de un lenguaje con las capacidades de abstracción y reutilización de código de la orientación a objetos y la eficiencia de C++. El lenguaje fue ganando popularidad a la par que se incluían numerosas características como genericidad, excepciones, etc.
- Java. Desarrollado por J. Gosling en 1993 como un lenguaje orientado a objetos para dispositivos electrónicos empujados. Alcanzó gran popularidad en 1996 cuando Sun Microsystems hizo pública su implementación. La sintaxis del lenguaje se inspira en la de C++, pero contiene un conjunto más reducido de características. Incluye un sistema de gestión de memoria y un mecanismo de tratamiento de excepciones y concurrencia.
- Python. Creado por Guido van Rossum en 1990 como un lenguaje orientado a objetos interpretado. Utiliza una sintaxis inspirada en C++ y contiene un sistema dinámico de tipos. El lenguaje ha ganado popularidad debido a su capacidad de desarrollo rápido de aplicaciones de Internet, gráficos, bases de datos, etc.
- C#. Creado por Microsoft para la plataforma .NET en 1999. Utiliza una sintaxis similar a la de Java y C++. También contiene un sistema de gestión dinámica de memoria y se apoya en la máquina virtual CLR. El lenguaje aprovecha muchas de las características de esta plataforma, como el mecanismo de excepciones, sistema de componentes, control de versiones.

Los lenguajes de programación tienen como objetivo la construcción de programas, normalmente escritos por personas humanas. Estos programas se ejecutarán por un computador que realizará las tareas descritas. La utilización de un lenguaje de programación requiere, por tanto, una comprensión mutua por parte de personas y máquinas. Este objetivo es difícil de alcanzar debido a la naturaleza diferente de ambos. En un lenguaje natural, el significado de los símbolos se establece por la costumbre y se aprende mediante la experiencia. Sin embargo, los lenguajes de programación se definen habitualmente por una autoridad, que puede ser el diseñador individual del lenguaje o un determinado comité.

Los lenguajes de programación son, por tanto, una solución de compromiso entre las necesidades del emisor (programador - persona) y del receptor (computador - máquina). De esa forma, las declaraciones, tipos, nombres simbólicos, etc. son concesiones de los diseñadores de lenguajes para que los humanos podamos entender mejor lo que se ha escrito en un programa. Por otro lado, la utilización de un vocabulario limitado y de unas reglas estrictas son concesiones para facilitar el proceso de traducción.



En 1938, C. Morris [Morris71] realiza una división del estudio de los signos (semiótica) en tres partes:

- Sintaxis: relación de los signos entre sí
- Semántica: Relación de los signos con los objetos a los que se aplican.
- Pragmática: Relación de los signos con sus intérpretes

Adaptando dichas definiciones al caso particular de lenguajes de programación, la sintaxis se refiere al formato de los programas del lenguaje, la semántica estudia el comportamiento de los programas y la pragmática estudia aspectos relacionados con las técnicas empleadas para la construcción de programas.

*Lenguajes de programación: diseño e implementación / Terrence. W. Pratt,
Marvin. V. Zelkowitz; traducido por Héctor Javier Escalona García.*

Cadenas, Alfabetos y Lenguajes.

Las cadenas son la unión de símbolos o concatenación desde cero símbolos expresados con la cadena vacía épsilon ϵ hasta un número infinito de símbolos: ∞ , por lo que las cadenas tienen una longitud que se especifica más adelante. Las cadenas por lo general se escriben con letras minúsculas como la u, v, w, etc. Los alfabetos son el conjunto de símbolos que definen un lenguaje formal. Los alfabetos son el conjunto de símbolos que definen un lenguaje formal. Los alfabetos se denotan con Σ y opcionalmente un subíndice.

Ejemplos de Alfabetos

$$\begin{aligned}\Sigma_1 &= \{0, 1\} & \Sigma &= \text{Alfabeto} \\ \Sigma_2 &= \{a, b, c\} & w &= \text{Cadena}\end{aligned}$$

Ejemplos de Cadenas

$$\begin{aligned}w &= 011100 & v &= 11 \\ u &= bcabb\end{aligned}$$

Cuando hay cadenas de un mismo Alfabeto, se pueden concatenar o pegar:

- $wv = 01110011$

La concatenación no es conmutativa.

- Cadena vacía ϵ
- $\epsilon X = X\epsilon = X$

La longitud de una cadena es el número de símbolos y tienen un valor absoluto $|x|$



Ejemplos:

- $|w| = 6$ Longitud 6
- $|\epsilon| = 0$ Longitud de ϵ es 0

Prefijos y subfijos.

Prefijo: Porción inicial de una palabra o cadena subyacente.

Subfijo: Cualquier porción final de una cadena o parte final de la palabra.

Para prefijos y subfijos a manejar, pueden tener un significado por sí solos. ϵ_1 es un prefijo.

b_1 es un prefijo, al igual que los siguientes:

- Bc Debido a que es cualquier porción inicial de la cadena u.
- bcab
- bcabb
- ϵ_1 es un sufijo

b es un sufijo de la cadena, al igual que los siguientes:

- Bb
- Abb
- Cabb
- bcabb

Cuando el prefijo no es igual a la palabra completa es un prefijo propio. Los sufijos propios son todos aquellos sufijos que no sean la palabra completa.

- Conjunto que se puede obtener a partir de cualquier alfabeto.

Σ^* Consta de todas las cadenas que se pueden formar con los símbolos de Σ y es infinito.

- Ejemplo:
- $\Sigma_1^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots, \infty\}$ (Siguen los de tres dígitos, cuatro, etcétera, por eso es infinito)

Lenguajes: L C Z^* Lenguaje sobre Σ

- Por lo que un lenguaje puede ser finito e infinito.



Lenguajes.

- $\emptyset$ El vacío es formalmente muy útil.
- $\{\epsilon\}$ Tiene el elemento de la palabra vacía, ausencia de símbolos y su longitud es cero (símbolos).
- Σ Lenguaje
- Σ^* Lenguaje Los elementos también pueden operarse dado que estos son conjuntos.

Operaciones entre lenguajes.

La concatenación.

- $L_1 L_2 = \{WV \mid W \in L_1, V \in L_2\} = u u u \dots u \text{ (n veces)} = \epsilon$

Cerradura de Kleene.

- Se denota como L^* (V^*)
- En L^* están concatenadas todas las palabras contra todas.

Cerradura positiva

- Siempre tendrá a ϵ como un elemento
- Tendrá a ϵ si L la tenía desde el principio.

Ejemplos:

- $L_1 = \{a, ab\}$, $L_2 = \{b, ba\}$
- $L_1 L_2 = \{ab, aba, abb, abba\} = \{\epsilon, a, ab, aa, aab, aba, abab, \dots\}$
- Se suprime ϵ porque no aparece $L_1 = \{a, ab, aa, aab, aba, abab, \dots\}$
- ϵ está en la cerradura positiva. () Si y sólo si ϵ está en L (el lenguaje) originalmente desde un principio.

1.3 Sistemas de traducción y compilación.

LENGUAJE DE MÁQUINA.

Es el sistema de códigos directamente interpretable por una máquina, como el procesador de un ordenador o computadora. Este lenguaje está compuesto por un conjunto de instrucciones que determinan acciones a ser tomadas por la máquina. Un programa de computadora consiste en una cadena de estas instrucciones de lenguaje de máquina (más datos).



Estas instrucciones son normalmente ejecutadas en secuencia, con eventuales cambios de flujo causados por el propio programa o eventos externos. El lenguaje de máquina es específico de cada máquina o arquitectura de la máquina, aunque el conjunto de instrucciones disponibles pueda ser similar entre ellas.

El lenguaje-máquina sólo utiliza dos signos. Y éstos se corresponden exactamente con las únicas dos constantes del álgebra booleana, y también con los dos estados exclusivos de un conmutador. Estos signos, constantes o estados de conmutación son el 1 y el 0, llamados dígitos binarios. Claude Elwood Shannon, en su *Analysis of Relay and Switching Circuits*, y con sus experiencias en redes de conmutación sentó las bases para la aplicación del álgebra de Boole a redes de conmutación. Una Red de Conmutación es un circuito de interruptores eléctricos que, al cumplir ciertas combinaciones booleanas con las variables de entrada, define el estado de la salida.

Este concepto es el núcleo de las Compuertas Lógicas, las cuales son por su parte, los ladrillos con que se construyen sistemas lógicos cada vez más complejos. Shannon utilizaba al “*relay*” como dispositivo físico de conmutación en sus redes. El “*relay*”, igual que el interruptor de una lámpara eléctrica, es 1 o es 0; está prendido o está apagado.

El desarrollo tecnológico ha permitido evolucionar desde las redes de *relays* electromagnéticos de Shannon, pasar a circuitos con tubos de vacío, luego a redes transistorizadas, hasta llegar a los modernos circuitos integrados. El Microprocesador de la CPU de la computadora u ordenador opera en lenguaje-máquina, ya que su repertorio de instrucciones consiste en la ejecución de conjuntos binarios. Por cierto, que Shannon fue quien aportó el término de **bit** para los guarismos 1 o 0, acrónimo de las palabras inglesas binary digit o el equivalente en español: "dígito binario".

LENGUAJE ENSAMBLADOR.

El lenguaje ensamblador consiste en una selección de abreviaturas (mnemónicos o mnemoclaves) tomadas de palabras del lenguaje inglés, las cuales dan una idea de su significado. Este lenguaje está muy cercano a la arquitectura de la computadora y se dan instrucciones indicando el direccionamiento para cada instrucción. Este es una versión mnemotécnica del código de máquina, donde se usan nombres (identificadores) en lugar de códigos binarios para operaciones, y también se usan nombres (también identificadores) para las direcciones de memoria.



Existe un lenguaje ensamblador para cada tipo de computadora en especial, por lo que se dice que son dependientes del hardware y de bajo nivel, por lo que suelen ser complejos dependiendo de la capacidad de la computadora, a mayor capacidad mayor complejidad en la elaboración del código en este tipo de lenguaje.

Un ejemplo puede ser lo que se muestra a continuación...

Dirección en hexadecimal	Instrucción en lenguaje ensamblador	Código de máquina
0000 0001	MOV a, R1	0001 01 00 00000000
0000 0010	ADD #2, R1	0010 01 10 00000010
0000 0011	MOV R1, a	0001 01 00 00000100

Se concibe una pequeña palabra de instrucción, en la que los cuatro primeros bits son el código de la instrucción, donde 0001 y 0010 representan hipotéticamente las instrucciones MOV (mover) y ADD (sumar) respectivamente, el 00 representa (también hipotéticamente) la dirección de la variable a, el 01 representa la dirección de la variable R1, de donde se concluye que las tres instrucciones en ensamblador corresponden a:

MOV a, R1 - Se mueve el contenido de la constante cuya dirección es a (00) a la variable R1 (cuya dirección en 10) y esto equivale a igualar el valor de la variable $a = R1$

ADD #2, R1 - Se incrementa el valor de la variable R1 en dos unidades (que convertidas a binario es 10) y equivale a $R1 = R1 + 2$

MOV R1, a El valor que tenga R1 se guarda en la variable a
(Igualándolos) $R1 = a$

De esta forma podemos imaginar que todo lo que se escribe en mnemónicos conlleva una traducción a binario. Según se mostró con un tipo muy particular (ya simplificado) de lenguaje ensamblador, solo con el objetivo de entender una traducción simulada de ensamblador a binario



LENGUAJE DE ALTO NIVEL

La programación en el lenguaje de la máquina o en lenguaje simbólico tiene ciertas ventajas:

- Mayor adaptación al equipo.
- Posibilidad de obtener la máxima velocidad con mínimo uso de memoria.

También tienen importantes inconvenientes:

- Imposibilidad de escribir código independiente de la máquina.
- Mayor dificultad en la programación y en la comprensión de los programas.

Por esta razón, a finales de los años 1950 surgió un nuevo tipo de lenguaje que evitaba los inconvenientes, a costa de ceder un poco en las ventajas. Estos lenguajes se llaman "de tercera generación" o "de alto nivel", en contraposición a los "de bajo nivel" o "de nivel próximo a la máquina".

Principales lenguajes de alto nivel

- Basic
- Pascal, Object Pascal
- C, C#, C++
- Python
- Rubi
- Perl
- Ada
- Cobol
- Java
- Fortran
- Lenguajes funcionales
 - Lisp
 - Haskell

Nota: Dentro de los lenguajes de nivel alto, el lenguaje C y sus derivados son considerados lenguajes de nivel intermedio.



TRADUCTORES

Los traductores son un conjunto de programas, necesarios para lograr la conversión de un código a otro, de un lenguaje a otro y de diferentes niveles. Antes de mostrar los diferentes tipos de traductores es importante remarcar la importancia de la traducción de un código a otro. Para poder lograr la comunicación hombre máquina, simplificando esta labor, se han creado herramientas “traductores” cada vez más cercanos al hombre y más lejanos del lenguaje de máquina, Por esta razón trataremos de definir brevemente que es un nivel.

Los niveles en los programas se consideran tomando en cuenta las características del lenguaje que se utiliza. En términos muy generales, los lenguajes pueden ser:

- De alto nivel. - Cercanos al hombre y lejanos a las características particulares de la computadora o al lenguaje de máquina.
- De bajo nivel. - Cercanos a la arquitectura de las computadoras y con gran detalle para el código que realiza el hombre debido a su cercanía con el tipo particular de computadora.
- De nivel intermedio. - Estos son poderosos como los lenguajes de bajo nivel (ensambladores) y accesibles al hombre, como son los lenguajes de alto nivel. El ejemplo clásico de este tipo de lenguajes de nivel medio, son los que tienen alguna relación con los diferentes tipos de lenguaje C, ya que estos tienen una gran variedad por la cantidad de derivados que se han generado, incluyendo a JAVA.

Con base a lo anterior, existen infinidad de niveles intermedios de traducción o interpretación, los cuales llevan diferentes nombres tales como:

Ensambladores. - Estos existen en gran variedad y en diferentes tipos, de acuerdo con sus particularidades, ya que, dentro de los mismos ensambladores, también incluyen varios tipos de niveles de traductores debido a que son de bajo nivel y dependen de las características específicas de cada computadora. Esto hace que se tenga un tipo especial de ensamblador para cada tipo de procesador en particular, dependiendo de su tamaño de palabra, el número de registros internos con los que cuenta, así como características específicas de sus tipos de direccionamiento.

Además de las características físicas con las que se diseña un lenguaje ensamblador, estos lenguajes pueden ser identificados de forma genérica, en los siguientes tipos: Los pseudo-ensambladores o emuladores de ensamblador, un ejemplo de este tipo diseñado para el procesador INTEL modelo 80286 es el programa que se ejecuta como comando externo, conocido como el DEBUG.



Otro tipo de ensambladores son los macroensambladores, también para procesadores INTEL, un ejemplo de este es MASM y los ensambladores en sí. Es importante hacer la diferencia entre el concepto de ensambladores como procesos y como lenguajes, ya que en el proceso de traducción de un lenguaje de programación de alto nivel se requiere traducir a bajo nivel y para esto se tiene un programa o procesador que por lo general proporcionan los sistemas operativos llamado ensamblador (ASSEMBLER), y este adecua el código de un lenguaje de alto nivel a otro de bajo nivel, mismo que es el que la computadora interpreta y ejecuta.

Los lenguajes ensambladores son a los que nos referimos en la clasificación de los niveles que aquí se describen, ya que un compilador puede incluir una fase de síntesis o ensamble que se describirá con más detalle en las siguientes unidades.

INTÉRPRETES O TRADUCTORES INMEDIATOS.

Estos tienen como principal característica la posibilidad de emitir una respuesta inmediata (en tiempo real), esto es, para cada instrucción se genera una acción o respuesta que se obtiene de la máquina al mismo tiempo que se realiza la interpretación del código escrito por el usuario programador. Este es un nivel de traducción eficiente y económica, debido a que no requiere gran capacidad de almacenamiento, ya que no genera código intermedio y por eso es eficiente al producir una respuesta inmediata. Su inconveniente es que deja el código fuente a disposición del usuario.

Existen lenguajes como BASIC y C de los cuales se han construido versiones compiladas e intérpretes con respuesta inmediata, la diferencia entre uno y otro es la cantidad de recursos que requiere cada tipo de herramienta y los niveles de traducción que a continuación se describen. Es importante ubicar a los traductores, intérpretes y compiladores como herramientas de desarrollo de programas para la utilización de computadoras cualquiera que sea su capacidad y complejidad. Por lo que se abrevian como herramientas de desarrollo o productividad, existiendo una gran variedad por la combinación de sus características.

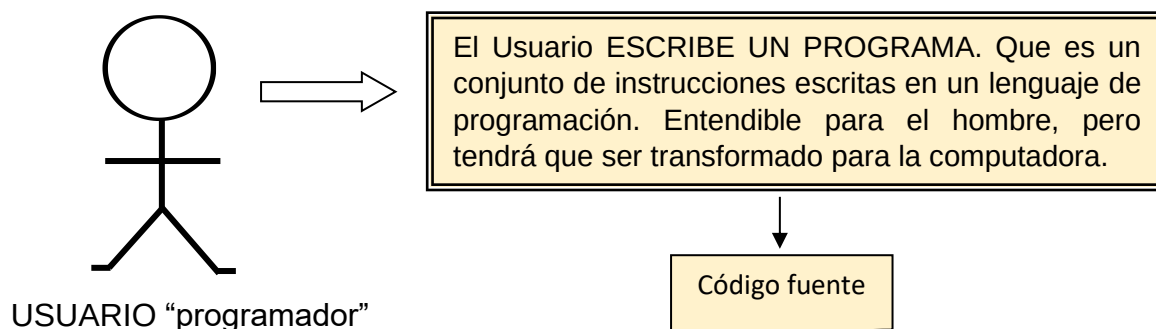
Niveles de traducción. - Recordemos la necesidad de realizar una comunicación hombre máquina para poder controlar los trabajos que realiza una computadora. Para esto ya se explicó que el nivel más alto de traducción es el lenguaje cercano al hombre, los cuales se conocen como lenguajes naturales tales como el inglés, español, francés, alemán, etc. Estos lenguajes son entendibles para el hombre, pero difíciles de definir en sus reglas gramaticales debido a la gran cantidad de excepciones, así como la variedad de palabras y distintos significados que se requieren para expresar ideas que por lo general están sujetas a un contexto.

Recapitulando lo anterior, para poder lograr que la máquina identifique palabras, se necesita establecer reglas gramaticales y realizar procesos tales como:

- Codificación, Análisis, Síntesis, estos tres conocidos como traducción y la parte de interpretación o ejecución del código conocido como tiempo de ejecución o RUN TIME.
- Codificación es una operación consistente en representar una información o instrucción mediante un código.
- Código es un conjunto de reglas que proporcionan una correspondencia biunívoca que permite representar datos, programas y otras informaciones con vistas a facilitar su tratamiento automático o su transmisión.
- Análisis es el estudio realizado para separar las distintas partes de un todo.
- Síntesis es la unión de esas partes (o la composición) del todo.

En la siguiente figura se presenta un diagrama esquemático de un proceso de codificación, en este se incluye un usuario programador, el cual escribe un código en algún lenguaje de programación. El código se puede introducir directamente en la computadora (de forma interprete) o bien grabar el código en un archivo llamado programa fuente.

La codificación es la que realiza el hombre, las cuales se guardan en un archivo conocido como programa fuente o código fuente. Este programa fuente entra a un proceso de análisis y síntesis conocido como compilación, donde además se realiza la labor de análisis de las palabras estructuradas en el código fuente para verificar que sean léxica, semántica y sintácticamente correctas.



Después de realizado lo anterior, si son correctas, pasan a la fase de análisis y después se aceptan para pasar a la siguiente etapa que es de síntesis y generación de un código que será el que la máquina interpretará y ejecutará.

De aquí que en esa cantidad de pasos intermedios se tienen una variedad de compiladores que en términos generales y basándonos en sus niveles de traducción, los podemos describir como:

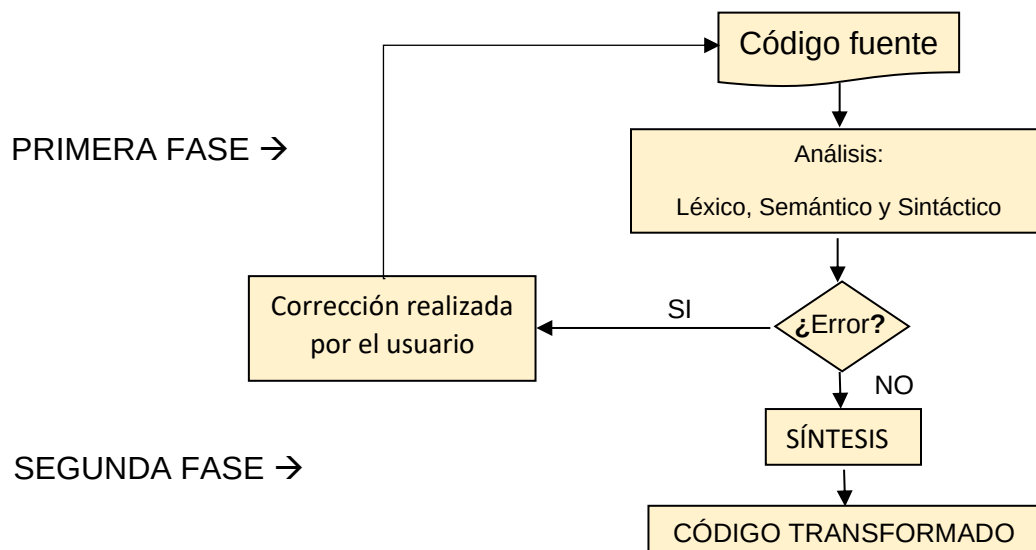
Precompiladores o Preprocesadores, Compiladores y Pseudocompiladores.

Estos conceptos se ilustran a continuación. Los compiladores también se pueden clasificar por el número de pasadas, pero todos estos tipos de compiladores requieren de otro proceso de invocación o ejecución conocido como RUN-TIME. Los dos tiempos que tiene un compilador: COMPILE-TIME y RUN-TIME es otra característica que diferencia a un compilador de un intérprete, ya que éste último solo tiene un tiempo que es una invocación y su inmediata ejecución.

Intérpretes. - Programa que conjunta el tiempo de revisión, análisis, síntesis y ejecución (RUN-TIME), en una sola orden o instrucción. Es eficiente y económico porque no requiere de grandes recursos de almacenamiento y se realiza todo en un solo tiempo.

Compiladores. - Como ya se describió, los diferentes nombres y tipos de compiladores están en función del nivel de traducción que se tiene del código fuente. Cuando el código fuente es de muy alto nivel, por ejemplo, herramientas que se conocen como 5GL (Quinta generación). Que se describirán como los diferentes tipos de lenguajes en la siguiente unidad.

Los lenguajes 5GL pueden requerir de un preproceso o etapa previa de adaptación o traducción agregada para disminuir el nivel de 5º a 4º nivel y con esto agregarle instrucciones que completen la información que la computadora requiere.





Fases y etapas que intervienen, en el proceso de compilación y elementos que intervienen en el proceso de codificación

El código transformado puede ser un conjunto de instrucciones agregadas al código fuente original, puede ser un código en alto nivel, y solo adecuarlo como es el caso de la compilación que se realiza parcialmente denominada precompilación o pseudocompilación o puede generarse un código conocido como objeto que es el lenguaje de máquina que corresponde al código fuente que el usuario programador escribió.

Sobre los niveles en los lenguajes de programación, ya se había descrito una clasificación sólo de tres niveles:

El alto nivel que corresponde a los lenguajes de programación conocidos como formales y el bajo nivel que corresponde a los lenguajes más cercanos a la arquitectura de la computadora. Pero, dada la diversidad en los lenguajes de programación actuales de los cuales ya son miles. Se han establecido niveles intermedios, como ya se mencionó el ejemplo de un nivel intermedio es el lenguaje C con la variedad de herramientas que se desprenden de este, tales como TURBO C, AWK, C++, gracias a que como ya se mencionó. El lenguaje C y sus derivados, se considera de alto nivel por su facilidad y flexibilidad y de bajo nivel por la posibilidad de administrar los espacios en memoria y su poderío en el uso de recursos computacionales, mismas características que hasta han servido como herramienta para desarrollar el sistema operativo más evolucionado con el que se cuenta actualmente, este es UNIX y sus cientos de clones (el más famoso y actual LINUX).

Otros niveles que a continuación se describen son a juicio particular y temporal, ya que este tema es polémico y no se ha llegado a criterios que se fijen como una norma o estándar para definir a los lenguajes por niveles. Algunos autores en vez de utilizar el término nivel, utilizan el concepto de generaciones de lenguajes de programación, en el entendido de que los lenguajes de programación son todos artificiales o creados por el hombre, diferentes a los lenguajes naturales que se les considera de origen Divino y muchos de los lenguajes naturales se forman de la combinación entre ellos.

Con base a lo anterior, sucede algo similar en los lenguajes de programación o artificiales, de los cuales surgen niveles intermedios a lo largo de la historia, mima que se trata de resumir a continuación:

Nivel cero o de más bajo nivel. Este nivel corresponde al lenguaje de máquina, el cual como ya se mencionó, consiste en unos y ceros integrados en códigos diversos, controlados de forma alambrada o bien con microinstrucciones.



Nivel uno corresponde al lenguaje ensamblador basado en abreviaturas, mnemónicos, mnemotécnicos o mnemoclaves del idioma inglés. Estos son dependientes del hardware y existe un tipo de lenguaje ensamblador para cada tipo de procesador.

Nivel dos o segundo nivel, corresponde al de los lenguajes de programación formales. Estos tienen estructuras y reglas gramaticales muy restrictivas, sólo permiten el uso de datos que se tienen que declarar con anterioridad a su uso (esta es una característica que se conoce como fuertemente “tipeado”) y comprenden un gran número de palabras reservadas o instrucciones particulares para cada lenguaje con formatos específicos, algunos ejemplos de estos lenguajes de segundo nivel son, el compilador de ALGOL, COBOL y FORTRAN. Aquí cabe hacer la aclaración de que algunos autores consideran a este nivel como de primera generación de los lenguajes de programación, considerando solo a los lenguajes de nivel alto como lenguajes, y los del nivel bajo tales como lenguaje de máquina y ensamblador, de bajo nivel, o bien como de nivel cero o generación cero.

Nivel tres, son lenguajes de programación informales que tienen como base la restrictiva formalidad del anterior nivel, pero incluyen estructuras gramaticales más flexibles y dinámicas, mismas que le dan mayor libertad al usuario programador. Aquí nace el lenguaje Basic, después Smalltalk, PASCAL y culmina con el lenguaje C. Al que se le considera también de nivel medio (entre alto y bajo nivel ya mencionado con anterioridad).

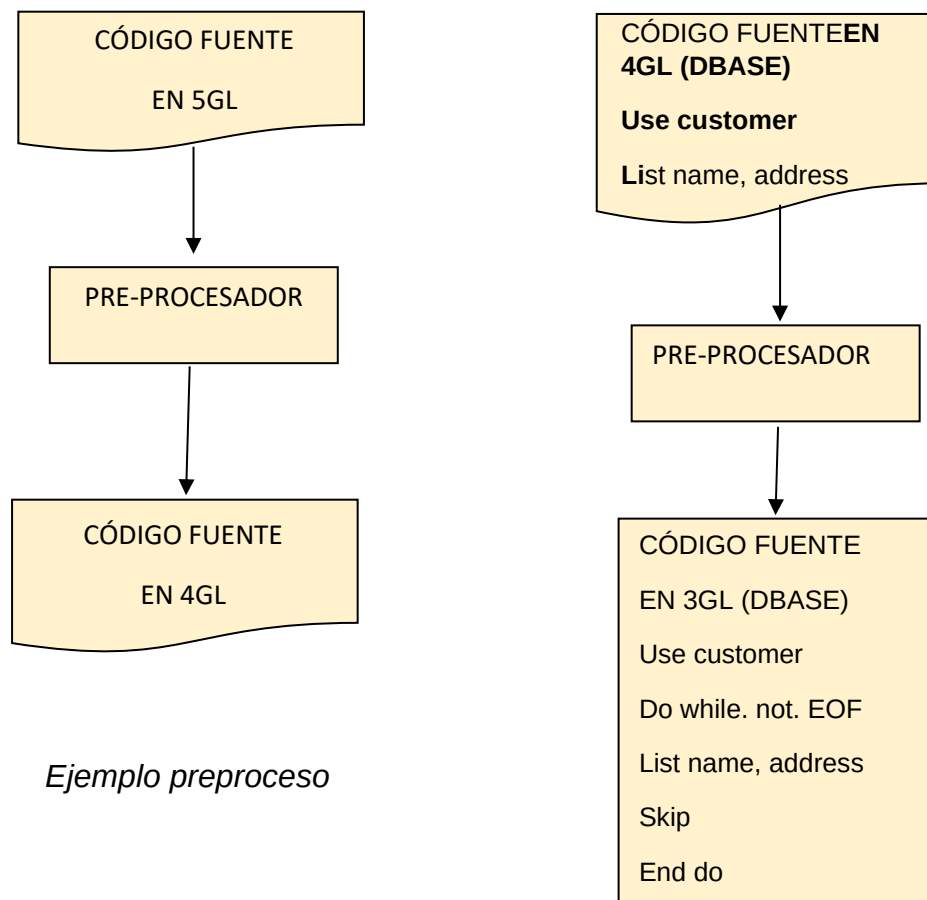
Nivel cuatro, conocidos como de cuarta generación, se abrevian como 4GL, estos lenguajes tienen como principal característica facilitarle la labor de codificar al usuario programador. Con esto se crean herramientas tales como: DBASE, SQL, INFORMIX, JAVACREATOR, BLUE JEEN, etc. Estas herramientas son generadoras de código automático. Cabe aclarar que, a partir de la tercera, cuarta y quinta generación existe mayor diversidad de opiniones en base a que no se tiene un consenso estándar sobre las características que guarda cada tipo de lenguaje, este tema se hace cada día más polémico y difícil de definir, se puede decir que al hablar de generaciones o niveles de los lenguajes de programación estamos entrando a un mundo extenso, en constante crecimiento y variabilidad.

Por lo anterior es importante aclarar que se pueden encontrar discrepancias en la forma de clasificar los lenguajes por niveles y generaciones mismas que se puede decir se manejan de forma indistinta en los libros sobre esta materia. Según apreciación personal, es más fina la clasificación de los lenguajes en base a paradigmas de programación y estos son descritos con mayor detalle después de los tipos de traductores sobre los lenguajes.

Es importante concluir que entre más alto es el nivel que alcanza el lenguaje, más pasos intermedios se deben incluir para lograr una traducción e interpretación de la computadora. Por lo anterior, quizá hasta sea necesario crear preprocesos esto es, procesos previos a la compilación conocidos como precompiladores. Este aspecto está abierto para futuros desarrollos los cuales no presentan un límite, sólo el que la creatividad del programador decida.

Ejemplo de un preprocesador o precompilador:

En el siguiente diagrama del lado izquierdo, sólo se esquematiza de forma genérica la función de un pre-proceso que es bajar el nivel del lenguaje a procesar. Este diagrama del lado derecho se puede observar en el código que se escribe para una herramienta 4GL, solo se tienen dos líneas de instrucciones y en una herramienta 3GL, se agregan más instrucciones para realizar la misma función descrita en la herramienta 4GL, que consiste en tomar la base de datos conocida como clientes “customer” y de ella obtener una lista con el nombre y la dirección de los clientes. Este ejemplo corresponde a: SQL Informix.



Ejemplo preproceso

El último ejemplo relacionado con los niveles de traducción es el caso de un pseudocompilador el cual se puede describir como un generador de código intermedio, esto es, se crea un código estándar para lograr algunas ventajas con respecto a la arquitectura de la computadora, de forma muy general tenemos en el siguiente diagrama de bloques un pseudocompilador o máquina virtual.

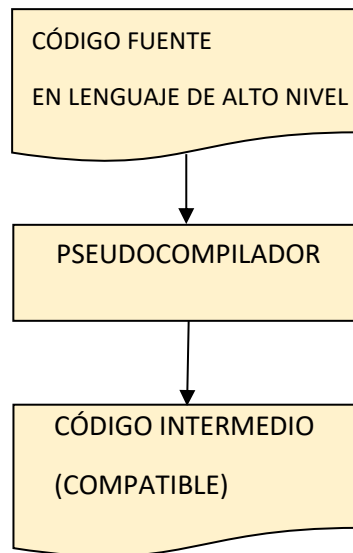


Diagrama esquemático de un pseudocompilador

En este caso el pseudocompilador es un generador de código intermedio, cuya principal ventaja es su compatibilidad. El ejemplo clásico, conocido y actual de este tipo de herramientas es la máquina virtual de JAVA, donde se tiene un generador de código intermedio conocido con BYTECODE el cual es independiente de la arquitectura de la computadora donde se corre para hacer posible que las aplicaciones corran en cualquier plataforma, lo mismo para UNIX, MACHINTOSH Y MICROSOFT. Una máquina virtual es una capa intermedia de software que se le agrega al hardware para lograr la compatibilidad y el enmascaramiento o bien el ocultamiento de las particularidades del hardware donde se ejecutan las aplicaciones diseñadas para la máquina virtual en cuestión.

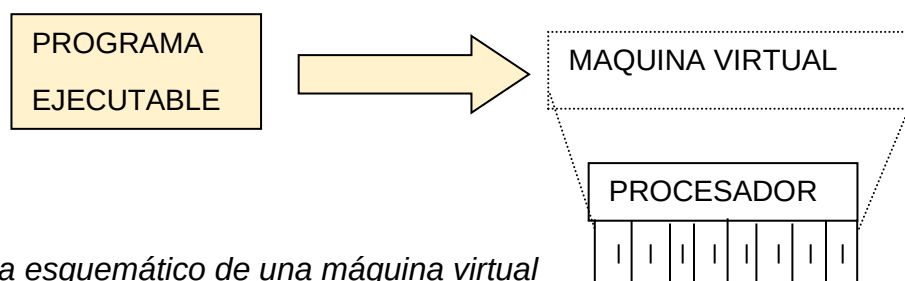


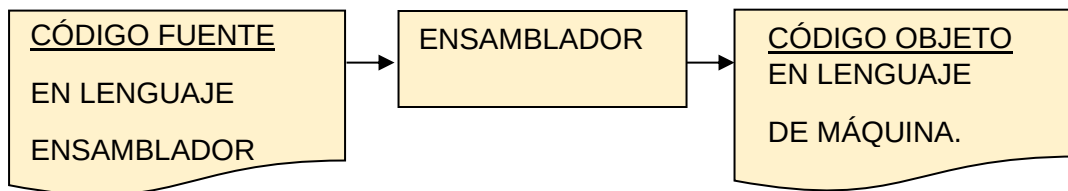
Diagrama esquemático de una máquina virtual

Gracias a los pseudocompiladores se tiene una compatibilidad y transportabilidad (fácil implementación en diferentes plataformas) de los programas que se desarrollen con generación de código intermedio. Antes de concluir con el tema de los niveles de traducción, el cual está relacionado con los diferentes tipos de interpretación, es importante remarcar que entre más bajo es el nivel del lenguaje que se utiliza en el programa fuente, más sencillo es el proceso que se realiza para obtener la traducción al lenguaje de máquina.

Otro ejemplo es el codificar un programa fuente al lenguaje ensamblador, donde se tiene solo un proceso para lograr obtener un código de máquina ejecutable. Es importante remarcar la importancia que tiene este tipo de herramientas de bajo nivel, pues siguen dando eficiencia y poderío en el uso de recursos computacionales.

Ya para concluir sobre los niveles de traducción es importante remarcar que, con el éxito de la computadora personal, se tienen muchas facilidades en cuanto a herramientas de desarrollo y paqueterías, pero es difícil, por ejemplo, pensar que se tienen microprocesadores con velocidades medibles en gigahertz y aún no se tiene un ensamblador para procesador INTEL PENTIUM. Por lo que en este tema se pretende aprovechar la necesidad de crear niveles de traducción más eficientes y realizarlos con herramientas de bajo nivel como son los ensambladores.

Ejemplo de un proceso de ensamble...



Paradigmas de los lenguajes de programación. - Para completar el tema sobre los niveles de traducción, ya se mencionó que una cualidad más detallada sobre los diferentes lenguajes que se han diseñado hasta la fecha son los paradigmas o modelos a los que responden los actuales lenguajes.

A continuación, se da una lista con los paradigmas o modelos que tienen los lenguajes actuales de programación, no sin olvidar que cada día se puede desarrollar un nuevo lenguaje o paradigma. Ya que los lenguajes que se conocen hasta la fecha corresponden a los modelos actuales o paradigmas conocidos con la tecnología que existe en este momento, pero estos generarán nuevos modelos, mismos que guardan características heredadas, tal como los hijos se parecen a sus padres.



Los paradigmas conocidos hasta la fecha son:

Imperativo. - Corresponden a la arquitectura de John Von Newmann “programa almacenado en memoria”, con instrucciones estructuradas, funciones y manejo de memoria dinámica con apuntadores a direcciones de memoria que controla el usuario. Ejecución señalizada. Ejemplos: ALGOL, COBOL y FORTRAN.

Funcional (operacional). - Los lenguajes que responden a este modelo optimizan la representación interna de los datos y le dan particular importancia al orden de las decisiones que se toman en la lógica del código. Este paradigma está basado en la optimización de los datos que se almacenan en memoria. El ejemplo más representativo de este tipo de paradigma es LISP.

J. McCarthy “Recursive Functions of Symbolic Expressions and Their Computation by Machine. Part 1. Comm ACM Vol. 3 No. 4. abril de 1960 páginas 184-195.

Orientado a objetos. - Este quizá es el paradigma más difundido y conocido para los nuevos programadores, ya que pretende facilitar la labor de codificar el detalle de cómo resolver un problema. Los lenguajes orientados a objetos proporcionan una gran cantidad de elementos con los cuales no se tiene que definir el cómo, sólo lo que se desea obtener del lenguaje y aprovechar el encapsulamiento (ocultamiento y protección del código), la inferencia, el paso de mensajes, las clases, subclases, la herencia, etc. El primer lenguaje representativo de este paradigma es el Smalltalk (en 1980), luego C++ (en 1986), el Object Pascal (en 1989), Object Cobol (en 1993) hasta el actual y exitoso Java (en 1995).

Asíncrono. - Este paradigma y el que se describe a continuación corresponde a las nuevas arquitecturas avanzadas de las computadoras, pueden optimizar las características de los microprocesadores con tipo de procesamiento paralelo o vectorial. Este paradigma corresponde a procesos asíncronos (en diferentes tiempos de ejecución) los cuales se hacen posibles mediante la comunicación entre procesos. El lenguaje de programación LINDA es un ejemplo representativo de uno de los lenguajes que agrega estas facilidades, tomando como base a los lenguajes C y FORTRAN.

Síncrono. - En este modelo se tiene un procesamiento paralelo con alto grado de acoplamiento. Debe de haber una sincronía en todos los procesos que se ejecutan al mismo tiempo. Un ejemplo de un lenguaje que corresponde a este paradigma es el ACTUS, Paralation LISP que proveen construcciones independientes de la arquitectura de la computadora.



Funcional (definicional). - Este paradigma ofrece mayores facilidades que el paradigma operacional, pues tiene una gran variedad de funciones que el usuario programador puede utilizar para simplificar su código. Un ejemplo de este tipo de lenguajes es el lenguaje de programación llamado Miranda y el Haskell. Este último es un polimorfismo estático (variedad de formas predefinidas) derivadas de Miranda y ambos lenguajes (Miranda y Haskell) se caracterizan por tener funciones de alto orden, usar datos algebraicos definidos por el usuario y un sistema de clases. También proporciona operadores con redireccionamiento dinámico (overloading), sistema funcional de entradas y salidas, arreglos funcionales y compilación por separado.

Transformacional. - Este paradigma permite definir reglas sobre argumentos y términos que se reescriben. El ejemplo representativo de este tipo de lenguaje es Bertrand, que corresponde a la característica de ser un lenguaje de especificación basado en reglas, sobre argumentos y términos que se pueden reescribir. En los lenguajes restrictivos el usuario programador debe especificar explícitamente los árboles de búsqueda y la propagación restrictiva que es la característica básica de este paradigma transformacional.

Lógico. - Este paradigma emplea la lógica y las reglas de inferencia, tratando de adecuarse al lenguaje natural. El ejemplo representativo de este paradigma es PROLOG y todos sus derivados.

Basado en formas (Form-based). - Este es el paradigma que corresponde a todas las hojas electrónicas de cálculo que se han diseñado, desde Visical, Lotus hasta MS-Excel, iWork, OpenOffice, Tables, Mesa, Mariner Calc, Papyrus Office, Ragtime 6, Neo Office Calc, ThinkFree Calc, KSpread 2.

De flujo de datos (dataflow). - Los lenguajes que corresponden a este paradigma se programan a través de elementos gráficos. Un ejemplo de este lenguaje es "Show and Tell".

Restringido (constraint). - Especifica la relación que debe permanecer. El usuario programador es el responsable de especificar la relación y el sistema es el encargado de mantenerla. Aunque no se tienen muchos lenguajes que respondan a este paradigma, un ejemplo que corresponde a este paradigma es el lenguaje visual ThigLab.

Demostracional. - Mejor conocido como de "Query by example". ¿En este paradigma se presenta el "qué quiero?" con un ejemplo y automáticamente se genera código, mismo que le indica a la computadora el cómo obtener lo que quiero con una generación inmediata de resultados.



Otro enfoque de clasificar a los traductores de forma genérica en ciencias de la computación es definir a los procesadores de lenguajes como aquellos programas destinados a trabajar sobre una entrada que por la forma como ha sido elaborada, pertenece a un lenguaje en particular reconocido o aceptado por el programa en cuestión. Los procesadores de lenguajes se clasifican como traductores o intérpretes.

Un traductor es un programa que recibe una entrada escrita en un lenguaje (el lenguaje fuente) a una salida perteneciente a otro lenguaje (el lenguaje de bajo nivel), conservando su significado. En términos computacionales esto significa que tanto la entrada como la salida sean capaces de producir los mismos resultados. Un intérprete, por otra parte, no lleva a cabo tal transformación; en su lugar obtiene los resultados conforme va analizando la entrada. Los traductores son clasificados en compiladores, ensambladores y preprocesadores.

- **Traductor:** Es una máquina teórica que tiene como entrada un texto escrito en un lenguaje L1 y como salida un texto escrito en un lenguaje L2. Habitualmente se denomina a L1 lenguaje fuente y a L2 lenguaje objeto. Las técnicas que se desarrollan en esta asignatura no sólo son válidas para la implementación de compiladores, sino que son aplicables en general a todos los sistemas de procesamiento de lenguajes y de traducción. Estos sistemas pueden ser de distintos tipos:
- **Traductores de lenguaje natural:** Serían los que tradujeran un lenguaje natural en otro (por ejemplo, español a inglés). Esto en la actualidad no se ha conseguido debido fundamentalmente a la ambigüedad del lenguaje natural. Los mayores logros en la materia siempre trabajan con un subconjunto del lenguaje natural, limitando las construcciones sintácticas válidas y/o el vocabulario. Este tema se aborda generalmente mediante técnicas de inteligencia artificial.
- **Compilador:** Es un traductor que convierte un texto escrito en un lenguaje fuente de alto nivel en un programa objeto en código máquina.
- **Intérprete:** Es un traductor que realiza la operación de compilación paso a paso. Para cada sentencia que compone el texto de entrada, se realiza una traducción, ejecuta dicha sentencia y vuelve a iniciar el proceso con la sentencia siguiente. La principal ventaja del proceso de compilación frente al de interpretación es que los programas se ejecutan mucho más rápidamente una vez compilados; por el contrario, es más cómodo desarrollar un programa mediante un intérprete que mediante un compilador puesto que en el intérprete las fases de edición y ejecución están más integradas. La depuración de los programas suele ser más fácil en los intérpretes que en los compiladores puesto que el código fuente está presente durante la ejecución.



Estas ventajas pueden incorporarse al compilador mediante la utilización de entornos de desarrollo y depuradores simbólicos en tiempo de ejecución. Preprocesadores: Procesan un texto fuente modificándolo en cierta forma previamente a la compilación. Por ejemplo, muchos compiladores admiten un conjunto de macroinstrucciones ajenas al lenguaje en sí que indican al compilador si tiene que incluir algún fichero externo, si ha de hacer o no un listado completo de la compilación, etc.

Conversores Fuente-Fuente: (LCP) Traducen un lenguaje fuente de alto nivel a otro (por ejemplo, PASCAL $\rightarrow$ C). Una aplicación interesante de la traducción fuente-fuente es el desarrollo e implementación de prototipos de nuevos lenguajes de programación. Así, por ejemplo, si se desea definir un lenguaje especializado puede implementarse rápidamente mediante su traducción a un lenguaje convencional de alto nivel.

Rutinas de análisis de instrucciones: El conjunto de instrucciones del entorno de un sistema operativo constituye un lenguaje que debe ser analizado previamente para realizar las acciones oportunas. Igualmente, ciertos programas como editores de texto, sistemas de diseño asistido, etc., utilizan instrucciones complejas que deben interpretarse adecuadamente. Ensambladores: Son compiladores cuyo lenguaje de entrada, llamado ensamblador, permite la traducción de cada sentencia fuente a una instrucción en código máquina. Compilador cruzado: Es el que genera un código objeto ejecutable en un ordenador distinto de aquél en el que se realiza la compilación.

Compilación-Montaje-Ejecución: En las aplicaciones grandes es conveniente fragmentar el programa a realizar en módulos que se compilan por separado, y una vez que estos estén compilados unirlos mediante un programa denominado montador para formar el módulo ejecutable. El montador se encarga, a su vez, de incluir las librerías donde están guardadas las funciones predefinidas de uso común.

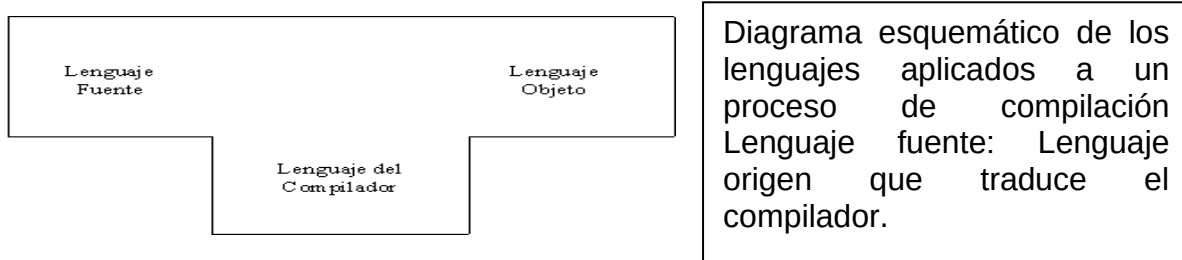
Compilación en una o varias pasadas: Se llama pasada a cada lectura que hace el compilador del texto fuente. Compilación incremental. Este compilador actúa de la siguiente manera. Compila un programa fuente. Caso de detectar errores al volver a compilar el programa corregido sólo compila las modificaciones que se han hecho respecto al primero.

Auto compilador: Es aquél que está escrito en el mismo lenguaje que se pretende compilar. Supongamos, por ejemplo, que queremos desarrollar la versión 2.0 de un compilador Pascal. Dicho compilador generará un código mucho más rápido y eficiente que el que generaba la versión anterior 1.0. Sin embargo, son ya muchas las máquinas (IBM 370, Serie 1, PDP 11, ...) que disponen del antiguo compilador, o que al menos tienen otro compilador Pascal.

La mejor opción consiste en escribir el nuevo compilador en Pascal, ya que así podrá (el compilador) ser compilado en las distintas máquinas por los compiladores Pascal ya existentes.

- **Metacompilador:** Es un traductor que tiene como entrada la definición de un lenguaje y como salida el compilador para dicho lenguaje.
- **Decompilador:** Es el que traduce código máquina a lenguaje de alto nivel. Los decompiladores más usuales son los desensambladores, que traducen un programa en lenguaje máquina a otro en ensamblador.
- **Bootstrapping.** Es una técnica muy usada actualmente para el desarrollo de compiladores de lenguajes de alto nivel, en especial si se quiere obtener un auto-compilador, o sea, un compilador que se compile a sí mismo.

Para describir el proceso de autocomparación se emplea la notación en T que representa gráficamente los tres lenguajes implicados en el proceso de compilación:



- **Lenguaje objeto:** Lenguaje meta, al cual traduce el compilador.
- **Lenguaje del compilador:** Lenguaje en el que está escrito el compilador.

Supongamos que se quiere implementar un nuevo lenguaje $A(N)$ en una máquina determinada. Disponemos solamente de un ensamblador para dicha máquina. En principio parece que la solución es escribir un compilador en lenguaje ensamblador que traduzca desde el lenguaje $A(N)$ al lenguaje máquina LM .

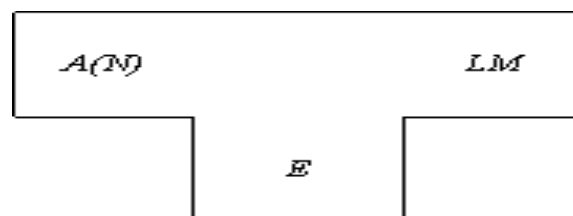


Diagrama esquemático.

Esto en la práctica resulta bastante complicado, dado que programar en ensamblador es muy laborioso.

Lo que se hace en estos casos es desarrollar un lenguaje restringido A (1), parecido al A(N) pero más simple, y para este lenguaje escribir el compilador en ensamblador, o en cualquier otro lenguaje soportado por la máquina.

Una vez construido este compilador, y dado que nuestra máquina es ya capaz de entender el lenguaje A (1), se puede desarrollar un compilador para otro lenguaje A (2) escribiéndolo en el lenguaje A (1), y así sucesivamente hasta llegar a obtener un auto compilador del lenguaje A(N). Esta técnica se conoce como bootstrapping.

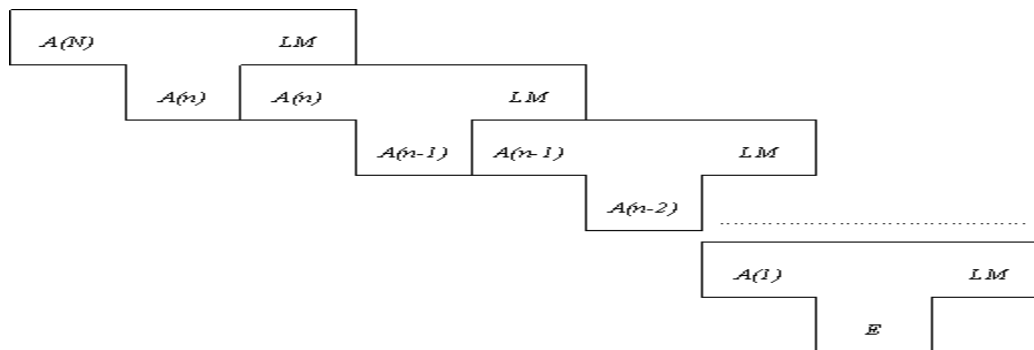


Diagrama esquemático de Lenguaje fuente: Lenguaje origen que traduce el compilador.

El primer compilador de Pascal desarrollado en Zurich por Wirth fue posible gracias a esta técnica. El conocido compilador C de GNU emplea también este mecanismo en tres pasos.

1.4 Fases y pasos del compilador.

Estructura de un compilador.

Un compilador es un programa, en el que pueden distinguirse dos subprogramas o fases principales: una fase de análisis, en la cual se lee el programa fuente y se estudia la estructura y el significado de este; y otra fase de síntesis, en la que se genera el programa objeto.

En un compilador pueden distinguirse, además, algunas estructuras de datos comunes, la más importante de las cuales es la tabla de símbolos, junto con las funciones de gestión de ésta y de los demás elementos del compilador, y de una serie de rutinas auxiliares para detección de errores.

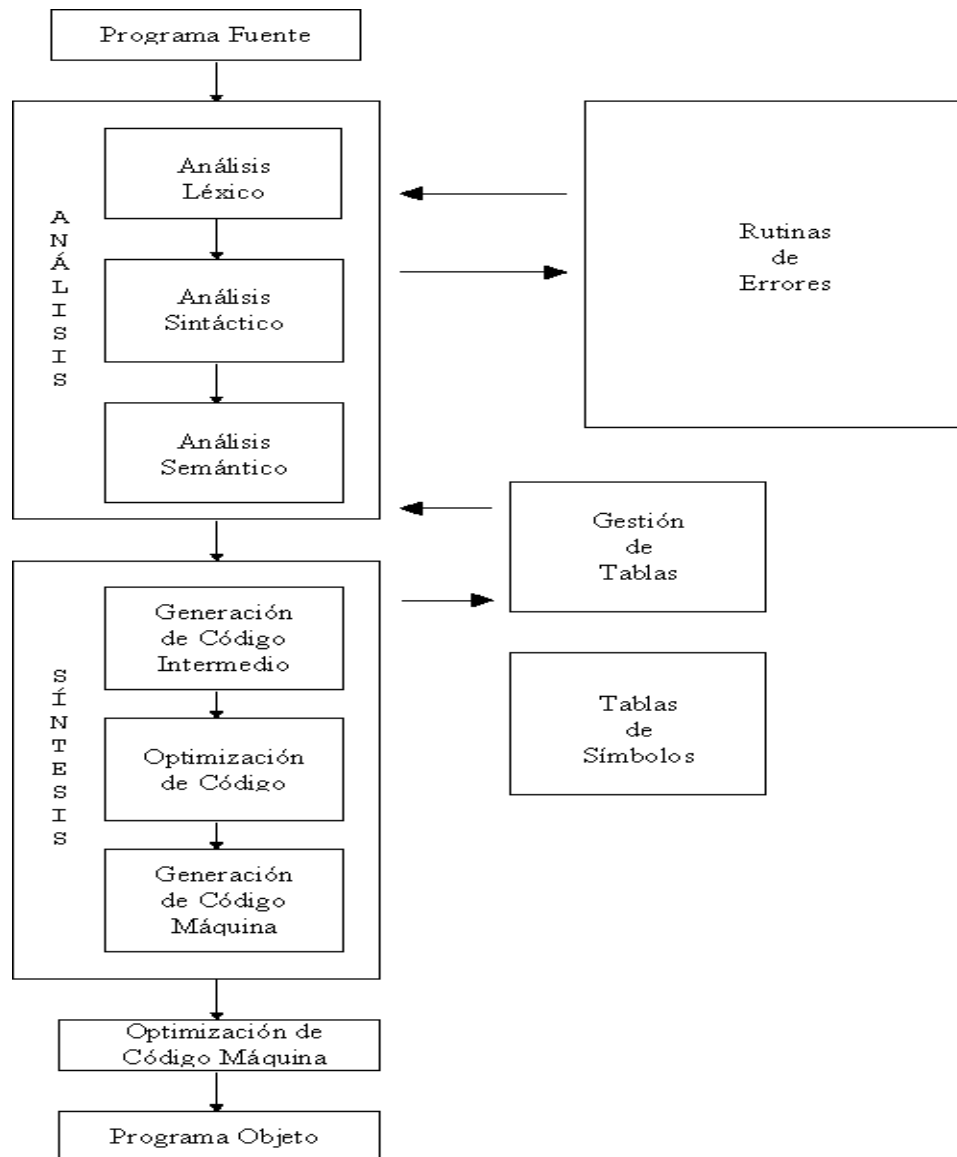


Diagrama esquemático de las fases y etapas que integran un compilador.



Esquema de un compilador.

Las funciones de estos módulos son las siguientes:

Analizador lexicográfico: Las principales funciones que realiza son:

- Identificar los símbolos.
- Eliminar los blancos, caracteres de fin de línea, etc...
- Eliminar los comentarios que acompañan al código fuente.
- Crear unos símbolos intermedios llamados **tokens**.
- Avisar de los errores que detecte.

Ejemplo: A partir de la sentencia en PASCAL:

nuevo: = viejo + RAZON*2

Genera un código simplificado para el análisis sintáctico posterior, por ejemplo:

<id1> <:=> <id2> <+> <id3> <*> <ent>

Nota: Cada elemento encerrado entre <> representa un único **token**. Las abreviaturas **id** y **ent** significan identificador y entero, respectivamente.

Analizador Léxico. Comprueba que las sentencias que componen el texto fuente son correctas en el lenguaje, creando una representación interna que corresponde a la sentencia analizada. De esta manera se garantiza que sólo serán procesadas las sentencias que pertenezcan al lenguaje fuente. Durante el análisis sintáctico, así como en las demás etapas, se van mostrando los errores que se encuentran.

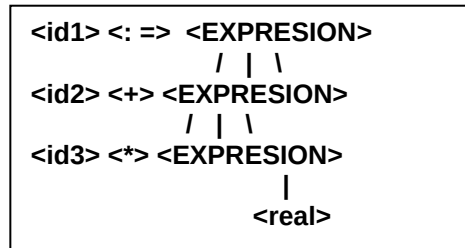
Ejemplo: El esquema de la sentencia anterior corresponde al de una sentencia de asignación del lenguaje Pascal. Estas sentencias son de la forma:

<id> <:=> <EXPRESION>

y la parte que se denomina <EXPRESION> es de la forma:

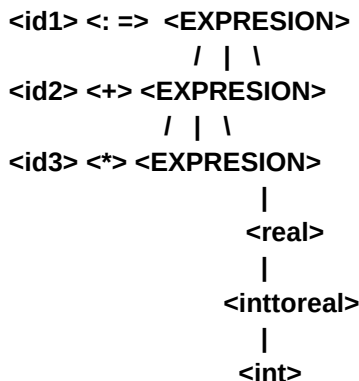
- **<id> <+> <EXPRESION> o bien**
- **<id> <*> <EXPRESION> o bien**
- **<real>**

La estructura de la sentencia queda, por tanto, de manifiesto mediante el siguiente esquema:



Análisis semántico: Se ocupa de analizar si la sentencia tiene algún significado. Se pueden encontrar sentencias que son sintácticamente correctas pero que no se pueden ejecutar porque carecen de sentido. En general, el análisis semántico se hace a la par que el análisis sintáctico introduciendo en éste unas *rutinas semánticas*.

Ejemplo: En la sentencia que se ha analizado existe una variable entera. Sin embargo, las operaciones se realizan entre identificadores reales, por lo que hay dos alternativas: o emitir un mensaje de error "Discordancia de tipos", o realizar una conversión automática al tipo superior, mediante una función auxiliar *inttoreal*.



Generador de código intermedio: El *código intermedio* es un código abstracto independiente de la máquina para la que se generará el código objeto. El código intermedio ha de cumplir dos requisitos importantes: ser fácil de producir a partir del análisis sintáctico, y ser fácil de traducir al lenguaje objeto. Esta fase puede no existir si se genera directamente código máquina, pero suele ser conveniente emplearla.



Ejemplo: Consideremos, por ejemplo, un *código intermedio de tercetos*, llamado así porque en cada una de sus instrucciones aparecen como máximo tres operandos. La sentencia traducida a este código intermedio quedaría:

temp1: = inttoreal (2) temp2: = id3 * temp1temp3: = id2 + temp2id1: = temp3

Optimizador de código: A partir de todo lo anterior crea un nuevo código más compacto y eficiente, eliminando por ejemplo sentencias que no se ejecutan nunca, simplificando expresiones aritméticas, etc. La profundidad con que se realiza esta optimización varía mucho de unos compiladores a otros. En el peor de los casos esta fase se suprime.

Ejemplo: Siguiendo con el ejemplo anterior, es posible evitar la función *inttoreal* mediante el cambio de 2 por 2.0, obviando además una de las operaciones anteriores. El código optimizado queda como sigue:

temp1: = id3 * 2.0id1: = id2 + temp1

Generador de código: A partir de los análisis anteriores y de las tablas que estos análisis van creando durante su ejecución produce un código o lenguaje objeto que es directamente ejecutable por la máquina. Es la fase final del compilador. Las instrucciones del código intermedio se traducen una a una en código máquina reubicable.

Nota: Cada instrucción de código intermedio puede dar lugar a más de una de código máquina.

Ejemplo: El código anterior traducido a ensamblador DLX quedaría:

LW R1, id3 MUL R1, R1,2 LW R2, id2 ADD R2, R2, R1 SW id1, R2

En donde id1, id2 y id3 representan las posiciones de memoria en las que se hallan almacenadas estas variables; R1 y R2 son los registros de la máquina; y las instrucciones LW, SW, MUL y ADD representan las operaciones de colocar un valor de memoria en un registro, colocar un valor de un registro en memoria, multiplicar en punto flotante y sumar, respectivamente.

La tabla de símbolos: Es el medio de almacenamiento de toda la información referente a las variables y objetos en general del programa que se está compilando.



Ejemplo: Hemos visto que en ciertos momentos del proceso de compilación debemos hacer uso de cierta información referente a los identificadores o los números que aparecen en nuestra sentencia, como son su tipo, su posición de almacenamiento en memoria, etc. Esta información es la que se almacena en la tabla de símbolos.

Rutinas de errores: Están incluidas en cada uno de los procesos de compilación (análisis lexicográfico, sintáctico y semántico), y se encargan de informar de los errores que encuentran en texto fuente.

Ejemplo: El analizador semántico podría emitir un error (o al menos un aviso) cuando detectase una diferencia en los tipos de una operación.

ENSAMBLADOR:

Un ensamblador es el programa encargado de llevar a cabo un proceso denominado de ensamble o ensamblado. Este proceso consiste en que, a partir de un programa escrito en lenguaje ensamblador, se produzca el correspondiente programa en lenguaje máquina (sin ejecutarlo), realizando:

- La integración de los diversos módulos que conforman al programa.
- La resolución de las direcciones de memoria designadas en el área de datos para el almacenamiento de variables, constantes y estructuras complejas; así como la determinación del tamaño de éstas.
- La identificación de las direcciones de memoria en la sección de código correspondientes a los puntos de entrada en saltos condicionales e incondicionales junto con los puntos de arranque de las subrutinas.
- La resolución de los diversos llamados a los servicios o rutinas del sistema operativo, código dinámico y bibliotecas de tiempo de ejecución.
- La especificación de la cantidad de memoria destinada para las áreas de datos, código, pila y montículo necesaria y otorgada para su ejecución.
- La incorporación de datos y código necesarios para la carga del programa y su ejecución.

Turbo Assembler. - De Borland Intl., es muy superior al Turbo Editasm. Trabaja de la misma forma, pero proporciona una interfaz mucho más fácil de usar y un mayor conjunto de utilerías y servicios.



Un precompilador, también llamado preprocesador, es un programa que se ejecuta antes de invocar al compilador. Este programa es utilizado cuando el programa fuente, escrito en el lenguaje que el compilador es capaz de reconocer (de aquí en adelante denominado lenguaje anfitrión-- en inglés “host language”), incluye estructuras, instrucciones o declaraciones escritas en otro lenguaje (el lenguaje empotrado-- en inglés “embeded language”). El lenguaje empotrado es siempre un lenguaje de nivel superior o especializado (Por ejemplo, de consulta, de cuarta generación, simulación, cálculo numérico o estadístico, etcétera). Siendo que el único lenguaje que el compilador puede trabajar es aquel para el cual ha sido escrito, todas las instrucciones del lenguaje empotrado deben ser traducidas a instrucciones del lenguaje anfitrión para que puedan ser compiladas. Así pues un precompilador también es un traductor.

Los precompiladores son una solución rápida y barata a la necesidad de llevar las instrucciones de nuevos paradigmas de programación (Por ejemplo, los lenguajes de cuarta generación), extensiones a lenguajes ya existentes (como el caso de C y C++) y soluciones de nivel conceptual superior (por ejemplo, paquetes de simulación o cálculo numérico) a código máquina utilizando la tecnología existente, probada, optimizada y confiable (lo que evita el desarrollo de nuevos compiladores). Facilitan la incorporación de las nuevas herramientas de desarrollo en sistemas ya elaborados (por ejemplo, la consulta a bases de datos relacionales substituyendo las instrucciones de acceso a archivos por consultas en SQL). Resulta común encontrar que el flujo de proceso en los lenguajes de cuarta generación o de propósito especial puede resultar demasiado inflexible para su implantación en los procesos de una empresa, flujos de negocio o interacción con otros elementos de software y hardware, de aquí que se recurra o prefiera la creación de sistemas híbridos soportados en programas elaborados en lenguajes de tercera generación con instrucciones empotradas de nivel superior o propósito especial.

Un pseudocompilador es un programa que actúa como un compilador, salvo que su producto no es ejecutable en ninguna máquina real sino en una máquina virtual. Un pseudocompilador toma de entrada un programa escrito en un lenguaje determinado y lo transforma a una codificación especial llamada código de byte. Este código no tendría nada de especial o diferente al código máquina de cualquier microprocesador salvo por el hecho de ser el código máquina de un microprocesador ficticio. Tal procesador no existe, en su lugar existe un programa que emula a dicho procesador, de aquí el nombre de máquina virtual.

La ventaja de los pseudocompiladores que permite tener tantos emuladores como microprocesadores reales existan, pero sólo se requiere un compilador para producir código que se ejecutará en todos estos emuladores. Este método es una de las respuestas más aceptadas para el problema del tan ansiado lenguaje universal o código portable independiente de plataforma.



Un intérprete es un programa que ejecuta cada una de las instrucciones y declaraciones que encuentra conforme va analizando el programa que le ha sido dado de entrada (sin producir un programa objeto o ejecutable). La ejecución consiste en llamar a rutinas ya escritas en código máquina cuyos resultados u operaciones están asociados de manera unívoca al significado de las instrucciones o declaraciones identificadas.

Los intérpretes son útiles para el desarrollo de prototipos y pequeños programas para labores no previstas. Presentan la facilidad de probar el código casi de manera inmediata, sin tener que recurrir a la declaración previa de secciones de datos o código, y poder hallar errores de programación rápidamente. Resultan inadecuados para el desarrollo de complejos o grandes sistemas de información por ser más lentos en su ejecución.

Los programas interpretados suelen ser más lentos que los compilados, pero los intérpretes son más flexibles como entornos de programación y depuración. Comparando su actuación con la de un ser humano, un compilador equivale a un traductor profesional que, a partir de un texto, prepara otro independiente traducido a otra lengua, mientras que un intérprete corresponde al intérprete humano, que traduce de viva voz las palabras que oye, sin dejar constancia por escrito. Los intérpretes de instrucciones pueden tener Interfaces Gráficas de usuario en inglés “GUI’s” (Graphical User Interface), se pueden usar desde el cmd (command) del sistema operativo o bien la consola de operación también en archivos por lotes o conjunto de instrucciones conocidos como “.bat” (batch).

COMPILADOR:

Un compilador es un programa que recibe como entrada un programa escrito en un lenguaje de nivel medio o superior (el programa fuente) y lo transforma a su equivalente en lenguaje ensamblador (el programa objeto), e inclusive hasta lenguaje máquina (el programa ejecutable) pero sin ejecutarlo. Un compilador es un traductor. La forma de como llevará a cabo tal traducción es el objetivo central en el diseño de un compilador.

1.5 Traductor simple orientado a la sintaxis y el análisis semántico.

CONCEPTO

Un *compilador* es un programa que recibe como entrada un programa escrito en un lenguaje X y deja otro programa escrito en un lenguaje Y.

Es usual que se realicen compiladores que logren la traducción de un lenguaje de nivel medio o superior (el programa fuente) y lo transforma a su equivalente en lenguaje ensamblador (el programa objeto), e inclusive hasta lenguaje máquina (el programa ejecutable) pero sin ejecutarlo. Un compilador es un traductor. La forma de cómo llevará a cabo tal traducción es el objetivo central en el diseño de un compilador.

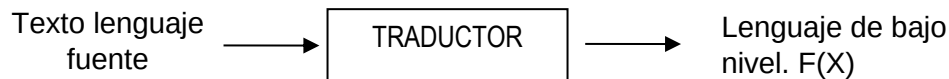


Diagrama esquemático, simplificado de un traductor

PROCESO DE INTERPRETACIÓN

Aunque es equivocado, es común encontrar referencias en documentación de productos, publicidad y textos (inclusive escuchar a la gente del medio informático), utilizando los términos **traductor, compilador e intérprete** de una forma libre e indistinta. Estas palabras no se utilizan para identificar de manera genérica a un programa que nos permitiría poder programar una computadora. Debemos ser precisos al emplear estas palabras, ya que se refieren a programas de distinta naturaleza que realizan labores encaminadas a un objetivo específico y particular. Aunque la conducta manifestada pueda ser similar, su comportamiento interno definitivamente es diferente.

Genéricamente hablando, en ciencias de la computación, los procesadores de lenguajes son aquellos programas destinados a trabajar sobre una entrada que, por la forma en la que ha sido elaborada, pertenece a un lenguaje en particular reconocido o aceptado por el programa en cuestión. Los procesadores de lenguajes se clasifican como traductores o **intérpretes**.

Un *traductor* es un programa que recibe una entrada escrita en un lenguaje (el lenguaje fuente), a una salida perteneciente a otro lenguaje (el lenguaje objeto), conservando su significado. En términos computacionales esto significa que tanto la entrada como la salida sean capaces de producir los mismos resultados. Un intérprete, por otra parte, no lleva a cabo tal transformación; en su lugar obtiene los resultados conforme va analizando la entrada. Los traductores son clasificados en **compiladores, ensambladores y preprocesadores**.



Un *ensamblador* es el programa encargado de llevar a cabo un proceso denominado de ensamble o ensamblado.

PROCESO DE COMPILACIÓN

Lo primero que hace es el reconocimiento de caracteres para identificar que las instrucciones que introdujo el usuario programador estén léxica y sintácticamente de acuerdo con el lenguaje de programación.

Después realiza un manejo de errores y consulta a la tabla de símbolos para avisar que el usuario programador realice su corrección.

Una vez corregidos los errores y comprobada la fase de análisis, se realiza una traducción o conversión de un código a otro cercano a la máquina en la fase de síntesis.

El código que se produce de este proceso es estático, pues el código que se genera en la traducción tiene posteriormente que invocarse o ejecutarse para obtener una acción o resultado.

En la invocación del código se tiene una alta eficiencia, pues éste ya está en lenguaje de máquina y adecuado para obtener una respuesta más eficiente, que la que se obtiene en un intérprete.

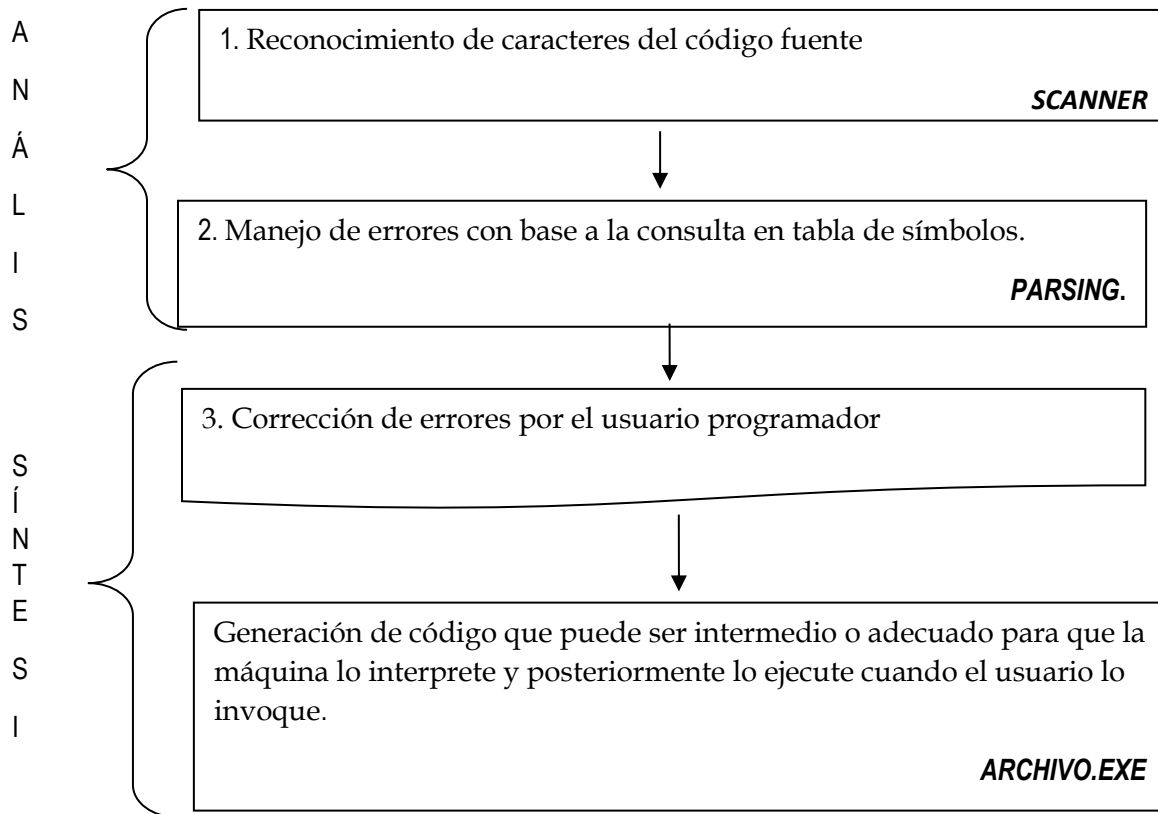
Lo anterior se trata de esquematizar en el siguiente diagrama.

En la fase de síntesis se realiza la generación de código y en algunos compiladores se da también en esta fase la optimización de código.

Independientemente de estas dos fases se puede requerir un pre-proceso para adecuar el código fuente. Y en ambas fases se consulta la tabla de símbolos para el manejo de errores. Esto se muestra en la siguiente Figura.

Fases y Componentes. - Como ya se describió un compilador puede tener dos fases: La fase de análisis y la de síntesis.

Dentro de la fase de análisis está el análisis léxico, sintáctico y semántico (conceptos que se describen más adelante). Con un manejo de errores y consulta en la tabla de símbolos. Diagrama esquemático de las dos fases de un compilador.



Análisis y Síntesis. - Estas dos actividades, dependiendo de la fuente de información que se consulte, se pueden encontrar definidas como fases o bien como etapas. Para efectos de este material, se describirán las dos como fases básicas, mismas que requiere un compilador. Dentro de la fase de análisis también definiremos a los tres tipos de análisis como etapas a cada una de ellas. Representadas con rectángulos en el diagrama anterior.

La fase de análisis permite separar cada uno de los símbolos o caracteres que integran el código fuente, les asigna un componente léxico conocido como **token**, que es código numérico que facilita el chequeo posterior; se toma de la tabla de símbolos para su posterior procesamiento; donde se realiza el análisis sintáctico con base a las gramáticas del lenguaje y posteriormente el análisis semántico que da un significado a los conjuntos de **tokens** para después realizar la conversión en la siguiente fase.

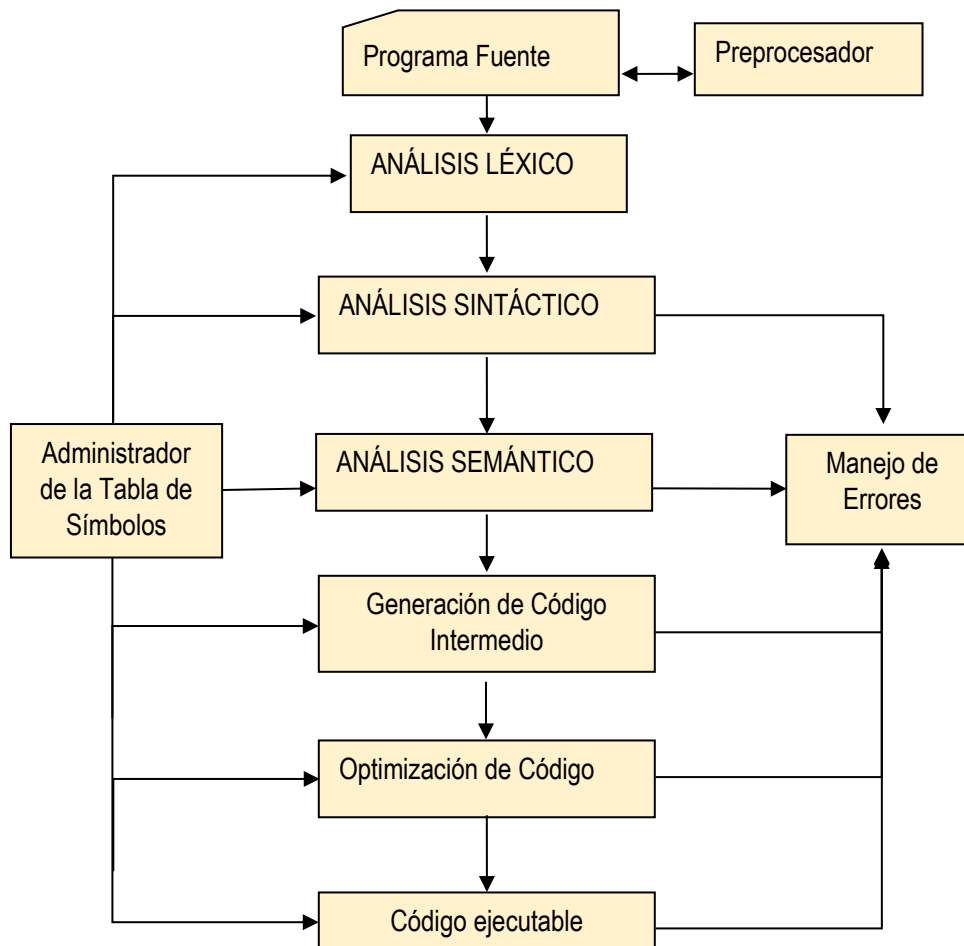


Diagrama esquemático de las fases y etapas de un compilador

En la fase de síntesis se toman los elementos por separado, ya aceptados, y se genera código con ellos, en los que se le agregan códigos adecuados a la máquina. Se tratará de esquematizar esos pasos de análisis y síntesis en el siguiente ejemplo.

Ejemplo. - Tomemos la siguiente cadena de una sencilla instrucción... PRINT X

- **Análisis Léxico.** - Es recibir la cadena PRINT y al identificar cada uno de los caracteres, consulta la tabla de símbolos, si es correcto y se encuentra en la tabla de símbolos: le genera un **token** para este caso, de manera hipotética se asigna como **token** el 503 y se da de alta el identificador X como variable, también asignándole un **token**, para este ejemplo es 001.
- **Análisis Sintáctico.** - Se tiene como entrada dos **tokens**...503 001. Con base a la búsqueda en el orden definido como parsing de la gramática que acepta el lenguaje, tenemos que la cadena puede ser aceptada y pasar al



siguiente análisis o bien, puede ser rechazada y generar un error de sintaxis (es decir, un orden no aceptado entre los dos elementos (el “PRINT” y la “X”). Un error de sintaxis puede ser X PRINT o bien PRNT X.

- **Análisis Semántico.** - Una vez que se pasa el análisis léxico y sintáctico se da un significado a los conjuntos de tokens aceptados y se le da un significado en otro lenguaje más cerca de la computadora. El significado se toma de la tabla de símbolos donde se obtiene un código equivalente pero más detallado.
- **Generación de Código.** - Para este ejemplo, de dos cadenas en una sola instrucción, se toma un código equivalente en un ensamblador ficticio que puede ser:

Load X,100	Carga el contenido de la variable X en la localidad de memoria numero 100
Load 100, BX	Carga el contenido de la localidad 100 en el registro interno de memoria a nivel segmento BX (según lo requiere la rutina de interrupción que emitirá la salida a impresión)
Int 21	Maneja la salida de la variable X.

- **Optimización de Código.** - Siguiendo con el mismo ejemplo de una sola instrucción de impresión y con una emulación simulada queda...

LOAD X, BX	Carga el contenido de la variable X en el registro base (a nivel segmento BX)
INT 21	Imprime el contenido del segmento base BX.

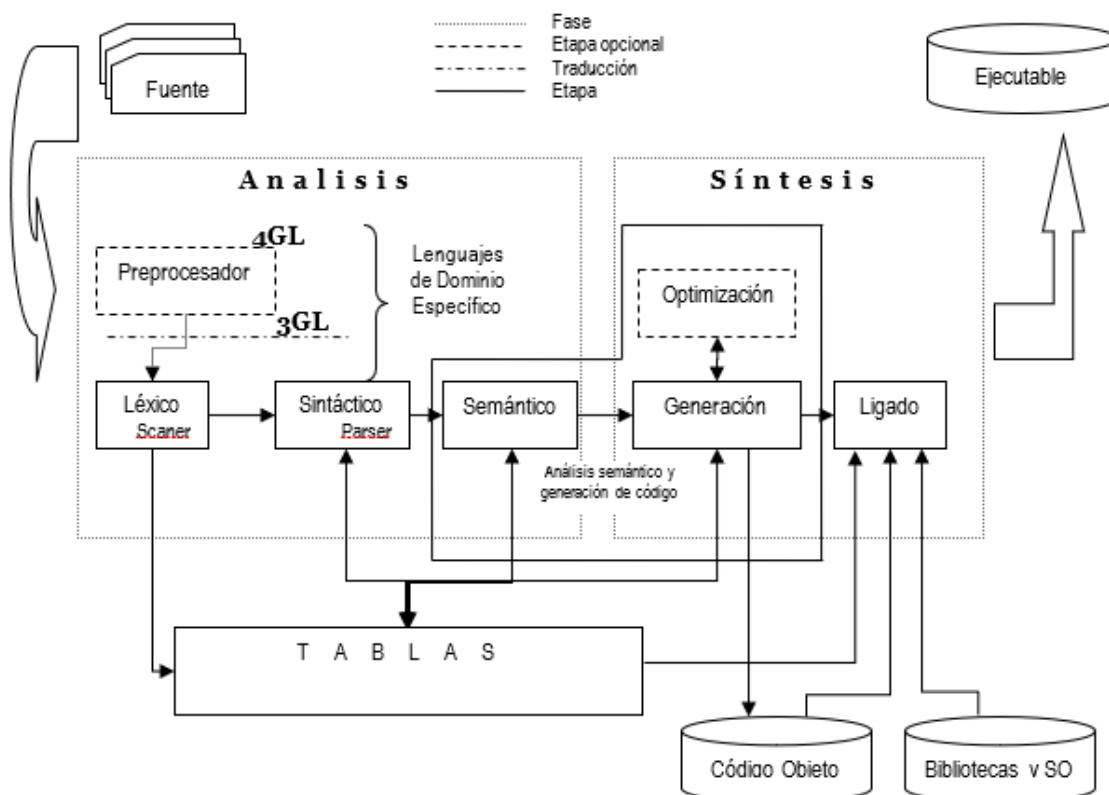
La siguiente figura presenta un diagrama que ilustra de forma general las dos fases que componen al proceso de compilación y cada fase desglosada en sus etapas componentes. En una primera fase de análisis el programa es descompuesto en sus elementos fundamentales.

En la posterior fase de síntesis es construido el programa ejecutable correspondiente a los elementos identificados en la fase previa. El proceso es irreversible e inclusive puede considerarse destructivo, ya que no hay forma de reconstruir el programa fuente a partir del ejecutable.

Etapas del proceso de compilación.

Únicamente por facilidad descriptiva, el proceso es presentado con las etapas que componen a cada fase perfectamente diferenciadas y separadas; aunque en la práctica los límites no son estrictamente bien definidos; en la práctica se ha demostrado que productos muy rápidos y eficientes pueden ser desarrollados alterando el orden de algunas etapas o entremezclándolas. La entrada a este proceso es por supuesto el programa fuente. Por lo general este es un archivo que es creado por el usuario como un texto ASCII con o sin un formato específico, aunque también puede ser el resultado de algún otro proceso. A partir de este archivo, diversos pasos pueden ser llevados a cabo:

Preprocesamiento. - Un preprocesador es la estrategia generalmente adoptada como solución a lenguajes huéspedes, extensiones, lenguajes 4GL, o lenguajes de dominio específico. El preprocesador es un traductor encargado de transformar dichas instrucciones a instrucciones del lenguaje anfitrión, sobre las cuales finalmente trabajará el compilador. Esta etapa es definitivamente opcional.



El anterior es un Diagrama esquemático completo de proceso de compilación

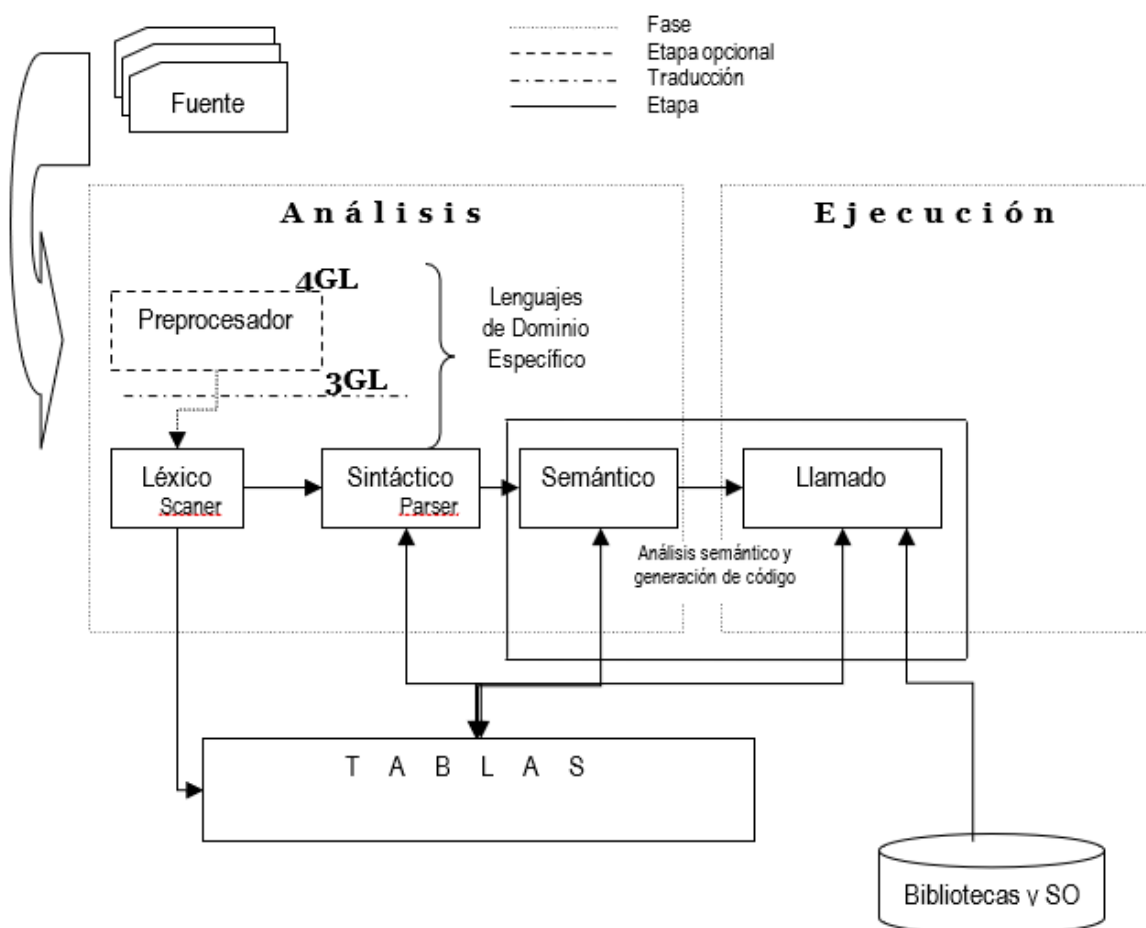


- **Análisis Léxico.** - En esta fase, la cadena de caracteres que conforma al programa fuente es despojada de comentarios, espacios en blanco y otros elementos superfluos. El programa encargado de hacer esto es conocido como un scanner, y de aquí que al proceso se le refiera comúnmente como *scanning* (exploración). Durante esta fase se identifican los elementos gramaticales usados en la creación del programa. Cada elemento identificado es substituido por un código numérico conocido como *token*.
- **Análisis Sintáctico.** - La cadena de *tokens* resultante es alimentada a un programa conocido como *parser*. El *parser* es el encargado de verificar que la secuencia y disposición de los *tokens* corresponda con la sintaxis del lenguaje. Este proceso de verificación sintáctica es conocido como *parsing* y es completamente guiado por la gramática del lenguaje.
- **Análisis Semántico y Generación de Código.** - Una vez que la secuencia de *tokens* ha sido validada, ésta es utilizada para identificar el sentido de la acción a realizar y generar el correspondiente código en lenguaje máquina. Algunos compiladores recurren a la creación de código intermedio para posteriormente generar la secuencia de instrucciones de máquina necesarias, mientras que algunos otros proceden a la generación directa del código máquina.
- **Optimización de Código.** - Esta es otra etapa opcional. La optimización de código es una actividad que raya en un arte dominado solamente por un experimentado programador de ensamblador y conocedor de la arquitectura del computador. Existen algunas técnicas desarrolladas al respecto, pero nada supera a la experiencia de un hábil programador. En esta etapa, ya sea posteriormente o trabajando al unísono con el generador de código, secuencias de instrucciones y estructuras de datos son examinadas buscando su substitución con secuencias, instrucciones o estructuras más cortas, rápidas o eficientes.
- **Ligado.** - Como paso final, todas las referencias pendientes de resolver sobre rutinas, módulos, bibliotecas y demás porciones de código necesarias para el funcionamiento del programa son cubiertas en esta parte. La resolución puede consistir desde el proporcionar meramente una dirección o llamado a una función hasta la inclusión de enormes porciones de código.

Al final, como producto de todo este proceso, lo que se obtiene es un programa escrito en código máquina que puede ser cargado en memoria y ejecutado.

El proceso seguido por un intérprete es ligeramente diferente, ya que mientras que cubre todas las etapas de análisis no cuenta con una fase de síntesis. Un intérprete no genera código, se limita a invocar rutinas ya escritas (proceso muchas veces llamado de interpretación). La siguiente figura ilustra esto el *diagrama esquemático de las etapas del proceso de interpretación*

En el caso de un pseudocompilador, cuyo caso mejor conocido es el de Java, la diferencia consiste en el código generado. Mientras que todas las etapas de un compilador son cubiertas, el programa ejecutable no es creado para ser ejecutado en un procesador "real" sino para uno "hipotético" o "imaginario" y conocido generalmente como *máquina virtual*. La máquina virtual es otro programa cuyo funcionamiento simula al de un procesador. Este procesador recibe de entrada el pseudocódigo creado por el compilador y procede a la ejecución de las instrucciones contenidas en éste; puede verse que no se trata más que de un intérprete muy sencillo.



La siguiente figura ilustra con mayor detalle lo que pasa en cada una de las etapas del proceso de compilación. El procesamiento de instrucciones de un lenguaje huésped (como puede ser SQL) correría a cargo del preprocesador, siendo transformadas instrucciones del lenguaje anfitrión. Durante la fase de análisis léxico el scanner se encarga de identificar cada uno de los elementos usados para escribir el programa fuente, substituyendo a cada uno de estos por un código numérico único (tokens).

En este proceso se eliminan comentarios y espacios en blanco. Los tokens son alimentados al analizador sintáctico que, valida, que su disposición está acorde a las reglas del lenguaje. Validado éste, el analizador semántico procede a identificar el propósito de las diversas secuencias de tokens, y buscará generar representaciones intermedias de cada acción o directamente código máquina.

Resultados obtenidos

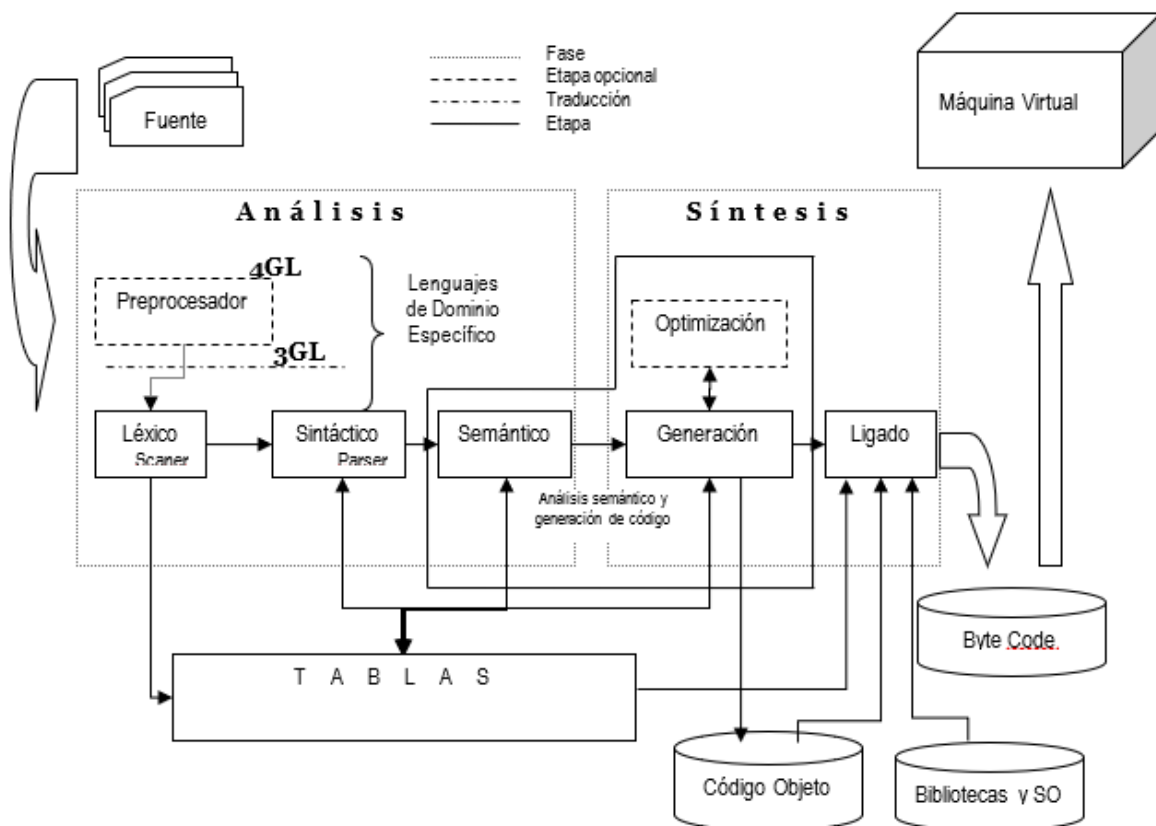


Diagrama esquemático del proceso de pseudocompilación

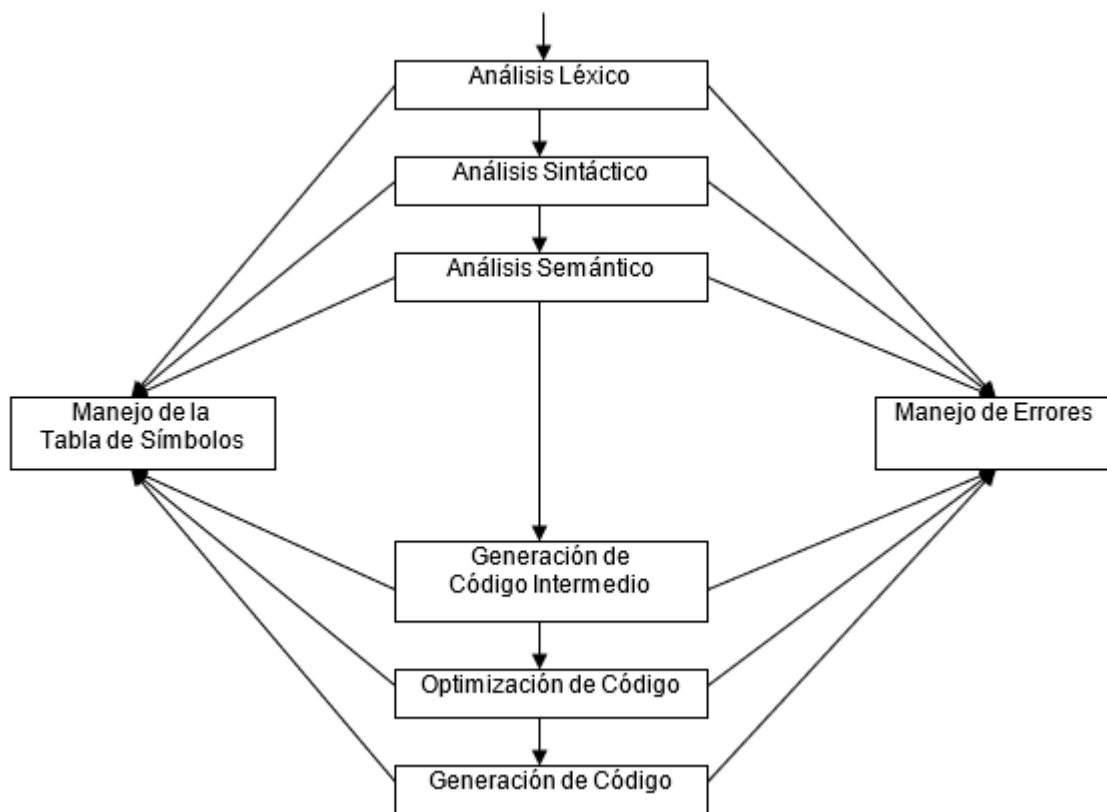
Este posteriormente es optimizado. El código optimizado es un nuevo código el cual contiene elementos que permiten la optimización de recursos computacionales, menor tiempo de respuesta, disminución del espacio de memoria para alojar direcciones y los resultados. Los algoritmos de asignación de memoria son implementados en la lógica del compilador.

Análisis: Dado un sistema, identificar los elementos constituyentes, las interrelaciones entre estos, y determinar la conducta creada.

Síntesis: Dado un conjunto de elementos y estableciendo un determinado número de interrelaciones entre éstos, crear uno de muchos posibles sistemas con una determinada conducta.

El "**descompilar**" un programa, sólo es posible si se cuenta con información de depuración, generada por el compilador. Mientras que es posible "desensamblar" un programa, sin información adicional que permita asociar secuencias de instrucciones ensamblador a instrucciones de nivel alto, es imposible reconstruir el programa fuente a partir del código ejecutable.

FASES Y COMPONENTES DE UN COMPILADOR



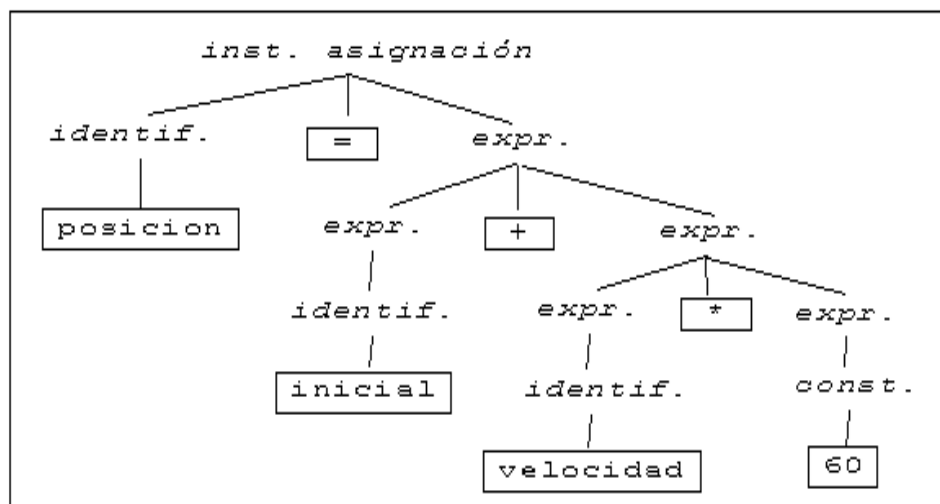
Conceptualmente un compilador opera en fases. Cada una de las cuales transforma el programa fuente de una representación a otra. En la siguiente figura se muestra una descomposición típica de un compilador. En la práctica se pueden agrupar fases y las representaciones intermedias entre las fases agrupadas no necesitan ser construidas explícitamente.

Las tres primeras fases, que forman la mayor parte de la porción de análisis de un compilador se analizan en la sección IX. Otras dos actividades, la administración de la tabla de símbolos y el manejo de errores, se muestran en interacción con las seis fases de análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio, optimación de código y generación de código. De modo informal, también se llamarán "fases" al administrador de la tabla de símbolos y al manejador de errores.

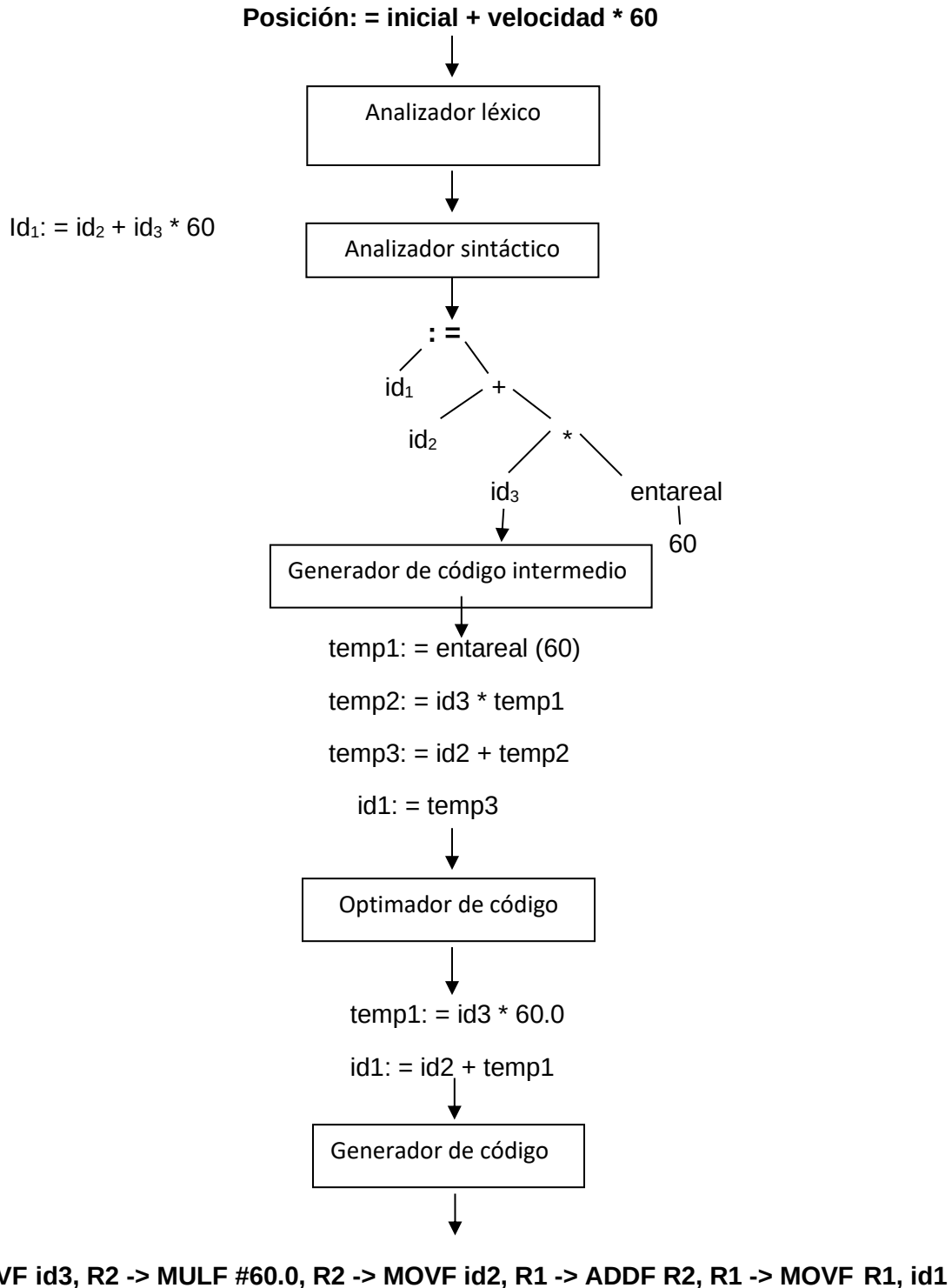
FASE DE ANÁLISIS Y SÍNTESIS DE UN COMPILADOR

En la compilación hay dos partes: Análisis y Síntesis. La parte del análisis divide al programa fuente en sus elementos componentes y crea una representación intermedia. De las dos partes, la síntesis es la que requiere la técnica más especializada.

Durante el análisis se determina las operaciones que implica el programa fuente y se registra en una estructura jerárquica llamada árbol. A menudo, se usa una clase especial de árbol llamado árbol sintáctico, donde cada nodo representa una operación y los hijos de un nodo son los argumentos de la operación. Por ejemplo, en la figura se muestra un árbol sintáctico para una proposición de asignación. Árbol sintáctico para: posición = inicial + velocidad * 60



Conforme avanza la traducción, la representación interna del programa fuente que tiene el compilador se modifica. Para ilustrar esas representaciones, considérese las traducciones en el siguiente ejemplo.



ANÁLISIS LÉXICO

La fase de análisis léxico lee los caracteres de un programa fuente y los agrupa en una cadena de componentes léxicos en los que cada componente representa una secuencia lógicamente coherente de caracteres, como un identificador, una palabra clave (if, while, etc.), un carácter de puntuación, o un operador de varios caracteres, como: =. La secuencia de caracteres que forman un componente léxico se denomina lexema del componente.

A ciertos componentes léxicos se les agregará un "valor léxico". Así, cuando se encuentra un identificador como velocidad, el analizador léxico no sólo genera un componente léxico (por ejemplo, id,) sino que también introduce el lexema velocidad en la tabla de símbolos, si aún no estaba allí. El valor léxico asociado con esta aparición de id, señala la entrada de la tabla de símbolos correspondiente a velocidad.

Usaremos id1, id2 e id3 para posición, inicial y velocidad, respectivamente, para enfatizar que la representación interna de un identificador es diferente de la secuencia de caracteres que forman el identificador. Por tanto, la representación de (1) después del análisis léxico queda sugerida por:

$$\text{id1} := \text{id2} + \text{id3} * 60 \quad \dots (2)$$

Se deberían construir componentes para el operador de varios caracteres: = y el número 60, para reflejar su representación interna. En la sección IX ya se introdujeron las fases segunda y tercera: los análisis sintáctico y semántico. El análisis sintáctico impone una estructura jerárquica a la cadena de componentes léxicos, que se representará por medio de árboles sintácticos, (como se muestra en la figura 1.47). Una estructura de datos típica para el árbol se da cuando un nodo interior es un registro con un campo para el operador y dos campos que contienen apuntadores a los registros de los hijos izquierdo y derecho. Una hoja es un registro con dos o más campos, uno para identificar el componente léxico de la hoja, y los otros para registrar información sobre el componente léxico. Se puede tener información adicional sobre las construcciones del lenguaje añadiendo más campos a los registros de los nodos.

ANÁLISIS SINTÁCTICO

El analizador sintáctico, también llamado parser, recibe como entrada los tokens que le pasa el Analizador Léxico (el analizador sintáctico no maneja directamente caracteres) y comprueba si esos tokens van llegando en el orden correcto



La salida "teórica" de la fase de análisis sintáctico sería un árbol sintáctico.

Así pues, sus funciones son:

- Aceptar lo que es válido sintácticamente y rechazar lo que no lo es.
- Hacer explícito el orden jerárquico que tienen los operadores en el lenguaje que se trate. Por ejemplo, la cadena $A/B * C$ es interpretada como $(A/B) * C$ en FORTRAN y como $A/(B * C)$ en APL.
- Guiar el proceso de traducción (traducción dirigida por la sintaxis).

ANÁLISIS SEMÁNTICO

El análisis semántico es posterior al sintáctico y mucho más difícil de formalizar que éste. Se trata de determinar el tipo de los resultados intermedios, comprobar que los argumentos que tiene un operador pertenecen al conjunto de los operadores posibles, y si son compatibles entre sí, etc. En definitiva, comprobará que el significado de lo que se va leyendo es válido.

La salida "teórica" de la fase de análisis semántico sería un árbol semántico. Consiste en un árbol sintáctico en el que cada una de sus ramas ha adquirido el significado que debe tener. En el caso de los operadores polimórficos (un único símbolo con varios significados), el análisis semántico determina cuál es el aplicable.

Por ejemplo, consideremos la siguiente sentencia de asignación:

$A = B + C$

En Pascal, el signo "+" sirve para sumar enteros y reales, concatenar cadenas de caracteres y unir conjuntos. El análisis semántico debe comprobar que B y C sean de un tipo común o compatible y que se les pueda aplicar dicho operador. Si B y C son enteros o reales los sumará, si son cadenas las concatenará y si son conjuntos calculará su unión.

Ejemplo

VAR

ch: CHAR; (*Un identificador no se puede utilizar si previamente no se ha definido*)

(* En Pascal no es válido, en C sí. *)

ent: INTEGER;

...

ch: = ent + 1;



GENERACIÓN DE CÓDIGO

Después de los análisis sintáctico y semántico, algunos compiladores generan una representación intermedia explícita del programa fuente. Se puede considerar esta representación intermedia como un programa para una máquina abstracta. Esta representación intermedia debe tener dos propiedades importantes; debe ser fácil de producir y fácil de traducir al programa objeto.

La representación intermedia puede tener diversas formas. Existe una forma intermedia llamada "código de tres direcciones", que es como el lenguaje ensamblador para una máquina en la que cada posición de memoria puede actuar como un registro. El código de tres direcciones consiste en una secuencia de instrucciones, cada una de las cuales tiene como máximo tres operandos.

El programa fuente de (1) puede aparecer en código de tres direcciones como

```
temp1: = entarea1(60)
temp2: = id3 * temp1 (2)
temp3: = id2 + temp2
id1: = temp3
```

Esta representación intermedia tiene varias propiedades. Primera, cada instrucción de tres direcciones tiene a lo sumo un operador, además de la asignación. Por tanto, cuando se generan esas instrucciones el compilador tiene que decidir el orden en que deben efectuarse, las operaciones; la multiplicación precede a la adición al programa fuente de (1), mostrado anteriormente.

Segunda, el compilador debe generar un nombre temporal para guardar los valores calculados por cada instrucción. Tercera, algunas instrucciones de "tres direcciones" tienen menos de tres operadores, por ejemplo, la primera y la última instrucción de (2).

OPTIMIZACIÓN DE CÓDIGO

La fase de optimación de código trata de mejorar el código intermedio de modo que resulte un código de máquina más rápido de ejecutar. Algunas optimaciones son triviales. Por ejemplo, un algoritmo natural genera el código intermedio (2) utilizando una instrucción para cada operador de la representación del árbol después del análisis semántico, aunque hay una forma mejor de realizar los mismos cálculos usando las dos instrucciones



```
Temp1: = id3 * 60.0          ... (3)
id1: =
id2 + temp1
```

Este sencillo algoritmo no tiene nada de malo, puesto que el problema se puede solucionar en la fase de optimización de código. Esto es, el compilador puede deducir que la conversión de 60 de entero a real se puede hacer en el momento de la compilación, de modo que la operación entera se puede eliminar. Además, temp3 se usa sólo una vez, para transmitir su valor a id1. Entonces resulta seguro sustituir a id1 por temp3, a partir de lo cual la última proposición de (2) no se necesita y se obtiene el código de (3).

Hay muchas variaciones en la cantidad de optimización de código que ejecutan los distintos compiladores. Los compiladores que realizan mucha optimización de código son llamados: "compiladores optimadores". Sin embargo, hay optimaciones sencillas que mejoran significativamente el tiempo del compilador que se ocupa en esta fase.



UNIDAD II

2.1 Conceptos de expresiones regulares y Automatas Finitos.

Conceptos de expresiones regulares.

Las expresiones regulares, aplicado a las ciencias computacionales, especifican la estructura de los datos integrados por cadenas, caracteres o símbolos. También conocido como un Lenguaje Regular o "Formal".

CONSTRUCCION DE EXPRESIONES REGULARES.	
Termino	DEFINICION/EJEMPLO
Unión	Partiendo de la teoría de conjuntos, la unión representa la mezcla o fusión de dos o más elementos. Un ejemplo de esto es código ASCII extendido, el cual es la unión de los caracteres del primer código y nuevos caracteres para extenderlo. Ejemplo: $a+b$ a,b $a b$ (se interpreta que puede ser una "a" o "b")
Concatenación	Concatenación, generalmente se refiere al acto de enlazar cosas. Ejemplo: ab también se expresa $[ab]$ o $/ab/$ Esto aplicado a la teoría computacional, se denota en este ejemplo. Dos caracteres formando una cadena de caracteres, en PHP esto es empleado con un punto, es decir "."
Clausura de Kleene.	Es una operación <i>unaria</i> , que se aplica sobre un conjunto de símbolos o caracteres, y representan el conjunto de cadenas tomando cualquier número de cadenas del conjunto inicial, pueden repetirse y concatenarse. Puede ser a^* o b^+
Alternación	Una barra vertical separa las posibles alternativas. Por ejemplo, "vivo muerto" Aquí se combina la concatenación de símbolos y unión.
Cuantificación	Un cuantificador tras un carácter especifica la <i>frecuencia</i> con la que éste puede ocurrir. Un ejemplo es la búsqueda general con asterisco en windows. Los cuantificadores más comunes son $+$, $?$ y $*$

Los patrones de caracteres que pueden describir expresiones regulares son los mismos que pueden ser descritos por autómatas finitos. Su estilo es muy diferente al de las gramáticas. Es a menudo llamada también patrón, es una expresión que ejemplifica un conjunto de cadenas sin enumerar sus elementos.

Más específicamente las expresiones regulares están constituidas por los operadores unión, concatenación y clausura de Kleene (también conocida como cerradura de Kleene positiva + y estrella o asterisco *). Además de esto, cada expresión regular tiene asociado un autómata finito. **IMPORTANTE:** Las expresiones regulares se pueden representar por Autómatas Finitos ya sea No Determinísticos con épsilon transiciones, sin épsilon transiciones y Determinísticos.

Concepto de Autómatas Finitos.

El autómata finito es un modelo matemático que consta de una quintupla o tabla con, cinco elementos y estos son: $M = \{Q, \Sigma, \delta, q_0, F\}$

- La **M** significa **Modelo** o **Máquina de estados finitos** también conocido como diagrama de transiciones.
- **Q** se define como el conjunto de estados que se pueden etiquetar con $q_0, q_1, q_2, q_3, \dots, q_f$ También pueden ser enumerados: **0, 1, 2, 3, 4, 5, 6, etc.**
 $Q = \{q_0, q_1, q_2, q_3, \dots, q_f\}$ Se representan con círculos.



- Σ (**sigma**) es el **alfabeto** o símbolos que se requieren para pasar de un estado a otro. Ejemplo: $\Sigma = \{a\}$
- Las transiciones son un eje cartesiano entre el conjunto de los estados y el alfabeto: $\delta = Q \times \Sigma \rightarrow Q$

Q	$\Sigma = a$
q ₀	q ₁
q ₁	q ₂
q ₂	q _f
q _f	$\emptyset$

- **F** es el conjunto de estados finales o un sub-conjunto de **Q** **Ejemplo: qf**
Se dibuja con un círculo con línea doble.



El diagrama de autómata e la tabla anterior es el siguiente:



Este autómata finito se considera determinista en base a que se tiene un solo símbolo del alfabeto esto es: $\Sigma = a$ que es el mismo que se utiliza en todas las transiciones para integrar un lenguaje regular que expresa una sola cadena.

$LR = \{aaa\}$

Para llegar desde q_0 a q_f necesitamos tres símbolos a (o bien la “ a ” tres veces)

PASOS PARA CONSTRUIR UN AUTOMÁTA FINITO

- **Paso 1** La palabra inicio de forma explícita
- **Paso 2** Los estados Q , etiquetados (con un número, letra mayúscula o $q_0, q_1, q_2, q_3, \dots q_f$)
- **Paso 3** Cada transición o flecha debe llevar una solo una y al menos una dirección y cada transición AL menos un símbolo del alfabeto o en ausencia de símbolos a la cadena vacía (épsilon).

Ejemplos correctos: $\xrightarrow{a}$ $\xleftarrow{a}$ $\xrightarrow{\epsilon}$

INCORRECTO: $\xleftrightarrow{a}$

- **Paso 4** EVITAR CRUZAR TRANSICIONES
- **Paso 5** Al menos un estado final etiquetado y con doble circulo.





El mecanismo de control de un elevador es un buen ejemplo de un sistema de estados finitos. El mecanismo no recuerda todas las demandas previas de servicio, sino solo el piso en el que se encuentra, la dirección de movimiento (hacia arriba o hacia abajo) y el conjunto de demandas de servicio aún no satisfechas.

Otro ejemplo que tenemos es que ciertos programas como los editores de texto y los analizadores léxicos que se encuentran en la mayoría de los compiladores son diseñados como sistemas de estado finito. En teoría, una Expresión Regular o Lenguaje Regular LR es posible representar con un Autómata que puede ser No Determinista con épsilon transiciones, No Determinista sin épsilon transiciones o bien un Autómata Determinista, para esto son los procesos de Transformación que se describen en esta unidad temática.

AUTÓMATA FINITO DETERMINISTA (AFD abreviando)

Entonces podemos definir que un autómata finito determinista **AFD** consiste en un conjunto finito de estados y un conjunto de transiciones de estado a estado, que se dan sobre el símbolo de entrada tomado de un alfabeto Σ . Un estado q_0 , que es el estado inicial, en el que el autómata comienza.

Para determinar los distintos símbolos de un lenguaje podemos construir un diagrama, este diagrama tiene la forma de un grafo dirigido con información adicional, y se llama diagrama de transición. Los nodos del grafo se llaman estados y se usan para señalar hasta que lugar se ha analizado la cadena.

Las aristas del grafo que se etiquetan con caracteres del alfabeto determinan las transiciones. Naturalmente nosotros debemos comenzar por un estado inicial, y cuando se hayan tratado todos los caracteres de la cadena correspondiente, necesitamos saber si la cadena es legal. Para ello se marcan ciertos estados como estados de aceptación o estados finales.

IMPORTANTE: Un Autómata Finito Determinista tiene los siguientes requisitos: UNA, SOLO UNA Y AL MENOS UNA TRANSICIÓN DESDE CADA ESTADO POR CADA UNO DE LOS SIMBOLOS DEL ALFABETO (Σ)

Ejemplo

En la siguiente página se ilustra el diagrama de transiciones de un Autómata Finito. El estado inicial q_0 está indicado por la flecha con la etiqueta “inicio”

(en este ejemplo existe solo un estado final, pero pueden existir más en otros autómatas), también q_0 en este caso indicado por el doble círculo. Este autómata acepta todas las cadenas de ceros y unos, en las que el número de 0's y el número de 1's son pares.

Dado este ejemplo, formalmente, un autómata finito determinista AFD es una colección de cinco elementos:

- 1 Un alfabeto de entrada Σ .
- 2 Una colección finita de estados Q .
- 3 Un estado inicial S .
- 4 Una colección F de estados finales o de aceptación.
- 5 Una función $\delta: Q \times \Sigma \rightarrow Q$, conocida como función de transición que determina el único estado siguiente para el par (q_i, σ) correspondiente al estado actual q_i y la entrada σ . (Donde "i" puede ser 0, 1, 2, 3, etc.)

NOTA IMPORTANTE: La "cadena vacía" o ausencia de símbolos puede representarse con el símbolo épsilon ϵ que tiene una longitud de 0 (símbolos). Pero también se puede representar con los símbolos "lambda" λ

También se puede representar con el símbolo vacío (teta) σ

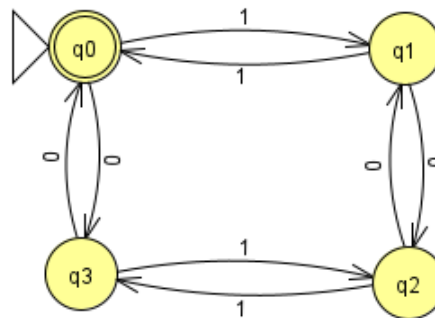


Diagrama de transiciones de un Autómata Finito Determinístico

Denominaremos a un AFD con M , por lo que usaremos $M = \{Q, \Sigma, S, F, \delta\}$ para indicar el conjunto de estados, el alfabeto, el estado inicial, el conjunto de estados finales y la función asociada con el AFD. Se dice que una cadena x es aceptada por un AFD.



Si $\delta(q_0, x) = p$ para algún p en F , es decir que la cadena termine en uno de los estados de aceptación o bien estados finales. El lenguaje aceptado por M , denominado $L(M)$, es el conjunto $\{x \mid \delta(q_0, x) \text{ está en } F\}$. Por lo que un lenguaje es regular, si es el conjunto de cadenas es aceptado por un autómata finito.

Ejemplo

Considerando el diagrama de nuestro ejemplo anterior, en notación formal, este AFD se denota como $M = \{Q, \Sigma, S, F, \delta\}$ de la siguiente manera:

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$S = q_0$$

$$F = \{q_0\}$$

δ = según la tabla 2.1

Estados	Entradas		
	0	1	Finales
q0	q3	q1	1
q1	q2	q0	0
q2	q1	q3	0
q3	q0	q2	0

Tabla 2.1.-Diagrama de transiciones δ del autómata.

Dada la cadena **110101** vemos que pertenece a **$L(M)$** . **Ejemplo**

El siguiente **AFD** que se representa mediante la siguiente manera

$$M = \{Q, \Sigma, S, F, \delta\}.$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{a, b\}$$

$$S = q_0$$

$$F = \{q_0\}$$

δ = según tabla 1.2

Entradas			
Estados	a	b	Finales
q_0	q_0	q_1	1
q_1	q_1	q_0	0

Tabla 1.2 Diagrama de transiciones δ de un autómata finito determinístico

La secuencia completa es $q_0^a q_0^b q_1^a q_1^b q_0^a q_0$ por lo tanto.

Dada la siguiente cadena **ababa**, vemos que pertenece a **L(M)**. (Está integrado por las letras o símbolos de las transiciones en la parte superior izquierda de la secuencia completa que se forma).

AUTÓMATA FINITO NO DETERMINISTA

Si se permite que desde un estado se realicen cero o más transiciones mediante el mismo símbolo de entrada, se dice que el autómata finito es no determinista **AFDN** o **AFN**, pero llegaremos a la conclusión de que cualquier lenguaje aceptado por un **AFN** también es aceptado por un **AFD**.

A veces es más conveniente diseñar **AFN** en lugar de **AFD**.

De manera formal denotamos a un **AFN** mediante una colección de cinco objetos **(Q, Σ , S, F, δ)**, donde:

- 1 **Q** es un estado finito de estados.
- 2 **Σ** es el alfabeto de entrada.
- 3 **S** es uno de los estados de **Q** designado como estado de partida.
- 4 **F** es una colección de estados de aceptación o finales.
- 5 **δ** es una relación sobre **(Q x Σ)** y se llama relación de transición o función de transición expresada en una tabla de transiciones que hacen objetiva la función de producto cartesiano.

Ejemplo

En la siguiente página se muestra un diagrama de transiciones de un AFN. Observe que existen dos aristas con la etiqueta 0 que salen del estado q_0 , uno que regresa a q_0 y otro que va al estado q_3 .

Por lo que su relación de transición δ está en la siguiente página.

Estados	Entradas		Finales
	0	1	
q_0	$\{q_0, q_3\}$	$\{q_0, q_1\}$	0
q_1	$\{\emptyset\}$	$\{q_2\}$	0
q_2	$\{q_2\}$	$\{q_2\}$	1
q_3	$\{q_4\}$	$\{\emptyset\}$	0
q_4	$\{q_4\}$	$\{q_4\}$	1

Tabla 1.3 Transiciones del diagrama de la siguiente **figura**.

AUTÓMATA ϵ -TRANSICIONES

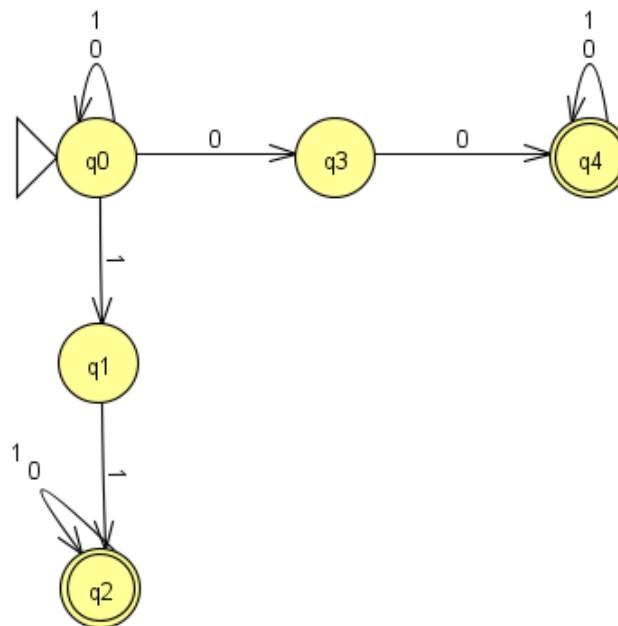
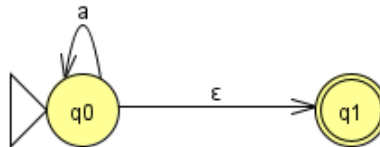


Diagrama de un autómata finito no determinístico.

Este autómata es también un AFN, que incluye transiciones sobre la cadena vacía ϵ , cuando hablamos de una cadena vacía conecta a dos estados, nos referimos a que no se consume ningún símbolo del alfabeto.

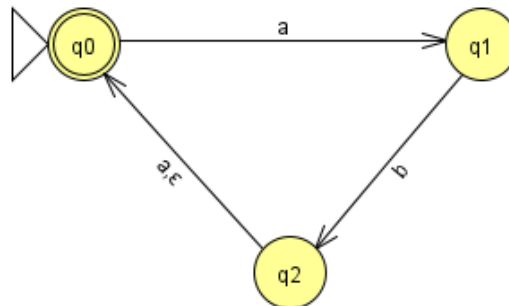
Como ejemplo de estos autómatas tenemos los siguientes, en este primer ejemplo tenemos a un autómata que puede cambiar su estado de q_0 a q_1 sin consumir nada en la entrada. Observemos que q_1 es el único estado de aceptación de este AFN. Si w es cualquier cadena de 0 o más a 's, este autómata cicla sobre q_0 hasta que consume las a 's. Una vez que la cadena se vacía, se desplaza a q_1 con ϵ y lo acepta.

A continuación, se muestra un diagrama de un autómata finito no determinístico con ϵ – transiciones



En esta figura el AFND puede moverse del estado q_0 al estado q_1 sin consumir nada en la entrada. En ambos AFND, la decisión de elegir una ϵ -transición se realiza de la misma forma que la de cualquier otra transición con elección múltiple que exista en un AFND. Según se muestra en la siguiente figura.

Diagrama de un autómata finito no determinístico con ϵ – transiciones



Este **AFND** asocia $Q \times (\Sigma \cup \{ \epsilon \}) \times Q$ con subconjuntos Q . Por lo que la figura anterior genera la siguiente tabla de transición:

Estados	Entradas			Finales
	a	b	ϵ	
q0	{q1}	{ $\emptyset$ }	{ $\emptyset$ }	1
q1	{ $\emptyset$ }	{q2}	{ $\emptyset$ }	0
q2	{q0}	{ $\emptyset$ }	{q0}	0

Tabla 1.4 de la figura anterior



2.2 Transformación de Expresiones Regulares a Autómatas Finitos No Deterministas con épsilon (ϵ) transiciones.

En teoría de la computación, la construcción Thompson es un algoritmo también llamado el algoritmo de McNaughton-Yamada-Thompson, es un método de transformación de una expresión regular en un equivalente Autómata Finito NO Determinista (AFND). Las expresiones regulares y autómatas finitos no deterministas son dos representaciones de lenguajes formales, de forma más explícita son también lenguajes “regulares” según la jerarquía de Noam Chomsky.

Por ejemplo, en los procesadores de texto son muy útiles las expresiones regulares para describir los patrones de búsqueda avanzada, pero un AFND son más adecuados para su ejecución en una computadora. Por lo tanto, este algoritmo es de interés práctico, ya que puede analizar expresiones regulares en AFND. Desde un punto de vista teórico, este algoritmo es una parte de la prueba que ambos aceptan exactamente los mismos lenguajes formales, es decir, los lenguajes regulares.

Un AFND puede hacerse determinista por la construcción de un conjunto potencia y luego se reduce al mínimo para obtener un autómata óptimo correspondiente a la expresión regular dada. Sin embargo, un AFND también puede interpretarse directamente.

Para decidir si dos expresiones regulares dadas describen el mismo lenguaje, cada uno se puede convertir en un mínimo equivalente autómata finito determinista a través de la construcción de Thompson, mediante la construcción de conjunto potencia, y la minimización del AFND. Si, y sólo si, los autómatas resultantes están de acuerdo hasta el cambio de nombre de los estados, PRODUCIENTO EL MISMO LENGUAJE o las mismas cadenas de las expresiones regulares de acuerdo.

Para entender lo anterior, vamos a empezar con un ejemplo muy sencillo, una expresión de un solo símbolo que representa un lenguaje regular. Los lenguajes regulares pueden ser finitos como el caso del diagrama de la cadena

W = aaa Es una sola cadena o palabra formada con un sólo símbolo (que se repita tres veces) formado un LENGUAJE FINITO (UNA sola palabra)

Ahora tenemos la expresión regular **a***

Con la cerradura de “Kleene” estrella o asterisco, incluye a la cadena vacía ϵ y al único símbolo que es la letra o símbolo “a”, se repite el símbolo “a” un número infinito de veces, creando también un número infinito de cadenas, por lo que el lenguaje regular que representa también es infinito y “formalmente” se puede representar de las siguientes formas:

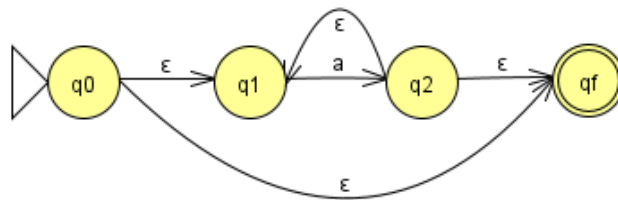
- 1) $LR = \{ \epsilon, a, aa, aaa, aaaa, aaaaa, aaaaaa, aaaaaaa, aaaaaaaa, aaaaaaaaa, aaaaaaaaaa, aaaaaaaaaa, \dots, \infty \}$
- 2) $LR = \{ \epsilon \mid a^n, n \geq 0 \}$

Se lee, que el lenguaje regular puede estar integrado por cadenas que incluyan a la cadena vacía ϵ o al símbolo “a” donde n significa un numero de repeticiones mayores o iguales a cero.

Nota: “n” NO REPRESENTA UN EXPONENTE SINO LAS VECES QUE REPITE EL SIMBOLO.

El autómata AFND- ϵ correspondiente para la expresión regular, se abrevia /ER/ es a^*

Uno de varios diagramas de Autómatas Finitos NO Deterministas (AFND) de la anterior /ER/ es el siguiente:



La definición formal de este diagrama es la siguiente:
AFND- ϵ que se representa LA **/ER/ = a^*** mediante **$M = \{Q, \Sigma, S, F, \delta\}$** .

$Q = \{q_0, q_1, q_2, q_f\}$

$\Sigma = \{ \epsilon, a \}$

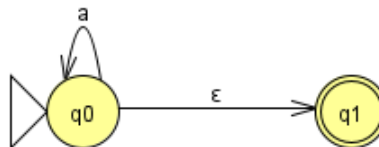
$S = q_0$

$F = \{q_f\}$

δ = según la siguiente tabla:

Estados	Entradas		Finales
	ϵ	a	
q_0	$\{q_1, q_f\}$	$\{\emptyset\}$	0
q_1	$\{\emptyset\}$	$\{q_2\}$	0
q_2	$\{q_1, q_f\}$	$\{\emptyset\}$	0
q_f	$\{\emptyset\}$	$\{\emptyset\}$	1

Otro diagrama de Autómatas Finitos NO Deterministas (AFND) de la anterior $/ER/ = a^*$ es el siguiente:



La definición formal de este diagrama es la siguiente:

AFND- ϵ que se representa la $/ER/ = a^*$ mediante $M = \{Q, \Sigma, S, F, \delta\}$.

$Q = \{q_0, q_1\}$

$\Sigma = \{\epsilon, a\}$

$S = q_0$

$F = \{q_1\}$

δ = según la siguiente tabla:

Estados	Entradas		Finales
	ϵ	a	
q_0	$\{q_1\}$	$\{q_0\}$	0
q_1	$\{\emptyset\}$	$\{\emptyset\}$	1

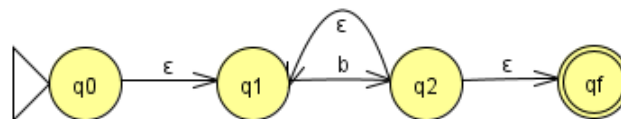
Otros ejemplos básicos de **AFND- ϵ** son: La expresión regular $/ER/ = b^+$

Se lee: Cerradura de Kleene positiva, la cual incluye a la cadena vacía, si y solo si está en el lenguaje originalmente y genera el siguiente Lenguaje Regular (LR) que se puede representar de las siguientes formas:

- 1) $LR = \{ b, bb, bbb, bbbb, bbbbbb, bbbbbbb, bbbbbbbb, bbbbbbbbb, bbbbbbbbbb, bbbbbbbbbb, bbbbbbbbbb, \dots, \infty \}$
- 2) $LR = \{ bn \mid n > 0 \}$

Los diagramas de Autómata Finitos NO Determinista (AFND- ϵ) de la anterior $/ER/ = b^+$ puede ser alguno de los siguientes (A), (B), o bien (C):

Diagrama (A)



La definición formal de este diagrama es la siguiente:
AFND- ϵ que se representa $LA /ER/ = a^+$ mediante **$M = \{Q, \Sigma, S, F, \delta\}$** .

$Q = \{q_0, q_1, q_2, q_f\}$

$\Sigma = \{ \epsilon, b \}$

$S = q_0$

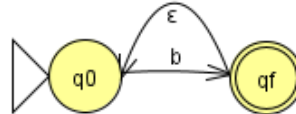
$F = \{q_f\}$

δ = según la siguiente tabla:

Estados	Entradas		Finales
	ϵ	b	
q_0	$\{q_1\}$	$\{\emptyset\}$	0
q_1	$\{\emptyset\}$	$\{q_2\}$	0
q_2	$\{q_1, q_f\}$	$\{\emptyset\}$	0
q_f	$\{\emptyset\}$	$\{\emptyset\}$	1

NOTA: Recuerde que la cadena: **$\epsilon b \epsilon = b$** y que al regresar con **ϵ** tenemos un número infinito de símbolos “b”

Diagrama (B) Equivalente al anterior porque genera el mismo lenguaje regular:
 $LR = \{bn \mid n > 0\}$



La definición formal de este diagrama **AFND-ε** es la siguiente:
AFND- ε que se representa LA **/ER/ = a*** mediante **M = {Q, Σ, S, F, δ}**.

Q = {q₀, q_f}

Σ = { ε, b}

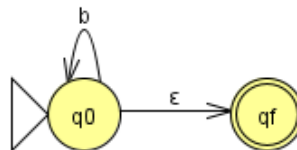
S = q₀

F = {q_f}

δ = según la siguiente tabla:

Estados	Entradas		Finales
	ε	b	
q ₀	{∅}	{q ₁ }	0
q _f	{q ₀ }	{∅}	1

Diagrama (C) Equivalente al anterior porque genera el mismo lenguaje regular:
L_R = {bn | n > 0} CON UN **MINIMO** NUMERO DE ESTADOS



NOTA IMPORTANTE: Es posible dibujar diferentes AFND-ε con diferente forma e incluir más estados conectados con la cadena vacía siempre y cuando se genere el **MISMO LENGUAJE** o las mismas cadenas, por lo que lo que represente el símbolo a, depende del creador del lenguaje regular. Los anteriores diagramas de AFND-ε son los más simples.



La definición formal de este diagrama **AFND- ϵ** es la siguiente:
AFND- ϵ que se representa LA **/ER/ = a^*** mediante **$M = \{Q, \Sigma, S, F, \delta\}$** .

$$Q = \{q_0, q_f\}$$

$$\Sigma = \{\epsilon, b\}$$

$$S = q_0$$

$$F = \{q_f\}$$

δ = según la siguiente tabla:

Estados	Entradas		Finales
	ϵ	b	
q_0	$\{q_f\}$	$\{q_0\}$	0
q_f	$\{\emptyset\}$	$\{\emptyset\}$	1

2.3 Transformación de Autómatas Finitos No Deterministas con épsilon (ϵ) transiciones a Autómatas Finitos No Deterministas sin épsilon (ϵ) transiciones.

El origen de los autómatas finitos probablemente se remonta a su uso implícito en máquinas electromecánicas, desde principios del siglo XX. En 1907, el matemático ruso Andréi Márkov formalizó un proceso llamado “cadena de Markov”, donde la ocurrencia de cada evento depende con una cierta probabilidad del evento anterior. Esta capacidad de “recordar” es utilizada posteriormente por los autómatas finitos, que poseen una memoria primitiva similar, en que la activación de un estado también depende del estado anterior, así como del símbolo o palabra presente en la función de transición.

Posteriormente, en 1943, surge una primera aproximación formal de los autómatas finitos con el modelo neuronal de McCulloch-Pitts. Durante la década de 1950 prolifera su estudio, frecuentemente llamando seles “máquinas de secuencia”; se establecen muchas de sus propiedades básicas, incluyendo su interpretación como “lenguajes regulares” y, su equivalencia con las “expresiones regulares” abreviadas /ER/. Al final de esta década, en 1959, surge el concepto de “autómata finito no determinista” abreviado AFND en manos de los primeros teóricos de la computación:



Michael Oser Rabin (nacido en 1931 en Breslavia, Alemania, hoy en día parte de Polonia, es un notable científico de la computación y ganador del Premio Turing, el galardón más prestigioso en el campo computacional) y Dana Stewart Scott (nacido en 1932 es el Profesor Emérito de Ciencias de la Computación, Filosofía y Lógica Matemática en Carnegie Mellon University).

En la década de 1960 se establece su conexión con las series de potencias y los sistemas de sobreescritura. Finalmente, con el desarrollo del sistema operativo Unix en la década de 1970, los autómatas finitos encuentran su nicho en el uso masivo de expresiones regulares para fines prácticos, específicamente en el diseño de analizadores léxicos (comando lex) y la búsqueda y reemplazo de texto (comandos ed y grep). A partir de ese tiempo, los autómatas finitos también se comienzan a utilizar en sistemas dinámicos.

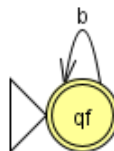
Las máquinas no deterministas se han convertido en un concepto clave en la teoría de la complejidad computacional.

Fuente de información: Wikipedia

Existen varios métodos de transformación de un Autómata Finito No Determinístico con épsilon transiciones a otro que no las tenga. Con la práctica se pueden diseñar o crear directamente de la expresión regular. Por ejemplo: De la expresión regular b^+ que representa el Lenguaje Regular (LR) con cadenas que incluyen solo al símbolo b que se repite desde la primera cadena de longitud uno (b), la segunda cadena de longitud dos (bb), de longitud 3 (bbb) y así sucesivamente hasta un número infinito de b 's. Aquí tenemos un AFND ya sin épsilon transiciones abreviado ϵ -transiciones.

El siguiente diagrama de autómata representa el $LR = \{b, bb, bbb, \dots, \infty\}$

Con la $/ER/ = b^+$



La definición formal de este diagrama **AFND** es la siguiente:

$M = \{Q, \Sigma, S, F, \delta\}$.

$Q = \{q_i\}$

$\Sigma = \{b\}$

$S = \{q_i\}$

$F = \{q_f\}$

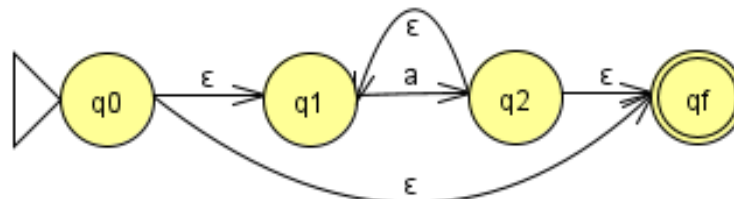
δ = según la siguiente tabla:

Estados	Entradas	
	b	Finales
q_i	$\{q_i\}$	1

Sin embargo para llegar a este autómata, también se puede realizar una Transformación con un método denominado “formal”, conocido como el método de la epsilon cerradura. Aquí es pertinente preguntar:

¿Qué es la epsilon cerradura?

También conocida como la cerradura de epsilon, abreviado: “ ϵ -C” o bien “C- ϵ ” Es identificar dentro del autómata finito con epsilon transiciones A todos aquellos estados que se conectan con epsilon y si no tienen a la cadena vacía o epsilon, se elige el mismo estado. Ejemplo, tomaremos el siguiente AFND- ϵ .



PASO 1. Se analiza la “ ϵ -C” de cada uno de los estados del autómata

ϵ -C	Aquí es el conjunto de estados	Si a este conjunto de estados se le agrega el símbolo “a” tenemos	FINALES
q_0	$\{q_0, q_1, q_f\}$	$\{q_2\}$	0
q_1	$\{q_1\}$	$\{q_2\}$	0
q_2	$\{q_1, q_2, q_f\}$	$\{q_2\}$	1
q_f	$\{q_f\}$	$\{\emptyset\}$	1

PASO 2. Se renombran los grupos de estados y se realiza un nuevo diagrama ya sin las épsilon transiciones.

$\{q_0, q_1, q_f\} = A$

$\{q_1\}$ Está contenido en el grupo A

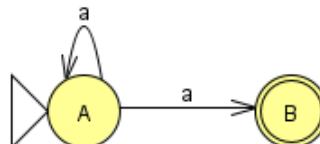
$\{q_1, q_2, q_f\} = B$

En este grupo está un estado final, por lo que el nuevo estado será también final. $\{q_f\}$ Está contenido en el grupo B.

PASO 3. Se desarrolla otra tabla de transiciones con los nuevos estados

ESTADOS RENOMBRADOS	Alfabeto con: a	FINALES
A	A	0
B	$\emptyset$	1

Con la $/ER/ = a^*$



Este es un AFND sin épsilon transiciones. Transformado mediante el método de la épsilon cerradura.

2.4 Transformación de Autómatas Finitos No Deterministas sin épsilon (ϵ) transiciones a Autómatas Finitos Deterministas.

Estas transformaciones nos permiten llegar a un autómata mínimo cuyo procesamiento resulta más objetivo y práctico, y recordemos que un autómata finito no determinista, NO incluye una transición para cada uno de los símbolos del alfabeto, sin embargo, un ejemplo sencillo de presentar esto es cuando se tiene como únicos símbolos del alfabeto al 0 y al 1 o bien la letra “a” o “b”.

La parte aplicativa de los Autómatas Finitos es el hecho de que nosotros podemos realizar procesos de abstracción sobre el significado de dos o más símbolos, así como integrar con varios símbolos palabras como Si, No, Encendido, Apagado, Abierto Cerrado, Nombres, cualquier dato que nosotros queramos procesar y abstraer de forma “discreta”. Dependiendo de los datos o conjunto de símbolos concatenados que deseamos procesar se pueden diseñar Autómatas que de forma simbólica o discreta logren abstraer el comportamiento de sistemas tan simples como el comportamiento de una puerta que se abre y se cierra, imaginando que la A significa abrir y la C significa cerrar la puerta.

Para este simple ejemplo de abstracción podemos definir el estado A (de Abierto) y el estado C (de Cerrado) para indicar que la puerta está abierta podemos agregar el símbolo del alfabeto 1 (uno) y para cerrar un 0 (cero). De esta forma es factible diseñar un autómata finito NO determinista (AFND), formalmente:

$$M = \{Q, \Sigma, S, F, \delta\}.$$

$$Q = \{A, C\}$$

$$\Sigma = \{0,1\}$$

$$S = \{A\}$$

$$F = \{A,B\}$$

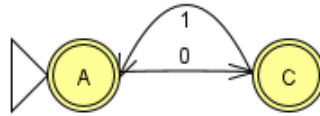
δ = según la siguiente tabla:

Entradas (alfabeto) = Σ

Estados	0	1	Finales
A	{C}	{ $\emptyset$ }	1
C	{ $\emptyset$ }	{A}	1

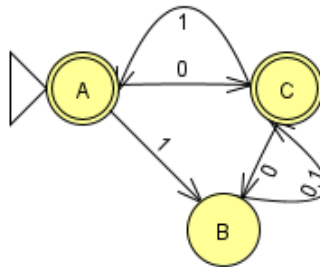
Esta tabla que muestra las transiciones, al dejar conjuntos con vacío deja en evidencia un Autómata Finito NO Determinista (AFND).

Aquí la $/ER/ = [0|1]^*$ (casualmente esta es la expresión regular de un lenguaje binario).



Aquí tenemos un ejemplo muy sencillo de la abstracción de una puerta que tiene sólo dos estados y en el paso de un estado a otro un alfabeto con dos símbolos 0 y 1. Recordemos que un Autómata Finito Determinista (AFD) incluye solo una transición, (de salida) desde cada uno de los estados A y C. Por lo que falta una transición con 1 (UNO) de salida de A y otra con un 0 (CERO) de salida de C. ¿Cómo lograr esto?

La alternativa más sencilla, de manera gráfica y NO formal, sin afectar el alfabeto es crear un estado NO FINAL que sirve como comodín para las transiciones que faltan. Dejando el autómata de la siguiente forma:



Aquí se acepta

$L = \{\epsilon, 0, 1, 01, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, \dots, \infty\}$

Este es un AFD donde el estado B puede ser una condición de ERROR o NO ACEPTADO, ya que es un estado No Final y según la definición de un AFD se tiene una transición con cada uno de los símbolos del alfabeto $\Sigma = \{0, 1\}$ desde cada uno de los estados $Q = \{A, B, C\}$

Con la siguiente definición formal.

$M = \{Q, \Sigma, S, F, \delta\}$.

$Q = \{A, B, C\}$

$\Sigma = \{0, 1\}$

$S = \{A\}$

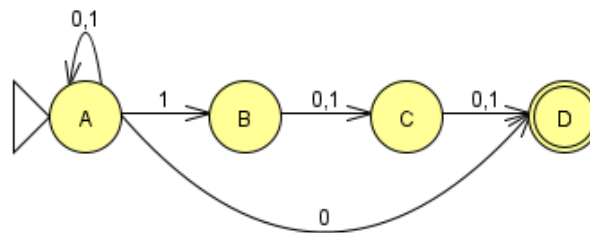
$F = \{A, B\}$

δ = según la siguiente tabla:

Entradas (alfabeto) = Σ

Estados	0	1	Finales
A	{C}	{B}	1
B	{C}	{C}	0
C	{B}	{A}	1

Ahora vamos a realizar una conversión con un método formal del siguiente AFND. Se puede observar que desde el estado A, tenemos dos transiciones con el símbolo 1 que van a los estados A y B; dos transiciones con el símbolo 0, una al estado A y otra al D; motivo por el que No se cumple el requisito de un AFD.



El Lenguaje regular lo podemos abstraer de la siguiente forma:

LR = {[0|1]*0|[0|1]*1 [0|1] [0|1]}

La definición formal es:

M = {Q, Σ , S, F, δ }.

Q = {A, B, C, D}

Σ = {0,1}

S = {A}

F = {D}

δ = según la siguiente tabla:

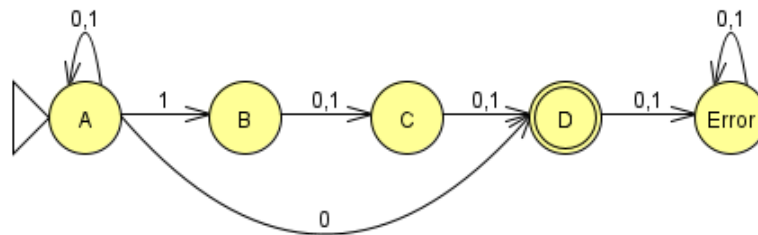
Entradas (alfabeto) = Σ

Estados	0	1	Finales
A	{A,D}	{A,B}	1
B	{C}	{C}	0
C	{D}	{D}	1
D	{ $\emptyset$ }	{ $\emptyset$ }	

El Lenguaje regular lo podemos abstraer de la siguiente forma Partimos del autómata finito no determinista (AFND) del anterior diagrama.

PASO 1. Añadimos un estado de error o “X” puede ser etiquetado con cualquier nombre según se muestra en el siguiente diagrama:

IMPORTANTE: El nuevo estado es NO FINAL y tiene una transición por cada uno de los símbolos del alfabeto. A este estado vamos a enviar las transiciones que sobran



PASO 2. Generamos una tabla considerando la transición de error y agrupando las nuevas transiciones, agregando a las nuevas transiciones los símbolos del alfabeto y nos queda la siguiente tabla:

Q = estados	0	1	F
{A}	{A,D}	{A,B}	0
{A,B}	{A,B,C,D}	{A,B,C}	0
{A,D}	{A,D,Error}	{A,D,Error}	1
{A,B,C}	{A,B,C,D}	{A,B,C,D}	0
{A,D,Error}	{A,D,Error}	{A,D,Error}	1
{A,B,C,D}	{A,B,C,D, Error}	{A,B,C,D,Error}	1
{A,B,C,D,Error}	{A,C,D, Error}	{A,B,C,D,Error}	1

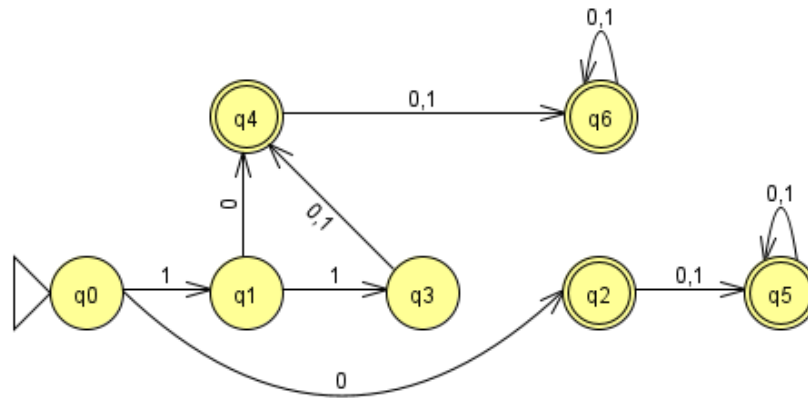
PASO 3. Se pueden renombrar los estados en grupos y se re-emplazan en la tabla para generar una nueva tabla.

Q = estados	0	1	F
{A}=q ₀	{A,D}=q ₂	{A,B}=q ₁	0
{A,B}=q ₁	{A,B,C,D}=q ₄	{A,B,C}=q ₃	0
{A,D}=q ₂	{A,D,Error}=q ₅	{A,D,Error}=q ₅	1
{A,B,C}=q ₃	{A,B,C,D}=q ₄	{A,B,C,D}=q ₄	0
{A,D,Error}=q ₅	{A,D,Error}=q ₅	{A,D,Error}=q ₅	1
{A,B,C,D}=q ₄	{A,B,C,D, Error}=q ₆	{A,B,C,D,Error}=q ₆	1
{A,B,C,D,Error}=q ₆	{A,B,C,D, Error}=q ₆	{A,B,C,D,Error}=q ₆	1

PASO 4. Con esta tabla se construye el autómata finito determinista (AFD), ya sin épsilon transiciones y dado que acepta el mismo lenguaje:

$LR = \{[0|1]^*0 \mid [0|1]^*1[0|1] [0|1]\}$ Es un autómata equivalente.

IMPORTANTE: Se conserva el estado D como estado final, considerándose como final el grupo de estados que contiene a D y también el nuevo estado de error es final de los símbolos del alfabeto.



PASO 5. La definición formal del AFD de la anterior página es:

$M = \{Q, \Sigma, S, F, \delta\}$.

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

$\Sigma = \{0, 1\}$

$S = \{q_0\}$

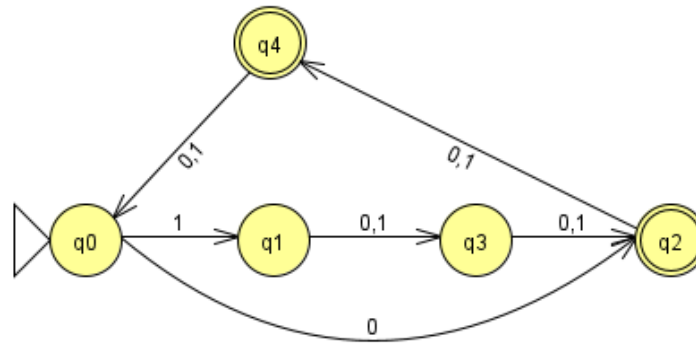
$F = \{q_4, q_5, q_6\}$

δ = según la siguiente tabla:

Entradas (alfabeto) = Σ

Estados	0	1	Finales
q_0	$\{q_2\}$	$\{q_1\}$	0
q_1	$\{q_4\}$	$\{q_3\}$	0
q_2	$\{q_5\}$	$\{q_5\}$	1
q_3	$\{q_4\}$	$\{q_4\}$	0
q_4	$\{q_6\}$	$\{q_6\}$	1
q_5	$\{q_5\}$	$\{q_5\}$	1
q_6	$\{q_6\}$	$\{q_6\}$	1

El anterior autómata lo podemos convertir en un AFD mínimo, para el mismo lenguaje regular: $LR = \{[0|1]^*0 \mid [0|1]^*1[0|1] [0|1]\}$ esto es, con el menor número de estados.



La definición formal de este AFD es la siguiente:

Q = estados	0	1	F
q_0	q_2	q_1	0
q_1	q_3	q_3	1
q_2	q_4	q_4	0
q_3	q_2	q_2	
q_4	q_0	q_0	1

$M = \{Q, \Sigma, S, F, \delta\}$.

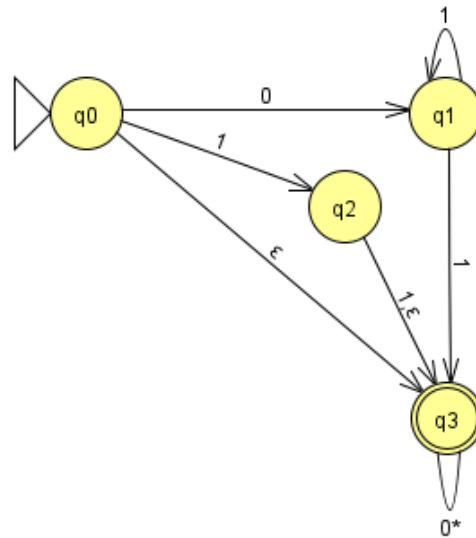
$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{0,1\}$

$S = \{q_0\}$

$F = \{q_2, q_4\}$

Para concluir con los temas de esta unidad, haremos un ejemplo completo de conversión de un **AFND- ϵ** a **AFND** sin épsilon transiciones y luego a un **AFD**.



Recordemos lo que es la épsilon cerradura: Son el conjunto de todos aquellos estados en los cuales se accede a ellos mediante la cadena vacía o épsilon cerradura. La épsilon cerradura de un estado, es al menos el estado en cuestión, por ejemplo:

La épsilon cerradura de q_0 es q_0 de una forma "formal" se especifica:

$$\varepsilon\text{-C}(q_0, \varepsilon) = \delta^*(q_0, \varepsilon) = \{q_0\}$$

Aclaremos que, para este material, los símbolos $\varepsilon\text{-C}$ se leen como la épsilon cerradura. El símbolo δ se le como la transición y δ^* es la transición modificada de la δ transición.

La descripción formal del autómata es:

$M = \{Q, \Sigma, \delta, S, F\}$ donde: $Q = \{q_0, q_1, q_2, q_3\}$

El alfabeto, que es el conjunto de símbolos que se utilizan para el lenguaje (de solo dos palabras de longitud uno) es: $\Sigma = \{\varepsilon, 0, 1\}$

Las transiciones son: $\delta = Q \times \Sigma$

Q = estados	ε	0	1	F
q_0	q_0	q_1	q_2	0
q_1	q_1	$\emptyset$	$\{q_1, q_3\}$	0
q_2	q_2	$\emptyset$	q_3	1
q_3	q_3	q_3	q_3	0
$S = \{q_0\}$		$F = \{q_3\}$		



El lenguaje aceptado para este autómata, son todas y cada una de las cadenas que llegan al estado final de q_3 y este lenguaje a su vez puede incluir una expresión regular o varias de éstas. Consideradas las expresiones regulares como metalenguajes, esto es, el lenguaje de los lenguajes y queda:

$$L = \{0^*, 1, 11, [01^*1|1|11]0^*\}$$

A continuación, vamos a obtener la épsilon cerradura de los estados que se definen en el autómata de la página anterior. Empezamos con la épsilon cerradura de todos y cada uno de los cuatro estados del autómata desde $q_0 \dots q_1 \dots q_2 \dots q_3 \dots$ primero con épsilon y luego con cada uno de los símbolos del alfabeto.

Que para este autómata el alfabeto es:

$$\Sigma = \{ \epsilon, 0, 1 \} \text{ (según descripción "formal" anterior) } \epsilon$$

$$\epsilon\text{-C}(q_0, \epsilon) = \delta^*(q_0, \epsilon) = \{q_0, q_3\}$$

$$\epsilon\text{-C}(q_1, \epsilon) = \delta^*(q_1, \epsilon) = \{q_1\}$$

$$\epsilon\text{-C}(q_2, \epsilon) = \delta^*(q_2, \epsilon) = \{q_2, q_3\}$$

$$\epsilon\text{-C}(q_3, \epsilon) = \delta^*(q_3, \epsilon) = \{q_3\}$$

$$\delta^*(q_0, 0) = \epsilon\text{-C}(\delta(\delta^*(q_0, \epsilon), 0)) = \epsilon\text{-C}(\delta(\{q_0, q_3\}, 0)) = \epsilon\text{-C}(\{q_1\} \cup \{q_3\}) = \epsilon\text{-C}(\{q_1, q_3\}) = \{q_1, q_3\}$$

$$\delta^*(q_1, 0) = \epsilon\text{-C}(\delta(\delta^*(q_1, \epsilon), 0)) = \epsilon\text{-C}(\delta(\{q_1\}, 0)) = \{\emptyset\}$$

$$\delta^*(q_2, 0) = \epsilon\text{-C}(\delta(\delta^*(q_2, \epsilon), 0)) = \epsilon\text{-C}(\delta(\{q_2, q_3\}, 0)) = \epsilon\text{-C}(\{q_3\}) = \{q_3\}$$

$$\delta^*(q_3, 0) = \epsilon\text{-C}(\delta(\delta^*(q_3, \epsilon), 0)) = \epsilon\text{-C}(\delta(\{q_3\}, 0)) = \epsilon\text{-C}(\{q_3\}) = \{q_3\}$$

$$\delta^*(q_0, 1) = \epsilon\text{-C}(\delta(\delta^*(q_0, \epsilon), 1)) = \epsilon\text{-C}(\delta(\{q_0, q_3\}, 1)) = \epsilon\text{-C}(\{q_2\}) = \{q_2, q_3\}$$

$$\delta^*(q_1, 1) = \epsilon\text{-C}(\delta(\delta^*(q_1, \epsilon), 1)) = \epsilon\text{-C}(\delta(\{q_1\}, 1)) = \epsilon\text{-C}(\{q_1, q_3\}) = \{q_1, q_3\}$$

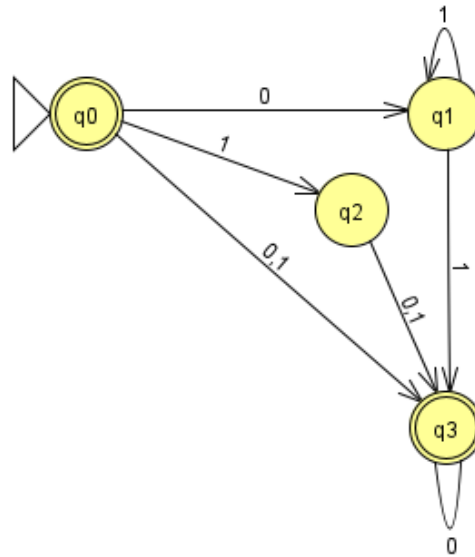
$$\delta^*(q_2, 1) = \epsilon\text{-C}(\delta(\delta^*(q_2, \epsilon), 1)) = \epsilon\text{-C}(\delta(\{q_2, q_3\}, 1)) = \epsilon\text{-C}(\{q_3\} \cup \{\emptyset\}) = \epsilon\text{-C}(\{q_3\}) = \{q_3\}$$

$$\delta^*(q_3, 1) = \epsilon\text{-C}(\delta(\delta^*(q_3, \epsilon), 1)) = \epsilon\text{-C}(\delta(\{q_3\}, 1)) = \epsilon\text{-C}(\{\emptyset\}) = \{\emptyset\}$$

Conviene comprobar en el modelo gráfico, si este autómata que se forma de las transiciones anteriores es ya un autómata finito no determinístico, pero sin la cadena vacía o épsilon dentro de sus transiciones, por lo que es evidente que ya no está, y también gráficamente podemos verificar si el lenguaje aceptado es el mismo que en el autómata anterior para validar que ambos son autómatas equivalentes.

$L = \{0^*, 1, 11, [01^*1|1|11]0^*\}$

Con todo lo anterior, obtenemos un nuevo AFND ya sin épsilon transiciones y su descripción es la que se muestra en el siguiente diagrama:



La descripción formal de éste AFND es:

$M = \{Q, \Sigma, \delta, S, F\}$ donde:

$Q = \{q_0, q_1, q_2, q_3\}$

El alfabeto, que es el conjunto de símbolos que se utilizan para el lenguaje (de solo dos palabras de longitud uno) es: $\Sigma = \{0, 1\}$ (este ya no incluye a la cadena vacía o épsilon)

Las transiciones son: $\delta = Q \times \Sigma$

Q = estados	0	1	F
q_0	$\{q_1, q_3\}$	$\{q_2, q_3\}$	1
q_1	$\{\emptyset\}$	$\{q_1, q_3\}$	0
q_2	$\{q_3\}$	$\{q_3\}$	0
q_3	$\{q_3\}$	$\{\emptyset\}$	1

$S = \{q_0\}$

$F = \{q_0, q_3\}$

Ejemplo de transformación de Autómata Finito no Determinista (AFND) a un Autómata Finito Determinista (AFD) EQUIVALENTE.

Como ya se explicó anteriormente, Estas transformaciones nos permiten llegar a un autómata mínimo cuyo procesamiento resulta más objetivo y práctico, y recordemos que un autómata finito no determinista, no incluye una transición para cada uno de los símbolos del alfabeto, sin embargo, un ejemplo sencillo de presentar es cuando se tiene como únicos símbolos del alfabeto al 0 y al 1. Es por esto la razón de que las computadoras se hayan diseñado con sistemas binarios en su lenguaje de máquina.

Partimos del autómata finito no determinista sin épsilon transiciones del diagrama anterior. **AFND**. Añadimos un estado de error o "X" puede ser etiquetado con cualquier nombre según se muestra en la siguiente **Tabla**. Generamos la tabla considerando la transición de error y agrupando nuevas transiciones, agregando las nuevas transiciones y nos queda la tabla en la siguiente página.

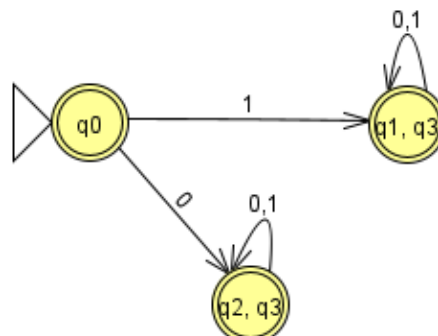
Las transiciones son: $\delta = Q \times \Sigma$

Q = estados	0	1	F
$\{q_0\}$	$\{q_1, q_3\}$	$\{q_2, q_3\}$	1
$\{q_1, q_3\}$	$\{q_3\}$	$\{q_1, q_3\}$	1
$\{q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$	1
$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	1

Con esta tabla se construye el autómata finito determinista ya sin épsilon transiciones y dado que acepta el mismo lenguaje regular:

$$L = \{0^*, 1, 11, [01^*1|1|11]0^*\}$$

Este es un AFD o Autómata Finito Determinístico equivalente. Se conserva el estado $\{q_3\}$ como estado final.





La descripción formal de éste AFD es:

$M = \{Q, \Sigma, \delta, S, F\}$ donde:

$Q = \{q_0, q_1, q_2, q_3\}$

El alfabeto, que es el conjunto de símbolos que se utilizan para el lenguaje (de solo dos palabras de longitud uno) es: $\Sigma = \{0, 1\}$ (este ya no incluye a la cadena vacía o épsilon)

$S = \{q_0\}$

$F = \{q_0, q_1, q_2, q_3\}$



UNIDAD III

3.1 Elementos de diseño de Analizadores Léxicos.

Los analizadores léxicos han tenido múltiples aplicaciones en el desarrollo de software de base, tales como los Sistemas Operativos y los protocolos de comunicación. Los recorridos del código fuente que se realizan dentro de los lenguajes ensambladores, intérpretes de instrucciones y los compiladores. Todas estas herramientas requieren de un analizador léxico. Además de las aplicaciones de software de base se tienen convertidores o traductores tan simples como indispensables que convierten una cantidad escrita con números a su correspondiente representación escrita con lenguajes tales como el inglés, español, francés, etc.

Dentro de los sistemas teleinformáticas de gestión se requiere de un proceso indispensable de validar o identificar los datos numéricos requeridos o cualquier otro tipo de dato que debe entrar al procesamiento para generar resultados. A este proceso se le considera como FILTROS, dependiendo de las necesidades o requerimientos que son identificados símbolo por símbolo, carácter por carácter, integrando palabras o cadenas que se conocen como los componentes léxicos del lenguaje o código de entrada también denominado código fuente.

Actualmente, cada vez son más requeridos los traductores en lenguajes naturales, el procesamiento de datos o información no estructura en la red, ya sea de datos digitales, voz o videos. Todos estos elementos también requieren de analizadores léxicos realizados con herramientas computacionales, tanto para aplicaciones en dispositivos móviles como todo tipo de aplicaciones que pueda crear el intelecto y la creatividad del diseñador de sistemas.

Las funciones de estos módulos de un analizador léxico de tipo digital son las siguientes:

- Analizador Lexicográfico: Las principales funciones que realiza son:
- Identificar los símbolos.
- Eliminar los blancos, caracteres de fin de línea, etc...
- Eliminar los comentarios que acompañan al código fuente.
- Crear símbolos intermedios por cada lexema, estos símbolos son llamados tokens.
- Avisar de los errores que detecte.
- Actualizar la tabla de errores



- Avanzar en la revisión sin importar el número de errores y sin caer en ciclos repetitivos “loop”.
- Actualizar la tabla de símbolos

Ejemplo: A partir de la sentencia en PASCAL:

nuevo:= viejo + RAZON*2

Esto genera un código simplificado para el análisis sintáctico posterior, por ejemplo:

<id1> <:=> <id2> <+> <id3> <*> <ent>

Nota: Cada elemento encerrado entre <> representa un único token. Las abreviaturas <id> y <ent> significan identificador y entero, respectivamente.

<id1> significa identificador 1 equivale a nuevo

<:=> separador

<id2> significa identificador 2 equivale a viejo

<+> es el signo de más

<id3> significa identificador 3 equivale a RAZON

<*> es el signo de multiplicación

<ent> significa el número entero 2

De los elementos más importantes del diseño de un analizador léxico son la tabla de símbolos y el manejador de errores. Mismos que se describen a continuación:

TABLAS DE SÍMBOLOS.

Las tablas de símbolos (también llamadas tablas de identificadores y tablas de nombres), realizan dos importantes funciones en el proceso de traducción:

- 1) Verificar que la semántica sea correcta y
- 2) Ayudar en la generación apropiada de código.

Ambas funciones se realizan insertando o recuperando desde la tabla de símbolos los atributos de las variables usadas en el programa fuente. Estos atributos, tales como: el nombre, tipo, dirección de almacenamiento y dimensión de una variable, usualmente se encuentran explícitamente en las declaraciones o más implícitamente a través del contexto en que aparecen los nombres de variables en el programa.

Una de las estructuras de datos que se encuentran relacionadas con las fases del proceso de compilación es la tabla de símbolos, la cual tiene como propósito registrar información que se comparte entre varias etapas y que permite administrar los recursos asociados a las entidades que manipulará el programa.

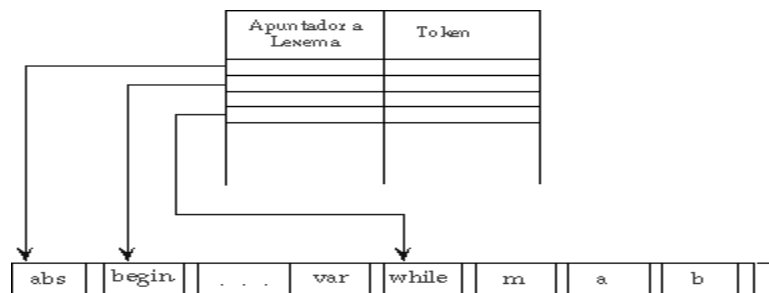
La tabla de símbolos tiene típicamente la estructura de la siguiente forma:

	Lexema	Token	Atributos
1			
2			
...			
n			

Primera propuesta de formato para el diseño de tabla de símbolos.

En donde podemos apreciar la designación de la entidad y su **token** -derivados del análisis de léxico- así como una serie de atributos (tipo de dato, dirección en memoria) que emanan de otras fases (análisis gramatical y semántico). Las consultas a la tabla de símbolos se realizan por medio del lexema con que se designa a la entidad.

Esta concepción de la tabla de símbolos es demasiado simple para fines prácticos, si consideramos que el lexema de la entidad es de longitud variable y se desea que la estructura sea homogénea. Una solución es considerar que en el campo lexema se tiene un apuntador (que siempre ocupa el mismo espacio) hacia donde se registrarán propiamente los lexemas. Eso evitará el desperdicio de memoria al tener el espacio justo para representar a cada lexema.



Propuesta de formato para el análisis de lexemas.



La creación de la tabla de símbolos compete inicialmente al analizador de léxico, quien registrará a las entidades (reconocidas bajo el patrón de Identificador) de manera única, por medio del binomio de operaciones Búsqueda-Inserción.

En el contexto de un programa las entidades pueden describir propiamente objetos manipulables por el lenguaje (por ejemplo, variables, constantes o funciones) o descriptores de acciones (las palabras reservadas); ambas situaciones son reconocidas bajo el mismo patrón de identificador y la tabla de símbolos se emplea para hacer su discriminación. El analizador de léxico funciona bajo el siguiente mecanismo:

REPITE

Token = AnaLex();

SI Token == Identificador

ENTONCES

Token= REVISA_RESERVADAS (lexema)

FSI

HASTA QUE Token == EOF

FREPITE

¿Cuándo construir la tabla de símbolos y cuando interactuar con ella?

El punto del procesador de traducción en el cual son invocadas las rutinas de manejo de la tabla de símbolos. Esto depende primeramente del número y la naturaleza de los pasos del compilador.

En un compilador multipasos, la tabla de símbolos es creada durante el paso de análisis léxico. Por medio de un índice se entra a la tabla de símbolos para ubicar la variable, a partir del token generado por el “scanner”.

Contenido de la tabla de símbolos.

Una tabla de símbolos puede conceptualizarse como una serie de renglones, cada uno de los cuales contiene una lista de valores de atributos que son asociados con una variable en particular. Las clases de los atributos que aparecen en una tabla de símbolos dependen en algún grado de la naturaleza del lenguaje de programación para el cual se escribe el compilador.

Por ejemplo, un lenguaje puede ser sin tipos, y por lo tanto el atributo tipo no necesita aparecer en la tabla. Similarmente, la organización de la tabla de símbolos variará dependiendo de las limitaciones de memoria y tiempo de acceso.

A continuación, se presenta un ejemplo de una tabla de símbolos típica.

Número	Nombre Variable	Dirección.	Tipo	Valor	Línea Declaración	Línea Referenciada	Liga
1	Empresa	0	2	1	2	9,14,25	3
2	X3	4	1	0	3	12,14	0
3	Formal	8	3	2	4	36,37,38	6
4	B	48	1	0	5	10,11,13,23	1
5	Resp	52	1	0	5	11,23,25	2
6	M	56	6	0	6	17,21	7
7	Primero	64	1	0	7	28,29,30,38	5

Éste es un ejemplo de tabla de símbolos

Los atributos que se manejan en la tabla anterior y que se describen enseguida, no son estrictamente necesarios para todos los compiladores; sin embargo, cada uno de tales atributos deberá ser considerado para la implementación de un compilador de un compilador en particular.

- Nombre de la variable.
- Dirección del código objeto.
- Tipo.
- Valor (o número de parámetros para uno procedimiento).
- Número de línea fuente donde fue declarada la variable.
- Números de línea fuente donde se hace referencia a la variable.
- Liga. Campo cuyos valores sirven para listar las variables en orden alfabético.

El nombre de la variable debe estar, en cualquier caso, formando parte de la tabla de símbolos, ya que es el medio por el cual una variable en particular es identificada en las etapas de análisis semántico y generación de código. Para proveer un acceso rápido, es conveniente manejar un tamaño predefinido pero lo suficientemente grande para los nombres de las variables. Una longitud igual o mayor que 16 caracteres es bastante adecuado. El identificador completo puede almacenarse, justificado a la izquierda, en un campo de longitud fija en la tabla de símbolos. Este criterio posibilita un acceso más rápido a la tabla de símbolos, a costa de no aprovechar eficientemente el espacio de almacenamiento en el caso de las variables con identificadores cortos.



Otro criterio para manejar los identificadores en la tabla de símbolos consiste en colocar una cadena descriptora en el campo Nombre Variable de la tabla. El descriptor contiene los subcampos posición y longitud. El subcampo posición es un apuntador que indica la posición del primer carácter del nombre de la variable en un área general de cadenas, y el subcampo describe el número de caracteres del nombre de la variable. Este enfoque produce un acceso más lento a la tabla de símbolos, pero ofrece un ahorro considerable de espacio de almacenamiento.

En el proceso de compilación, una dirección de código objeto debe asociarse con todo variable en un programa. Esta dirección establece la ubicación relativa para los valores de la variable en tiempo de ejecución. La dirección del código objeto se coloca en la tabla de símbolos cuando la variable es declarada (o encontrada por primera vez). Esta dirección es reinvocada desde la tabla cuando la variable es referenciada en el programa fuente. La dirección es luego utilizada en una instrucción objeto que accede al valor de esa variable (carga o almacena).

El atributo tipo se almacena en la tabla de símbolos cuando los lenguajes compilados tienen tipos de datos explícitos o implícitos. Tradicionalmente, el tipo de dato de una variable es almacenado codificada en la tabla de símbolos; por ejemplo 1 puede representar el tipo real, 2 el tipo integer, etc.

Los atributos `número_de_dimensiones` y `número_de_parámetros` son importantes en la fase de análisis semántico. En las referencias a los arreglos, el número de dimensiones debe coincidir con el especificado en la declaración del arreglo, y esto debe ser verificado en la fase de análisis semántico. El número de dimensiones se utiliza también como parámetro en el cálculo de la dirección de un elemento particular del arreglo. El número de parámetros de la invocación a un procedimiento también debe coincidir con el número usado en la declaración. En la construcción de la tabla de símbolos, es conveniente considerar el número de parámetros de un procedimiento como su número de dimensiones y así combinar estos dos atributos en uno. Además de conveniente, este enfoque también es consistente, ya que el tipo de verificación semántica para ambos atributos es similar.

El atributo `liga` se ha incluido en la tabla de ejemplo simplemente para facilitar la producción de un listado de referencias cruzadas ordenado alfabéticamente por nombre de variable. La tabla de símbolos debe incluir cada uno de los elementos o cadenas aceptadas por el lenguaje, conocidos como elementos terminales, para asignarle el token o identificador numérico que le corresponda a cada elemento terminal. Los elementos no terminales, se deberán dar de alta en la tabla de símbolos y reasignarle el mismo token o identificador cada vez que se utilicen.



Ejemplo:

Diseño de Tablas de símbolos por carácter para el lenguaje STOP. "Stack Opertion", operador de pila.

Tabla para **un** carácter

Ch.	token
`	100
"	102
[	104
]	106
+	108
-	110
*	112
/	114
\	116
^	118
=	120
<	122
>	124
&	126
	128
!	129

Tabla para **dos** caracteres

Instr.	token
DO	200
IF	202
UP	204
<=	206
=>	208
><	210

Tabla para **tres** caracteres

Instr.	token
ALL	300
END	302
FOR	304
IFF	306
POP	308
TOP	310

Tabla de **cuatro** caracteres

Instr.	token
DOWN	400
DROP	402
ELSE	404
OPEN	406
OVER	408
PICK	410
PUSH	412
READ	414
ROLL	416
THEN	418
TYPE	420
SIZE	422

Tabla de **cinco** caracteres

Instr.	token
BELOW	500
CLEAR	502
CLOSE	504
ENTER	506
STORE	508
UNTIL	510
WHILE	512
WRITE	514

Tabla de **siete** caracteres

Instr.	token
INCLUDE	700



Tabla de **ocho** caracteres

Instr.	token
INDIRECT	800
EVALUATE	802

Tabla de **seis** caracteres

Instr.	token
CHOOSE	600
INSERT	602
MODULE	604
OPTION	606
REPEAT	608
ROTATE	610

Tabla de **nueve** caracteres

Instr.	token
DUPLICATE	900
TRANSFORM	902

MANEJO DE ERRORES

Algunos errores que se presentan en el análisis léxico son como escribir mal un identificador, palabra clave u operador.

Hay diversas razones por la que se separa la fase léxica de la sintáctica son:

El diseño del analizador sintáctico es más fácil de esa forma, ya que este no ha de preocuparse de leer el archivo de entrada, ni de saltar blancos, ni comentarios, ni de recibir caracteres inesperados, puesto que todo ello ha sufrido un filtrado previamente por el analizador lexicográfico. Por tanto, el diseño consiguiente se hace más claro y comprensible.

Se mejora la eficiencia del compilador en su conjunto. La lectura del programa fuente suele requerir gran parte del tiempo de compilación, que se ve reducido si el analizar lexicográfico incorpora técnicas especializadas de lectura, o está realizando un ensamblador. Aumenta la portabilidad del compilador, ya que de todas las diferencias que se produzcan en el alfabeto de entrada, o en el dispositivo de almacenamiento, pueden ser reducidas al analizador lexicográfico, dejando al análisis sintáctico intacto. Se pueden resolver fácilmente casos dudosos.

El analizador lexicográfico, además, realiza ciertas tareas adicionales como:

- Eliminar comentarios del programa fuente.
- Eliminar los blancos, saltos de página, tabuladores, retornos de carro, y demás caracteres propios del dispositivo de entrada.
- Reconocer las variables y asignarles una posición en la tabla de símbolos.
- Relacionar los mensajes de error que produce el compilador en sus diversas fases con la parte correspondiente del programa fuente



Si el programa fuente incorpora meta-nociones o macros, el analizador lexicográfico puede incorporar un preprocesador, avisa de los errores lexicográficos que detecta.

Hay diversas razones por las que se separa la fase de análisis de un compilador en análisis lexicográfico y análisis sintáctico. Son las siguientes:

- El diseño del analizador sintáctico es más fácil de esta forma, ya que éste no ha de preocuparse de leer el archivo de entrada, ni de saltar blancos, ni comentarios, ni de recibir caracteres inesperados, puesto que todo ello ha sido filtrado previamente por el analizador lexicográfico. Por tanto, el diseño consiguiente se hace más claro y comprensible.
- Se mejora la eficiencia del compilador en su conjunto. La lectura del programa fuente suele requerir gran parte del tiempo de compilación, que se ve reducido si el analizador lexicográfico incorpora técnicas especiales de lectura, o está realizado en ensamblador.
- Aumenta la portabilidad del compilador, ya que todas las diferencias que se produzcan en el alfabeto de entrada, o en el dispositivo de almacenamiento, pueden ser reducidas al analizador lexicográfico, dejando al analizador sintáctico intacto. Supóngase que el compilador lee un texto ASCII y nos interesa construir uno que sea capaz de leer un texto escrito bajo el estándar EBCDIC. Bastaría con modificar el analizador lexicográfico.

Se pueden resolver fácilmente casos dudosos como la siguiente sentencia FORTRAN:

DO5I=1.25 (Compárese con DO5I=1,25)

Hasta que no se reconoce el carácter "." o el carácter "," no podemos saber si se trata de una sentencia de asignación o de una instrucción DO. Para comprender mejor por qué se realiza un explorador veamos el siguiente ejemplo, que corresponde a una definición muy restringida del lenguaje de programación PASCAL:

```
<SENT>  -> if <EXP-BOOL> then <SENT> [else <SENT>]<SENT>  -> <VAR> :  
= <EXP><EXP-BOOL>-> <VAR-BOOL> | <EXP> <COMP> <EXP><COMP>  -> <  
| > | <> | <= | >= | =< | => | =<EXP>      -> <EXP> <OP> <FACTOR> |  
<FACTOR><FACTOR>  -> ( <EXP> ) | <VAR> | <CTE><CTE>      -> <DIGITO>  
[<DIGITO>]<VAR>      -> <LETRA> [<ALFANUM>]<VAR-BOOL>-> <LETRA>  
[<ALFANUM>]<ALFANUM> -> <LETRA> | <DIGITO><LETRA>  -> a | b | ..... |  
z<DIGITO>  -> 0 | 1 | ..... | 9<OP>      -> + | - | * | / | **
```



En esta gramática serían símbolos terminales los siguientes:

$$T = \{ , , a, b, c, \dots z, (,), +, -, *, /, <, >, =, : \}$$

y los símbolos no terminales que se han empleado son:

$$N = \{ \langle \text{SENT} \rangle, \langle \text{EXP-BOOL} \rangle, \langle \text{VAR} \rangle, \langle \text{EXP} \rangle, \langle \text{VAR-BOOL} \rangle, \langle \text{COMP} \rangle, \langle \text{OP} \rangle, \langle \text{FACTOR} \rangle, \langle \text{CTE} \rangle, \langle \text{LETRA} \rangle, \langle \text{ALFANUM} \rangle, \langle \text{DIGITO} \rangle \}$$

Esta gramática puede ser descompuesta o transformada en otras dos, como éstas:

Primera:

$$\begin{aligned} \langle \text{PRES-IF} \rangle &\rightarrow \text{if} \langle \text{PRES-THEN} \rangle \rightarrow \text{then} \langle \text{PRES-ELSE} \rangle \rightarrow \text{else} \langle \text{COMP} \rangle \rightarrow < | \\ &> | < > | < = | > = | = < | = > | = \langle \text{CTE} \rangle \rightarrow \langle \text{DIGITO} \rangle [\langle \text{DIGITO} \rangle] \langle \text{VAR} \rangle \rightarrow \\ &\langle \text{LETRA} \rangle [\langle \text{ALFANUM} \rangle] \langle \text{VAR-BOOL} \rangle \rightarrow \langle \text{LETRA} \rangle [\langle \text{ALFANUM} \rangle] \langle \text{ALFANUM} \rangle \\ &\rightarrow \langle \text{LETRA} \rangle | \langle \text{DIGITO} \rangle \langle \text{LETRA} \rangle \rightarrow a | b | \dots | z \langle \text{DIGITO} \rangle \rightarrow 0 | 1 | \dots | \\ &9 \langle \text{OP} \rangle \rightarrow + | - | * | / | ** \langle \text{ASIG} \rangle \rightarrow := \langle \text{ABREPAR} \rangle \rightarrow (\langle \text{CIERRAPAR} \rangle \rightarrow) \end{aligned}$$

Cuyos símbolos terminales y no terminales serían:

$$T = \{ , , a, b, c, \dots z, (,), +, -, *, /, <, >, =, : \}$$

$$N = \{ \langle \text{PRES-IF} \rangle, \langle \text{PRES-THEN} \rangle, \langle \text{PRES-ELSE} \rangle, \langle \text{COMP} \rangle, \langle \text{CTE} \rangle, \langle \text{VAR} \rangle, \langle \text{VAR-BOOL} \rangle, \langle \text{LETRA} \rangle, \langle \text{DIGITO} \rangle, \langle \text{ALFANUM} \rangle, \langle \text{OP} \rangle, \langle \text{ASIG} \rangle, \langle \text{ABREPAR} \rangle, \langle \text{CIERRAPAR} \rangle \}$$

Segunda:

$$\begin{aligned} \langle \text{SENT} \rangle &\rightarrow \langle \text{pres-if} \rangle \langle \text{EXP-BOOL} \rangle \langle \text{pres-then} \rangle \langle \text{SENT} \rangle [\langle \text{pres-else} \rangle \\ &\langle \text{SENT} \rangle] \langle \text{SENT} \rangle \rightarrow \langle \text{var} \rangle \langle \text{asig} \rangle \langle \text{EXP} \rangle \langle \text{EXP-BOOL} \rangle \rightarrow \langle \text{VAR-BOOL} \rangle | \\ &\langle \text{EXP} \rangle \langle \text{comp} \rangle \langle \text{EXP} \rangle \langle \text{EXP} \rangle \rightarrow \langle \text{EXP} \rangle \langle \text{op} \rangle \langle \text{FACTOR} \rangle | \\ &\langle \text{FACTOR} \rangle \langle \text{FACTOR} \rangle \rightarrow \langle \text{abrepar} \rangle \langle \text{EXP} \rangle \langle \text{cierrapar} \rangle | \langle \text{var} \rangle | \langle \text{cte} \rangle \end{aligned}$$

Cuyos símbolos terminales y no terminales serían:

$$T = \{ \langle \text{pres-if} \rangle, \langle \text{pres-then} \rangle, \langle \text{pres-else} \rangle, \langle \text{var} \rangle, \langle \text{asig} \rangle, \langle \text{var-b} \rangle, \langle \text{comp} \rangle, \langle \text{op} \rangle, \langle \text{abrepar} \rangle, \langle \text{cierrapar} \rangle, \langle \text{cte} \rangle \}$$

$$N = \{ \langle \text{SENT} \rangle, \langle \text{EXP-BOOL} \rangle, \langle \text{EXP} \rangle, \langle \text{FACTOR} \rangle \}$$



La primera de estas dos gramáticas es la que utiliza el explorador o analizador lexicográfico, mientras que la segunda corresponde a la gramática del parser o analizador sintáctico. Como puede apreciarse, la primera gramática es de tipo 3 según la clasificación de Chomsky (es decir, es una gramática regular), mientras que la segunda es una gramática de contexto libre (o sea, de tipo 2).

Debe hacerse notar que las meta-nociones (símbolos no terminales) de la gramática del analizador lexicográfico, o bien no aparecen en la gramática del analizador sintáctico, o son símbolos terminales en dicha gramática.

Para la implementación de un analizador lexicográfico existen en la actualidad tres vías. La principal ventaja de cada una de ellas respecto a las siguientes es la mayor comodidad y facilidad de programación. Por el contrario, la implementación en un lenguaje más cercano a la máquina hace posible una mejor gestión de las entradas/salidas, con la consiguiente rapidez de ejecución.

Escribir el analizador lexicográfico en lenguaje ensamblador directamente. Escribir el analizador lexicográfico en un lenguaje de alto nivel, usando las facilidades de entrada/salida que éste tenga. Usar un generador de analizadores lexicográficos, capaz de producir un analizador lexicográfico a partir de las expresiones regulares que definen los distintos tokens.

En este caso, el generador incorpora sus propias rutinas de entrada/salida. Uno de los generadores de analizadores lexicográficos cuyo uso está más generalizado es el programa Lex. Mismo que se describe en el ejercicio de este capítulo.

3.2 Diseño o descripción de una aplicación que requiera el uso de un lenguaje de programación.

CASO PRÁCTICO

Usted se encuentra trabajando para una compañía desarrolladora de software a la medida. Su jefe se acaba de enterar que está llevando una unidad de aprendizaje de Teoría de Lenguajes y Compiladores, así que decide que es la persona ideal para el proyecto de uno de sus principales clientes. Se le asigna a asistir a una entrevista con el usuario a fin de obtener las especificaciones finales del último módulo del producto.



En la entrevista el cliente comenta:

“Para el módulo de análisis necesitamos generar gráficas y que se conserven para posteriores análisis, pero requerimos que tanto los datos como los parámetros de graficación se guarden, ya que pasado el tiempo la gráfica pierde sentido y el analista sólo puede saber qué hacer si cuenta con los datos y los parámetros usados para generar la gráfica. Guardar datos y especificaciones no nos sirve, todo debe quedar en un mismo archivo. Todas nuestras gráficas son con dos variables (bidimensionales). ¿Qué sugiere?”

Tras un minuto de meditación, usted responde: “Hmmm... Hasta donde veo la única forma es... Creando una forma de especificación para generar la gráfica... Como un pequeño lenguaje que iría al inicio del archivo de datos. De esta forma se puede alimentar el sistema y siempre generar la gráfica deseada”

Su cliente lo piensa un rato... Acepta la idea... Pero conociendo como son sus analistas él le indica como debe ser este lenguaje:

“- Frecuentemente tenemos visitantes o clientes extranjeros que utilizan el sistema... Por lo que las palabras de este lenguaje deben ser en inglés.

- La entrada será un archivo a través de la pantalla en la que cada línea puede ser un dato o una instrucción de graficación.

- Los datos podrán aparecer en pares de números, el primero será la abscisa y el segundo la ordenada, o sólo un número... en cuyo caso el valor de la abscisa se asigna por defecto (1, 2, 3, etc.) ... Y un carácter opcional de graficación puede seguir a cada dato.

- Los datos pueden ser asignados e incluir exponentes científicos.

- Se debe especificar el título de la gráfica, rangos de abscisa y ordenada (en la misma línea), las marcas de cada eje... El largo y ancho de la gráfica. Las especificaciones pueden aparecer en cualquier orden, y todas deben ser opcionales...

Una por línea y no debe ser necesario indicar el tamaño de la muestra...”

“Y eso sería todo. Sólo le pido que todas las instrucciones lleven una cantidad de argumentos fijos, analizadores lexicográficos, capaz de producir un analizador salvo las marcas que podrán ser libres”

La entrevista termina.



SE PIDE: Desarrolle la especificación de este lenguaje en notación BNF o BNFE.

*Redacción Dr. Eduardo René Rodríguez Ávila, IEEE computer Society Branch &
Neural Networks Council*

<https://eravila.wordpress.com/home/intereses/apuntes/>

¿Qué son los BNF y BNFE?

La Notación de Backus-Naur, también llamado BNF extendido o EBNF (del inglés Extended Backus–Naur Form) es un metalenguaje usado para expresar gramáticas libres de contexto. Esta es una extensión de la Notación de Backus-Naur. *Fuente de esta definición: Wikipedia*

Para comprender mejor este concepto, nos adelantaremos un poco a describir breve y coloquialmente lo que es una Gramática libre de contexto abreviada (GLC).

Una GLC es la definición formal de una gramática “formal” que también puede ser regular como las expresiones regulares ya descritas con anterioridad. Dentro de la definición tenemos que dentro de una GLC se tienen cuatro tipos de variables: Variables No Terminales (NT) y dentro de estas La variable inicial No terminal (“Start” con el símbolo S), Variables Terminales (con el símbolo T) y las Reglas de producción, denominadas Producciones, reglas generadoras y un sinfín de nombres que se pueden encontrar dependiente de la fuente de información, pero para efectos de estos apuntes las definiremos con el símbolo P.

Queda: $GLC = \{S, NT, T, P\}$

Ejemplo: Si decidimos crear una instrucción simple como “dibujar” esta palabra DIBUJAR puede servir para crear una Gramática Libre de Contexto. Si definimos que podemos dibujar tres figuras geométricas: Cuadrado, Triangulo y Círculo de tamaño 1 a 10 centímetros y en colores, azul, rojo, amarillo, blanco y negro. Para esto podemos definir un BNF

$\langle \text{dibujar} \rangle = \text{DIBUJA } \langle \text{figuras} \rangle \langle \text{tamaño} \rangle \langle \text{color} \rangle$

$\langle \text{figuras} \rangle = \langle \text{cuadrado} \rangle | \langle \text{triangulo} \rangle | \langle \text{circulo} \rangle$

$\langle \text{tamaño} \rangle = 1|2|3|4|5|6|7|8|9|10$

$\langle \text{color} \rangle = \text{azul} | \text{rojo} | \text{amarillo} | \text{blanco} | \text{negro}$

$\langle \text{cuadrado} \rangle = \square$ $\langle \text{circulo} \rangle = \bigcirc$

$\langle \text{triangulo} \rangle = \Delta$



Los elementos NO terminales se pueden definir con mayúsculas o bien entre “tags” $\langle y \rangle$, las variables terminales son símbolos alfabéticos, numéricos, alfanuméricos o especiales. Estos signos especiales: = separan la variable NO terminales (a la izquierda) de las Terminales a la derecha. No existe un estándar de este signo que se lee: PRODUCE ya que puede ser un guion intermedio -, puede ser también una flecha de izquierda a derecha $\rightarrow$ un símbolo de =, etc.

La siguiente definición se escribió anteriormente y también se puede configurar como un BNF:

$\langle \text{SENT} \rangle: = \text{if } \langle \text{EXP-BOOL} \rangle \text{ then } \langle \text{SENT} \rangle [\text{else } \langle \text{SENT} \rangle]$

$\langle \text{SENT} \rangle: = \langle \text{VAR} \rangle$

$\langle \text{VAR} \rangle: = \langle \text{EXP} \rangle \langle \text{EXP-BOOL} \rangle$

$\langle \text{EXP-BOOL} \rangle: = \langle \text{VAR-BOOL} \rangle \mid \langle \text{EXP} \rangle \langle \text{COMP} \rangle$

$\langle \text{EXP} \rangle \langle \text{COMP} \rangle: = < \mid > \mid < > \mid < = \mid > = \mid < < \mid = > \mid =$

$\langle \text{EXP} \rangle: = \langle \text{EXP} \rangle \langle \text{OP} \rangle \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle$

$\langle \text{FACTOR} \rangle: = (\langle \text{EXP} \rangle) \mid \langle \text{VAR} \rangle \mid \langle \text{CTE} \rangle$

$\langle \text{CTE} \rangle: = \langle \text{DIGITO} \rangle [\langle \text{DIGITO} \rangle] \langle \text{VAR} \rangle$

$\langle \text{VAR-BOOL} \rangle: = \langle \text{LETRA} \rangle [\langle \text{ALFANUM} \rangle] \langle \text{ALFANUM} \rangle$

$\langle \text{ALFANUM} \rangle: = \langle \text{LETRA} \rangle \mid \langle \text{DIGITO} \rangle$

$\langle \text{LETRA} \rangle: = a \mid b \mid \dots \mid z$

$\langle \text{DIGITO} \rangle: = 0 \mid 1 \mid \dots \mid 9$

$\langle \text{OP} \rangle: = + \mid - \mid * \mid / \mid **$

Esta es la definición de un BNFE el cual tiene a las variables NO terminales con mayúscula y entre “tags” $\langle \dots \rangle$ y por cada NO terminal, la definición de sus variables Terminales.

3.3 Diseño de un lenguaje simple de programación.

La teoría sin la práctica se olvida. El aprendizaje además de ser un proceso intrapersonal, conviene que sea significativo, esto es, el que nos es útil para la vida o ganarnos un medio de ingresos económicos.



Usted como estudiante tiene creatividad, inteligencia, habilidad matemática y lo único que hace falta es que ponga en práctica lo aprendido en la mayor cantidad de unidades de aprendizaje, como la ingeniería de software, programación, estructura de datos, bases de datos, lenguajes de programación, tecnología digital, idiomas como el inglés, etc. Esa es la justificación de que realice un proyecto real donde se aplique la teoría de Lenguajes y compiladores, y logre un software conocido como traductor.

RESUMEN EJECUTIVO.

Este proyecto se elabora desde el principio del periodo escolar ya que cuenta para tres evaluaciones, una por cada periodo. En el primer periodo es el diseño conceptual con respecto a un analizador que puede ser léxico, sintáctico, semántico, un traductor o compilador parcial o completo.

El proyecto se puede realizar de forma individual o en equipos, el número de integrantes del equipo dependiente de la complejidad del proyecto. Si se elabora en equipo, se tendrá que adjuntar desde la primera etapa o entrega, elaborar la asignación de actividades para cada elemento del equipo sin duplicar las actividades asignadas a cada responsable.

Primera etapa

“Diseño conceptual de un lenguaje experimental”

SE ENTREGA IMPRESO EXCLUSIVAMENTE. EN EL HORARIO DE CLASE, EN LA CLASE ANTERIOR A LA FECHA DEL EXAMEN DEL PRIMER PERIODO.
(Oportunidad en la entrega 2 puntos)

Se entrega la programación, pruebas y presentación, su contenido, además de los contenidos de la primera etapa, se agrega al menos tres páginas más, con lo siguiente:



1. Manual de usuario o descripción de ¿Cómo se ejecuta el software? Proceso de instalación y utilización. (Dos puntos)
2. Descripción técnica del proyecto. Recordar que estamos elaborando un software de base y se detalla de lo general a lo particular como lo requiere la programación estructurada. (Dos puntos) Incluir: Diagramas, mapas mentales, conceptuales, redes semánticas, diagramas de autómatas, formatos de Backus y Naur (BNF's), etc. Los diagramas son creados específicamente para el programa en cuestión. (Dos puntos)
3. CODIGO FUENTE (sintético) con documentación interna, esto es, comentarios en cada línea de código (dos puntos) En congruencia con los diagramas.
4. EVIDENCIA DE LA EJECUCIÓN O CIFRAS DE CONTROL **(dos puntos)**.
Valor total de la etapa **(diez puntos)**.

Si el proyecto tiene una evaluación menor a 8 (ocho), se considera SOLO como DERECHO para presentar el tercer examen parcial y el examen extraordinario.

Tercera etapa

“Presentación del proyecto completo”

Considerar todos los elementos del proyecto desde el PRIMER PARCIAL. Si el proyecto tiene una evaluación mayor o igual a 8 (ocho), se corrige y se prepara una presentación con material de apoyo y ya corregido se asignarán fechas de presentación de los mejores proyectos presentados y agregando las correcciones al diseño.

- 1) Preparación de presentación para el grupo, mismas que se evalúan con una co-evaluación. La calificación de las presentaciones se promedian y complementan el otro 50% con la evaluación de la presentación realizada por parte del profesor. Al final se suman ambas evaluaciones y se tiene la calificación para el tercer parcial.



- 2) Apegarse a la fecha, hora de la presentación por todos los integrantes del equipo. Esto es un requisito para no perder el derecho a la calificación del tercer parcial. Elaborado por: Sara Méndez García. En la Ciudad de México el 3 de abril de 2020.

ELEMENTOS IMPORTANTES PARA LA PRESENTACIÓN O DISCURSO (3ª. etapa):

Cualquier persona necesita comunicarse con sus semejantes, el uso adecuado de las palabras, principalmente al hablar; con correcta entonación, dicción y el uso impecable de las palabras nos genera muchas ventajas como: inspirar confianza, transmitir ideas de forma asertiva, convencer, conseguir un buen empleo. También ayuda a evitar conflictos, puede ser un medio para ganarse la vida como los políticos, locutores, profesores, vendedores, etc. A todos nos conviene ser buenos comunicadores.

Para aprender a nadar no hay alternativa, sólo cambiar el temor por preparación, tirarnos al agua y practicar; para caminar se aprende caminando y también para hablar en público necesitamos ejercitar con un programa académico pertinente, de calidad y la constancia; conjuntamente con el deseo y la práctica; las evaluaciones respetuosas y el conocimiento de nuestras áreas de oportunidad nos permitirán de forma consiente lograr que todos seamos comunicadores competentes.

RESUMEN EJECUTIVO.

Para su primer proyecto de oratoria, se presentará a los asistentes con información acerca de su vida, trayectoria académica, intereses, ambiciones, habilidades, presentación de un proyecto, etc. Practique dando su discurso y escríbalo antes de presentarlo. Evite leerlo y explíquelo en 5 a 20 minutos.

OBJETIVOS:

Comenzar a hablar ante un público. Descubrir las habilidades de oratoria que ya posee y las habilidades que necesita fortalecer sus áreas de oportunidad.

DESARROLLAR SU DISCURSO, primero por escrito –el escrito no se entrega, solo se explica al público de forma opcional como evaluación continua- con la siguiente estructura.



Introducción. - Debe captar inmediatamente la atención del público y decirle al público algo de lo siguiente:

- Una pregunta sorprendente o un enunciado desafiante.
- Una cita, ilustración o historia apropiada de forma breve.
- Un despliegue de algún objeto o figura.
- Una generalización atractiva que esté vinculada con su personalidad.

Evitar introducciones deficientes: Un enunciado apologético; Un cuento o chiste que no está relacionado con su tema; Una observación común; Un enunciado o historia larga o lenta y Una pregunta trivial, tal como ¿Alguna vez te has detenido a pensar...?

Desarrollo. - No perder el punto principal, después dejar claro el subpunto y se puede llevar material de apoyo fácil de mostrar.

Conclusión. - Puede ser la revisión de lo descrito, un resumen, llamado a la acción o enunciado digno de recordarlo.

Evaluación: 1=Necesita atención 2=Puede mejorar 3=Satisfactorio 4=Superior 5=Excelente

***Valor del discurso _____ Comentarios:_____**

***Preparación _____ Sugerencias:_____**

***Organización _____ Correcciones:_____**

***Introducción _____ Se completó lo solicitado:_____**

***Desarrollo _____ Áreas de oportunidad:_____**

***Conclusión _____ Se apegó a lo solicitado _____**

***Transiciones _____ Fueron aceptables _____**

***Tiempo _____ Se respetó de 5 a 20 minutos:_____**

SUMA DE PUNTOS: _____ Máximo $8 \times 5 = 40$ puntos máximo = 10 de calificación (20 puntos = 5 de calificación 24 = 6, 28=7, 32=8, 36=9)



3.4 Definición de Léxico mediante expresiones regulares.

DESARROLLAR un Analizador léxico puede ser algo tan trivial o sencillo como diseñar una expresión regular; hacer un traductor de cantidades numéricas a algún lenguaje que puede ser en español o en inglés de las cantidades numéricas, esta traducción requiere de una tabla de símbolos y un manejador de errores, mismo que podemos afirmar tiene un nivel medio de complejidad; Algo más complejo como crear un lenguaje de programación nuevo, pero; ¿Porque crear nuevos lenguajes? cuando ya casi son 3000 los lenguajes de programación de computadoras.

En un principio se pensaba que la teoría de lenguajes y compiladores era el área exclusiva de los “gurús” de la computación, pero actualmente se pueden crear un sin fin de herramientas que requieren un analizador léxico que, con las herramientas adecuadas, como los intérpretes de instrucciones, nos permiten tener resultados inmediatos. Aquí vamos a presentar el planteamiento de analizadores léxicos que se pueden realizar con una sola línea de código que incluya una expresión regular para seleccionar de un archivo de texto, las cadenas, palabras o elementos que deseamos seleccionar. Para esto, se solicita que también se incluya el autómata Finito para cada expresión regular y sus resultados.

Paso 1.- Existe un archivo en formato de texto que contiene información relacionada con los lenguajes de programación de computadoras recopilados el:

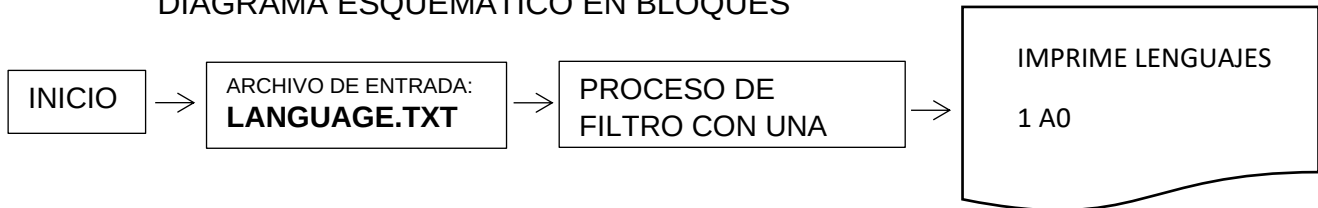
23 de enero de 1995. Disponible en:

<https://sites.google.com/site/upiicsasaramendez/Inicio/compiladores>

Paso 2.- Descargue este archivo, analícelo y seleccione solo el nombre de los lenguajes enumerándolos, desplegarlos en pantalla, imprimiendo el número de lenguajes seleccionados para aproximarnos a los 2350 lenguajes de programación de computadoras que incluye dicho archivo, ordenados desde la letra A a la Z. Este proceso se le conoce como “filtro” dentro del procesamiento de datos.

Paso 3.- Desarrolle el Diagrama del Autómata Finito No determinista con épsilon transiciones de la expresión que más se aproxime a la selección solicitada en el paso anterior.

DIAGRAMA ESQUEMÁTICO EN BLOQUES



Para poder ejecutar este paso se requiere lo siguiente:

Si se trabaja en ambiente UNIX o LINUX el lenguaje AWK se puede ejecutar directamente desde el intérprete de instrucciones del sistema operativo. También trabaja igual en ambientes de MAC.

Si se trabaja en ambientes Microsoft será necesario descargar el lenguaje intérprete AWK.EXE o GAWK.EXE y tener este archivo en la misma carpeta que el archivo de texto. - LANGUAGE.TXT para poder ejecutar alguna de las siguientes expresiones regulares: / - / Esta expresión regular de un solo símbolo (guion intermedio) Servirá como prueba para ejecutar por primera vez, de la siguiente forma en el intérprete de instrucciones: **awk "/ - /" LANGUAGE.TXT**

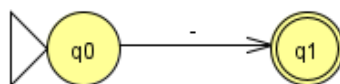
La primera diagonal le indica a AWK el inicio de una expresión regular y la segunda diagonal, el final de la misma expresión regular. Sobre el archivo de entrada.

IMPORTANTE: Dependiendo de la versión del sistema operativo, puede requerir una doble comilla o comilla simple, así que se verifica que los dos archivos estén en la misma carpeta. Si No da resultados la expresión regular, se prueba con comilla sencilla.

La expresión regular de un solo símbolo queda así:

r₁ = -

Conversión a un autómata finito no determinista:





Existe otra forma de ejecutar el AWK y es mediante un código guardado en un archivo de texto: En este caso será nombrado **L.awk** cuyo contenido es lo siguiente:

```
# UPIICSA

# TERORIA DE LENGUAJES Y COMPILADORES

# PROF. SARA MENDEZ GARCIA

BEGIN { print "LISTA DE LENGUAJES RECOPIADOS POR BILL
KINNERSLEY.."

    print "LISTA DE LOS LENGUAJES DE 1900 A 1999"

    FS = "  "}

# LISTA DE LOS LENGUAJES DE 1900 A 1999

/19[0-9] [0-9]/ {
    if(length ($2) != 0) {
        n=n+1

        print n, $2
    }

    END { print "-----"

    print "proceso terminado, suma de lenguajes...", n

    }
```

En este código la expresión regular queda limitada a seleccionar una fecha:

```
/19[0-9][0-9]/
```

Si se desea ejecutar este código, necesitamos teclear lo siguiente en el intérprete de instrucciones:

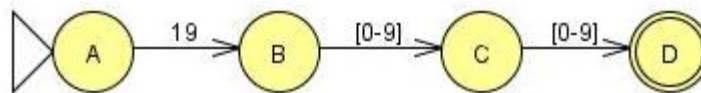
```
awk -f /19[0-9][0-9]/ l.awk language.txt
```

También podemos incluir el redireccionamiento del resultado de esta expresión regular que funciona como analizador léxico con el símbolo > (mayor que) para guardar la salida en otro archivo, que en este caso es: “salida.txt” que si existe lo reemplaza y si no existe lo crea.

awk – f /19[0-9][0-9]/ l.awk language.txt > salida.txt

Conversión a un autómata finito no determinista:

$r_2 = 19[0-9][0-9]$

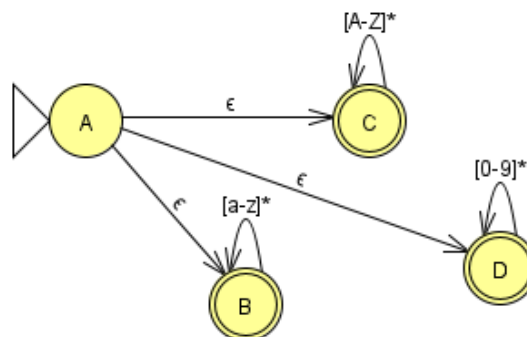


En este autómata se ha simplificado la concatenación de 4 símbolos correspondientes a una fecha que corresponde al formato del siglo XX 19.

Los dos últimos dígitos al este autómata representado con un rango de valores, se incluye del cero al nueve y corresponde a todas las posibilidades desde 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9.

IMPORTANTE: También es posible incluir en la ejecución de una expresión regular instrucciones del lenguaje awk para desde el intérprete de instrucciones con una sola línea de código generemos el desplegado de la selección con el número de lenguaje seleccionado de la siguiente forma:

awk /[a-z|A-Z|0-9] *] - / language.txt {n++; print n, \$2}





Este autómata representa la posibilidad de tener cadenas escritas con mayúsculas, minúsculas o bien cadenas numéricas o la cadena vacía.

EJERCICIO.

Desarrolle un autómata que represente el comportamiento de una calculadora, así como su analizador léxico. El autómata (AFND con épsilon transiciones) tomará en consideración las operaciones que se realizan en el teclado de la calculadora, mismas que se muestra en la siguiente figura, con las siguientes consideraciones:

La Tecla CE/C. Si se pulsa una vez, borra la última entrada (*clear*). Si se pulsa dos veces (*Clear all*), borra todo lo introducido hasta regresar al estado de iniciar nuevamente. El contenido del *display* se toma como independiente, esto es, la salida de resultados es un aspecto que no se incluye dentro del autómata. Existen teclas de doble función como El ON/OFF. La primera vez que se pulsa la tecla ON/OFF se interpreta como ON. La segunda que se oprime la tecla ON/OFF representa un OFF. Esta calculadora cuando se pone la pila siempre espera el pulsar el ON/OFF.

Otras teclas que incluyen doble función son las que tienen el sombreado, todas éstas se activan con la tecla gris. Ejemplo: Para oprimir la función 2 al cubo (2 a la 3), se oprime primero el 2, luego la tecla gris, posteriormente se oprime la tecla Y a la x para luego oprimir el signo de igual y obtener el resultado.

Los paréntesis sólo funcionan una vez. En el caso de las funciones y operaciones. Las funciones de logaritmo y anti-logaritmo (no necesitan la tecla gris) La tecla EE, permite introducir el exponente de ingeniería base 10.

Existen funciones unarias, mismas que requieren un solo operador (operan con el último valor). También existen funciones binarias, las cuales requieren de dos valores (un ejemplo de funciones binarias son e^x y y^x).

La calculadora realiza operaciones aritméticas como son la suma, resta, multiplicación y división. Se aceptan funciones trigonométricas, Seno, Coseno, Tangente, el número PI y la notación científica. Así como las funciones alternas del inverso del seno, inverso del coseno e inverso de la tangente.

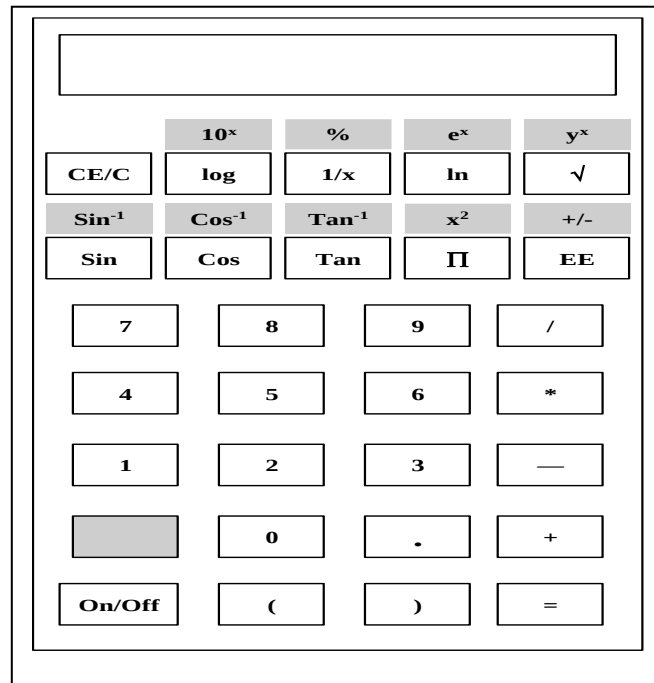


Diagrama esquemático de una calculadora

3.5 Implementación de un Analizador Léxico mediante una herramienta automatizada.

La herramienta automatizada que se va a utilizar para las prácticas de esta unidad de aprendizaje es AWK. Es un lenguaje intérprete completo de aplicación general. Dentro de las herramientas automatizadas las más versátiles son las que se conocen como "OPEN SOURCE", esto significa que se encuentra el código fuente disponible de forma gratuita. Si a esto agregamos que AWK es multiplataforma ya que se diseñó para trabajar de forma "nativa" en los ambientes UNIX.



Es muy parecido a lenguaje en C el cual permite integrar en el código expresiones regulares: **C + /ER/ = awk**

PRONTUARIO DE AWK EN EXPRESIONES REGULARES /ER/

En AWK, una expresión regular /ER/, es una de las siguientes:

- Un carácter, diferente de cualquier meta-carácter, como A, que empata consigo misma.
- La definición especial de un carácter o grupo de estas mediante meta-caracteres.
- La combinación de una o más /ER/ mediante operaciones.
- La sintaxis de las /ER/ en los patrones, comparaciones y funciones de cadena y búsqueda son:
 - 1) /ER/
 - 2) Expresión ~ /ER/
 - 3) Expresión ~/ER/
 - 4) /ER/, /ER/

METACARACTERES

\ Secuencia de escape para presentar un carácter especial.

^ Al inicio de una cadena

\$ Al final de una cadena

. Cualquier carácter

[] Clase o rango de caracteres

| Operador de alternación

() Delimitadores para el uso de operadores y expresiones

***Cerradura de Kleene**

+ Cerradura positiva

? Cerradura 0 o 1



METACARACTERES DE CLASE

Dentro de una clase, todos los caracteres tienen un significado literal (entre corchetes) excepto

- (guion intermedio) Entre dos caracteres representa un rango de caracteres
- \ (diagonal invertida) Conserva su comportamiento de meta-caracteres
- ^ (circunflejo) Al inicio, representa el complemento de la clase de caracteres o de rango indicado.

EJEMPLOS:

/^C/ Empata con cualquier cadena con una C al inicio

/C\$/ Empata con cualquier cadena con una C al final

/^C\$/ Empata con una cadena con que sólo tenga una letra C

/^.\$/ Empata con una cadena que sólo tenga carácter cualquiera

/^.../ Empata con una cadena que consiste en tres caracteres

/.../ Empata con cualquier cadena consecutivos

\/.\$/ Empata con cualesquiera consecutivos que tenga un punto final

/^c/ Empata con cualesquiera consecutivos que empieza con c

/[AEIOU]/ Empata con la cadena que tenga las letras interiores

/[ABC]/ Empata con la cadena que empieza con A, B o C.

/^[^]/ Cualquier carácter excepto el circunflejo al inicio de la cadena

/B*/ Cadena vacía, B, BB, BBB,..., ∞

/AB*C/ Genera: AC, ABC, ABBC,..., ∞

/AB+C/ Genera: ABC, ABBC, ABBBC,..., ∞



UNIDAD IV

4.1 Gramáticas Libres de Contexto.

Introducción

Las gramáticas se utilizan para describir lenguajes. Existe una primera clasificación para las gramáticas, y ésta estará en función a los tipos de lenguajes que genere, esto es; ya sean lenguajes naturales o lenguajes formales.

Un lenguaje natural es el idioma como el español o el inglés o como cualquier otro lenguaje de comunicación entre personas, en donde la estructura de las frases, se describen por medio de una gramática que agrupa las palabras en categorías sintácticas tales como sujetos, predicados, frases preposicionales, etcétera. Estas construcciones gramaticales surgen como un intento de explicar las formas admitidas por el lenguaje, con las cuales se construyen las frases, aunque en su definición se presentarán excepciones gramaticales. ($L \Rightarrow G$)

Un Lenguaje formal, por el contrario, surge a partir de su gramática, y por lo tanto no presenta excepciones en su definición. Esto es así, porque los Lenguajes formales son los que se utilizan para que se comuniquen los hombres con las máquinas. De esta manera a partir de las gramáticas formales es como surgen los lenguajes de programación. ($G \Rightarrow L$)

Ya que hemos tocado el tema de los lenguajes, hemos descrito también sus paradigmas, continuaremos viendo los lenguajes formales y su representación matemática.

4.2 Conceptos básicos relacionados con gramáticas.

LENGUAJES FORMALES

Podemos definir a un lenguaje formal, como aquel que se construye en base a reglas gramaticales, el cual tiene una sintaxis, misma que corresponde a la forma de los programas y una semántica o significado del lenguaje. Para especificar la sintaxis de un lenguaje, se presenta una notación muy usada llamada gramática independiente del contexto (GIC), también definida como Gramáticas Libres de Contexto (GLC) que son lo mismo.



La simplicidad de la definición de una GLC está “formalizada” en cuatro variables que también se representa de forma sintética en lo que se conocen como los BNF's. Esta representación gramatical se abrevia como BNF (forma Backus-Naur) o bien BNFE (forma Extendida de Backus-Naur).

Mencionaremos que una gramática independiente del contexto (GIC) tiene cuatro componentes:

- Un conjunto de símbolos terminales (T). También se identifican como variables o símbolos terminales Σ_T
- Un conjunto de símbolos no terminales (NT). También se identifican como variables o símbolos NO terminales Σ_{NT}
- Un conjunto de producciones (P, donde cada producción tiene un símbolo no terminal en su parte izquierda y un conjunto de terminales o no terminales en su parte derecha).
- Un símbolo inicial o generador de la gramática (S, no terminal).

Con las notaciones disponibles hoy en día, es mucho más difícil describir la semántica de un lenguaje que su sintaxis. Por lo que para especificar la semántica del lenguaje se usarán descripciones informales con ejemplos.

Por semántica entenderemos que es la definición del significado de un elemento del lenguaje que se puede realizar de diferentes formas. Y como sintaxis entenderemos que es la que tiene como función, especificar cómo se pueden construir los programas en ese lenguaje, permitiéndose solo el uso de determinadas combinaciones de símbolos seleccionados y palabras clave.

Lenguaje de programación.

Anteriormente se dio una definición de lo que es un lenguaje de programación, ahora es conveniente dar una definición formal que se presenta a continuación de lo que es un lenguaje de programación y esta es: Un lenguaje es un conjunto de elementos, en el entendido de que dicho conjunto tiene...

- **Una descripción**, misma que puede ser explícita o implícita. Si es explícita se tiene que para el conjunto A que representa los diez dígitos del sistema decimal tenemos: $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Ejemplo de una definición implícita para el mismo conjunto A ya definido tenemos: $A = \{0, 1, 2, \dots, 9\}$



También se puede hacer una descripción a través de predicados de donde el mismo conjunto A queda:

$$A = \{x \mid x \text{ es un dígito}\} \text{ o también } A = \{x \mid 0 \leq x \leq 9 \text{ y } x \in \mathbb{N}\}$$

Esto se lee, como el conjunto de elementos donde “x” es el elemento de A tal que “x” es un dígito. O también como el conjunto donde “x” es el elemento de A tal que está comprendido entre 0 y 9 inclusive y que además pertenece al conjunto de los números Naturales

- **Tiene propiedades**, tales como $A \subseteq B$ $B \supseteq A$

Si $A \subseteq B$ y $B \supseteq A$ entonces $A = B$

- **Tiene operaciones:**

- Unión “o” $A \cup B = \{x \mid x \in A \vee x \in B\}$
- Intersección “y” $A \cap B = \{x \mid x \in A \wedge x \in B\}$
- exclusión “-” $A - B = \{x \mid x \in A \wedge x \notin B\}$
- producto cartesiano: $A \times B = \{(x,y) \mid x \in A \wedge y \in B\}$
- exponenciación o conjunto potencia $2^A = \{x \mid x \subseteq A\}$
- Los conjuntos pueden ser indexados a sus elementos.

$$\bigcup_{i=1}^n A_i = A_1 \cup A_2 \cup A_3 \dots \cup A_n$$

- **Tiene cardinalidad**, que es la cantidad de elementos con los que consta un conjunto.

Ejemplos de operaciones:

Se tiene que $A = \{a, b\}$ y $B = \{r, s, a\}$, entonces

$$A \cup B = \{a, b, r, s\}$$

$$A \cap B = \{a\}$$

$$A - B = \{b\}$$

$$A \times B = \{(a, r), (a, s), (a, a), (b, r), (b, s), (b, a)\}$$

$$2^A = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$$



Gramáticas. Una gramática es la descripción de forma natural de la estructura jerárquica de muchas construcciones de los lenguajes de programación. Los componentes que tiene son:

1. Componentes léxicos terminales.

Son todas aquellas cadenas o conjuntos de caracteres que conforman a los elementos terminales, se ubican del lado izquierdo de las reglas de producción.

2. Componentes léxicos no terminales.

Pueden ser:

a) Lo que se conoce como instrucciones, palabras clave o palabras reservadas que se aceptan con cadenas o grupos de caracteres, mismos que conforman las palabras que las identifican con un grupo de parámetros en el orden determinado por la sintaxis del lenguaje.

b) También pueden ser identificadores o ciertos parámetros y elementos que se verificarán como elementos léxicos ubicados en la nomenclatura BNF como el lado derecho de la producción. Estas son todas aquellas cadenas (separadas por espacios) los cuales necesitan una definición detallada posterior en los elementos terminales mismas que se ubican en el lado izquierdo de las reglas de producción.

3. Reglas de producción.

Consta de tres elementos, el primero es el elemento léxico no terminal (lado izquierdo) la simbología que representa lo que produce cada elemento no terminal y el conjunto de elementos no terminales (lado derecho).

4. La denominación de uno de los no terminales.

Para identificar los cuatro elementos anteriores tenemos el siguiente ejemplo:

Mueve A, B $\rightarrow$ Esto significa que el contenido de la dirección A será asignado al contenido de la dirección B.

Para representar la gramática de esta única instrucción de: “Mueve” entre las variables “A” y “B” considerados como identificadores o direcciones tenemos la definición gramatical siguiente:

$\langle \text{Mueve} \rangle ::= \langle \text{Mueve} \rangle \langle \text{literal} \rangle, \langle \text{literal} \rangle \dots (1)$

$\langle \text{Mueve} \rangle ::= \langle \text{Mueve} \rangle \langle \text{mueve} \rangle \langle \text{MUEVE} \rangle \dots (2)$

$\langle \text{literal} \rangle ::= \{A, B, C, \dots, Z \mid a, b, c, \dots, z\} \dots (3)$

Según esta breve notación BNF, la primera regla de producción (1) significa que en la construcción gramatical de la instrucción “MUEVE” se acepta la palabra reservada seguida de una literal, una como requerida y otra literal. La regla de producción (2) indica que se aceptan las cadenas de caracteres “Mueve” (iniciando con una mayúscula y luego minúscula), “mueve” (solo con minúsculas) y “MUEVE” (solo con mayúsculas) si se introduce cualquier otra cadena, el lenguaje la rechazará.

La regla de producción (3) indica que como identificadores de literal se acepta de forma implícita cualquier carácter alfabético de la A a la Z tanto con mayúsculas como con minúsculas. En las tres reglas de producción se tiene un lado derecho del símbolo: = y un lado izquierdo.

El lado derecho representa el primer elemento no terminal, mismo que se tiene que escribir con detalle en las siguientes reglas de producción. Este primer elemento léxico no terminal tendrá su descripción detallada en las siguientes reglas de producción al lado derecho hasta llegar al nivel terminal de dígitos o caracteres o símbolos que integran las cadenas no terminales.

- **Jerarquía de Chomsky**



Noam Chomsky es profesor de lingüística y activista político estadounidense, nacido el 7 de diciembre de 1928 en Filadelfia, Pennsylvania, en 1955 se doctoró en la universidad de Pennsylvania y fue profesor e investigador del MIT (*Massachusetts Institute of Technology*)

Se le considera fundador de la gramática generativa transformacional, que es un sistema de análisis del lenguaje y que ha revolucionado la lingüística. Chomsky cree que el lenguaje es consecuencia de una facultad humana innata y que, por lo tanto, la finalidad de la lingüística consiste en determinar qué propiedades universales existen y también en establecer la "gramática universal" que pudiera explicar el amplio espectro que abarca todas las lenguas humanas posibles.



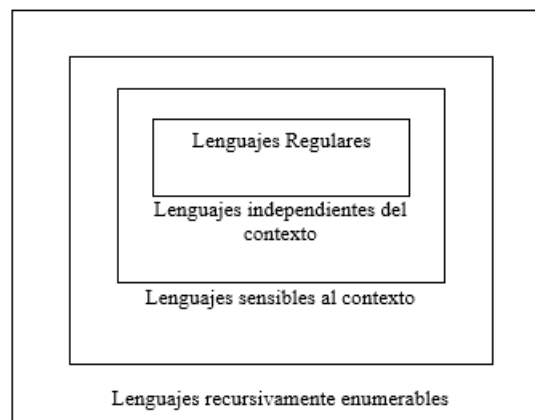
Chomsky con sus trabajos contribuyó en la sistematización de la sintaxis de los lenguajes de programación y métodos de análisis sintáctico, Chomsky hizo una clasificación jerárquica de distintos tipos de gramáticas formales que generan lenguajes formales.

Tipo	Lenguaje	Autómata
0	Recursivamente enumerables (LRE)	Máquina de Turing (MT)
1	Dependiente del contexto (LSC)	Autómata linealmente acotado
2	Independiente del contexto (LLC)	Autómata de pila
3	Regular (RL)	Autómata finito

Clasificación de gramáticas formales según Noam Chomsky

1. Lenguajes recursivamente enumerables (de tipo 0). - No existen reglas compresoras, salvo, opcionalmente, la que deriva el axioma a la palabra vacía.
2. Lenguajes dependientes del contexto (sensibles al contexto, de tipo 1). - Existen reglas en las que un símbolo no terminal puede derivar a formas sentenciales distintas, según los símbolos que aparezcan a su alrededor
3. Lenguajes Independientes del contexto (de contexto libre, de tipo 2). - La mayoría de los lenguajes de programación entran en esta categoría.
4. Lenguajes Regulares (de tipo 3). - Se pueden expresar también mediante expresiones regulares.

Teorema de la Jerarquía: Sobre un alfabeto dado, el conjunto de los *lenguajes recursivamente enumerables* (tipo 0) contienen propiamente al conjunto de los *lenguajes sensibles al contexto* (tipo 1), que contiene propiamente al conjunto de los *lenguajes independientes del contexto* (tipo 2), que a su vez contienen propiamente a los lenguajes regulares (tipo 3).





Especialmente resaltaremos que en el año de 1956 Chomsky introdujo las gramáticas independientes del contexto como parte de un estudio sobre los lenguajes naturales. Su utilización para especificar la sintaxis de los lenguajes de programación surgió independientemente, mientras trabajaba en el borrador de ALGOL 60, John Backus “adaptó de inmediato (las producciones de Emil Post) a ese uso”. La notación resultante fue una variante de las gramáticas independientes del contexto.

Lenguajes naturales

- Concepto

Podemos definir como lenguajes naturales a todos aquellos que utilizan los humanos para comunicarse, tales como el chino, Japonés, Español, Inglés, Francés, Italiano, Portugués, etc. Estos tienen reglas gramaticales regidas por la sintaxis y la semántica de cada uno, ésta siempre ligada a un contexto. Por su gran cantidad de lenguajes naturales, expresiones idiomáticas, y excepciones a las reglas antes mencionadas, se tiene gran complejidad, misma que es motivo de estudio de la inteligencia artificial.

- Ambigüedades

En el manejo de cualquier lenguaje natural se tiene este problema, la ambigüedad significa que de una palabra se pueden tener distintas interpretaciones y su significado queda incierto, dudoso y poco claro, como un ejemplo podemos decir de la palabra “papa”. En los lenguajes formales, la ambigüedad surge cuando dos árboles de derivación uno por la izquierda y otro por la derecha genera la misma cadena.

- COMUNICACIÓN HOMBRE MÁQUINA

Por todo lo que hemos visto, podemos decir que la comunicación hombre máquina se logra creando los lenguajes de programación, los cuales son un tipo muy especial de software que permite al hombre dar instrucciones a la máquina de una forma factible, estructurada, ordenada, siguiendo las reglas gramaticales que el mismo hombre diseña para su fácil utilización y para lograr resolver cualquier tipo de programas. Gramáticas para describir lenguajes naturales.



Para poder describir a los lenguajes naturales, podemos escribir reglas como las que enumeramos en el siguiente conjunto.

```
Pn = {<oración> → <sujeto> <predicado>,
      <sujeto> → <sustantivo> | <artículo> <sustantivo> | <artículo> <sustantivo>
      <adjetivo>,
      <predicado> → <verbo> | <verbo> <modificador>,
      <artículo> → el | la | los | las | un,
      <sustantivo> → niño | madre | árbol | vehículos,
      <adjetivo> → pequeño | alto | joven,
      <verbo> → aprende | corre | juega | crece,
      <modificador> → velozmente | risueño | temprano}
```

Vale la pena destacar que esta es “una versión muy simplificada” de la gramática castellana, y si bien da lugar a oraciones que son correctas en nuestro idioma, también puede generar oraciones que resulten incorrectas desde el punto de vista tanto semántico (de su significado), como sintáctico (de su estructura).

Este es el caso de:

1. El árbol juega (incorrecta semánticamente)
2. La madre come risueño (incorrecta sintácticamente)

Estos errores se solucionan en la lengua castellana agregando reglas que regulan y restringen las combinaciones de terminales y que corresponden al análisis morfológico y semántico del lenguaje. (En los lenguajes formales no existe el análisis morfológico que es el de género, número y persona, es decir el que controla la concordancia entre sujeto y verbo o la concordancia entre artículo, sustantivo y adjetivo)

Gramáticas formales

Estas gramáticas permitirán en forma intencional describir un determinado lenguaje; esto se hará definiendo el alfabeto sobre el que se construirán sus palabras, denominadas símbolos terminales; un símbolo inicial del que se partirá para la obtención de cualquier de las palabras del lenguaje llamado axioma inicial, cabecera de producción o un conjunto de símbolos especiales denominados no terminales, los que permitirán expresar representaciones o estados intermedios en el proceso de generación de las palabras del lenguaje; y un conjunto de reglas de producción o de reescritura, que serán las que permitan realizar las transformaciones necesarias, partiendo desde el axioma inicial, produciendo los reemplazos de símbolos no terminales, mediante la utilización de las reglas de producción hasta obtener las palabras del lenguaje.



Definición

Chomsky: $G_3 \subset G_2 \subset G_1 \subset G_0$

Una gramática formal **G**, es una 4-tupla, que queda definida de la siguiente manera:

$$G = (\Sigma_T, \Sigma_N, S, P)$$

En donde:

Σ_T Conjunto de símbolos que representan el alfabeto de símbolos terminales, en donde toda palabra del lenguaje generado por esta gramática estará formada por símbolos o caracteres definidos en este conjunto.

Σ_N Conjunto de símbolos que representan el alfabeto de símbolos NO terminales. Este conjunto de símbolos será utilizado como símbolos auxiliares en la derivación de cadenas, pero no formarán parte de las cadenas del lenguaje.

De las definiciones anteriores se observa lo siguiente:

$$\Sigma = \Sigma_T \cup \Sigma_N \quad \text{y} \quad \Sigma_T \cap \Sigma_N = \Phi$$

S: Símbolo NO terminal especial, que pertenece al conjunto de símbolos NO terminales ($S \in \Sigma_N$), denominado símbolo inicial o axioma de la gramática.

P: Conjunto finito de reglas o producciones que tienen como única restricción que en la parte izquierda debe haber al menos un símbolo no terminal.

Ejemplo: dada la gramática

$$G_1 = (\{0, 1\}, \{A, B\}, A, P)$$

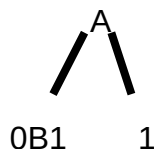
Donde

$$P = \{ (A := 0B1), (A := 1), (B := 0A0), (B := 01), (B := 0AB) \}$$

También podríamos escribirlas:

$$A := 0B1|1$$

$$B := 0A0|01|0AB$$



Producciones

Una producción, o regla de producción es un par ordenado de palabras (x, y), con $x \in \Sigma^*$, donde la presencia de x se encuentra como parte integrante de cualquier otra palabra, puede ser sustituida por y ; lo que permite transformar palabras en otras.

Como notación suele utilizarse $x := y$, denominada notación BNF (por *Backus - Naur Form*, por sus creadores).

La palabra x es denominada la parte izquierda o primer miembro de la producción; y la palabra y , la parte derecha o segundo miembro.

Simbología: $=$ o bien $:=$ también $\rightarrow$ o simplemente un guion intermedio.

Serán producciones el conjunto P de la gramática de ejemplo G_1 :

$\{ (A := 0B1), (A := 1), (B := 0A0), (B := 01), (B := 0AB) \}$

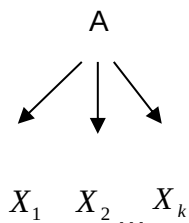
4.1.2 Árboles de derivación y derivación por la derecha e izquierda.

Árboles de Derivación

Es un Árbol formado a partir de la derivación de una gramática. Sirve para estudiar la ambigüedad (Es la representación de una misma palabra o cadena con más de un árbol de derivación por la izquierda y derecha, por tanto, tienen dos interpretaciones)

Características:

- Cada nodo interior es variable.
- Cada nodo hoja es Terminal o épsilon ϵ
- Si existe una producción $A \mapsto X_1 X_2 \dots X_k$, El árbol de derivación es el siguiente:



- Para que el X_i sea ϵ debe existir la producción $X_i \mapsto \epsilon$

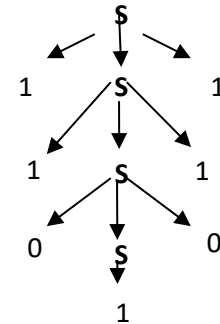


La cadena resultado del árbol: las hojas del árbol concatenadas de izquierda a derecha, recorrido en preorden.

Por ejemplo, en la GLC anterior (binario palíndromo), donde

$G = (\{S\}, \{0,1\}, P, S)$, y $P = \{ S \rightarrow \epsilon \mid 0 \mid 1 \mid 1S1 \mid 0S0 \}$

La derivación anterior de la cadena 1101011, se obtiene el siguiente árbol de derivación:



La derivación:

$S \Rightarrow^4 1S1 \Rightarrow^4 11S11 \Rightarrow^2 110S011 \Rightarrow^3 1101011$

Ejercicios:

1. $L = \{0^n 1^n, n \geq 1\}$
 $S \rightarrow 01 \mid 0S1$
2. $L = \{0^n 1^n, n \geq 0\}$
 $S \rightarrow \epsilon \mid 01 \mid 0S1$
3. $L = \{0^n 1^m, n \geq m\}$
 $S \rightarrow 0 \mid 0S \mid 0S1$

Derivación directa

Es la aplicación directa de una producción ($x := y$), a una determinada palabra v para convertirla en w .

Simbología: $\rightarrow$

Ejemplo: dado $v = z.x.u$; y aplicando ($x := y$) obtenemos $w = z.y.u$

Con $v, w, z, u \in \Sigma^*$.

Para el caso de la gramática G_1 , podemos, partiendo de una palabra 0B1, obtener, aplicando las reglas de producción ($B := 0A0$) lo siguiente: $0B1 \rightarrow 00A01$

Derivación

Es la aplicación de una secuencia de producciones a una palabra.

Simbología: $\rightarrow^*$



Ejemplo: En el ejercicio anterior continuaremos aplicando reglas de producción y pasaremos de una palabra 0B1 a una palabra 0000101

$0B1 \rightarrow 00A01 \rightarrow 000B101 \rightarrow 00001101$

Descripción: En 0B1 se reemplaza la B por 0A0 y queda $0(0A0)1 := 00A01$

En 00A01 se reemplaza la A por 0B1 y queda $00(0B1)01 := 000B101$

y finalmente En 000B101 se reemplaza la B por 01 y queda:

$000B101 := 000(01)101$

Según reglas de producción:

$\{ (A := 0B1), (A := 1), (B := 0A0), (B := 01), (B := 0AB) \}$

4.1.3 Recursividad por la derecha e izquierda.

Derivaciones utilizando una gramática

Aplicamos las producciones de un GIC para inferir que determinadas cadenas pertenecen al lenguaje de una cierta variable. Para llevar a cabo esta inferencia hay disponibles dos métodos. El más convencional de ellos consiste en emplear las reglas para pasar del cuerpo a la cabeza.

Procedimiento de interferencia cursiva

- Tomamos cadenas que sabemos que pertenecen al lenguaje de cada una de las variables del cuerpo.
- Concatenar en el orden apropiado con cualquier símbolo terminal.
- Inferir que la cadena resultante pertenece al lenguaje de la variable de la cabeza.

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

Imaginemos una gramática con estas dos reglas:

1. $A \rightarrow bA$
2. $A \rightarrow c$

El elemento en mayúsculas es el *símbolo inicial*. Los elementos en minúsculas son los *símbolos terminales*. Para generar cadenas de caracteres, la idea es sustituir el símbolo inicial de la izquierda por los símbolos de la derecha, y luego repetir el proceso hasta que sólo haya símbolos terminales. Por ejemplo:

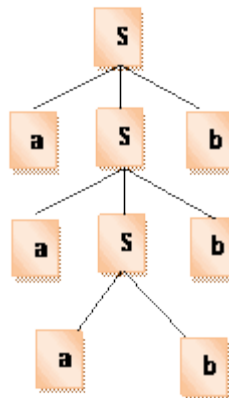
$$A \rightarrow bA \rightarrow bbA \rightarrow bbbA \rightarrow bbbbc$$

Esta gramática da lugar a un lenguaje formal que consiste en el conjunto de todas las cadenas de caracteres que pueden ser generadas por medio ellas. Por ejemplo: bbbbc, bbbbbbbbc, c, bc, etc.

Ejemplo de un árbol de derivación.

Sea $G = (N, T, S, P)$ una GLC con $P: S \rightarrow ab|aSb$

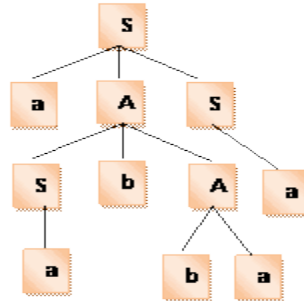
La derivación de la cadena: aaabbb será: $S \rightarrow aSb \rightarrow aaSbb \rightarrow aaabbb$ y el árbol de derivación se muestra en el siguiente árbol de derivación



La palabra está formada por los nodos hojas tomados de izquierda a derecha y de arriba para abajo. Relación entre derivaciones y árboles. Si leemos las etiquetas de las hojas de izquierda a derecha tenemos una sentencia. Llamamos a esta cadena la producción del árbol de derivación.

Teorema. Sea $G=(N, T, S, P)$ una GLC. Entonces $S \rightarrow^* \alpha$ (de S se deriva α) si y sólo si hay un árbol de derivación en la gramática G con la producción α .

Si w es una cadena de $L(G)$ para la gramática libre de contexto G , entonces w tiene al menos un árbol de derivación. Referido a un árbol de derivación particular, w tendrá una única derivación a la izquierda y otra única a la derecha. El ejemplo se muestra en la derivación a la izquierda:



$S \rightarrow Aas \rightarrow aSbAS \rightarrow aabbaS \rightarrow aabbbaa$

Aquí la derivación se toma en la primera cadena aAS , el nodo hoja de la izquierda con los dos nodos intermedios no terminales (AS , con mayúscula). La segunda cadena: $aSbAS$, es el primer nodo hoja de la izquierda con el nodo intermedio unido al nodo intermedio de A que es la S y su nodo hoja también de A , para terminar con el tercer nodo de A que también es una A y el último nodo intermedio S . La tercera cadena $aabAS$ toma todos los nodos hojas desde S (raíz) y deja los dos últimos nodos intermedios de la derecha que es la AS . La cuarta cadena, $aabbaS$, toma todos los nodos hojas desde S (raíz) y solo deja el último nodo intermedio de la derecha que es la misma S . Por último, la quinta cadena, toma todos los nodos hojas desde S (raíz) y queda $aabbbaa$.

Conclusión derivación de izquierda a derecha, derivando nodos intermedios generadores de nodos hojas, desde el nodo raíz a las hojas.

Derivación a la derecha: $S \rightarrow aAS \rightarrow aAa \rightarrow aSbAa \rightarrow aSbbbaa \rightarrow aabbbaa$

Aquí la derivación se toma en la primera cadena aAS , al igual que en la derivación por la izquierda.

La segunda cadena: $aSbAS$, es el primer nodo hoja de la izquierda con el nodo intermedio unido al nodo intermedio de A que es la S y su nodo hoja también de A , para terminar con el tercer nodo de A que también es una A y el último nodo intermedio S .



La tercera cadena aabAS toma todos los nodos hojas desde S (raíz) y deja los dos últimos nodos intermedios de la derecha que es la AS.

La cuarta cadena, aabbaS, toma todos los nodos hojas desde S (raíz) y solo deja el último nodo intermedio de la derecha que es la misma S.

Por último, la quinta cadena, toma todos los nodos hojas desde S (raíz) y queda aabbbaa.

Conclusión derivación de izquierda a derecha, derivando nodos intermedios generadores de nodos hojas, desde el nodo raíz a las hojas.

Recursividad por la derecha e izquierda

La recursividad es la característica de una gramática formal, la cual puede ser recursiva o repetitiva cuando existe una producción que se incluye a sí misma dentro de las reglas de producción de la gramática. Ejemplo, $S \rightarrow aS$ (Aquí la regla inicial de S también produce la cadena “a” de longitud 1 y la S que al repetirse a sí misma se convierte en infinita).

La recursividad tiene como característica principal la sensación de infinito, de algo que es continuo y que por tanto no puede ser delimitado en el espacio o el tiempo porque se sigue replicando y multiplicando de manera lógica y matemática. Así, es común encontrar casos de recursividad por ejemplo en imágenes de espejos que hacen que la imagen sea replicada al infinito, una dentro de otra hasta que deja de verse pero no por eso deja de existir.

Otro caso típico de recursividad en las imágenes es cuando encontramos una publicidad en la que el objeto tiene la propaganda de sí mismo en su etiqueta y así al infinito, o cuando una persona está sosteniendo una caja de un producto en cuya etiqueta aparece esa misma persona sosteniendo el mismo producto y así hasta el infinito. En estos casos, la recursividad pasa por el hecho de que se busca definir algo con la misma información que ya se tiene.

En la definición de una gramática recursiva (repetitiva hasta el infinito), se presentan algunos problemas en la derivación de palabras que sea de fácil derivación o generación. Es por esto que se tiene la necesidad de eliminar la recursividad por la izquierda o por la derecha para tener palabras definidas en el alfabeto.



Una gramática G se llama recursiva en A , $A \in \Sigma_N$

$$A \rightarrow^+ xAy$$

– Si x es la palabra vacía, se dice que la gramática es recursiva a la izquierda

$$A \rightarrow^+ Ay$$

– Si y es la palabra vacía, se dice que la gramática es recursiva a la derecha

$$A \rightarrow^+ xA$$

• Se dice que una producción es recursiva si

$$A ::= xAy$$

– La producción es recursiva a izquierdas si $x = \epsilon$ (cadena o palabra vacía)

$$A ::= Ay$$

– Será recursiva a derechas si $y = \epsilon$ (cadena o palabra vacía)

$$A ::= xA$$

• Si un lenguaje es infinito, la gramática que lo representa ha de ser recursiva.

Eliminación de la recursividad por la izquierda en producciones de un mismo símbolo no terminal.

$$\forall A \in \Sigma_N$$

Si $P1 = (A ::= A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m)$ entonces //crear un símbolo nuevo

$$A'_{\Sigma_N} = \Sigma_N \cup \{A'\};$$

$$P = (P - P1) \cup \{A ::= \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'; A' ::= \alpha\}$$

Donde $\beta_1 A' \mid \alpha_i$ no comienza por A_2

$$A' \mid \alpha$$

$$A' \mid \epsilon$$

fsi

f $\forall$



Eliminación de la recursividad por la izquierda en más de un paso

1. Disponer los Σ_N en algún orden $A_1, A_2, \dots, A_n$
2. Para $i:=1$ hasta n

Para $j:=1$ hasta n

Si $i \neq j$ entonces

Reemplazar cada producción $A_i ::= A_{j\theta}$ por:

$$A_i ::= \gamma_{1\theta} \mid \gamma_{2\theta} \mid \dots \mid \gamma_{k\theta}$$

Donde $A_j ::= \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_{k_{ij}}$ son todas las reglas de A

fsi

Eliminar la recursividad por la izquierda de las A

fpara

fpara

$G = (\{i, +, *, (), \}, \{E, T\}, E, P = \{E ::= T + E \mid T * E \mid i ; T ::= E(E)\})$

1. $A_1 = E; A_2 = T$

2. Bucles:

$i=1 (A_1=E); j=1 (A_1=E)$. Se reemplazarían $E ::= E_\alpha$. No hay.

$i=1 (A_1=E); j=2 (A_2=T)$. Se reemplazan $E ::= T_\alpha$. El nuevo P' es

$E ::= E + E \mid E * E \mid (E) + E \mid (E) * E \mid i$

$T ::= E \mid (E)$

• Se elimina la recursión en E quedando P'' :

$E ::= (E) + EE' \mid (E) * EE' \mid i E'$

$E' ::= +EE' \mid *EE' \mid \epsilon$

$T ::= E \mid (E)$

$i=2 (A_2=T); j=1 (A_1=E)$. Se reemplazan $T ::= E_\alpha$.

El nuevo P''' es



$E ::= (E)+EE' \mid (E)*EE' \mid i E'$

$E' ::= +EE' \mid *EE' \mid \lambda$

$T ::= (E)+EE' \mid (E)*EE' \mid i E' \mid (E)$

- Se eliminaría la recursión en T si la hubiera.

$i=2 (A_2=T); j=2 (A_2=T)$. Se reemplazarían $T::=T_\alpha$.

No hay el conjunto final de producciones es P'''

Ejemplo de Gramáticas Regulares o de estado finito TIPO 3.

Sus producciones son de la forma:

Lineal por la derecha: $A \rightarrow aB$ o $A \rightarrow a$, donde $A, B \in \Sigma_N, a \in \Sigma_T$

Lineal por la izquierda: $A \rightarrow Ba$ o $A \rightarrow a$, donde $A, B \in \Sigma_N, a \in \Sigma_T$

Se permiten producciones de la forma $S \rightarrow \varepsilon$

Los lenguajes representados por este tipo de gramáticas se denominan lenguajes regulares.

Ejemplos:

$G1 = (\{0, 1\}, \{A, B\}, A, \{A ::= B1 \mid 1, B ::= A0\})$

Gramática lineal por la izquierda que describe el lenguaje

$L1 = \{1, 101, 10101, \dots\} = \{1(01)^n \mid n = 0, 1, 2, \dots\}$

$G2 = (\{0, 1\}, \{A, B\}, A, \{A ::= 1B \mid 1, B ::= 0A\})$

Gramática lineal derecha que genera el mismo lenguaje que la gramática anterior.
Para cada Gramática lineal por la derecha existe una Gramática lineal izquierda que genera el mismo lenguaje y viceversa.



Algoritmo:

1. Se transforma la gramática de forma que no haya ninguna regla en cuya parte derecha esté el axioma. Para lo cual:
 - 1.1. Se crea un nuevo $S' \in \Sigma_N$
 - 1.2. $\forall$ regla $S ::= x$ (con S axioma y $x \in \Sigma^*$) se crea una nueva regla $S' ::= x$.
 - 1.3. Cada regla $A ::= xSy$, se transforma en $A ::= xS'y$.
2. Se crea un grafo G dirigido:
 - 2.1. $\forall A \in \Sigma_N \cup \{\epsilon\}$ se crea un nodo.
 - 2.2. $\forall$ producción $(A ::= aB) \in P$ se crea un arco etiquetado con a que va del nodo A al B .
 - 2.3. $\forall$ producción $(A ::= a) \in P$ se crea un arco etiquetado con a que va del nodo A al $\lambda(F)$.
 - 2.4. Si $\exists S ::= \lambda$ se crea un arco sin etiqueta que va del nodo del axioma al nodo λ .
3. Se crea otro grafo G' a partir de G :
 - 3.1. Se intercambian las etiquetas del axioma y λ .
 - 3.2. Se invierte la dirección de todos los arcos.
4. Se transforma en un conjunto de reglas:
 - 4.1. $\forall$ nodo, se crea un símbolo no terminal excepto para el nodo ϵ .
 - 4.2. $\forall$ arco etiquetado con a que va del nodo A al nodo $B \in \Sigma_N \cup \{\epsilon\}$, se crea una producción $A ::= Ba$.
 - 4.3. Si $\exists$ un arco del nodo del axioma al nodo λ se crea una regla $S ::= \epsilon$

El algoritmo para el paso inverso es similar al descrito.

Ejemplo: Gramática lineal derecha:

$$G1 = (\{0, 1\}, \{A, B\}, A, \{A ::= 1B \mid \epsilon, B ::= 0A, B ::= 0\})$$

1. Se transforma la gramática para que no haya regla como: $B ::= 0A$
 - 1) Se crea un nuevo A'
 - 2) Se crean las reglas $A' ::= 1B \mid \epsilon$
 - 3) Se crea la regla $B ::= 0A'$
 - 4) Se borra la regla $B ::= 0A$

La Gramática resultante es:

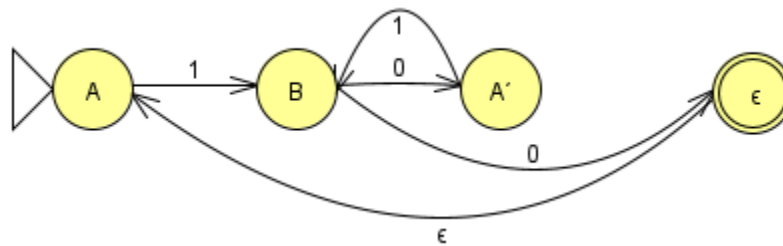
$$G_{11} = (\{0, 1\}, \{A, B, A'\}, A, \{A ::= IB \mid \epsilon, A' ::= IB \mid \epsilon, B ::= 0A', B ::= 0\})$$

Se elimina $A' ::= \epsilon$ quedando:

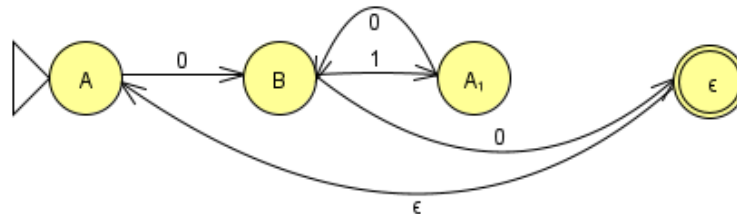
$$G_{12} = (\{0, 1\}, \{A, B, A'\}, A, \{A ::= IB \mid \epsilon, A' ::= IB, B ::= 0A', B ::= 0\})$$

$$G_{12} = (\{0, 1\}, \{A, B, A'\}, A, \{A ::= IB \mid \epsilon, A' ::= IB, B ::= 0A', B ::= 0\})$$

1. Se crea un grafo G (Autómata Finito NO determinístico con épsilon transiciones) que representa las transiciones de la gramática según se muestra a continuación



2. Se crea el grafo G' invirtiendo G según se muestra



3. Se transforma en un conjunto de reglas, en la Gramática lineal por la izquierda equivalente:

$$G_2 = (\{0, 1\}, \{A, B, A'\}, A, \{A ::= B0 \mid \epsilon, A' ::= B0, B ::= A'1, B ::= 1\})$$

Para cada Gramática lineal por la izquierda existe una Gramática lineal derecha que genera el mismo lenguaje y viceversa.

Algoritmo:

1. Se transforma la gramática de forma que no haya ninguna regla en cuya parte derecha esté el axioma. Para lo cual:
 - 1.1. Se crea un nuevo $S' \in \Sigma_N$
 - 1.2. $\forall$ regla $S ::= x$ (con S axioma y $x \in \Sigma^*$) se crea una nueva regla $S' ::= x$.
 - 1.3. Cada regla $A ::= xSy$, se transforma en $A ::= xS'y$.
2. Se crea un grafo G dirigido:
 - 2.1. $\forall A \in \Sigma_N \cup \{F\}$ se crea un nodo.



- 2.2. $\forall$ producción $(A ::= Ba) \in P$ se crea un arco etiquetado con a que va del nodo A al B .
- 2.3. $\forall$ producción $(A ::= a) \in P$ se crea un arco etiquetado con a que va del nodo A al F .
- 2.4. Si $\exists S ::= \varepsilon$ se crea un arco con etiqueta ε que va del nodo del axioma al nodo F .
3. Se crea otro grafo G' a partir de G :
 - 3.1. Se intercambian las etiquetas del axioma y F .
 - 3.2. Se invierte la dirección de todos los arcos.
4. Se transforma en un conjunto de reglas:
 - 4.1. $\forall$ nodo, se crea un símbolo no terminal excepto para el nodo F .
 - 4.2. $\forall$ arco etiquetado con a que va del nodo A al nodo $B \in \Sigma_N \cup \{F\}$, se crea una producción $A ::= aB$.
 - 4.3. Si $\exists$ un arco etiquetado con λ del nodo del axioma al nodo F se crea una regla $S ::= \varepsilon$

Ejemplo: Gramática lineal izquierda:

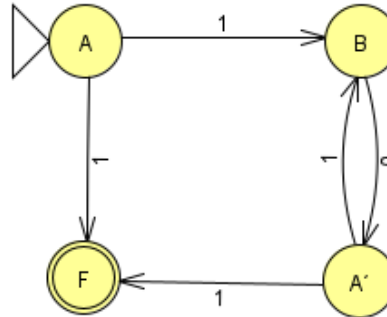
$$G_I = (\{0, 1\}, \{A, B\}, A, \{A ::= B1 \mid 1, B ::= A0\})$$

1. Se transforma la gramática para que no haya regla como: $B ::= A0$
 - 1.1. Se crea un nuevo A'
 - 1.2. Se crean las reglas $A' ::= B1 \mid 1$
 - 1.3. Se crea la regla $B ::= 0A'0$
 - 1.4. Se borra la regla $B ::= A0$

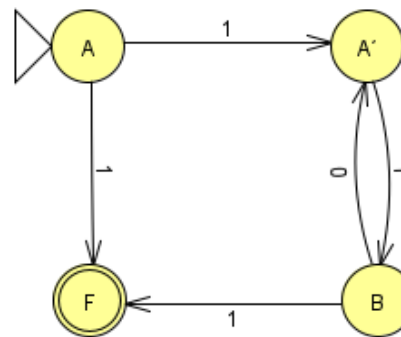
La Gramática resultante es:

$$G_{II} = (\{0, 1\}, \{A, B, A'\}, A, \{A ::= B1 \mid 1, A' ::= B1 \mid 1, B ::= A'0\})$$

2. Grafo G para las transiciones de la gramática, según se muestra a continuación:



3. Se crea el grafo G' invirtiendo y se muestra a continuación:



4. Se transforma en las reglas, de la Gramática lineal por la derecha equivalente:

$$G2 = (\{0, 1\}, \{A, B, A'\}, A, \{A ::= 1A' \mid 1, A' ::= 0B, B ::= 1A' \mid 1\})$$

Gramática regular asociada a un AFD

Si L es aceptado por un AFD entonces L puede generarse mediante una gramática regular.

Sea $M = (Q, \Sigma, f, q_0, F)$, la gramática regular equivalente es aquella definida como $G = (Q, \Sigma, P, q_0)$, donde P viene dado por:

Si $f(q, a) = p$ entonces añadir a P la producción $q \rightarrow ap$.

Si $f(q, a) = p$ y $p \in F$ entonces añadir a P la producción $q \rightarrow a$.

Si $q_0 \in F$ entonces añadir a P la producción $q_0 \rightarrow \epsilon$

Ejemplo: De AFD a Gramática regular:

Dado un lenguaje regular L reconocido por un **AFD** $M=(Q,\Sigma,f,q_0,F)$, se puede obtener una gramática regular $G=(\Sigma_N,\Sigma_T,S,P)$, que genere el mismo lenguaje de la siguiente forma:

$$\Sigma_N = Q$$

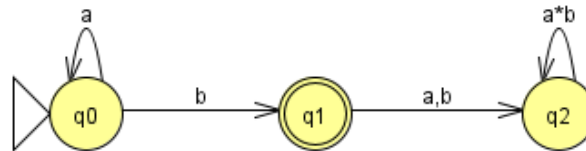
$$\Sigma_T = \Sigma$$

$$S = q_0$$

$$P=\{(q,ap)|f(q,a)=p\}\cup\{(q_0, \epsilon)|p,q_0 \in F\}$$

Ejemplo. - Considera el siguiente **DFA** que acepta el lenguaje a^*b

El autómata se muestra a continuación:



La gramática será:

$$q_0 \rightarrow aq_0|bq_1|b$$

$$q_1 \rightarrow aq_2|bq_2$$

$$q_2 \rightarrow aq_2|bq_2$$

Autómatas finitos y gramáticas regulares

AFD asociado a una gramática regular

Si L es un lenguaje generado por una gramática regular, entonces existe un **AFD** que lo reconoce:

Sea la gramática regular $G=(\Sigma_N, \Sigma_T, P, S)$, se construye el **AFND- ϵ** equivalente como:

$M=(\Sigma_N \cup \{F\}, \Sigma_T, f, S, \{F\})$, donde f viene dado por:

Si $A \rightarrow aB$ entonces $f(A,a)=B$

Si $A \rightarrow a$ entonces $f(A, a) = F$

Si $S \rightarrow \varepsilon$ entonces $f(S, \varepsilon) = F$

F es un “nuevo” símbolo no terminal.

Convertir el $AFND-\varepsilon$ obtenido a un AFD .

Ejemplo: De Gramática regular a AFN :

Dado un lenguaje regular L generado por una gramática regular $G = (\Sigma_N, \Sigma_T, S, P)$, se puede obtener un $AFD M = (Q, \Sigma, f, q_0, F)$ que reconozca el mismo lenguaje de la siguiente forma:

$$Q = \Sigma_N \cup \{F\},$$

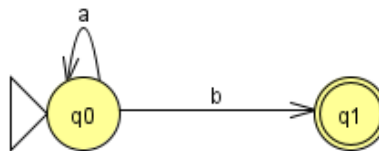
$$\Sigma = \Sigma_T$$

$$q_0 = S$$

Ejemplo. - Considera la siguiente gramática:

$S \rightarrow aS \mid b$ que acepta el lenguaje a^*b :

El AFD equivalente se muestra en la siguiente figura



Las expresiones regulares son una notación especial que se utiliza habitualmente para describir los lenguajes de tipo regular. La notación más utilizada para especificar patrones son las expresiones regulares, sirven como nombres para conjuntos de cadenas. Los usos más habituales de las expresiones regulares son en la creación de analizadores léxicos para compiladores, en la búsqueda de patrones dentro de los editores de texto, en la descripción de redes neuronales y circuitos electrónicos, etcétera.

En las expresiones regulares pueden aparecer símbolos de dos tipos:

Símbolos base: son los del alfabeto del lenguaje que queremos describir, λ para describir la palabra vacía y $\emptyset$ para describir un lenguaje vacío. Símbolos de operadores: “+” o unión, o concatenación y “*” para la clausura.



En ocasiones se utiliza para la unión el símbolo “|” y el punto se omite en la concatenación de expresiones regulares. El lenguaje de las expresiones regulares se construye de forma inductiva a partir de símbolos para expresiones regulares elementales y operadores.

4.2 Obtener la forma positiva de una gramática.

Obtener la forma positiva de una gramática.

“Una gramática es positiva si ninguna de sus producciones es de la forma $A \rightarrow \varepsilon$; a estas producciones se les conoce como ε -producciones.

Se demostrará que, si G es una Gramática Libre de Contexto (GLC), existe una gramática positiva G' tal que $L(G') = L(G) - \{\varepsilon\}$. Para esto, se define el núcleo de una GLC G , escrito como Kernel de G o $\text{Ker}(G)$, como el conjunto de variables A tales que $A \Rightarrow \varepsilon$. El siguiente lema proporciona un algoritmo para determinar $\text{ker}(G)$.

Lema 4.1. Sea G una GLC. Defínase, recursivamente, los conjuntos VN_i (i Variables No terminales) como sigue:

- i) $VN_0 = \{A | A \rightarrow \varepsilon \text{ es una producción de } G\}$ (Desde la Variable No terminal cero VN_0 que es la primera, A representa otra Variable No terminal o VN o solo V que produce una variable terminal que produce la cadena vacía ε)
- ii) $VN_{i+1} = VN_i \cup \{A | A \rightarrow \infty \text{ es una producción de } G, \infty \in VN_i^*\}$ (Todas y cada una de las Variables No terminales que se unen a las variables A que producen la cadena vacía hasta el infinito)

Entonces, $\text{ker}(G) = VN_k$, si $VN_{k+1} = VN_k^*$

Demostración

Es claro que, por definición, $VN_i \subseteq \text{Ker}(G)$ para toda i . Recíprocamente, se demostrará que, si $A \in \text{Ker}(G)$, entonces $A \in VN_k$. Se hará esta demostración por inducción en el número de pasos en una derivación de ε en G .

Base ($j=1$). En este caso, $A \Rightarrow \varepsilon$, así que $A \rightarrow \varepsilon$ es una producción de G y $A \in VN_0$.

Paso de inducción ($j > 1$). Supóngase que el resultado es válido para derivación con menos de j pasos.

Sea $A \Rightarrow \infty_1 \Rightarrow \infty_2 \Rightarrow \dots \Rightarrow \infty_{j-1} \Rightarrow \varepsilon$



Lo anterior es una derivación de j pasos en G . Las palabras $\alpha_1, \alpha_2, \dots, \alpha_{j-1}$ deben consistir, únicamente, de variables, ya que no es posible eliminar terminales con las producciones de una GLC.

Sea $\alpha_1 = A_1 A_2 \dots A_r$; entonces $A_i \Rightarrow \epsilon, i = 1, 2, \dots, r$, en menos de j pasos. Por hipótesis de inducción, cada A_r pertenece a VN_k . Dado que G tiene entre sus producciones a $A \rightarrow A_1 A_2 \dots A_r$, y $\alpha_1 \in VN_k$, se tiene que:

$A \in VN_{k+1} = VN_k$. Lo que queda demostrado (QED).

A continuación, se presenta el teorema que da la equivalencia, salvo por la palabra vacía, entre gramáticas libres de contexto y gramáticas positivas.

Teorema 4.1 Sea $L = L(G)$ para una GLC $G = (VN, VT; P; S)$, entonces existe una gramática positiva G' tal que:

$$L(G') = L - \{ \epsilon \}.$$

Demostración

Se determina, primeramente, $\ker(G)$ con ayuda del Lema 4.1. Se construye un conjunto de producciones P' como sigue. Si $A \rightarrow X_1 X_2 \dots X_n \in P$, entonces, se incluyen en P' todas las producciones $A \rightarrow \alpha_1 \alpha_2 \dots, \alpha_n$ tales que:

- Si X_i no pertenece a $\ker(G)$, entonces $\alpha_i = X_i$
- Si X_i pertenece a $\ker(G)$, entonces α_i es, o bien X_i , o bien ϵ
- No todas las α_i son ϵ

Sea $G' = (VN, VT, P', S)$. Se afirma que, para toda $A \in VN$ y $w \in VT^*$, $A \Rightarrow w$ si, y sólo si, $w \neq \epsilon$ y $A \Rightarrow w$.

Suficiencia. Supóngase que $A \Rightarrow w$ y $w \neq \epsilon$. Se prueba, por inducción sobre i , que $A \Rightarrow w$. La base, $i = 1$, es trivial porque $A \Rightarrow w$ debe ser una producción de P . Dado que $w \neq \epsilon$, también es una producción de P' , por construcción de este último conjunto. Para el paso de inducción, sea $i > 1$. Entonces $A \Rightarrow X_1 X_2 \dots X_n \Rightarrow w$. Escribáse $w = w_1 w_2 \dots w_n$ tal que, para cada j , $X_j \Rightarrow w_j$ en menos de i pasos. Si $w_j \neq \epsilon$ y X_j es una variable, entonces, por la hipótesis de inducción, se tiene $X_j \Rightarrow w_j$. Si $w_j = \epsilon$, entonces X_j pertenece a $\ker(G)$. Así, $A \Rightarrow \beta_1 \beta_2 \dots \beta_n$ es una producción en P' ,



donde $\beta_j = X_j$. Si $w \neq \epsilon$ y $\beta_j = \epsilon$ si $w_j = \epsilon$. Dado que $w \neq \epsilon$, no todas las β_j son ϵ . Por tanto se tiene una derivación:

$$A \Rightarrow \beta_1 \beta_2 \dots \beta_n \Rightarrow w_1 \beta_2 \dots \beta_n \Rightarrow w_1 w_2 \dots w_n, \text{ en } G'.$$

Necesidad. Supóngase que $A \Rightarrow w$. Seguramente $w \neq \epsilon$ ya que G' no tiene ϵ -producciones. Se prueba, por inducción sobre i , que $A \Rightarrow w$. Para la base, $i = 1$, obsérvese que $A \rightarrow w$ es una producción en G' . Debe haber una producción $A \rightarrow \infty$ en P tal que, si se cancelan, en ∞ , algunas variables pertenecientes a $\ker(G)$, queda w . Entonces, hay una derivación $A \Rightarrow \infty \Rightarrow w$, donde la derivación $\infty \Rightarrow w$, involucra derivar ϵ a partir de los símbolos de ∞ , pertenecientes a $\ker(G)$, que se cancelaron para obtener w .

Para el paso inductivo, sea $i > 1$. Entonces $A \Rightarrow X_1 X_2 \dots X_n \Rightarrow w$. Debe haber una producción $A \rightarrow \infty$ en P tal que $X_1 X_2 \dots X_n$ se encuentra con la cancelación de algunos símbolos de β pertenecientes a $\ker(G)$. Así $A \Rightarrow X_1 X_2 \dots X_n$. Escribáse $w = w_1 w_2 \dots w_n$ tal que, para cada j , $X_j \Rightarrow w_j$ con menos de i pasos. Por la hipótesis de inducción, $X_j \Rightarrow w_j$. Si X_j es una variable. Ciertamente, si X_j es una terminal, entonces $w_j = X_j$, y $X_j = w_j$ es trivialmente cierta. Así, $A \Rightarrow w$.

Dado que el resultado fue probado para cualquier variable de G , es válido, en particular, para el símbolo inicial de la gramática, S . Por tanto, $S \Rightarrow w$ si, y sólo si, $S \Rightarrow w$ y $w \neq \epsilon$; por ejemplo, $L(G') = L(G) - \{\epsilon\}$. Lo que queda demostrado (QED).

En vista del resultado probado en el teorema anterior, se supondrá, de aquí en adelante, que todas las gramáticas son positivas.

Ejemplo 4.3 Sea G la gramática cuyas producciones son:

$$S \rightarrow ABBA | aA$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

Se encontrará una gramática positiva G' equivalente a G . Para ello, se determinará, primeramente, $\ker(G)$. En este caso, $V_0 = \{A, B\}$. Como $ABBA$ pertenece a V_0^* y $S \rightarrow ABBA$ es una producción de G , se tiene que $V_1 = V_0 \cup \{S\}$. Dado que V_1 incluye todas las variables de G , se tiene que $V_2 = V_1$ y, por tanto, $\ker(G) = \{S, A, B\}$.

Se aplica, ahora, el algoritmo del Teorema 4.1 y se obtienen las siguientes producciones para G' .



$$S \rightarrow ABBA|BBA|ABA|ABB|BA|AA|AB|BB|A|B$$

$$S \rightarrow aA|a$$

La gramática G' , así obtenida, es una gramática positiva equivalente a G^{n1} .

Simplificación de Gramáticas Libres de Contexto

1. Eliminar producciones ϵ
2. Eliminar producciones unitarias
3. Eliminar los símbolos inútiles

1- Eliminar ϵ producciones ($A \mapsto \epsilon$)

Primero se revisa si el lenguaje contiene a la cadena vacía ϵ :

- Si ϵ no está en L no necesitamos ϵ producciones, por tanto aplicamos el algoritmo.
- Si ϵ en L , entonces generamos una GIC para $L - \{\epsilon\}$ y después al final hacer que:

$$G' = (V \cup \{S'\}, T, P', S') \text{ y } V \cap S' = \emptyset \text{ y } P' = P \cup (S' \mapsto S \mid \epsilon)$$

Existe un algoritmo para detectar símbolos “*nulificables*”

A es “*nulificable*” si $A \xRightarrow{*} \epsilon$

- Si $A \Rightarrow \epsilon$ en P entonces **A es “*nulificable*”**
- Si $A \Rightarrow \beta$ en P y todos los símbolos de β son “*nulificables*” $\Rightarrow A$ es “*nulificable*”.

ALGORITMO para eliminar producciones ϵ :

INICIO

i= 0

$$N^0 = \{A \mid A \rightarrow \epsilon \text{ una } P \text{ y } A \text{ en } V\}$$

REPETIR

i= i +1

$$N^i = N^{i-1} \cup \{A \mid A \rightarrow \beta \text{ en } P \text{ y } \beta \text{ en } (N^{i-1})^+\}$$



HASTA ($N^i = N^{i+1}$)

$P' =$ Si $A \rightarrow X_1 X_2 \dots X_n$ en P , entonces agregamos a P' las producciones $A \rightarrow \beta_1 \beta_2 \dots \beta_n$:

- Si X^i no esta en N^i entonces $\beta_i = X_i$
- Si X^i esta en N^i entonces $\beta_i = X_i$ o $\beta_i = \epsilon$
- No todos los β_i son ϵ

FIN

Ejemplo: Dada la GLC $G = (\{S, A, B, C, D\}, \{a, b\}, P, S)$, eliminar sus producciones ϵ

$$S \rightarrow ABCBD$$

$$A \rightarrow CD$$

$$B \rightarrow Cb$$

$$C \rightarrow a \mid \epsilon$$

$$D \rightarrow bD \mid \epsilon$$

Calcular N^i

$$N^0 = \{C, D\}$$

$$N^1 = N^0 \cup \{A\} = \{C, D\} \cup \{A\} = \{C, D, A\}$$

$$N^2 = N^1 \cup \phi = \{C, D, A\} \cup \phi = \{C, D, A\}$$

$$P' = \left\{ \begin{array}{l} S \rightarrow ABCBD \mid BCBD \mid ABCB \mid ABBD \mid ABB \mid BBD \mid BCB \mid BB \\ A \rightarrow CD \mid C \mid D \\ B \rightarrow Cb \mid b \\ C \rightarrow a \\ D \rightarrow bD \mid b \end{array} \right.$$



2- Eliminar producciones unitarias ($A \mapsto B$)

Son producciones de la forma $A \mapsto B$, donde A y B en V

Definición de “derivable de A ”:

Si $A \rightarrow B$ es una producción, entonces B es derivable de A .

- Si C es derivable de A , $C \rightarrow B$ es una producción y $B \neq A$, entonces B es derivable de A .

ALGORITMO para eliminar producciones unitarias:

INICIO

PRECONDICION = { La GLC $G = (V, T, P, S)$ no contiene ε -producciones }

1. $P_1 = P$
2. Para todos A en $V = \{ B \mid B \text{ es derivable de } A \}$
3. Para cada par (A, B) tal que B es derivable de A y cada producción no unitaria $B \rightarrow \beta$, añadir la producción $A \rightarrow \beta$ a P_1 sino está presente ya en P_1 .
4. Eliminar todas las producciones unitarias de P_1 .

FIN

Ejemplo: Tomamos el resultado del ejercicio anterior.

$$1- P_1 = P = \left\{ \begin{array}{l} S \rightarrow ABCBD \mid BCBD \mid ABCB \mid ABBD \mid ABB \mid BBD \mid BCB \mid BB \\ A \rightarrow CD \mid C \mid D \\ B \rightarrow Cb \mid b \\ C \rightarrow a \\ D \rightarrow bD \mid b \end{array} \right.$$



2- $S = \Phi$

$A = \{C, D\}$

$B = \Phi$

$C = \Phi$

$D = \Phi$

3- Pares (A, C), Como $C \rightarrow a$ y $A \rightarrow C$ entonces $A \rightarrow a$

(A, D), Como $D \rightarrow bD$ y $D \rightarrow b$ y $A \rightarrow D$ entonces $A \rightarrow bD$ y $A \rightarrow b$

$$4- P' = P_1 \cup \left\{ \begin{array}{l} A \rightarrow a \\ A \rightarrow bD \text{ y } A \rightarrow b \end{array} \right\} - \left\{ \begin{array}{l} A \rightarrow C \\ A \rightarrow D \end{array} \right\}$$

Resultando la siguiente GLC:

$S \rightarrow ABCBD \mid BCBD \mid ABCB \mid ABBD \mid ABB \mid BBD \mid BCB \mid BB$

$A \rightarrow CD \mid a \mid bD \mid b$

$B \rightarrow Cb \mid b$

$C \rightarrow a$

$D \rightarrow bD \mid b$

3- Eliminar símbolos inútiles

Los símbolos útiles (símbolos generadores de terminales y alcanzables desde S).

Sea $G = (V, T, P, S)$, X es un símbolo útil si $S \xRightarrow{*} \beta X \mu \xRightarrow{*} w$ para β y μ en $(V \cup T)^*$ y en $w \in T^*$

ALGORITMO para eliminar variables no generadoras

VV: Variables Viejas



VN: Variables Nuevas

INICIO

$$VV = \phi$$

$$VN = \{ A \mid A \rightarrow w \text{ en } P \text{ y } w \text{ en } T^* \}$$

MIENTRAS $VN \neq VV$ HACER

$$VV = VN$$

$$VN = VV \cup \{ A \mid A \rightarrow \beta \text{ en } P \text{ y } \beta \text{ en } (VV \cup T)^* \}$$

FINMIENTRAS

$$V' = VN$$

$$P' = \{ A \rightarrow \beta \mid A \text{ en } V' \text{ y } \beta \text{ en } (V' \cup T)^* \}$$

FIN

Se eliminan todas las producciones en donde intervienen 1 o más de los símbolos no generadores de terminales. Las $V' = VN$ son solo variables generadoras.

Ejemplo: Elimine las variables no generadoras de la siguiente

GLC $G = (\{S, A\}, \{a\}, P, S)$, con las producciones P :

$$S \rightarrow AB \mid a$$

$$A \rightarrow a$$

(0)

$$VV = \phi, T = \{a\}$$

$$VN = \{A, S\}$$

(1)

$$VV = \{A, S\}$$

$$VN = \{A, S\} \cup \{A, S\} = \{A, S\}$$



Parada ($VV = VN$) $\Rightarrow \{A, S\} = \{A, S\}$

$V' = \{A, S\}$

$P' = \begin{cases} S \rightarrow \alpha \\ A \rightarrow \alpha \end{cases}$

Decimos que X es “generador” si $X \Rightarrow^* w$ (con sucesivas derivaciones puede derivar una cadena de terminales) para alguna cadena Terminal w. Todo símbolo Terminal es generador, ya que w puede ser ese mismo símbolo Terminal, que se deriva en cero pasos.

ALGORITMO para determinar si son alcanzables

INICIO

$VV = TV = TN = \phi$

$VN = \{S\}$

MIENTRAS ($VV \neq VN$) o ($TV \neq TN$) HACER

$VV = VN;$

$TV = TN;$

$VN = VV \cup \{X \mid A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \text{ en } P, X \text{ en } V, X \text{ en algún } \beta_i \text{ y } A \text{ en } VV\}$

$TN = TV \cup \{X \mid A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \text{ en } P, X \text{ en } T, X \text{ en algún } \beta_i \text{ y } A \text{ en } VV\}$

FINMIENTRAS

$V' = VN$

$T' = TN$

$P' = \{A \rightarrow \beta \mid A \text{ en } V', \beta \text{ en } (V' \cup T')^*, A \rightarrow \beta \text{ en } P\}$

P' es el conjunto de producciones P contenido de símbolos de $(V' \cup T')$

FIN



Ejemplo: Elimine los símbolos no alcanzables de la siguiente GLC $G = (\{S, A, B\}, \{a\}, P, S)$, con las producciones:

$$S \rightarrow Aa \mid a$$

$$A \rightarrow a$$

$$B \rightarrow SA$$

$$0- VN = \{S\}$$

$$1- VV = \{S\}$$

$$TV = \Phi$$

$$VN = \{S, A\}$$

$$TN = \{a\}$$

$$V' = \{S, A\}$$

$$T' = \{a\}$$

$$P' = \begin{cases} S \rightarrow Aa \mid a \\ A \rightarrow a \end{cases}$$

$$2- VV = \{S, A\}$$

$$TV = \{a\}$$

$$VN = \{S, A\}$$

$$TN = \{a\}$$

Teorema: Si eliminamos

1ª los símbolos que NO son generadores $X \Rightarrow^* w$, para algún w en T^*

2ª los símbolos que NO son alcanzables $S \Rightarrow \beta X \mu$ para algún β y μ en $(V \cup T)^*$

Solo quedan los símbolos útiles de la GLC.

4.3 Hacer admisible y arborescente una gramática.

Gramáticas admisibles

Se dice que una gramática positiva $G = (VN, VT, P, S)$ es admisible si:

- i) Cada $A \in VN$ deriva alguna cadena terminal
- ii) Cada $a \in VT$ aparece en alguna cadena terminal
- iii) Cada derivación de una forma oracional ∞ puede completarse hasta obtener una cadena terminal



iv)

Los siguientes lemas y teoremas prueban que siempre es posible encontrar una gramática positiva admisible equivalente a una gramática positiva dada.

Lema 4.2 Sean $G = (VN, VT, P, S)$ una gramática positiva y VN' el conjunto de variables de G que derivan alguna cadena terminal en G . Defínanse, recursivamente, los conjuntos VN_i como sigue:

- i) $VN_0 = \{A | A \rightarrow w, \text{ para alguna } w \in VT^*\}$
 - ii) $VN_{i+1} = VN_i \cup \{A | A \rightarrow \alpha, \text{ para alguna } \alpha \in (VN_i \cup VT)^*\}$
- Entonces, $VN' = VN_k$, si $VN_{k+1} = VN_k$, para alguna k .

Demostración

Es claro que, por definición, $VN_i \subseteq VN'$ para cada i . Se probará, entonces, por inducción sobre el número de pasos de una derivación $A \Rightarrow w$ que $VN' \subseteq VN_k$, si $VN_{k+1} = VN_k$, para alguna k .

Base ($j=1$). Si la derivación consta, únicamente, de un paso, entonces $A \rightarrow w$ es una producción de G y A pertenecen a $VN_0 \subseteq VN_k$.

Paso de inducción ($j>1$). Sea $A \rightarrow X_1 X_2 \dots X_n \Rightarrow w$ una derivación de j pasos. Puede escribirse, entonces, $w = w_1 w_2 \dots w_n$, donde $X_i \rightarrow w_i$, $i=1, 2, \dots, n$, en menos de j pasos. Por hipótesis por inducción, aquellas X_i que son variables pertenecen a VN_k . Dado que $A \rightarrow X_1 X_2 \dots X_n$, es una producción de G y $X_1 X_2 \dots X_n \in (VN_k \cup VT)^*$, se tiene que $A \in VN_{k+1} = VN_k$. Así, $VN' \subseteq VN_k$ y, por tanto, $VN' = VN_k$. Lo que queda demostrado (QED).

Lema 4.3 Sea $G = (VN, VT, P, S)$ una gramática positiva con $L(G) \neq \emptyset$. Es posible encontrar, entonces, una gramática positiva $G' = (VN', VT, P', S')$ tal que $L(G) = L(G')$ y para cada $A \in VN'$ existe $w \in VT^*$ para la cual $A \Rightarrow w$.

Demostración

Calcúlese VN' con el algoritmo del lema anterior y sea P' el conjunto de producciones cuyos símbolos pertenecen a $VN' \cup VT'$. Es claro, por esta construcción, que si $A \in VN'$, entonces $A \Rightarrow w$, para alguna w . Como toda derivación en G' es una derivación de G , se tiene que $L(G') \subseteq L(G)$. Supóngase, ahora, que existe alguna variable en $VN - VN'$ o una producción en $P - P'$. Pero, entonces, hay una variable en $VN - VN'$ que deriva una cadena terminal, lo cual contradice la definición de VN' . Por tanto, no existe tal w y $L(G) = L(G')$. Lo que queda demostrado.

Lema 4.4 Dada una gramática positiva $G = (VN, VT, P, S)$, puede encontrarse una gramática positiva equivalente $G' = (VN', VT', P', S)$ tal que, para cada $X \in VN' \cup VT'$, existen $\alpha, \beta \in (VN' \cup VT')^*$ para las cuales $S \Rightarrow \alpha X \beta$

Demostración

El conjunto $VN' \cup VT'$ de símbolos que aparecen en formas oracionales de G , se construye con el siguiente proceso iterativo. Colóquese S en VN' . Si A es una variable ya colocada en VN' y sus producciones son $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_k$, entonces colóquese en VN' las variables que aparecen en las α_i y en VT' , los terminales que aparecen en ellas. El conjunto P' de producciones de G' consta de aquellas producciones de P que involucran, únicamente, a símbolos en $VN' \cup VT'$. Lo que queda demostrado.

Si se aplica, primeramente, el Lema 4.2, después el Lema 4.3 y, por último, el Lema 4.4 es posible transformar una gramática positiva en una gramática positiva admisible. Vale la pena hacer la observación de que si se aplica, en primer lugar, el lema 4.4 y, a continuación, los lemas 4.2 y 4.3, la gramática obtenida puede no ser admisible.

Teorema 4.3 Todo lenguaje libre de contexto (LLC) sin ϵ es generado por una gramática positiva admisible.

Demostración

Sea $L = L(G)$ un LLC no vacío sin ϵ . Por el Teorema 4.1, puede suponerse que $G = (VN, VT, P, S)$ es positiva. Sea $G_1 = (VN_1, VT, P_1, S)$ el resultado de aplicar los Lemas 4.2 y 4.3 a G y sea $G_2 = (VN_2, VT_2, P_2, S)$, el resultado de aplicar el lema 4.4 a G_1 . Se probará que G_2 es admisible y positiva.

Sea $A \in NV_2$. Ya que la construcción del lema 4.4 no agrega variables nuevas a VN_1 , se tiene que $VN_2 \subseteq VN_1$ y, por tanto, $A \in NV_1$. Por los lemas 4.2 y 4.3 se tiene que $A \rightarrow w$, para alguna $w \in VT_2$.

- i) Sea $a \in VT_2$. Por el lema 4.4, existen $\alpha, \beta \in (VN_2 \cup VT_2)^*$ tales que $S \Rightarrow \alpha a \beta$. Si α, β involucran variables, por los lemas 4.2 y 4.3, éstas derivan cadena terminales, así que existen $w_1, w_2 \in VT_2^*$ tales que $\alpha \Rightarrow w_1, \beta \Rightarrow w_2$, en G_2 . Por tanto, $S \Rightarrow \alpha a \beta \Rightarrow w_1 a w_2$, en G_2 , y aparece en una cadena terminal. Si α, β no involucran variables, claramente a aparece en una cadena terminal.



- ii) Sea $\infty \in (VN_2 \cup VT_2)^*$ tal que $S \rightarrow \infty$. Por los argumentos usados en el inciso anterior, debe existir $w \in VT_2^*$ tal que $\infty \rightarrow w$, en G_2 . Por tanto, cada derivación de una forma oracional en G_2 puede completarse a una derivación de una cadena terminal en G_2 .

De i), ii), iii) se sigue que G_2 es admisible y, como las construcciones de los Lemas 4.2, 4.3 y 4.4 no agregan producciones, se tiene que $\ker(G_2) = \emptyset$ por ejemplo G_2 es positiva. Lo que queda demostrado.

Ejemplo 4.6 Considérese la gramática cuyas producciones son:

$S \rightarrow BA|a$

$A \rightarrow a$

Si se aplican los lemas 4.2 y 4.3 se obtiene lo siguiente

$VN_0 = \{S, A\} \quad (VN_0 \cup VT)^* = \{S, A, a\}^*$

$VN_1 = \{S, A\} \cup VN_0 = VN_0$, ya que las únicas producciones en la gramática cuyo lado derecho es una cadena de $\{S, A, a\}^*$ son aquellas que tienen a "S" o bien a "A" en su lado izquierdo.

Así, el nuevo conjunto de producciones queda como sigue:

$S \rightarrow a \qquad A \rightarrow a$

Si se aplica el Lema 4.4 a la gramática obtenida arriba, se llega a

$S \rightarrow a$

Si, por el contrario, se aplica, en primer lugar, el Lema 4.4, se obtiene el siguiente conjunto de producciones:

$S \rightarrow BA|a$

$A \rightarrow a$

Aplicando, ahora, a esta gramática (que es la dada originalmente) los Lemas 4.2 y 4.3 se obtiene como nuevo conjunto de producciones a

$S \rightarrow a \qquad A \rightarrow a$

Como puede verse, A no aparece en ninguna forma oracional de la gramática; por tanto, ésta no es admisible.

Gramáticas arborescentes.

Sea $G = (VN, VT, P, S)$ una gramática; se dice que G es arborescente si no contiene producciones de la forma $A \rightarrow B$. A este tipo de producciones se les llama producciones unitarias.

El siguiente teorema demuestra que todo lenguaje libre de contexto puede generarse a partir de una gramática positiva, admisible y arborescente.

Teorema 4.4 Todo LLC sin ϵ es generado por una gramática positiva, admisible y arborescente.



Demostración

Sea L un LLC sin ϵ generado por una gramática $G = (VN, VT, P, S)$; por el Teorema 4.1 puede suponerse que G es positiva.

Supónganse que $A \Rightarrow B$, para algunas $A, B \in VN$. Esto puede verificarse fácilmente, ya que G es una gramática positiva, y sucede que:

$$A \Rightarrow B_1 \Rightarrow B_2 \Rightarrow \dots \Rightarrow B_n \Rightarrow B, \text{ en } G$$

Y alguna variable aparece dos veces en la sucesión, es posible encontrar una sucesión más corta de producciones unitarias que lleva $A \Rightarrow B$; así que basta con considerar, sólo, aquellas sucesiones de producciones unitarias que no repiten ninguna de las variables G .

Constrúyase, ahora, un nuevo conjunto de P' de producciones a partir de P , incluyendo, en primer lugar, todas las producciones no unitaria de P , agréguense a P' todas las producciones de la forma: $A \Rightarrow \infty$

Se tiene, ahora, una gramática modificada $G' = (VN, VT, P', S)$. Es claro que, si $A \Rightarrow \infty$ es una producción en P' , entonces $A \Rightarrow \infty$, en G . Así que, si $S \Rightarrow w$, en G' , entonces $S \Rightarrow w$, en G ; por ejemplo $L(G') \subseteq L(G)$.

Supónganse que $w \in L(G)$ y considérese una derivación izquierda de w en G , por ejemplo

$$S = \infty_0 \Rightarrow \infty_1 \Rightarrow \dots \Rightarrow \infty_n = w$$

Si, para $0 \leq i < n$, $\infty_i \Rightarrow \infty_{i+1}$ en G , por una producción no unitaria, entonces $\infty_i = \infty_{i+1}$ en G' . Supóngase, ahora, que:

$$\infty_i \Rightarrow \infty_{i+1} \Rightarrow \dots \Rightarrow \infty_j, \text{ en } G,$$

Por producciones unitarias y que $\infty_{i-1} \Rightarrow \infty_i$ y $\infty_j \Rightarrow \infty_{j+1}$, en G , por producciones no unitarias, o que $i=0$. Entonces, $\infty_i, \infty_{i-1}, \dots, \infty_j$ son todas de la misma longitud y, dado que la derivación izquierda, el símbolo reescrito en cada una de éstas debe ocupar, siempre, la misma posición. Pero, en este caso, se tiene que $\infty_i = \infty_{j-1}$, en G' , por una producción en $P'-P$; así que, $S \Rightarrow w$, en G' y, por tanto, $L(G) = L(G')$. Claramente, G' es positiva y arborescente. Si se usan los Lemas 4.2, 4.3 y 4.4, para transformar a G' en una gramática admisible, no se agregan producciones, así que, el resultado de aplicar estas construcciones a G' es una gramática que satisface el teorema. QLD.



4.4 Obtener la forma normal de Chomsky de una gramática Libre de Contexto.

Formas Normales de las GLC

Una gramática GLC, está en la FNC si cada una de sus producciones es de los tipos siguientes:

$$A \rightarrow BC$$

$$A \rightarrow a$$

Donde A, B y C son variables (en V) y a es un símbolo Terminal (en T)

Teorema: Cualquier GLC sin ϵ -producciones puede ser transformada a una gramática equivalente en donde las producciones son de la forma

$$A \rightarrow BC \text{ o } A \rightarrow a$$

Antes de aplicar el algoritmo hay que eliminar:

- 1- producciones ϵ
- 2- producciones unitarias
- 3- símbolos inútiles.

Forma Normal de Chomsky (FNC) (Chomsky, 1959)

Una gramática GLC, está en la FNC si cada una de sus producciones es de los tipos siguientes:

$$A \rightarrow BC$$

$$A \rightarrow a$$

Donde A, B y C son variables (en V) y a es un símbolo Terminal (en T)

Teorema: Cualquier GLC sin ϵ -producciones puede ser transformada a una gramática equivalente en donde las producciones son de la forma:

$$A \rightarrow BC \text{ o } A \rightarrow a$$



Antes de aplicar el algoritmo hay que eliminar:

- 4- producciones ϵ
- 5- producciones unitarias
- 6- símbolos inútiles.

ALGORITMO para llevar a la Forma Normal de Chomsky

INICIO

Sea $A \rightarrow X_1 X_2 \dots X_m$ en P

1. Crear producciones del tipo $A \rightarrow a$ adecuadamente

Si X_i es Terminal y $X_i = a$ entonces agregamos a P' la producción $C_a \rightarrow a$ y reemplazamos X_i por C_a . Entonces todas las producciones son de forma:

$$A \rightarrow B_1 B_2 \dots B_m \text{ y } A \rightarrow a$$

2. Cuando creamos producciones $A \rightarrow BC$ adecuadamente

Para los casos en que $m \geq 3$, hacemos:

$$A \rightarrow B_1 D_1; D_1 \rightarrow B_2 D_2; \dots, D_{m-2} \rightarrow B_{m-1} B_m$$

Agregamos las respectivas variables y producciones

$$V' = V \cup \{ D_1, D_2 \dots D_{m-2} \}$$

FIN

Ejemplo: Dada la GLC $G = (\{S, A, B\}, \{a, b\}, P, S)$, obtener su FNC

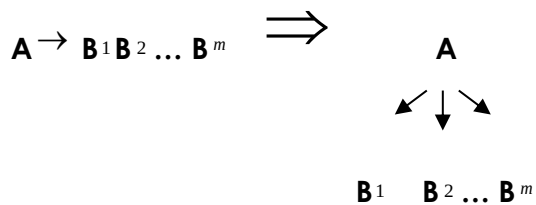
$$P \left\{ \begin{array}{l} S \rightarrow bA \mid aB \\ A \rightarrow bAA \mid aS \mid a \\ B \rightarrow aBB \mid bS \mid B \end{array} \right.$$

$$\begin{aligned} (1) \quad & A \rightarrow a \\ & S \rightarrow C_b A \mid C_a B \\ & A \rightarrow C_b AA \mid C_a S \mid a \\ & B \rightarrow C_a BB \mid C_b S \mid b \end{aligned}$$



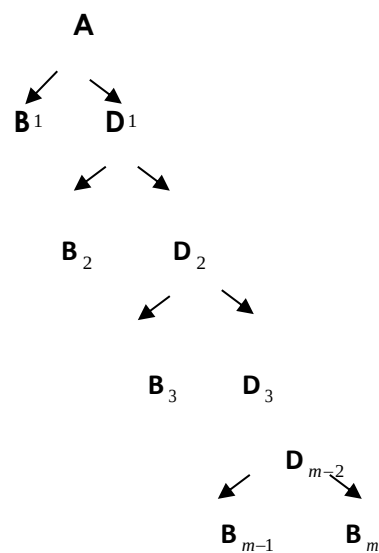
- $C_a \rightarrow a$
 $C_b \rightarrow b$
(2) $S \rightarrow C_b A \mid C_a B$
 $A \rightarrow C_b D_1 \mid C_a S \mid a$
 $B \rightarrow C_a D_2 \mid C_b S \mid b$
 $D_1 \rightarrow AA$
 $D_2 \rightarrow BB$
 $C_a \rightarrow a$
 $C_b \rightarrow b$

FNC no parece tener aplicaciones importantes en lingüística natural, aunque si tiene aplicación como una forma eficiente de comprobar si una cadena pertenece a un LLC. Los árboles de derivación son binarios.



$A \rightarrow B_1 D_1; D_1 \rightarrow B_2 D_2;$

$D_2 \rightarrow B_3 D_3; \dots D_{m-2} \rightarrow B_{m-1} B_m \Rightarrow$





4.5 Obtener la forma normal de Greibach de una gramática Libre de Contexto y generar su Autómata de Pila (AP).

Se dice que una gramática está en forma normal de *Greibach* (FNG) si todas sus producciones son de la forma:

$A \rightarrow a^\infty$, donde ∞ es una cadena, posiblemente vacía, de variables.

Los siguientes lemas y teoremas demuestran que siempre es posible encontrar una GLC en FNC que genere a una LLC dado, que no contenga a ϵ .

Lema 4.5 Sea $G = (VN, VT, P, S)$ una GLC. Sea $A \rightarrow \infty_1 B \infty_2$, una producción en P y sea $B \rightarrow \beta_1 | \beta_2 | \dots | \beta_r$ todas las B -producciones en P . Sea $G_1 = (VN, VT, P_1, S)$ la gramática obtenida, a partir de G , suprimiendo la producción $A \rightarrow \infty_1 B \infty_2$ de P y agregando las producciones $A \rightarrow \infty_1 B_1 \infty_2 | \infty_1 B_2 \infty_2 | \dots | \infty_1 B_r \infty_2$.

Entonces $L(G) = L(G_1)$.

Demostración

Es claro que si $A \rightarrow \infty_1 B \infty_2$ se usa en una derivación en G_1 , entonces $A \Rightarrow \infty_1 B \infty_2 \Rightarrow \infty_1 B_i \infty_2$ puede usarse en G ; así que $L(G_1) \subseteq L(G)$. Dado que $A \rightarrow \infty_1 B \infty_2$ es la única producción en G que no está en G_1 , si es usada en alguna derivación en G , la variable B debe reescribirse, en algún paso posterior, usando una producción de la forma $B \rightarrow \beta_i$; estos dos pasos pueden reemplazarse por el único paso $A \Rightarrow \infty_1 B_i \infty_2$, en G_1 . Lo que queda demostrado.

Lema 4.6 Sea $G = (VN, VT, P, S)$ una GLC. Sea $A \rightarrow A \infty_1 | A \infty_2 | \dots | A \infty_r$ el conjunto de A -producciones cuyo lado derecho inicia con A y sean $A \rightarrow \beta_1 | \beta_2 | \dots | \beta_s$ las restantes A -producciones. Sea $G_1 = (VN \cup \{B\}, VT, P, S)$ la GLC obtenida al agregar la variable B a V y reemplazar todas las A -producciones por las producciones:

- | | | |
|------------------------------|----------------------------|-------------------|
| i) $A \rightarrow \beta_i$ | $A \rightarrow \beta_i B$ | $1 \leq i \leq s$ |
| ii) $B \rightarrow \infty_i$ | $B \rightarrow \infty_i B$ | $1 \leq i \leq r$ |

Entonces, $L(G) = L(G_1)$.

Demostración

En una derivación izquierda, una sucesión de producciones de la forma $A \rightarrow A^{\infty_i}$ debe terminar, en algún momento, con una producción de la forma:

$A \rightarrow \beta_j$. La sucesión de reescrituras

$A \Rightarrow A^{\infty_{i1}} \Rightarrow A^{\infty_{i2}} \Rightarrow \dots \Rightarrow A^{\infty_{ip}} \dots \infty_{i2}^{\infty_{i1}} \Rightarrow \beta_j^{\infty_{ip}} \dots \infty_{i2}^{\infty_{i1}}$

en G , puede reemplazarse en G_1 por

$A \Rightarrow \beta_j B \Rightarrow \beta_j^{\infty_{ip}} B \Rightarrow \dots \Rightarrow \beta_j^{\infty_{ip}} \dots \infty_{i2}^{\infty_{i1}} B \Rightarrow \beta_j^{\infty_{ip}} \dots \infty_{i2}^{\infty_{i1}}$

Claramente, la transformación inversa también es posible. Por lo tanto,
 $L(G_1) = L(G)$. Lo que queda demostrado.

Teorema 4.6 (Forma normal de Greibach FNG) Todo LLC sin ϵ es generado por una gramática donde todas las producciones son de la forma $A \rightarrow a^{\infty}$, donde ∞ es una cadena de variables, posiblemente vacía.

Demostración

Sea $G = (VN, VT, P, S)$ una gramática en FNC que genera al LLC L y supóngase que $V = \{A_1, A_2, \dots, A_m\}$. El primer paso es modificar las producciones de tal modo que, si $A_i \rightarrow A_{j\zeta}$ es una producción, entonces $j > i$. Para lograr esto, se procede, recursivamente, como sigue:

Supóngase que se han modificado las producciones de tal modo que, para $1 < i < k$, $A_i \rightarrow A_{j\zeta}$ es una producción sólo si $i < j$; se procede, ahora, a modificar las A_k -producciones.

Si $A_k \rightarrow A_{j\zeta}$ es una producción con $k > 1$, se genera un nuevo conjunto de producciones sustituyendo A_j por el lado derecho de cada A_j -producción, de acuerdo con el Lema 4.5. Repitiendo este proceso, a lo más, $k-1$ veces, se obtienen producciones de la forma $A_k \rightarrow A_{j\zeta}$, $1 > k$. Las producciones con $1 = k$ se reemplazan, de acuerdo con el Lema 4.6, introduciendo una nueva variable B_k .

Repitiendo el proceso anterior para cada variable original, se tienen únicamente, producciones de las formas:

- | | | |
|------|------------------------------|---|
| i) | $A_i \rightarrow A_{i\zeta}$ | $j > i$ |
| ii) | $A_i \rightarrow a\zeta$ | $a \in T$ |
| iii) | $B_i \rightarrow \zeta$ | $\zeta \in (V \cup \{B_1, B_2, \dots, B_{i+1}\})^*$ |



Nótese que el lado derecho de cualquier producción para A_m , debe iniciar con un terminal, ya que A_m es la variable con mayor índice. El símbolo inicial en el lado derecho de cualquier A_{m-1} -producciones debe ser A_m o un terminal. Si es A_m , pueden generarse nuevas producciones, reemplazando A_m por el lado derecho de las A_m -producciones, de acuerdo con el Lema 4.5; estas producciones deben tener lados derechos que inician con un símbolo terminal.

Como último paso, se examinan las producciones para las nuevas variables $B_1, B_2, \dots, B_m$. Dado que se inició con una gramática en FNC, es fácil probar, por inducción sobre el número de aplicaciones de los Lemas 4.5 y 4.6, que el lado derecho de cada A_i -producciones, $i \leq m$, inicia con un terminal o con $A_i A_k$, para algunos j, k . Por tanto, en la producción $A_k \rightarrow A_k \infty$, ∞ no puede ser vacía o iniciar consiguiente, todas las B_i -producciones tienen lados derechos que inician con terminales o con alguna A_i , y una aplicación más del Lema 4.5, para cada B_i -producción, completa la construcción, lo que queda demostrado.

Ejemplo 4.8 Sea G la gramática cuyas producciones son:

$$A_1 \rightarrow A_2 A_1 | c$$

$$A_2 \rightarrow A_3 A_2 | a$$

$$A_3 \rightarrow A_1 A_3 | b$$

Se construirá una gramática equivalente en FNG.

La única producción cuyo lado derecho inicia con una variable de índice menor que la variable en el lado izquierdo es $A_3 \rightarrow A_1 A_3$. Como primer paso se le reemplaza por las producciones que se obtienen de sustituir A_1 por los lados derechos de sus producciones. El nuevo conjunto de producciones queda como sigue:

$$A_1 \rightarrow A_2 A_1 | c$$

$$A_2 \rightarrow A_3 A_2 | a$$

$$A_3 \rightarrow A_2 A_1 A_3 | c A_3 | b$$



En el siguiente paso se reemplaza A_2 , en el lado derecho de la producción $A_3 \rightarrow A_2A_1A_3$, por los lados derechos de las A_2 -producciones y se elimina $A_3 \rightarrow A_2A_1A_3$. Se obtiene el siguiente conjunto de producciones:

$$A_1 \rightarrow A_2A_1|c$$

$$A_2 \rightarrow A_3A_2|a$$

$$A_3 \rightarrow A_3A_2A_1A_3|aA_1A_3|cA_3|b$$

A continuación se introduce la nueva variable B_3 y se reemplazan las A_3 -producciones, utilizando el Lema 4.6, y se obtiene el nuevo conjunto de producciones

$$A_1 \rightarrow A_2A_1|c$$

$$A_2 \rightarrow A_3A_2|a$$

$$A_3 \rightarrow aA_1A_3|cA_3|b$$

$$A_3 \rightarrow aA_1A_3B_3|cA_3B_3|bB_3$$

$$B_3 \rightarrow A_2A_1A_3|A_2A_1A_3B_3$$

Ahora, todos los lados derechos de las A_3 -producciones inician con un terminal; por tanto, se utilizan para transformar la producción $A_2 \rightarrow A_3A_2$ en seis nuevas producciones, que inician sus lados derechos con un terminal. A su vez, las nuevas A_2 -producciones. El conjunto de producciones queda, entonces, como sigue

$$A_1 \rightarrow aA_1A_3A_2A_1|cA_3A_2A_1|bA_2A_1|aA_1|c$$

$$A_1 \rightarrow aA_1A_3B_3|cA_3B_3A_2A_1|bB_3A_2A_1$$

$$A_2 \rightarrow aA_1A_3A_2|cA_3A_2|bA_2|a$$

$$A_2 \rightarrow aA_1A_3B_3A_2|cA_3B_3A_2|bB_3A_2$$

$$A_3 \rightarrow aA_1A_3|cA_3|b$$

$$A_3 \rightarrow aA_1A_3B_3|cA_3B_3|bB_3$$

$$B_3 \rightarrow aA_1A_3A_2A_1A_3|cA_3A_2A_1A_3|bA_2A_1A_3|aA_1A_3$$

$$B_3 \rightarrow aA_1A_3B_3A_2A_1A_3|cA_3B_3A_2A_1A_3|bB_3A_2A_1A_3$$

$$B_3 \rightarrow aA_1A_3A_2A_1A_3B_3|cA_3A_2A_1A_3B_3|bA_2A_1A_3B_3|aA_1A_3B_3$$



Autómatas de Pila (AP)

Los autómatas de pila que desde este momento los abreviamos como: AP son similares a los autómatas finitos ya que se tienen autómatas de pila determinísticos o no determinísticos, también se pueden utilizar para aceptar cadenas de un lenguaje definido sobre un alfabeto A.

Los AP pueden aceptar lenguajes que no pueden aceptar los autómatas finitos. Como ya se ha estudiado, los autómatas finitos no determinísticos (AFND) reconocen los mismos lenguajes que los autómatas finitos determinísticos (AFD). Sin embargo, no ocurre lo mismo con autómatas de pila no determinísticos (APND) y autómatas de pila determinísticos (APD). Algunos lenguajes sólo pueden ser reconocidos por un APND pero no por un APD.

Un AP cuenta con una cinta de entrada y un mecanismo de control que puede encontrarse en uno de entre un número finito de estados. Uno de estos estados se designa como estado inicial, y además algunos estados se llaman de aceptación o finales. A diferencia de los autómatas finitos, los autómatas de pila cuentan con una memoria auxiliar llamada pila. Los símbolos (llamados símbolos de pila) pueden ser insertados o extraídos de la pila, de acuerdo con el manejo el último en entrar es el primero en salir (Last-In-First-Out LIFO).

Las transiciones entre los estados que ejecutan los autómatas de pila dependen de los símbolos de entrada y de los símbolos de la pila. El autómata acepta una cadena X si la secuencia de transiciones, comenzando en estado inicial y con pila vacía, conduce a un estado final, después de leer toda la cadena X.

Autómata de pila reconocedor determinístico

$APD = \langle E, A, P, \delta, e_0, Z_0, F \rangle$ de donde:

E: Conjunto finito de estados $E = \{e_1, e_2, e_3, e_4, \dots, e_i, \dots, e_j\}$

A: Alfabeto o conjunto finito de símbolos de la cinta de entrada,

P: Alfabeto o conjunto finito de símbolos de la Pila. $P \cap A = \emptyset$

δ : función de transición de estados

e_0 : Estado inicial $e_0 \in E$.

Z_0 : Símbolo distinguido $Z_0 \in P$



F: Conjunto de estados finales o estados de aceptación. $F \subseteq E$.

La función de transición definida como: $\delta: E \times (A \cup \{\epsilon\}) \times P \rightarrow E \times P^*$

$$1) \delta(e_i, a, X) = (e_j, \alpha)$$

$$2) \delta(e_i, \epsilon, X) = (e_j, \alpha) \text{ donde } a \in A; X \in P; \alpha \in P^*; e_i, e_j \in E.$$

Nota: Si existe transición de tipo (2), sólo se garantiza que AP es determinístico si $\forall s \in A, \delta(e_i, s, X)$ está indefinida.

Descripción instantánea:

Una configuración de un AP es una tripla $\langle e, \sigma, \pi \rangle$ donde

e: estado_actual, por ejemplo e_i, e_j ;

σ : cadena de entrada a ser leída;

π : contenido de la pila.

Luego, se define una relación de transición $\rightarrow$ en el espacio de posibles configuraciones del AP, tanto si:

(1)

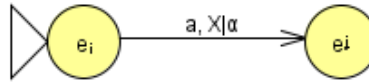
$\langle e_i, a\omega, X\beta \rangle \rightarrow \langle e_j, \omega, \alpha\beta \rangle$ Si existe la transición tipo (1), el AP pasa al estado por ejemplo, avanza la cabeza lectora y reemplaza el tope X por α .

(2)

$\langle e_i, a\omega, X\beta \rangle \rightarrow \langle e_j, a\omega, \alpha\beta \rangle$ Si existe la transición tipo (2), el AP pasa al estado por ejemplo, NO avanza la cabeza lectora y reemplaza el tope X por α .

Donde $a \in A; \omega \in A^*; X \in P; \alpha, \beta \in P^*; e_i, e_j \in E$

La función de transición de estados de un AP puede ser representada por un diagrama donde los nodos representan los estados y los arcos transiciones. Si existe transición tipo (1), el arco queda rotulado de la manera en lo que muestra el siguiente diagrama



Si el estado actual es e_i y la cabeza lectora apunta un símbolo a , y el tope de la pila es X , entonces cambiar al nuevo estado e_j , avanzar la cabeza lectora, y sustituir el símbolo del tope X en la pila por la cadena α .

Por ejemplo:

Si $\alpha = ZYX$ deja X , apila Y , y apila Z (nuevo tope Z), donde $X, Y, Z \in P$

Si $\alpha = XX$ deja X y apila X (nuevo tope X).

Si $\alpha = X$ deja X como el mismo tope (no altera la pila)

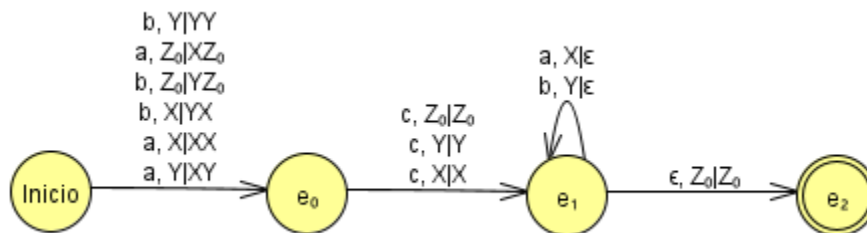
Si $\alpha = \epsilon$ elimina X , y el nuevo tope es el símbolo por debajo (desapila)

Ejemplo 4.9

$A = \{a, b, c\}$

$L_1 = \{\omega c \omega^R \mid \omega \in \{a, b\}^*\}$

APD_1 es un autómata de pila que reconoce L_1 . Según el siguiente AP



$APD_1 = \langle \{e_0, e_1, e_2\}, \{a, b, c\}, \{X, Y, Z_0\}, \delta_1, e_0, Z_0, \{e_2\} \rangle$

Ejemplos de cadenas aceptadas ó **no** aceptadas por AP_1

$\delta^*(e_0, abcba) = e_2$ $abcba \in L_1$

$\delta^*(e_0, c) = e_2$ $c \in L_1$

$\delta^*(e_0, abcab) = e_1$ $abcab \notin L_1$

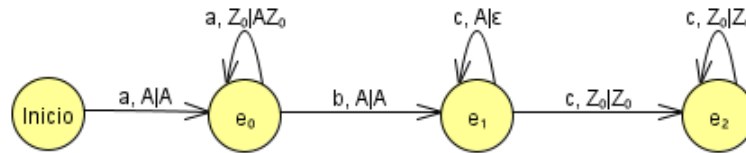
$\delta^*(e_0, a) = e_0$ $a \notin L_1$

Ejemplo 4.10

$$L_2 = \{a^i b c^k / i, k \geq 1 \text{ y } i < k\}$$

$$APD_2 = \langle \{e_0, e_1, e_2\}, \{a, b, c\}, \{A, Z_0\}, \delta_2, e_0, Z_0, \{e_2\} \rangle$$

δ_3 :

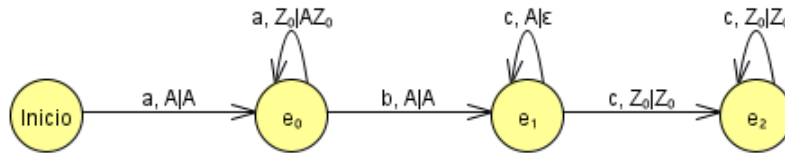


Ejemplo 4.11

$$L_3 = \{a^i b c^k / i, k \geq 1 \text{ y } i \leq k\}$$

$$PD_3 = \langle \{e_0, e_1, e_2\}, \{a, b, c\}, \{A, Z_0\}, \delta_3, e_0, Z_0, \{e_2\} \rangle \text{ Se muestra en el siguiente AP}$$

δ_3 :

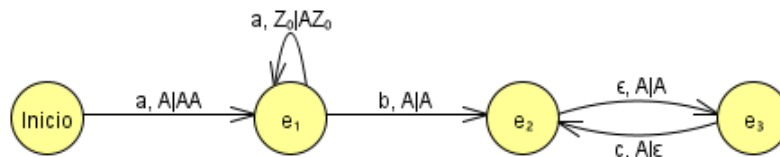


Ejemplo 4.12

$$L_4 = \{a^i b c^k / i, k \geq 1 \text{ y } i > k\}$$

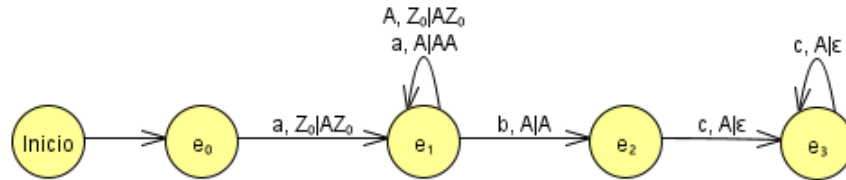
$$APD_{41} = \langle \{e_0, e_1, e_2, e_3\}, \{a, b, c\}, \{A, Z_0\}, \delta_{41}, e_0, Z_0, \{e_3\} \rangle \text{ Se muestra en el siguiente AP}$$

δ_4 :



$APD_{42} = \langle \{e_0, e_1, e_2, e_3\}, \{a, b, c\}, \{A, Z_0\}, \delta_{42}, e_0, Z_0, \{e_3\} \rangle$ Se muestra en el siguiente AP

δ_5 :

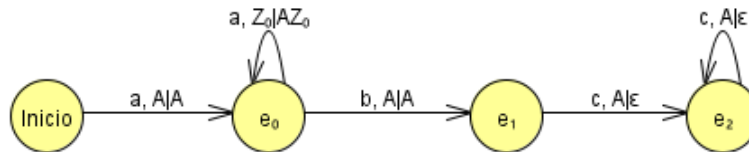


Ejemplo 4.13

$L_5 = \{a^i b c^k / i, k \geq 1 \text{ y } i \geq k\}$

$APD_5 = \langle \{e_0, e_1, e_2\}, \{a, b, c\}, \{A, Z_0\}, \delta_5, e_0, Z_0, \{e_2\} \rangle$

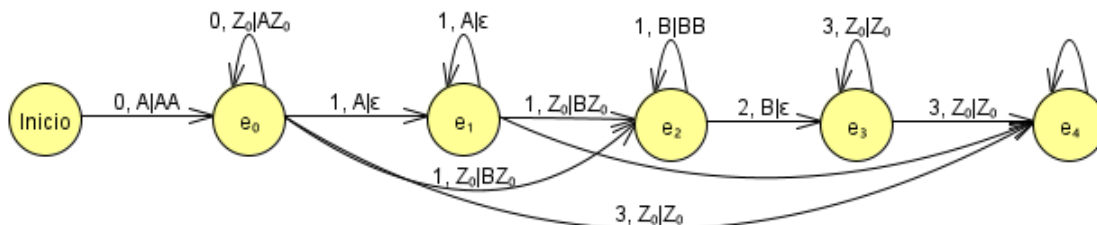
Ver el diagrama del AP para δ_5 :



Ejemplo 4.14

$_6 = \{0^i 1^{i+k} 2^k 3^{n+1} / i, k, n \geq 0\}$

$APD_6 = \langle \{e_0, e_1, e_2, e_3, e_4\}, \{0, 1, 2, 3\}, \{A, B, Z_0\}, \delta_6, e_0, Z_0, \{e_4\} \rangle$ Ver diagrama de **AP** para δ_6



Ejemplo 4.15

$$L_7 = \{h^n g^j e^{2n} d^{3i} / i, j, n \geq 0\}$$

$$APD_7 = \langle \{e_0, e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9\}, \{h, e, g, d\}, \{H, Z_0\}, \delta_7, e_0, Z_0, \{e_0, e_4, e_8, e_9\} \rangle$$

CASOS

Si $n, i, j > 0$

Si $n = 0$ y $i, j > 0$

Si $i = 0$ y $i, j > 0$

Si $j = 0$ y $n, i > 0$

Si $n, i = 0$ y $j > 0$

Si $n, j = 0$ y $i > 0$

Si $n, j = 0$ y $n > 0$

Si $n, i, j = 0$

CADENAS DE L_7

$$h^n g^j e^{2n} d^{3i}$$

$$g^j d^{3i}$$

$$h^n g^j e^{2n}$$

$$h^n e^{2n} d^{3i}$$

$$g^j$$

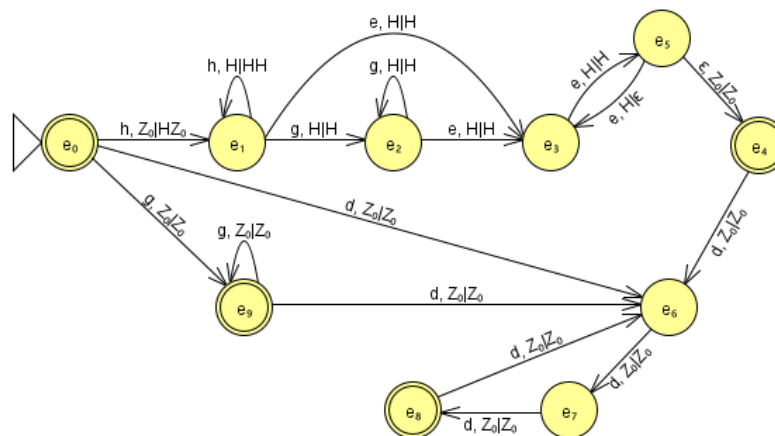
$$d^{3i}$$

$$h^n e^{2n}$$

ϵ

Ver el **Diagrama de Autómata de Pila no determinístico para δ_7**

δ_7 :



Autómata de pila no determinístico (APND)

Descripción.

$APND = \langle E, A, P, \delta, e_0, Z_0, F \rangle$, donde las componentes E, A, P, e_0, Z_0, F quedó definido como:



E: Conjunto finito de estados $E = \{e_1, e_2, e_3, e_4, \dots, e_i, \dots, e_j\}$

A: Alfabeto o conjunto finito de símbolos de la cinta de entrada,

P: Alfabeto o conjunto finito de símbolos de la Pila. $P \cap A = \emptyset$

δ : función de transición de estados

e_0 : Estado inicial $e_0 \in E$.

Z_0 : Símbolo distinguido $Z_0 \in P$

F: Conjunto de estados finales o estados de aceptación. $F \subseteq E$.

(Repetición de la definición de autómatata de pila).

La función de transición se define como:

$$\delta: E \times (A \cup \{\epsilon\}) \times P \rightarrow P_f(E \times P^*)$$

$$1) \delta(e_i, a, X) = \{(e_j, \alpha_1), (e_k, \alpha_2), \dots\}$$

$$2) \delta(e_i, \epsilon, X) = \{(e_j, \alpha_1), (e_k, \alpha_2), \dots\} \text{ donde } a \in A; X \in P; \alpha_1, \alpha_2 \in P^*; e_i, e_j, e_k \in E.$$

Como ya se ha estudiado, repetimos, los autómatas finitos no determinísticos (AFND) reconocen los mismos lenguajes que los autómatas finitos determinísticos (AFD). Sin embargo no ocurre lo mismo con autómatas de pila no determinísticos (APND) y autómatas de pila determinísticos (APD). Algunos lenguajes sólo pueden ser reconocidos por un APND, pero no por un APD.

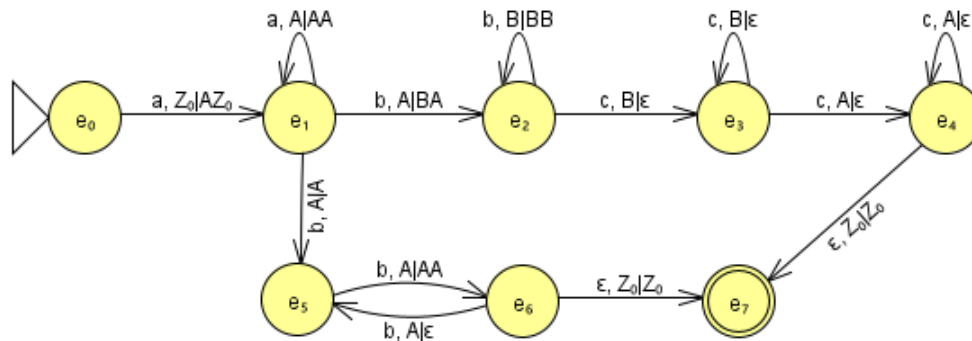
Ejemplo 4.16

$$L_8 = \{a^m b^p c^{p+m} / m, p \geq 1\} \cup \{a^{2i} b^i / i \geq 1\}$$

L_8 sólo se puede reconocer con un APND

$APND_8 = \langle \{e_0, e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9\}, \{a, b, c\}, \{A, B, Z_0\}, \delta_8, e_0, Z_0, \{e_9\} \rangle$ Ver el siguiente diagrama de un AP para δ_8

δ_9 :



Lenguajes aceptados por los autómatas de pila

Una cadena $w \in A^*$ es aceptado por $AP = \langle E, A, P, \delta, e_0, Z_0, F \rangle$

Si y sólo si $\langle e_0, w, Z_0 \rangle \rightarrow \langle e_r, \epsilon, \alpha \rangle$

El autómata de pila AP , comienza en el estado e_0 , con pila vacía luego de leer toda la cadena w , llega a un estado $e_r \in F$, y en la pila queda cualquier cadena $\alpha \in P^*$

El lenguaje aceptado por AP , es el conjunto de todas las cadenas que son aceptadas por AP .

$$L(AP) = \{w | \langle e_0, w, Z_0 \rangle \rightarrow \langle e_r, \epsilon, \alpha \rangle \text{ y } w \in A^* \text{ y } e_r \in F \text{ y } \alpha \in P^*\}$$

Los lenguajes aceptados por los AP^* se denominan lenguajes libres de contexto.

Autómata de pila traductor AP_T

Un AP_T es simplemente como una tupla de 9 elementos.

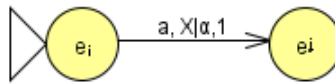
$$AP_T = \langle E, A, P, \delta, e_0, Z_0, F, \lambda, S \rangle$$

Donde $E, A, P, \delta, e_0, Z_0, F$ ya se definieron anteriormente y se agregan los componentes.

S : Alfabeto o conjunto finito como $\lambda: E \times (A \cup \{\epsilon\}) \times P \rightarrow S^*$

La λ está definida siempre que δ está definida.

En el diagrama de transición de AP_T puede describirse como una extensión de la notación usada para AP . Si existe $\delta(e_i, a, x) = t$; luego el arco queda rotulado según se observa en el siguiente diagrama para: $a, X | \alpha, 1$



Donde $e_i, e_j \in E$; $a \in A$; $x \in P$; $\alpha \in P^*$; $t \in S^*$

Función de transición para cadenas

El autómata solo define la traducción, si el autómata AP subyacente “acepta”. Es decir la traducción $T(w): A^* \rightarrow S^*$ asociado a AP, está definida como $T(w)$ es válida $\Leftrightarrow \langle e_0, w, Z_0 \rangle$

Ejemplo

$$L = \{0^i 1^{i+k} 2^k 3^{n+1} / i, k, n \geq 0\}$$

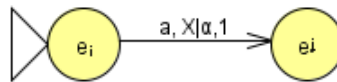
Traducir las cadenas de L $0^i 1^{i+k} 2^k 3^{n+1}$ como $a^{i+k} b^{2k} c^{3n}$

L_9 sólo se puede reconocer con un APD_T

$$APD_T = \langle \{e_0, e_1, e_2, e_3, e_4\}, \{0, 1, 2, 3\}, \{A, B, Z_0\}, \delta, e_0, Z_0, \{e_4\}, \lambda, \{a, b, c\} \rangle$$

Ver Diagrama de pila para δ y λ a continuación.

δ y λ :



Donde $e_i, e_j \in E$; $a \in A$; $x \in P$; $\alpha \in P^*$; $t \in S^*$

Función de transición para cadenas

El autómata solo define la traducción, si el autómata AP subyacente “acepta”. Es decir la traducción $T(w): A^* \rightarrow S^*$ asociado a AP, está definida como $T(w)$ es válida $\Leftrightarrow \langle e_0, w, Z_0 \rangle$

Ejemplo

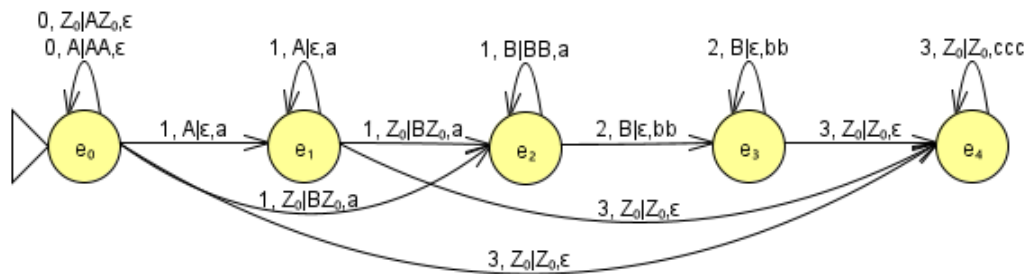
$$L = \{0^i 1^{i+k} 2^k 3^{n+1} / i, k, n \geq 0\}$$

Traducir las cadenas de L $0^i 1^{i+k} 2^k 3^{n+1}$ como $a^{i+k} b^{2k} c^{3n}$

L_9 sólo se puede reconocer con un APD_T

$$APD_T = \langle \{e_0, e_1, e_2, e_3, e_4\}, \{0, 1, 2, 3\}, \{A, B, Z_0\}, \delta, e_0, Z_0, \{e_4\}, \lambda, \{a, b, c\} \rangle$$

Ver Diagrama de pila para δ y λ a continuación.



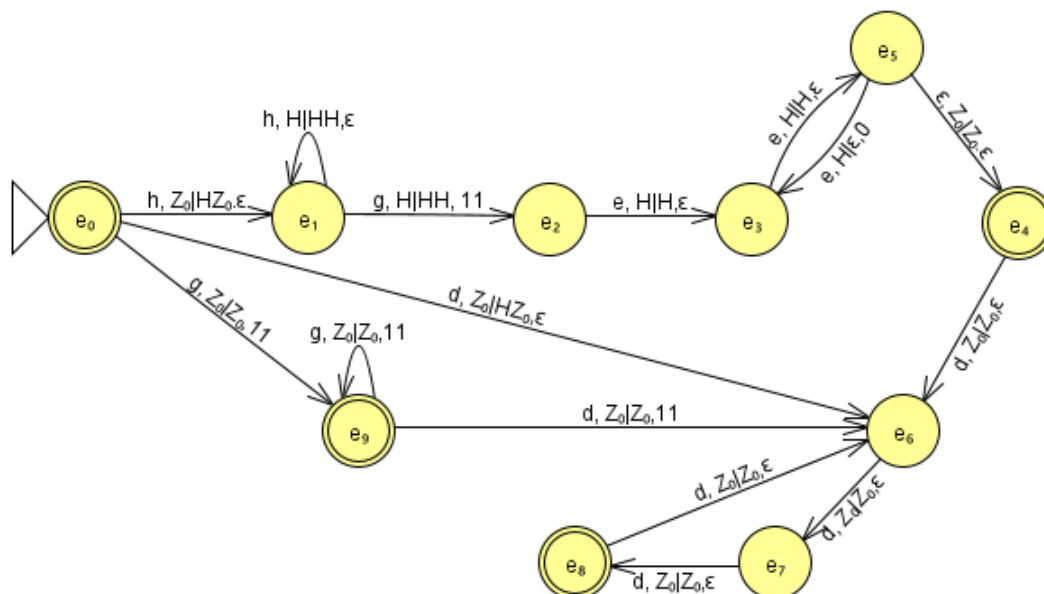
Último Ejemplo:

$$L = \{h^n g^j e^{2n} d^{3i} / i, j, n \geq 0\}$$

Traducir las cadenas de L $h^n g^j e^{2n} d^{3i}$ como $1^{2j} 0^n 2^i$

$$\text{APD}_T = \langle \{e_0, e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9\}, \{h, e, g, d\}, \{H, Z_0\}, \delta, e_0, Z_0, \{e_0, e_4, e_8, e_9\}, \lambda, \{0, 1, 2\} \rangle$$

Ver el diagrama de autómeta de pila para δ y λ

 $\delta y \lambda:$ 



UNIDAD V

Estructura general del lenguaje y gramáticas independientes del contexto.

El análisis jerárquico se denomina análisis sintáctico, este implica agrupar los componentes léxicos del programa fuente en frases gramaticales que el compilador utiliza para sintetizar la salida. La estructura jerárquica de un programa normalmente se expresa utilizando reglas recursivas. Las construcciones léxicas no requieren recursión mientras que las construcciones sintácticas suelen requerirla.

Normalmente se reconocen los identificadores por el simple examen del flujo de entrada esperando hasta encontrar un carácter que no sea ni letra ni dígito y agrupando después todas las letras y dígitos encontrados hasta este punto en un componente léxico identificador. Los caracteres así agrupados se registran en una tabla llamada tabla de símbolos.

Descripción de árboles sintácticos.

Un árbol sintáctico es una representación objetiva y compacta del análisis sintáctico en el que los operadores aparecen como los nodos interiores (*,+,-,/) y los operandos (datos) de un operador son los hijos del nodo para ese operador

Función del análisis sintáctico.

Analizar sintácticamente una *tira* o cadena de tokens no es más que encontrar para ella el árbol sintáctico o de derivación que tiene como raíz el axioma de la gramática, y como nodos terminales la sucesión ordenada de símbolos que componen la cadena analizada. En caso de no existir este árbol sintáctico, la cadena no pertenecerá al lenguaje, y el analizador sintáctico ha de emitir el correspondiente mensaje de error.

Existen dos formas de analizar sintácticamente una cadena:

- ***Análisis descendente:*** Partiendo del axioma inicial de la gramática se va descendiendo, utilizando las derivaciones izquierdas, hasta llegar a construir la cadena analizada.



- **Análisis ascendente:** Se va construyendo el árbol desde sus nodos terminales. Es decir, se construye desde los símbolos de la cadena hasta llegar al axioma de la gramática. En este caso, se emplean normalmente las derivaciones más a la derecha hasta la localización de la raíz.

Los principales métodos de análisis sintáctico son:

- Análisis descendente:
 - Análisis descendente con retroceso.
 - Análisis de gramáticas **LL**.
- Análisis ascendente:
 - Análisis ascendente con retroceso.
 - Análisis de gramáticas de precedencia simple.
 - Análisis de gramáticas de precedencia generalizada.
 - Análisis de gramáticas por el método mixto.
 - Análisis de gramáticas de precedencia de operador.
 - Análisis de gramáticas **LR**

Los *análisis con retroceso* se basan en la prueba sistemática de todas las alternativas posibles, dando marcha atrás tan pronto como se detecte que el camino seguido es erróneo. Pueden usarse para cualquier gramática de contexto libre, aunque tienen tres grandes inconvenientes: *Primero*, emplean mucho más tiempo para el análisis que los demás analizadores, dependiendo éste incluso de la ordenación de las reglas gramaticales; *Segundo*, no dan un buen diagnóstico de los errores que encuentran; *Tercero*, complican la generación de código cuando ésta se realiza al par que el análisis sintáctico.

Los métodos más eficientes de análisis (tanto ascendente como descendente) no funcionan para todas las gramáticas de contexto libre, sino sólo para las gramáticas que cumplen unas determinadas condiciones. Afortunadamente, en la mayoría de los casos, pueden encontrarse para los lenguajes de programación gramáticas de tipo **LL** o **LR** que los generen.

Para representar el árbol sintáctico que conduce hasta una cadena se asigna a cada regla de la gramática un número. Se define el *parse* como la secuencia ordenada de números (de reglas) aplicadas para construir dicho árbol.

Hay dos tipos de *parse*, que son:

- El **parse-izquierdo**: Son los números de las reglas de derivación izquierda utilizadas para generar la cadena a partir del axioma, por tanto, correspondiente a un *análisis descendente*.
- El **parse-derecho**: Son los números de las reglas de derivación derecha utilizadas para generar la cadena a partir del axioma, en orden inverso. El tomar el orden inverso viene condicionado por ser el análisis ascendente el que normalmente utiliza las reglas de derivación derecha, con lo que el orden en el que aparecen al realizar el análisis es **invertido**.

Ejemplos:

Dada la gramática

1. $E \rightarrow T$
2. $E \rightarrow T + E$
3. $T \rightarrow F$
4. $T \rightarrow F * T$
5. $F \rightarrow a$
6. $F \rightarrow b$
7. $F \rightarrow (E)$

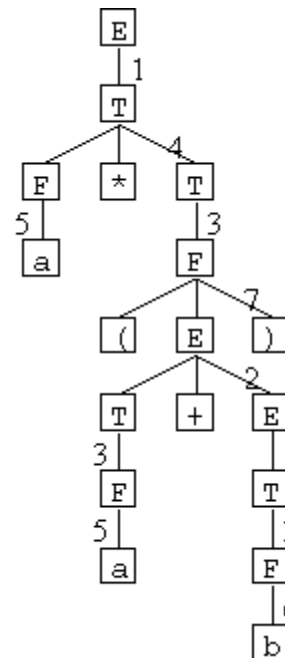
y la sentencia "a*(a+b)"

El parse izquierdo es:

1-4-5-3-7-2-3-5-1-3-6

y el derecho:

5-5-3-6-3-1-2-7-3-4-1



Simultáneamente a la fase de análisis sintáctico, además de reconocer las secuencias de tokens, y analizar su estructura, pueden realizarse una serie de tareas adicionales, como:

- 1) Recopilar información de los distintos tokens y almacenarla en la tabla de símbolos.
- 2) Realizar algún tipo de análisis semántico, tal como la comprobación de tipos.
- 3) Generar código intermedio.
- 4) Avisar de los errores que se detecten



5.1 Análisis Sintáctico Descendente Predictivo y gramáticas LL.

ANÁLISIS SINTÁCTICO DESCENDENTE

El análisis sintáctico descendente recursivo es un método en el que se ejecuta un conjunto de procedimientos recursivos para procesar la entrada. A cada elemento no terminal de la gramática se le asocia un procedimiento. Aquí, se considera una forma especial de análisis sintáctico descendente recursivo, llamado análisis sintáctico predictivo, en el que el símbolo de pre-análisis determina sin ambigüedad el procedimiento seleccionado para cada elemento no terminal. La secuencia de procedimientos llamados en el procesamiento de la entrada define implícitamente un árbol de análisis sintáctico para la entrada.

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

El símbolo de preanálisis guía la selección de la producción que desea divisar. Si el lado derecho de una producción empieza con un componente léxico, entonces la producción se usa cuando el símbolo de preanálisis coincide con el componente léxico.

Se puede considerar el análisis sintáctico descendente como un intento de encontrar una derivación por la izquierda para una cadena de entrada. También se puede considerar un intento de construir un árbol de análisis sintáctico para la entrada comenzando desde la raíz y creando los nodos del árbol en orden previo.

En muchos casos, escribiendo con cuidado una gramática, eliminando su recursión por la izquierda, se puede obtener una gramática analizable con un analizador sintáctico predictivo. Para construir un analizador sintáctico predictivo, se debe conocer, dado el símbolo actual A de entrada y el no terminal A a expandir.

Ejemplo: Considérese la siguiente gramática:

$S \rightarrow cAd$

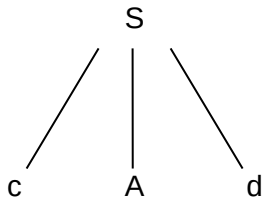
$A \rightarrow Ab \mid a$



Para construir un árbol de análisis sintáctico descendente para la cadena

$$S = cAd$$

Primero se forma un árbol con un solo nodo etiquetado con el símbolo inicial (S). Un apuntador a la entrada que apunta a C el primer símbolo de w. Después se utiliza la primera producción de S para expandir el árbol.



Se empareja o se compara la hoja situada más a la izquierda, etiquetada con c, con el primer símbolo de la cadena de entrada y a continuación se aproxima el apuntador de la entrada al siguiente carácter, y se compara con la siguiente hoja etiquetada con A. Entonces se puede expandir o derivar A utilizando la primera alternativa para obtener un nuevo árbol. Como ya se tiene una concordancia para el segundo símbolo de la entrada se lleva el apuntador de entrada al tercer símbolo y se compara con la hoja siguiente, etiquetada con b. Como b no concuerda con d, se indica que fallo y se regresa a A para saber si existe alternativa de A que no se haya intentado.

Al regresar a A Se debe restablecer el apuntador de entrada a la posición 2, debe almacenar el valor del apuntador a la entrada en una variable local para evitar repeticiones innecesarias. Se intenta a continuación la segunda alternativa de A y nuevamente obtenemos un árbol distinto.

Se empareja la hoja a con el segundo símbolo de la cadena de entrada, si se encuentra concordancia, se busca la siguiente hoja a la derecha para hacer la comparación del siguiente símbolo. Si esta no existe se considera la hoja derecha, pero de un nivel superior. Como ya se ha producido el árbol de análisis sintáctico para la cadena de entrada se separa y se avisa el éxito del análisis descendente.

Una gramática recursiva por la izquierda puede hacer que un analizador sintáctico por descenso recursivo, entre en un lazo infinito. Es decir, cuando se intenta expandir A, puede que de nuevo se esté intentando expandir A sin haber consumido ningún símbolo de entrada.

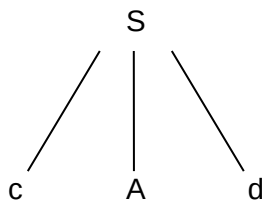
Análisis Sintáctico Descendente Recursivo

El análisis sintáctico descendente busca encontrar una derivación por la izquierda para una cadena de entrada. Se puede considerar como un intento por construir un árbol de análisis sintáctico para la entrada comenzando desde la raíz y creando los nodos del árbol hacia abajo del árbol.

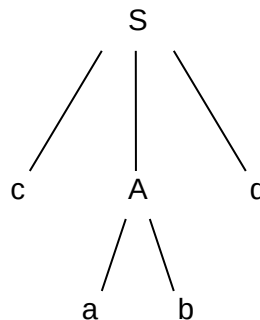
El análisis sintáctico descendente recursivo permite re-examinar la entrada (de forma recursiva). Se muestran los pasos que se siguen en el análisis sintáctico descendente para la gramática: $S \mid cAd \mid Aaba$

El algoritmo con el que se implementa esta gramática es:

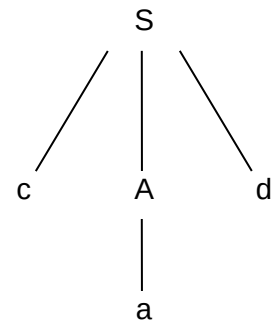
```
S() { if(token(c))  
      if(A())  
        if(token(d)) return TRUE; return FALSE;  
    A(){ if(token(ab)) return TRUE; if(token(a)) return TRUE; return FALSE;}
```



(1)



(2)



(3)

Una gramática recursiva por la izquierda puede hacer que un analizador sintáctico descendente recursivo entre en un ciclo infinito, por lo que es necesario eliminar la recursión de la gramática.



RECUSIÓN POR LA IZQUIERDA

Una gramática es recursiva por la izquierda si tiene un no-terminal A tal que exista una derivación $S \Rightarrow^* A$ para alguna cadena $cAd|ab|a$. Los métodos de análisis sintáctico descendente no pueden manejar gramáticas recursivas por la izquierda, por lo que se necesita una transformación que elimine la recursión por la izquierda.

La siguiente regla se utiliza para eliminar la recursión por la izquierda de una gramática:

Independientemente de cuántas producciones de A existan, se puede eliminar de ellas la recursión directa por la izquierda mediante la siguiente técnica.

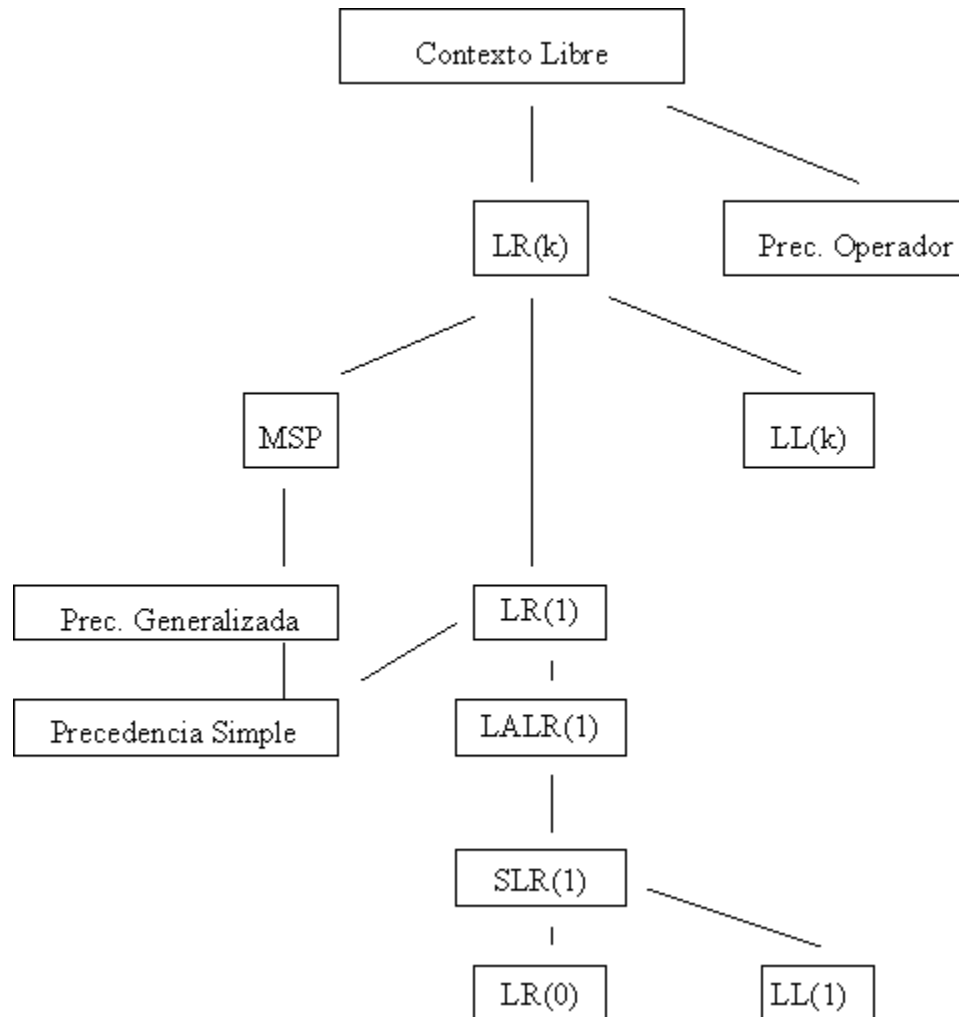
Método

Para encontrar la forma normal conjuntiva en su mínima expresión de una fórmula original se desarrolló un procedimiento para validar que la fórmula esté bien formada; si el procedimiento tiene éxito, construye un árbol de sintaxis, que es procesado (cambiando el contenido y añadiendo nuevos nodos) para reducir la fórmula a la forma normal conjuntiva en su mínima expresión. Como parte del proceso de reducción, el programa determina si la fórmula capturada es válida o es inconsistente.

El método aplicado se puede dividir en tres partes: Especificación de gramática, Validación de la fórmula bien formada y conversión a la forma normal conjuntiva.

Comparación entre los métodos de análisis LR Y LL.

Vamos a hacer una comparación no exhaustiva entre los distintos métodos de análisis sintáctico que se ha visto, atendiendo a distintos aspectos de los mismos. Algunas de las características que resumimos aquí han sido ya explicadas convenientemente dentro de su contexto.



Análisis sintáctico descendente, por la izquierda.

Como se ha visto, no todos los analizadores sirven para todas las gramáticas, ya que para poder construir las tablas de análisis correspondientes hay que imponer ciertas restricciones a la gramática. En este sentido, podemos establecer una clasificación entre las gramáticas que pueden ser analizadas según cada uno de los métodos:

Este grafo indica la inclusión de un tipo de gramática en otras. Sin embargo, no es en sí indicativo de la amplitud de la clase de gramáticas que abarca cada nodo en comparación con otro que le sea disjunto.



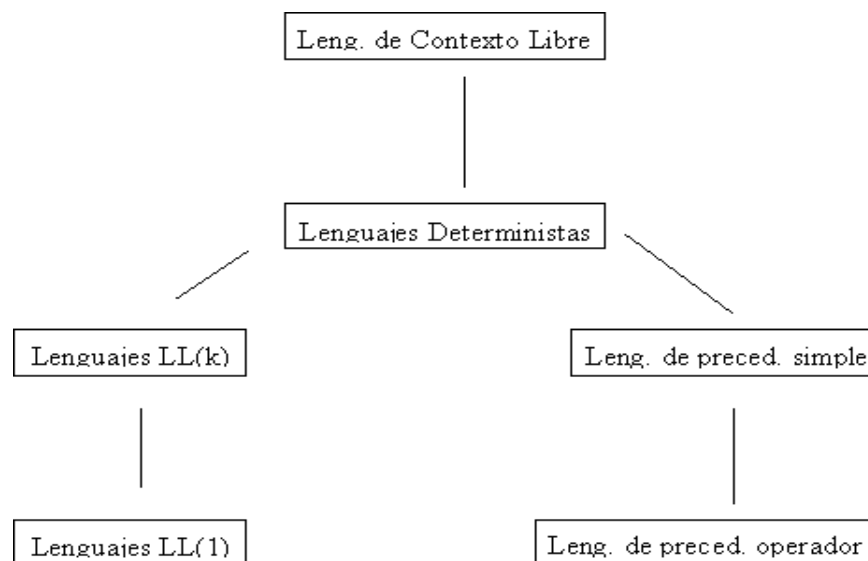
Entre todos los métodos existentes, el que cubre un mayor espectro de gramáticas en la práctica es sin duda el método LR. Los métodos basados en precedencia, por lo general, imponen unas condiciones muy estrictas a la gramática, por lo que en la actualidad son menos usados que los LR o LL.

En cuanto a los lenguajes, se ha demostrado que todos aquellos que pueden ser reconocidos mediante análisis LR(k), también pueden ser reconocidos mediante análisis LR (1), o mediante un análisis LALR (1) o SLR(1). Es decir, que para toda gramática que acepta un análisis LR(k), existe una gramática equivalente que acepta un análisis SLR (1). De hecho, existe un algoritmo de transformación que realiza dicha conversión (a estos lenguajes se les llama lenguajes deterministas, y son un subconjunto propio de aquellos que pueden ser generados por gramáticas de contexto libre).

Del mismo modo se ha demostrado que los lenguajes que admiten un análisis mediante precedencia generalizada pertenecen a esta clase, pero los lenguajes analizables mediante precedencia simple son un subconjunto propio de ella, y éstos a su vez incluyen a los lenguajes para los que existe una gramática que cumple las condiciones necesarias para un análisis de precedencia de operador.

Por otra parte, puede demostrarse que los lenguajes LL(k) son un subconjunto propio de los lenguajes LR(k), y que para todo k existen lenguajes que son LL(k+1) pero no LL(k).

El siguiente árbol resume la estructuración de los lenguajes:





Facilidad de construcción. La dificultad de construcción de las tablas de análisis ha quedado ya plasmada con suficiente claridad al explicar cada uno de los métodos. La construcción más laboriosa es sin duda la de las tablas LALR, LR y SLR, por este orden, seguidos de los análisis LL. Los métodos de precedencia suelen ser más simples de construir (especialmente el método de precedencia de operador) ya que la gramática es más sencilla, y no contiene producciones nulas. Sin embargo, tal como se indicó en el punto anterior, puede ser más difícil (si no imposible) la tarea de hallar la gramática que cumpla las restricciones necesarias. Éste es el principal problema del método LL.

Facilidad de depuración.

Debido a su complejidad, en las tablas LR se hace más difícil la resolución de las ambigüedades locales o de los pequeños ajustes finales necesarios para el correcto funcionamiento del analizador. Esta tarea es más fácil en los análisis basados en precedencia, y más aún en los análisis LL, dada la claridad con que resultan los programas escritos siguiendo este tipo de análisis.

Detección de errores.

En cuanto a la detección y recuperación de los errores, los mejores métodos son los LR y los LL, ya que detectan el error tan pronto como es posible, y posibilitan un correcto diagnóstico de los mismos. De entre ellos el mejor es el LR(1) canónico. Los métodos basados en relaciones de precedencia dan un diagnóstico muy pobre, y en algunos casos no detectan el error más que tras haber leído una serie de símbolos adicionales, con la dificultad que esto entraña para la adecuada recuperación del proceso de análisis.

Utilización de la memoria.

Todos los analizadores sintácticos pueden ser representados mediante tablas. Las más pequeñas son las de los métodos basados en precedencia de operador (más pequeñas aún si existen funciones de precedencia) que tienen tamaño t^2 , siendo t el número de símbolos terminales de la gramática. Los análisis de precedencia simple y el método mixto (MSP) requieren unas tablas de tamaño proporcional a $(n+t)^2$, siendo n el número de símbolos no terminales de la gramática. Las tablas de precedencia generalizada necesitan, por lo general, tablas mayores, aún en los casos en los que sólo se almacenen los elementos no nulos de las mismas.



Los análisis SLR(1) y LALR(1) utilizan tablas del mismo tamaño, mucho menores que las necesarias para el análisis LR(1) canónico de la misma gramática. En cuanto a los análisis LL(1), suelen generar tablas de longitud menor que las correspondientes LALR(1) o SLR(1). Sin embargo, por lo general, las gramáticas que se emplean para el análisis LL(1) contienen mayor número de producciones que las que definen el mismo lenguaje y puedan ser analizadas según LALR(1). Esto se debe a que la eliminación de la recursividad izquierda, o la factorización necesaria para encajar la gramática en la condición LL(1), introducen reglas adicionales, por lo que en la práctica se equipara la necesidad de memoria para un análisis LL(1) o LALR(1).

Velocidad del análisis.

En lo que se refiere a la rapidez del análisis los métodos LR y LL aventajan considerablemente a los métodos basados en precedencia, quedando entre medias de ambos el método mixto MSP. El método LL(1) es algo más lento que el LR(1) debido al mayor número de reglas de producción, y el consiguiente mayor número de movimientos necesarios para alcanzar la cadena final a partir del axioma.

Dentro de cada uno de los métodos estudiados existen diferentes "trucos" para agilizar la ejecución de los programas, por lo que la comparación no debe hacerse tanto entre los métodos como entre determinadas aplicaciones de los mismos. Existen varios estudios prácticos en este sentido.

Gramáticas atribuidas

A diferencia del análisis sintáctico, en donde en la actualidad existe un marco teórico ampliamente aceptado, el tema del análisis semántico suele ser tratado de diversas formas. La teoría que seguiremos para la formulación del análisis semántico y la posterior generación de código se basa en las gramáticas atribuidas. Antes de dar una definición formal intentaremos dar una idea intuitiva de las mismas.

Una gramática atribuida, no es más que una gramática de contexto libre a cuyos símbolos X (terminales y no terminales) asociamos un conjunto de atributos. Cada atributo es como una variable que representa una determinada propiedad del símbolo X , y puede tomar un valor cualquiera de un determinado conjunto de valores posibles. Denotaremos a este atributo con un nombre precedido del símbolo al que está asociado: $X.a$, $X.valor$, $X.tipo$, etc.... Al conjunto de todos los atributos asociados al símbolo X se le llama *conjunto de atributos* de X , que denotaremos por $A(X)$.



Ejemplo: Sea la gramática que representa las asignaciones

- (1) ASIGNACION $\rightarrow$ VARIABLE igual EXPRESION
- (2) EXPRESION $\rightarrow$ EXPRESION OPERADOR numero
- (3) EXPRESION $\rightarrow$ numero
- (4) VARIABLE $\rightarrow$ identificador
- (5) OPERADOR $\rightarrow$ mas

Esta gramática genera el lenguaje que contiene las siguientes cadenas de tokens:
Identificador igual número identificador igual número más número identificador igual
número más número más número, etc.

Estas cadenas de tokens son formadas por el analizador lexicográfico a partir de un
fichero de entrada que contendría los lexemas correspondientes como ya vimos en
el capítulo dedicado a analizadores lexicográficos.

Por ejemplo:

Cuatro := 2.0 + 2

Alfa := 3 + 4.5 + 1.5

Vamos a considerar los atributos siguientes, cada uno de los cuales representa una
propiedad determinada del símbolo al que está asociado.

numero.tipo	Tipo del número (entero o real)
numero.valor	Valor que el número representa
identificador.tipo	Tipo del identificador. Es decir, tipo con el que dicho identificador ha sido definido
variable.tipo	Tipo de la variable que interviene en la sentencia de asignación
variable.valor	Valor que toma dicha variable
expresion.tipo	Tipo de una expresión
expresion.valor	Valor que obtenemos al evaluarla
operador.tipo	Clase de operación (suma entera o real)

Los atributos asociados a X tomarán una serie de valores determinados en cada
nodo del árbol sintáctico de una sentencia del lenguaje generado por la gramática
en el que esté situado el símbolo X.

Estos valores serán establecidos de acuerdo con un conjunto de reglas semánticas asociadas a alguna de las reglas sintácticas de la gramática de las que intervengan en el nodo. Al conjunto de reglas semánticas asociado a una regla sintáctica lo llamaremos $R(p)$. Las reglas semánticas vienen dadas en función de los atributos de los demás símbolos que componen la regla.

Ejemplo: Continuando con el ejemplo anterior, vamos a formar el árbol sintáctico correspondiente al análisis de la cadena de caracteres:

Cuatro: $= 2.0 + 2$

Tras realizar el análisis lexicográfico obtenemos la cadena de tokens:
identificador igual número más numero

Además, el analizador lexicográfico ha realizado las correspondientes acciones semánticas y nos da los valores de algunos de los atributos de los tokens (símbolos terminales de nuestro análisis). Estos atributos, como ya veremos, reciben el nombre de atributos intrínsecos.

Sean estos:

identificador.tipo = Real

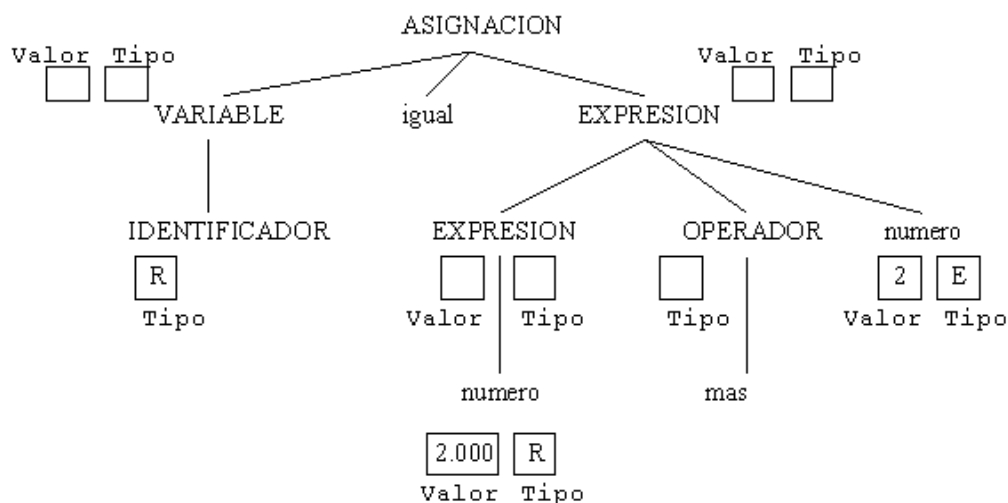
```
numero.tipo = Real
```

```
numero.valor = 2.000
```

```
numero.tipo = Entero
```

```
numero.valor = 2
```

Cada nodo del árbol estará ocupado por un símbolo de la gramática. Aquellos símbolos que tengan atributos asociados los representaremos con ellos:





Los conjuntos de reglas semánticas asociadas a cada regla sintáctica serían:

- (1) ASIGNACION --> VARIABLE igual EXPRESION
VARIABLE.valor := EXPRESION.valor
- (2) EXPRESION --> EXPRESION OPERADOR numero
OPERADOR.tipo:=Mayor_tipo(EXPRESION(2).tipo,numero.tipo)
EXPRESION(1).valor:=Suma(mas.tipo,EXPRESION(2).tipo,numero.tipo,
EXPRESION(2).valor,numero.valor)
EXPRESION(1).tipo:=OPERADOR.tipo
- (3) EXPRESION --> numero
EXPRESION.valor:=numero.valor
EXPRESION.tipo:=numero.tipo
- (4) VARIABLE --> identificador
VARIABLE.tipo:=identificador.tipo
- (5) OPERADOR --> mas
(No tiene ninguna regla asociada)

En donde la función Mayor_tipo se define como:

```
function Mayor_tipo (tipo1,tipo2) ---> tipo
begin
if (tipo1=Real or tipo2=Real) then tipo := Real;
else tipo := Entero;
end;
```

Y la función Suma como:

```
function Suma (tipo_suma,tipo1,tipo2,valor1,valor2) ---> valor
begin
if tipo_suma = Real then
  begin
    if tipo1 = Entero then Ent_Real(valor1);
    if tipo2 = Entero then Ent_Real(valor2);
    Suma := Suma_real (valor1,valor2);
  end;
else
  begin
    if tipo1 = Real then Real_Ent(valor1);
    if tipo2 = Real then Real_Ent(valor2);
    Suma := Suma_entera (valor1,valor2);
  end;
end;
```

Vamos a suponer que las funciones Suma_real y Suma_Entera, y los procedimientos Ent_Real y Real_Ent (que convierten un entero en un real, y viceversa) pertenecen ya al nivel inferior que es capaz de ejecutar la máquina.

Por lo general, pueden intervenir dos reglas en un nodo de un árbol sintáctico de una cadena del lenguaje, ocupado por el símbolo x :

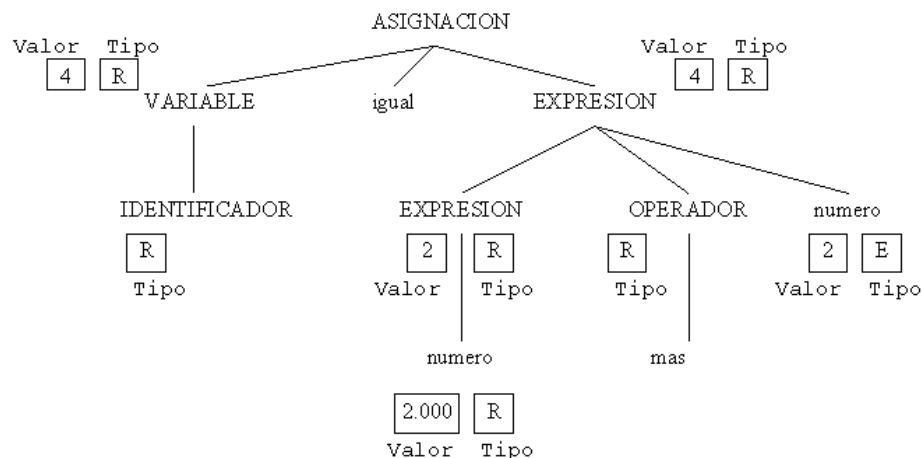
$$S \Rightarrow \dots \Rightarrow \alpha Y \beta \Rightarrow \alpha \tau X \gamma \beta \Rightarrow \alpha \tau \tau \gamma \beta \Rightarrow \dots t \quad (p) \ (q)$$

$$p: Y \dashrightarrow \tau X \gamma \quad X \dashrightarrow \tau$$

Algunos de los valores de los atributos del símbolo x en dicho nodo se calculan mediante reglas semánticas asociadas a la regla anterior (p), mientras que otros se calculan mediante reglas asociadas a la regla posterior (q). En el primero de los casos diremos que el atributo correspondiente a x es heredado, mientras que en el segundo caso diremos que el atributo es sintetizado. Al conjunto de atributos heredados lo llamaremos $AH(x)$, y al conjunto de atributos derivados $AS(x)$. Los conjuntos $AH(x)$ y $AS(x)$ son disjuntos.

Por último, y para garantizar la corrección semántica de una sentencia se establecen unas condiciones que han de cumplir los atributos de cada uno de los símbolos de una misma regla sintáctica. Llamamos a este conjunto de condiciones $B(p)$.

Ejemplo: Continuando con el ejemplo anterior, aplicando las reglas correspondientes en cada caso podemos calcular los atributos de todos los símbolos en todos los nodos del árbol sin más que aplicar la regla semántica que necesitamos. Nótese que sólo una regla semántica es aplicable para el cálculo de un determinado atributo de un símbolo en un nodo. El árbol sintáctico quedaría:





Dependiendo de la producción a la que esté asociada la regla que empleamos para calcular los atributos, podemos clasificar a estos en *atributos sintetizados* y *atributos heredados*. Se va a considerar que los símbolos terminales sólo tienen atributos sintetizados, que son proporcionados por el analizador lexicográfico. Los siguientes atributos son sintetizados:

EXPRESION.tipo
EXPRESION.valor
VARIABLE.tipo
numero.tipo
numero.valor
identificador.tipo

Y los siguientes serían atributos heredados:

VARIABLE.valor
OPERADOR.tipo

Por último, podemos establecer algunas condiciones para que las sentencias analizadas sean correctas. En este caso, sólo impondremos una condición asociada a la primera regla:

$B(1) := (VARIABLE.tipo = EXPRESION.tipo)$

A continuación vamos a definir formalmente el concepto de gramática atribuida:

Definición: Una *gramática atribuida* es una cuádrupla,

$GA = \{G, A, R, B\}$, en donde:

- $G = \{N, T, P, S\}$ es una gramática de contexto libre.
- $A = \bigcup_x A(x)$ es el conjunto de atributos asociados a cada símbolo x de la gramática (terminal o no terminal), tales que cumplen que: "para todo símbolo x e y (terminal o no terminal) $A(x) \cap A(y) = \emptyset$ ". Es decir, los atributos de cada símbolo son disjuntos.



- Dado un símbolo X , vamos a denotar los atributos asociados a él como $X.a$, $X.b$, etc..., o con las letras minúsculas v, w para casos generales.

- $R = \bigcup R(p)$ es un conjunto finito de reglas, o acciones semánticas, asociadas a cada una de las reglas sintácticas de la gramática G . Sea la regla p de la forma:

$$X_0 \rightarrow X_1 X_2 \dots X_n$$

Entonces, las reglas semánticas asociadas a esta regla sintáctica son de la forma $v := f(w_1, w_2, \dots, w_k)$, en donde:

- f es una función.
- $v, w_1, \dots, w_n$ pertenecen a $\bigcup A(X_i)$. Es decir, son atributos de los símbolos que componen la regla p .

- $B = \bigcup B(p)$ es un conjunto finito de condiciones asociadas a cada una de las reglas sintácticas de la gramática G . Sea la regla p de la forma $X_0 \rightarrow X_1 X_2 \dots X_n$. Entonces, las condiciones son de la forma $g(w_1, w_2, \dots, w_k)$, donde:

- g es una función booleana
- $w_1, w_2, \dots, w_k$ pertenecen a $\bigcup A(X_i)$

Cuando las funciones de las reglas semánticas pueden tener efectos laterales, tales como imprimir un valor, introducir una variable en una tabla de símbolos, etc..., la gramática atribuida pasa a ser una *definición atribuida*, también denominada *definición dirigida por sintaxis*.

Atributos sintetizados y heredados: Si p es una regla sintáctica de la forma:

$$p: X_0 \rightarrow X_1 X_2 \dots X_n$$

entonces se define el conjunto de *atributos calculados* $AC(p)$ mediante la regla p , como:

$$AC(p) = \{X_i.a / X_i.a = f(\dots) \text{ pertenece a } R(p)\}$$



Dada una gramática atribuida, se dice que un atributo asociado a un símbolo x es *sintetizado* si existe una regla sintáctica de la forma $x \rightarrow \tau$ y una regla semántica que lo calcula a partir de los atributos de los símbolos del consecuente de dicha regla. Al conjunto de los atributos sintetizados asociados a x lo denotaremos mediante $AS(x)$.

$$AS(X) = \{X.a \mid \text{Existe } X \rightarrow \tau \text{ perteneciente a } P \text{ y } X.a \text{ pertenece a } AC(p)\}$$

Dada una gramática atribuida, se dice que un atributo asociado a un símbolo x es *heredado* si existe una regla sintáctica de la forma $Y \rightarrow \alpha x \beta$, y una regla semántica que lo calcula a partir de los atributos de los demás símbolos que forman la regla. Al conjunto de los atributos heredados asociados a x lo denotaremos mediante $AH(x)$.

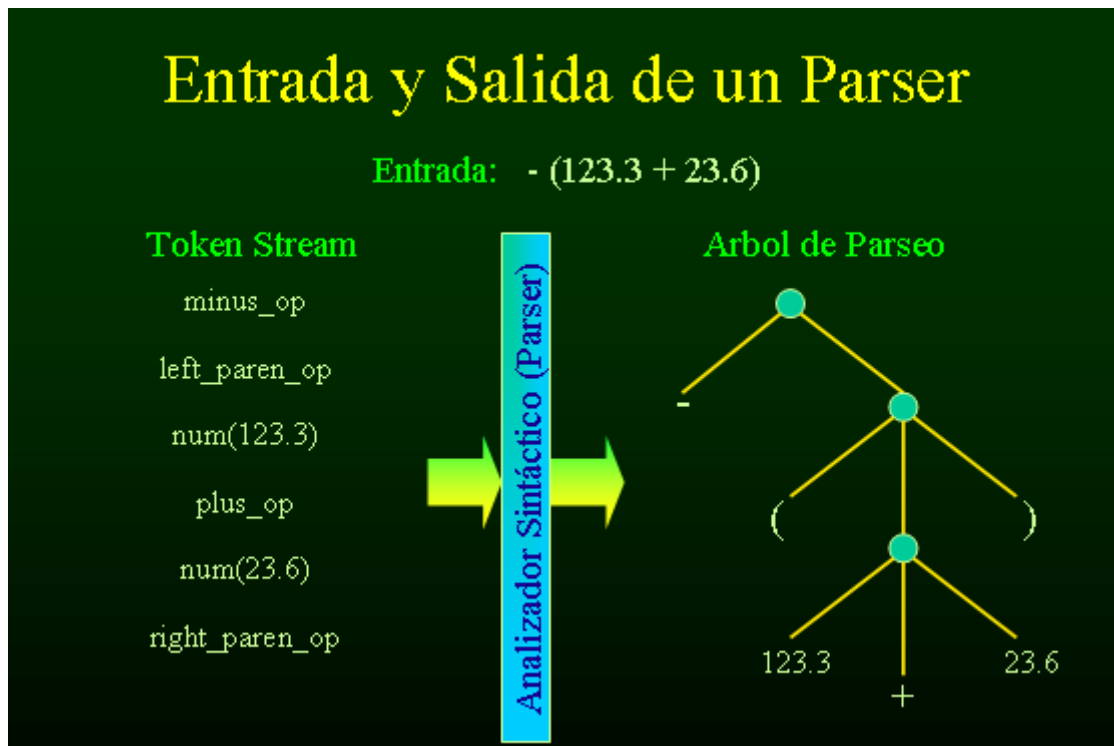
$$AH(X) = \{X.a \mid \text{Existe } Y \rightarrow \alpha X \beta \text{ perteneciente a } P \text{ y } X.a \text{ pertenece a } AC(p)\}$$

Gramática atribuida completa: Se dice que una gramática atribuida es *completa* si se cumplen las cuatro condiciones siguientes para todo símbolo x .

1. Para toda regla $p: X \rightarrow \tau$, $AS(x)$ está incluido en $AC(p)$.
2. Para toda regla $q: Y \rightarrow \alpha x \beta$, $AH(x)$ está incluido en $AC(q)$.
3. $AS(x) \cup AH(x) = A(x)$
1. $AS(x) \cap AH(x) = \emptyset$

De estas condiciones se deduce que el axioma de la gramática (S) no puede tener más que atributos sintetizados (es decir, $AH(S) = \emptyset$). Los atributos de los símbolos terminales se consideran como sintetizados (también se les denomina *atributos intrínsecos*, ya que tienen un valor en sí mismos y no es necesario calcularlo).

Gramáticas atribuidas bien definidas: Una gramática atribuida se dice que está *bien definida* si para toda sentencia de $L(G)$ todos los atributos pueden ser computados. A este proceso se le denomina *evaluación de la gramática*. Una sentencia del lenguaje se dice que está *correctamente atribuida* si se verifican todas las condiciones $B(p)$ asociadas a las reglas que la generan.



Especificación de la gramática

Para interpretar la fórmula capturada es necesario primero establecer cuales son todos los elementos que pueden formar una fórmula (elementos léxicos) y cuál es la forma en que pueden agruparse para dar tener significado (elementos sintácticos).



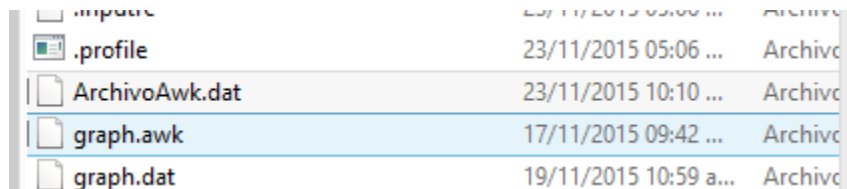
5.2 Herramienta automatizada para el desarrollo de un analizador sintáctico.

La mejor herramienta automatizada que consideramos para desarrollar un analizador sintáctico y semántico es el lenguaje interprete AWK. Mismo que se introdujo desde el desarrollo de un analizador léxico. A continuación mostramos un

“AWK sobre la marcha ”

PRIMER PROGRAMA EN AWK. Supongamos que tenemos un archivo llamado ArchivoAwk.dat con el contenido siguiente:

```
Jose 4.00 0
Pedro 7.00 1
Rosa 23.30 5
Maria 28.10 6
Juan 29.32 0
```



ArchivoAwk.dat contiene los nombres de 5 personas (Jose, Pedro, Rosa, Maria, Juan), el salario por hora que reciben (4.00, 7.00, 23.30, 28.10, 29.32) y el número de horas que han trabajado (0, 1, 5, 6, 0).

Supongamos que queremos imprimir el nombre y el salario total (número de horas por el salario por hora) de todas las personas que laboraron más de 0 horas. Ese es el tipo de trabajo para el que AWK está hecho. Basta con escribir lo siguiente en la consola:

```
awk '$3 > 0 {print $1, $2 * $3}' ArchivoAwk.dat
```

Y se obtendrá como salidas

Pedro 7

Rosa 116.5

Maria 168.6



La instrucción consta de 3 elementos:

- 1) una llamada al comando Unix AWK: Basta con imprimir el comando awk
- 2) un programa Todo lo que está contenido dentro de las comillas simples ('') es el programa en AWK
- 3) el archivo de entrada. El archivo se enlaza simplemente escribiendo su nombre o ruta completa

```
awk '$3 > 0 {print $1, $2 * $3}' ArchivoAwk.dat
```

La primera parte es un condicional. En awk : `$3 > 0`

Que quiere decir...

¿Es el valor de la columna 3 mayor que 0?

Para hacer referencia a una columna se usa el signo de pesos \$.

Así, \$3 hace referencia a la columna 3, \$1 hace referencia a la columna 1; \$0 **hace referencia a TODA la línea**. Lo que viene a continuación (encerrado entre llaves) es lo que ha de realizarse en caso de que la condición se cumpla:

```
{print $1, $2 * $3}'
```

La instrucción print (imprimir, en inglés) es clara. Se debe imprimir algo, pero.

¿Qué se imprime? Lo que viene justo a continuación, es decir, el valor de la **columna 1**, o sea el **nombre del trabajador y el salario**, el cual es el **producto del salario por hora (columna 2) por el número de horas (columna 3)**.

Columna 1 (nombre)	Columna 2 (salario)	Columna 3 (numero de hrs)
Pedro	7.00	1
Rosa	23.30	5
Maria	28.10	6



En AWK el signo para indicar **multiplicación es el asterisco (*)**. La coma que aparece justo después del número 1, indica que ha de **dejarse un espacio** en blanco entre el valor de la columna 1 y el producto de las columnas 2 y 3.

Resolviendo el producto de Columna 2 * Columna 3 queda :

Pedro	7.00
Rosa	116.5
Maria	168.6

```
Gabriela@GABBA ~  
$ awk '$3 > 0 {print $1, $2 * $3}' ArchivoAwk.dat  
Pedro 7  
Rosa 116.5  
Maria 168.6  
  
Gabriela@GABBA ~  
$ .....
```

AWK repite **la instrucción entre corchetes para todas las líneas del archivo**, por lo tanto el resultado desplegado arriba muestra, como queríamos, el salario de los trabajadores que sí recibirán una paga.

¿Qué pasa si más bien queremos imprimir los nombres de las personas que no recibirán salario?

Basta con escribir lo siguiente:

```
awk '$3 == 0 {print $1}' ArchivoAwk.dat
```

De nuevo, esta instrucción consta de tres partes: La llamada a awk, el programa y el archivo de entrada

La parte **condicional** es .

En awk :

```
'$3 == 0 {print $1}'
```

Que quiere decir ...

¿Es el valor de la columna 3 igual a 0?



Mostrando en pantalla.

```
Gabriela@GABBA ~  
$ awk '$3 == 0 {print $1}' ArchivoAwk.dat  
Jose  
Juan
```

COMANDOS USADOS EN ESTA PRÁCTICA

<u>COMANDO</u>	<u>FUNCIÓN</u>
\$	Referencia a una columna se usa el signo de pesos
\$0	Hace referencia a TODA la línea.
(*)	Multiplicación es el asterisco
(,)	Indica que ha de dejarse un espacio en blanco entre el valor de una columna y otra
(')	El programa va encerrado entre las comillas simples
{ }	Entre corchetes van las instrucciones a ejecutar

Bien, después de este programa básico en AWK podemos decir que :

La estructura de uso de AWK está compuesta por :

```
awk [condicion] { comandos }
```

Donde:

- **[condición]** representa una condición de líneas o parámetros.
- **{comandos}** una serie de órdenes a ejecutar como:

<u>SIGNO</u>	<u>FUNCIÓN</u>
\$	Referencia a una columna se usa el signo de pesos
\$0	Hace referencia a TODA la línea.
\$1-\$N	Mostrar los campos (columnas) de la línea especificados.
FS	Field Separator (Espacio o TAB por defecto)
NF	Número de campos (fields) en la línea actual
NR	Número de líneas (records) en el stream/fichero a procesar.
OFS	Output Field Separator (" ").
ORS	Output Record Separator ("\n").
RS	RS → Input Record Separator ("\n").



BEGIN	RS → Input Record Separator ("\\n").
END	Define sentencias a ejecutar tras acabar el procesado.
Length	Longitud de la línea en proceso.
FILENAME	Nombre del fichero en procesamiento.
ARGC	Número de parámetros de entrada al programa.
ARGV	Valor de los parámetros pasados al programa.

Las funciones utilizables en las condiciones y comandos son, entre otras:

<u>FUNCIONES PREDEFINIDAS</u>
Close (fichero_a_reiniciar_desde_cero) Cos (x) sin(x) index() int(num) length(string) substr(str,pos,len) tolower() toupper() system(orden_del_sistema_a_ejecutar) printf()

Los diferentes tipos de operadores:

<u>OPERADORES ARITMÉTICOS</u>
+ Suma - Resta * Multiplicación / División % Resto ^ Exponenciación Esp Concatenación



OTRO TIPO DE OPERADORES

!	Negación
A ++	Incrementar positivamente en una unidad
A --	Incrementar negativamente en una unidad
A += N	Incrementar positivamente en N unidades
A -= N	Incrementar negativamente en N unidades
A *= N	Multiplicar por N
A /= N	Dividir por N
A %= N	Reemplazar por el resto de la división por N
>	Mayor que
>=	Mayor o igual que
<	Menor que
<=	Menor o igual que
==	Igual a
!=	Distinto de
?:	Formato alternativo de decisión (if/else)

Ahora veamos otro ejemplo para aplicar estas funciones y operadores...

- Creamos el archivo **poema.txt**

```
Archivo Edición Formato Ver Ayuda
Uno llegar e incorporarse al día
Dos respirar para subir la cuesta
Tres no jugarse en una sola apuesta
Cuatro escapar de la melancolía

Cinco aprender la nueva geografía
Seis no quedarse nunca sin la siesta
Siete el futuro no será una fiesta
Y ocho no amilanarse todavía

Nueve vaya a saber quién es el fuerte
Diez no dejar que la paciencia ceda
Once cuidarse de la buena suerte

Doce guardar la última moneda
Trece no tutearse con la muerte
Catorce disfrutar mientras se pueda.
```



Si queremos visualizar el contenido en el prompt lo que tenemos que hacer es usar el comando **cat fichero *.txt *.dat** y nos imprimirá el documento de esta manera.

```
Gabriela@GABBA ~  
$ cat poema.txt  
Uno llegar e incorporarse al día  
Dos respirar para subir la cuesta  
Tres no jugarse en una sola apuesta  
Cuatro escapar de la melancolía  
  
Cinco aprender la nueva geografía  
Seis no quedarse nunca sin la siesta  
Siete el futuro no ser una fiesta  
Y ocho no amilanarse todavía  
  
Nueve vaya a saber quién es el fuerte  
Diez no dejar que la paciencia ceda  
Once cuidarse de la buena suerte  
  
Doce guardar la última moneda  
Trece no tutearse con la muerte  
Catorce disfrutar mientras se pueda.
```

Y podemos corroborar que nos muestra el mismo contenido que nuestro **archivo txt**.

- Ahora veamos para que se usa el famoso comando **NR**

```
$ awk '{print NR, $0}' poema.txt  
1 Uno llegar e incorporarse al día  
2 Dos respirar para subir la cuesta  
3 Tres no jugarse en una sola apuesta  
4 Cuatro escapar de la melancolía  
5  
6 Cinco aprender la nueva geografía  
7 Seis no quedarse nunca sin la siesta  
8 Siete el futuro no ser una fiesta  
9 Y ocho no amilanarse todavía  
10  
11 Nueve vaya a saber quién es el fuerte  
12 Diez no dejar que la paciencia ceda  
13 Once cuidarse de la buena suerte  
14  
15 Doce guardar la última moneda  
16 Trece no tutearse con la muerte  
17 Catorce disfrutar mientras se pueda.
```

- En AWK, NR hace referencia al número de renglón y \$0 hace referencia a toda la línea.
- Lo que hace que nos despliegue del lado izquierdo las líneas del contenido del texto numeradas, en este caso son 17.



Y si ahora aplicamos una operación de búsqueda usando **/cadena/** → **Búsqueda de cadena.**

Para este ejemplo haré que busque la palabra “en” y desplegará.

```
Gabriela@GABBA ~  
$ awk "/en/" poema.txt  
Tres no jugarse en una sola apuesta  
Cinco aprender la nueva geografia  
Nueve vaya a saber quien es el fuerte  
Diez no dejar que la paciencia ceda  
Once cuidarse de la buena suerte  
Catorce disfrutar mientras se pueda.
```

Y si juntamos la operación de búsqueda y el comando NR

¿Qué pasará? Tecleamos lo siguiente

```
awk "/en/" '{print NR, $0}' poema.txt
```

```
Gabriela@GABBA ~  
$ awk "/en/" '{print NR, $0}' poema.txt  
3 Tres no jugarse en una sola apuesta  
6 Cinco aprender la nueva geografia  
11 Nueve vaya a saber quien es el fuerte  
12 Diez no dejar que la paciencia ceda  
13 Once cuidarse de la buena suerte  
17 Catorce disfrutar mientras se pueda.
```

Nos muestra en que frases del texto aparece la palabra “en” indicando en que línea se encuentra. Veamos qué pasa si usamos el comando NF en el mismo texto de ejemplo poema.txt.

Tecleamos `awk '{ print $NF }' poema.txt`

Deberá imprimir la última palabra de cada línea y lo podemos comprobar comparando el archivo txt y lo que arrojó nuestro programa.

Archivo	Edición	Formato	Ver	Ayuda
<pre>Uno llegar e incorporarse al dia Dos respirar para subir la cuesta Tres no jugarse en una sola apuesta Cuatro escapar de la melancolia Cinco aprender la nueva geografia Seis no quedarse nunca sin la siesta Siete el futuro no sera una fiesta Y ocho no amilanarse todavia Nueve vaya a saber quien es el fuerte Diez no dejar que la paciencia ceda Once cuidarse de la buena suerte Doce guardar la ultima moneda Trece no tutearse con la muerte Catorce disfrutar mientras se pueda.</pre>				

```
Gabriela@GABBA ~
$ awk '{ print $NF }' poema.txt
dia
cuesta
apuesta
melancolia

geografia
siesta
fiesta
todavia

fuerte
ceda
suerte

moneda
muerte
pueda.

Gabriela@GABBA ~
$
Gabriela@GABBA ~
$
```

Les mostraré otro comando muy interesante `gsub` que es una función de sustitución. Nos servirá para reemplazar una palabra por otra.

Tecleamos

`awk '{ gsub(/en/, "###"); print }' poema.txt`

En este caso sustituiré la palabra “en” de nuevo por el signo de gato (#) para detectarlas más rápido, aunque también podemos usar letras y dígitos.

Veamos que nos imprime...

```
Gabriela@GABBA ~  
$ awk '{ gsub(/en/,"###"); print }' poema.txt  
Uno llegar e incorporarse al día  
Dos respirar para subir la cuesta  
Tres no jugarse ### una sola apuesta  
Cuatro escapar de la melancolía  
  
Cinco apr###der la nueva geografía  
Seis no quedarse nunca sin la siesta  
Siete el futuro no será una fiesta  
Y ocho no amilanarse todavía  
  
Nueve vaya a saber qui### es el fuerte  
Diez no dejar que la paci###cia ceda  
Once cuidarse de la bu###a suerte  
  
Doce guardar la última moneda  
Trece no tutearse con la muerte  
Catorce disfrutar mi###tras se pueda.
```

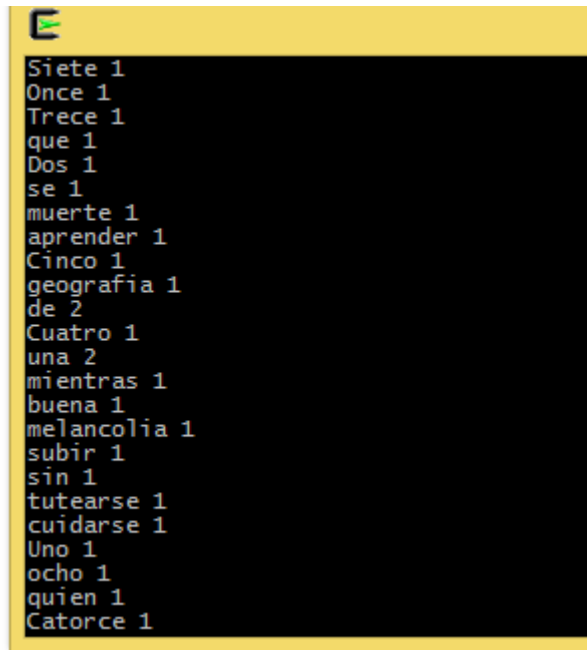
Un último ejemplo para concluir el tema de textos y líneas y ver más ejemplos con columnas. Hagamos la práctica del analizador léxico. En este ejemplo el programa imprimirá la lista de las palabras usadas junto con su frecuencia.

Crearemos el siguiente programa en el bloc de notas y lo guardaremos como **analizador.awk**

```
Archivo Edición Formato Ver Ayuda  
{  
    for (k=1; k<=NF; k++) {  
        cuenta[$k]++  
    }  
}  
  
END {  
    for (pal in cuenta) {  
        print pal, cuenta[pal]  
    }  
}
```

Seguiremos usando el mismo fichero de **poema.txt**, así que invoquemos el programa en awk, tecleando

\$ awk -f analizador.awk poema.txt



```
Siete 1
Once 1
Trece 1
que 1
Dos 1
se 1
muerte 1
aprender 1
Cinco 1
geografía 1
de 2
Cuatro 1
una 2
mientras 1
buena 1
melancolía 1
subir 1
sin 1
tutearse 1
cuidarse 1
Uno 1
ocho 1
quien 1
Catorce 1
```

Nos mostrará todas las palabras del texto y del lado izquierdo su frecuencia.

5.3 Diferentes notaciones para especificar la estructura o sintaxis de un lenguaje de programación y la tabla de símbolos.

Elementos Léxicos

Los símbolos que se aceptan como elementos de una fórmula bien formada se dividen en componentes léxicos (terminales).

Componentes léxicos de una fórmula bien formada

Los componentes léxicos son:

- Las conectivas conectivas lógicas.
- Los identificadores, que son cualquier letra del alfabeto reconocida por la computadora que representa una proposición, incluyendo al 0 para una contradicción y al 1 para una tautología.
- Los paréntesis, que permiten agrupar proposiciones compuestas y aclaran cuestiones de precedencia.
- La tabla muestra los elementos léxicos aceptados para una fórmula bien formada, así como símbolos que lo pueden representar.



Elementos Léxicos	Símbolos Aceptados
Doble Implicación	$\leftrightarrow =$
Implicación	$\rightarrow >$
Conjunción	$\& *$
Disyunción	$ +$
Negación	$\neg \sim -$
Paréntesis	$() [] \{ \}$
Identificador	$[a..z]$
Fórmula válida	1
Contradicción	0

Elementos léxicos de una fórmula bien formada.

Componentes sintácticos

Los componentes sintácticos se establecen construyendo la gramática de las fórmulas. En nuestro caso, la gramática sirve para:

- Especificar la forma correcta en que se espera la construcción de una fórmula.
- Establecer la precedencia y asociatividad de las conectivas lógicas.
- La gramática que se define para una fórmula bien formada es una gramática libre de contexto en BNF es la siguiente:

Exp $\rightarrow$ Exp " $\leftrightarrow$ " ExpOp1

| Exp "=" ExpOp1

| ExpOp1

ExpOp1 $\rightarrow$ ExpOp1 " $\rightarrow$ " ExpOp2

| ExpOp1 ">" ExpOp2

| ExpOp2

ExpOp2 $\rightarrow$ ExpOp2 "|" ExpOp3

| ExpOp2 "+" ExpOp3

| ExpOp3

ExpOp3 $\rightarrow$ ExpOp3 "&" ExpOp4



| ExpOp3 "*" ExpOp4

| ExpOp4

ExpOp4 -> "¬" ExpOp4

| "~" ExpOp4

| "-" ExpOp4

| ExpOp5

ExpOp5 -> Identificador

| ExpOp6

ExpOp6 -> "(" Exp ")"

| "[" Exp "]"

| "{" Exp "}"

El orden en que aparecen las reglas en la gramática establece la precedencia de operadores y la recursión por la izquierda indica la asociatividad por la izquierda de operadores con la misma precedencia. La gramática libre de contexto en BNF para formulas lógicas, sin recursión por la izquierda, en un parser recursivo descendente, es la siguiente:

Exp -> ExpOp1 ExpP

ExpP -> "<->" ExpOp1 ExpP

| "=" ExpOp1 ExpP | □

ExpOp1 -> ExpOp2 ExpP1

ExpP1 -> "->" ExpOp2 Exp1

| ">" ExpOp2 Exp1 | □

ExpOp2 -> ExpOp3 ExpP2

ExpP2 -> "|" ExpOp3 ExpP2



INSTITUTO POLITÉCNICO NACIONAL
Unidad Interdisciplinaria en Ingeniería y Ciencias Sociales y Administrativas.
APUNTES DE: Teoría de lenguajes y Compiladores
| "+" ExpOp3 ExpP2 | □



ExpOp3 -> ExpOp4 ExpP3

ExpP3 -> "&" ExpOp3 ExpP2

| "*" ExpOp3 ExpP2 | □

ExpOp4 -> "¬" ExpOp4

| "~" ExpOp4

| "-" ExpOp4

| ExpOp5

ExpOp5 -> Identificador

| ExpOp6

ExpOp6 -> "(" Exp ")"

| "[" Exp "]"

| "{" Exp "}"

Para simplificar la representación interna de las conectivas lógicas y las proposiciones, cada una tiene un equivalente como token.

Elementos Léxicos	Token
Doble Implicación	=
Implicación	>
Conjunción	&
Disyunción	
Negación	¬
Variable	[a..z]
Cláusula válida	1
Cláusula vacía	0

Tokens para elementos léxicos.

Preprocesamiento de la cadena de entrada

La fórmula para evaluar se recibe en una cadena de caracteres (string), la cual tiene un procesamiento previo antes de iniciar su validación. El reprocesamiento consiste en eliminar cualquier espacio en blanco de la cadena y convertir cualquier símbolo en minúsculas a mayúsculas. Es importante mencionar que esta conversión convierte al programa en un programa no sensible al caso (es decir, significa lo mismo "p" que "P"); si no existiera esta conversión la gramática diseñada si es sensible (representarían proposiciones diferentes "p" que "P").

En el proceso de validación de la fórmula bien formada se aplica conjuntamente el análisis léxico y sintáctico de la cadena recibida como fórmula. La figura muestra la forma que se enlazan estos procesos para evaluar la fórmula capturada.

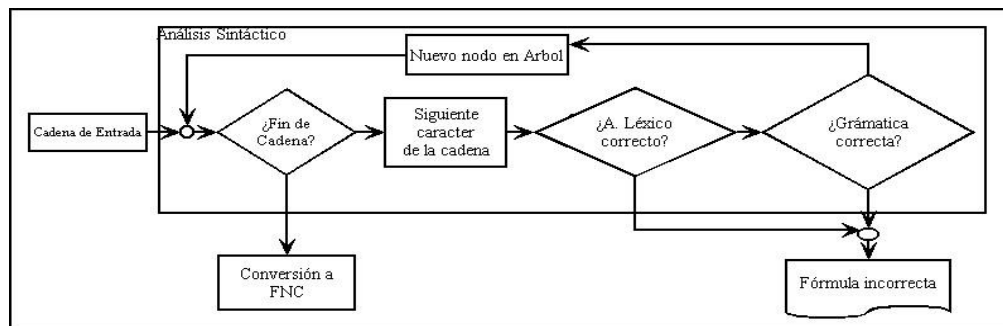


Diagrama a bloques del proceso de validación de fórmula bien formada.

El análisis sintáctico hace uso del análisis léxico para la verificación de los caracteres; si el análisis léxico es correcto, se evalúa cada regla de la gramática diseñada para fórmulas lógicas, para determinar si el carácter está colocado en una posición correcta. Si el carácter está colocado correctamente, de acuerdo a la gramática, entonces se crea un nuevo nodo en el árbol.

Análisis Sintáctico

El análisis sintáctico está compuesto de las siguientes funciones:

ExpOp1(), *ExpOp2()*, *ExpOp3()*, *ExpOp4()* y *ExpOp5()* se usan reconocer los operandos de la fórmula; al aceptar al operando, cada función adiciona un nodo al árbol.

ExpOp6() se utiliza para reconocer los paréntesis. Es importante aclarar que los paréntesis no se incluyen en el árbol ya que la estructura del árbol contiene implícitamente el orden de evaluación de los operadores.

ExpP() identifica al operando doble implicación ("=").

ExpP1() identifica al operando implicación (">").

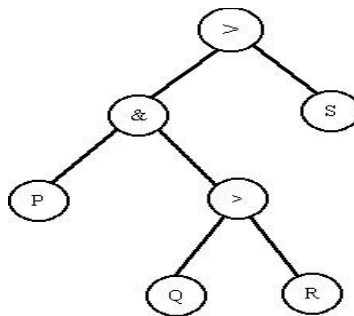
ExpP2() identifica al operando disyunción ("|").

ExpP3() identifica al operando conjunción ("&").

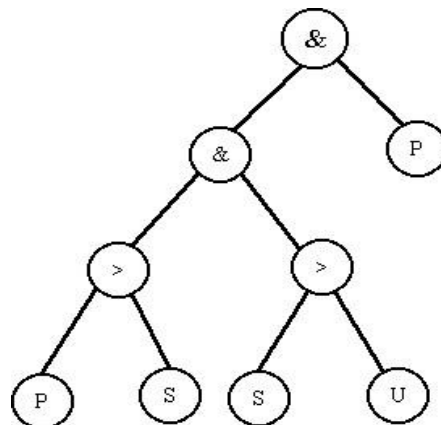
ExpP4() identifica al operando negación ("¬").

Árboles producidos

A continuación, se muestran algunos ejemplos de los árboles producidos por este paso de validación de una fórmula bien formada. La figura muestra el árbol generado para la fórmula $(P \& (Q > R)) > S$ y la figura muestra el árbol generado para la fórmula $(P > S) \& (S > U) \& P$.



Árbol para $(P \& (Q > R)) > S$



Árbol para $(P > S) \& (S > U) \& P$.

AUTÓMATA SINTÁCTICO.- NOTACIÓN POLACA



Gramática Para Expresiones Infijas

Una expresión infija es una combinación de operadores +, -, *, /, div y mod, y operandos formados por números e identificadores, donde los operadores van en medio de los operandos. Los operadores div y mod también funcionan como *palabras reservadas*. Una palabra es reservada si el programador del lenguaje no puede identificar variables con estos nombres. Las palabras reservadas tienen un significado especial predeterminado por el diseñador del lenguaje.

Un ejemplo de expresión infija es: suma + 120 div x Una expresión Postfija es aquella donde los operadores van después de los operandos. Por ejemplo suma 120 x div + (este el equivalente postfijo de la expresión infija anterior).

Para realizar una conversión de una expresión infija a postfija hay que tener en cuenta la precedencia y asociatividad de los operadores. La precedencia es el orden en que se realizan las operaciones y la asociatividad, que puede ser a la izquierda o a la derecha, expresa si un operando es tomado primero por el operador de su izquierda o derecha al evaluar la expresión.

La precedencia y asociatividad de expresiones que implementa el traductor de este artículo es la siguiente:

Operador Precedencia Tipo de Asociatividad

*, /, div, mod Mayor Izquierda

+, - Menor Izquierda

Los paréntesis violan la precedencia y asociatividad antes establecida.

La gramática libre de contexto que representa la estructura de las expresiones infijas es la siguiente:

(1) $\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{term} \rangle$

Propiedades de Los Árboles de Análisis Sintácticos

Se puede verificar la correctitud (es decir, si está bien construido) de un árbol de parser mediante las siguientes propiedades: Dada una gramática G y una cadena de tokens, su correspondiente árbol de parser satisface:



- a) La raíz del árbol es el símbolo de inicio de la gramática
- b) Las hojas del árbol son tokens de la gramática o símbolos terminales
- c) Los nodos interiores del árbol son no terminales

Si un nodo interior etiquetado $\langle A \rangle$ tiene como hijos de izquierda a derecha a $X_1, X_2, X_3, \dots, X_n$, donde cada X_i (con i entre 1 y n) representa un terminal o una no terminal, entonces debe existir una producción en la gramática de la forma $\langle A \rangle ::= X_1 X_2 X_3 \dots X_n$. En otras palabras, ésta última propiedad indica que cada paso en la derivación del árbol sintáctico debe estar justificado por una producción de la gramática; el recíproco de esta propiedad no es cierto porque pueden existir producciones en la gramática que no se utilicen para construir un árbol de parseo en particular. Por ejemplo, la producción (7) $\langle \text{term} \rangle ::= \langle \text{term} \rangle \text{ mod } \langle \text{factor} \rangle$ no se utilizan para construir ningunos de los árboles anteriores.

Esquema de Traducción

Es una notación que se utiliza para especificar la traducción de una cadena de entrada X en una cadena de salida Y . En este caso, la cadena de entrada X es una expresión en notación infija y la cadena de salida Y es una expresión en notación postfija.

Un esquema de traducción está conformado por producciones que representan la estructura de la cadena de entrada; en el lado derecho de la producción se insertan unas instrucciones conocidas como acciones semánticas.

Esquema de Traducción que Convierte Expresiones Infijas en postfijas

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{term} \rangle \{ \text{print} (" + ") \}$

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle - \langle \text{term} \rangle \{ \text{print} (" - ") \}$

$\langle \text{expr} \rangle ::= \langle \text{term} \rangle$

$\langle \text{term} \rangle ::= \langle \text{term} \rangle * \langle \text{factor} \rangle \{ \text{print} (" * ") \}$

$\langle \text{term} \rangle ::= \langle \text{term} \rangle / \langle \text{factor} \rangle \{ \text{print} (" / ") \}$

$\langle \text{term} \rangle ::= \langle \text{term} \rangle \text{ div } \langle \text{factor} \rangle \{ \text{print} (" \text{div} ") \}$

$\langle \text{term} \rangle ::= \langle \text{term} \rangle \text{ mod } \langle \text{factor} \rangle \{ \text{print} (" \text{mod} ") \}$

$\langle \text{term} \rangle ::= \langle \text{factor} \rangle$

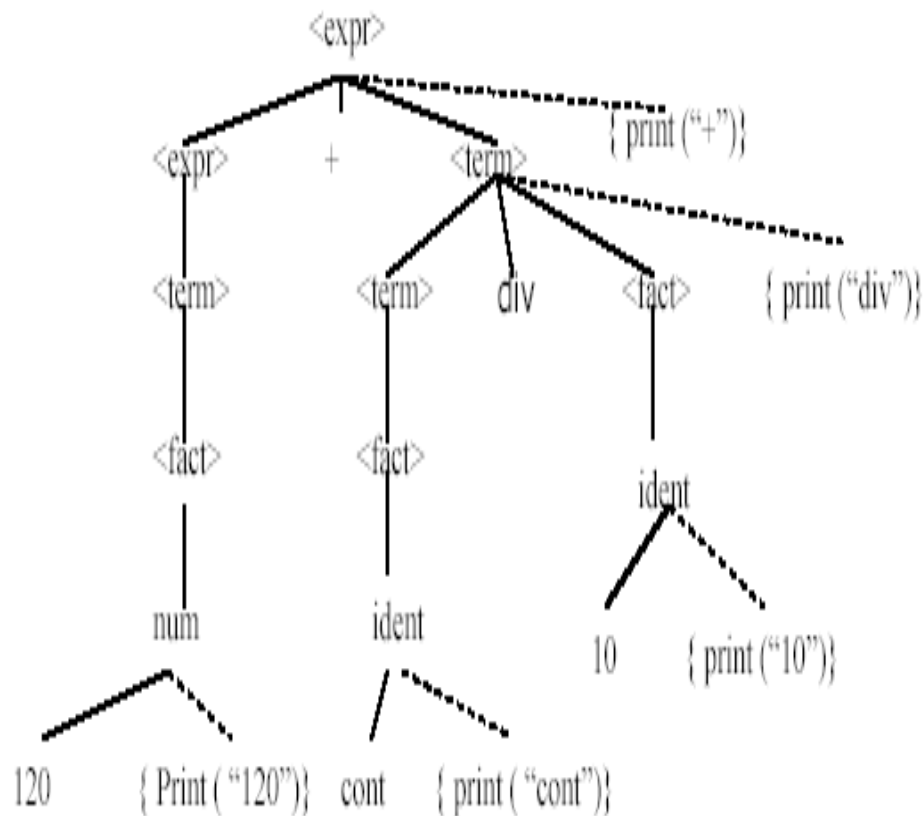
$\langle \text{factor} \rangle ::= (\langle \text{expr} \rangle)$

$\langle \text{factor} \rangle ::= \text{num} \{ \text{print}(\text{num.Valor}) \}$

$\langle \text{factor} \rangle ::= \text{ident} \{ \text{print}(\text{ident.Lexema}) \}$

Árbol de Parser Anotado

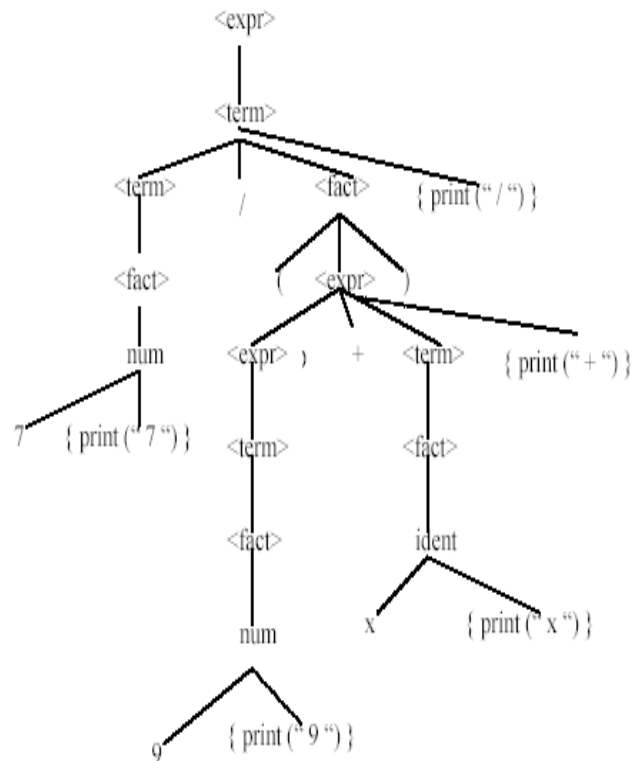
Es un árbol de parser donde se incluyen unas instrucciones conocidas como acciones semánticas. Lo interesante es que la cadena de salida se forma al aplicar las acciones semánticas en los nodos que las contienen cuando se recorre el árbol



en postorden (en el recorrido en postorden primero se visita la raíz y luego todos sus nodos hijos de izquierda a derecha). A continuación, se muestra el árbol de parser anotado para la cadena de tokens 120 + cont div 10

Al recorrer el árbol en postorden se genera la cadena de tokens de salida:

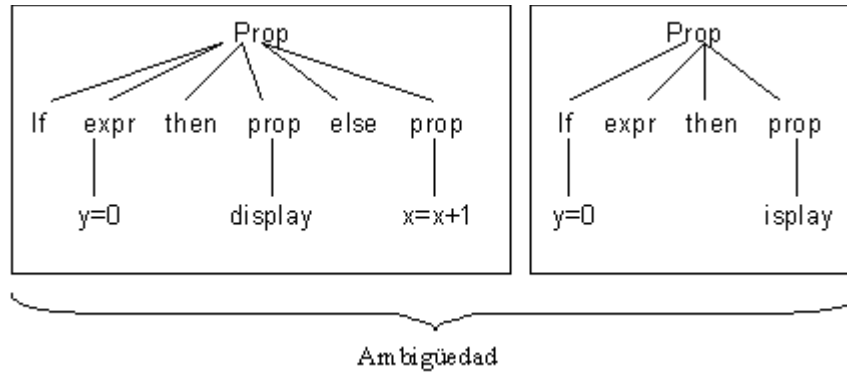
120 cont 10 div +



El árbol de parser anotado para la expresión infija de entrada: 7 / (9 + x) es: La cadena de tokens de salida es: 7 9 x + /

Ambigüedad Sintáctica

La gramática de un lenguaje es ambigua sintácticamente, o simplemente ambigua, si alguna cadena de su lenguaje tiene mas de un árbol de análisis sintáctico. Los lenguajes de programación se describen por lo general con gramáticas no ambiguas. Si hay ambigüedades, se resuelven median el establecimiento de convenciones que descartan todos los árboles de análisis sintáctico excepto uno para cada cadena.



Notación sufija

Llamada también postfija o polaca inversa, se usa para representar expresiones sin necesidad de paréntesis.

Ejemplos:

$a*b$ $ab*$
 $a*(b+c/d)$ $abcd/+*$
 $a*b+c*d$ $ab*cd*+$

Los identificadores aparecen en el mismo orden. Los operadores en el de evaluación (de izquierda a derecha).

Problema: operadores monádicos (unarios). O bien se transforman en diádicos (binarios) o se cambia el símbolo.

Ejemplo: $-a$ se convierte en $0-a$ o en $@a$

$a*(-b+c/d)$ $ab@cd/+*$

Existen dos problemas principales:

- Construir la notación sufija a partir de la infija.
- Analizar la notación sufija en el segundo paso de la compilación.



5.4 Procedimientos para la generación de la tabla de análisis sintáctico (primero, siguiente y otros).

Rutina semántica para transformar de infijo a sufijo

Si el analizador sintáctico es bottom-up, hacemos la siguiente suposición: "Cuando aparece un no terminal V en el asidero, la cadena polaca correspondiente a la subcadena que se redujo a V ya ha sido generada".

Se utiliza una pila donde se genera la salida, inicialmente vacía. Las acciones semánticas asociadas a las reglas son:

$E ::= E + T$	Push +
$E ::= E - T$	Push -
$E ::= T$	
$T ::= T * F$	Push *
$T ::= T / F$	Push /
$T ::= F$	
$F ::= i$	Push i
$F ::= (E)$	
$F ::= - F$	Push @

Análisis de la notación sufija

La gramática completa que permite analizar la notación sufija es:

```
<Operando> ::= id |cte |  
<Operando> <Operando> <Operador diádico> |  
<Operando> <Operador monádico>  
<Operador diádico> ::= + | - | * | / | ...  
<Operador monádico> ::= @ | ...
```

Algoritmo de evaluación de una expresión en notación sufija que utiliza una pila:

Si el próximo símbolo es un identificador, se pasa a la pila. Corresponde a la aplicación de la regla

$\langle \text{Operando} \rangle ::= \text{id}$

Si el próximo símbolo es una constante, se pasa a la pila. Corresponde a la aplicación de la regla

$\langle \text{Operando} \rangle ::= \text{cte}$



Si el próximo símbolo es un operador diádico, se aplica el operador a los dos operandos situados en lo alto de la pila y se sustituyen éstos por el resultado de la operación. Corresponde a la aplicación de la regla

$\langle \text{Operando} \rangle := \langle \text{Operando} \rangle \langle \text{Operando} \rangle \langle \text{Operador diádico} \rangle$

Si el próximo símbolo es un operador *monádico*, se aplica el operador al operando situado en lo alto de la pila y se sustituye éste por el resultado de la operación. Corresponde a la aplicación de la regla

$\langle \text{Operando} \rangle := \langle \text{Operando} \rangle \langle \text{Operador monádico} \rangle$

Extensión de la notación sufija a otros operadores

La asignación, teniendo en cuenta que podemos no querer valor resultante. Además, no interesa tener en la pila el valor del identificador izquierdo, sino su dirección.

- $a := b * c + d \quad abc * d + :=$

La transferencia (GOTO).

- $\text{GOTO } L \quad L \text{ TR}$

La instrucción condicional

- $\text{if } p \text{ then inst1 else inst2}$

Se convierte en $p \ L1 \ \text{TRZ} \ \text{inst1} \ L2 \ \text{TR} \ \text{inst2}$

$L1: \ L2:$

Subíndices:

- $a[\text{exp1}; \text{exp2}; \dots; \text{expn}]$

Se convierte en $a[\text{exp1}, \text{exp2}, \dots, \text{expn}] \ \text{SUBIN-n}$

TABLAS DE PALABRAS RESERVADAS Y DE USUARIO

Una palabra reservada es una palabra que tiene un significado especial para algún lenguaje de programación y no la puedes utilizar (su nombre) para otro propósito, es decir, no puedes usarla como identificador (nombre de variable, por ejemplo).

Las tablas de palabras reservadas son equivalentes a las palabras terminales y a las no terminales que acepta el lenguaje.



MANEJO DE ERRORES

Si un compilador tuviera que procesar solo programas correctos, su diseño e implantación se simplificarían mucho. Pero los programadores a menudo escriben programas incorrectos, y un buen compilador debería ayudar al programador a identificar y localizar errores. Sin embargo pocos lenguajes han sido diseñados teniendo en cuenta el manejo de errores. La mayoría de las especificaciones de los lenguajes de programación no describen como debe responder el compilador a los errores.

Los programas pueden contener errores de diversos tipos:

- **Léxicos:** Como escribir mal un identificador, palabra clave u operador.
- **Sintácticos:** Como una expresión aritmética con paréntesis no equilibrados.
- **Semánticos:** Como un operador aplicado a un operando incorrecto.
- **Lógicos:** Como una llamada infinitamente recursiva.
- El manejador de errores de un analizador sintáctico tiene objetivos fáciles de establecer:
- Debe informar la presencia de errores con claridad y exactitud.
- Se debe recuperar de cada error con la suficiente rapidez como para detectar errores posteriores.
- No debe retrasar de manera significativa el procesamiento de programas correctos.

5.5 Implementación de un Analizador Sintáctico.

El siguiente código en AWK es un ejemplo completo de un analizador léxico, sintáctico y semántico de una calculadora RPN. (Reverse Polish Notation)

```
# calc1 - reverse-Polish calculator, version 1  
  
# input: arithmetic expressions in reverse Polish  
  
# output: values of expressions
```



```
{ for (i = 1; i <= NF; i++) #Since field, enter by the user, until last one...
    if ($i ~ /^[+-]?([0-9]+[.]?[0-9]*[.]?[0-9]+)$/) { # field is numeric?
        stack[++top] = $i #if it is save in the stack
    } else if ($i == "+" && top > 1) { # Else, is it an add?
        stack[top-1] += stack[top]; top-- #Add numeric field in the top of stack
    } else if ($i == "-" && top > 1) { # Else, is it a subtract?
        stack[top-1] -= stack[top]; top-- #Subtract numeric field in the stack
    } else if ($i == "*" && top > 1) { # Else, is it a multiply?
        stack[top-1] *= stack[top]; top-- #Multiply numeric field in the stack
    } else if ($i == "/" && top > 1) { # Else, is it a divide?
        stack[top-1] /= stack[top]; top-- #Divide numeric field in the stack
    } else if ($i == "^" && top > 1) { # Else, is it an exponent?
        stack[top-1] ^= stack[top]; top-- #Elevate by numeric field in the stack
    } else {
        printf("error: cannot evaluate %s\n", $i) #Else not is numeric field
        top = 0
        next
    }
}

if (top == 1) #There is only one numeric datum in the stack
    printf("\t%.8g\n", stack[top--])
else if (top > 1) { #Else, there are more than a couple of numeric datum
    printf("error: too many operands\n") # It has more than a couple of data
    top = 0 # re-star the stack
}

}
```

Este código fuente en AWK sirve para ejemplificar un analizador léxico y una Gramática Libre de Contexto de tipo expresiones numéricas con notación polaca inversa. Diagrama de bloques de este programa: Calculadora

Lenguaje Regular:

$L_R = \{[0-9]^*, +, -, *, /, ^, (E)\}$

BNF de la GLC

$(E) \rightarrow (E) (E) +$

$(E) \rightarrow (E) (E) -$

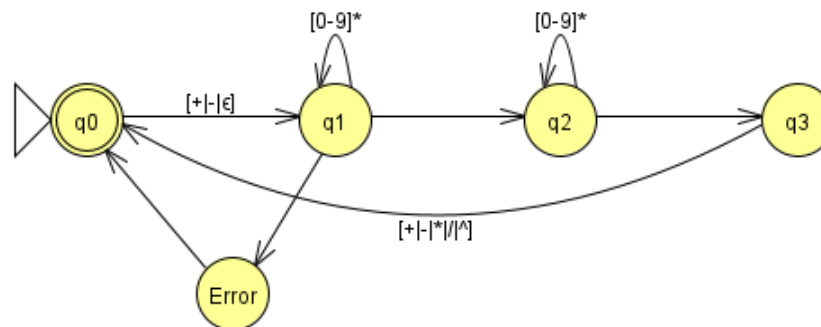
$(E) \rightarrow (E) (E) *$

$(E) \rightarrow (E) (E) /$

$(E) \rightarrow (E) (E) ^$

$(E) \rightarrow (E)$

Diagrama de Transiciones: AFND sin épsilon transiciones.



FUENTES:

Aho, Sethi, Ullman. (2014). *Compiladores, principios, técnicas y herramientas*. México: Pearson.

Autores varios. (2005). Diccionario de la lengua española. México: Trillas. DISPONIBLE EN INTENET: <http://lema.rae.es/drae/?val=lenguaje>.

Hopcroft, Motwani, Ullman. (2008). *Teoría de autómatas, lenguajes y computación*. España: Pearson.

Kinnersley, Bill (2015). Base de datos sobre lenguajes de programación
DISPONIBLE EN: <http://people.ku.edu/~nkinners/LangList/Extras/langlist.htm>

McCarthy, J. (1960). Recursive Functions of Symbolic Expressions and Their Computation by Machine. Part 1. Comm ACM Vol. 3. No. 4. Pp 184.195

Morris, C. (1938). Foundations of the theory of signs. International Encyclopedia of United Science. University of Chicago Press.

Pratt, Terrence. W., Marvin. V. Zelkowitz ; traducido por Héctor Javier Escalona García. (1987). lenguajes de programación. Mexico: Prentice Hall. Traducido por Héctor Javier Escalona García.

ABSTRACT **DISPONIBLE** **EN:**
http://www.researchgate.net/publication/31737166_Lenguajes_de_programacin_diseo_e_implementacin_T.W._Pratt_M.V._Zelkowitz_tr_por_Hctor_Javier_Escalona_Garca

Silva Mendoza Edith. [2010]. Sistemas Organizacionales. México, D., F.: IPN. Página 19.



Cibergrafía:

<http://people.ku.edu/~nkinners/LangList/Extras/langlist.htm>

RÚBRICAS DE ACTIVIDADES DE APRENDIZAJE

PRIMERA: “Diario de clase”

Objetivo de esta primera actividad: Ejercitar la lectura y escritura del estudiante, la comprensión y análisis de temas técnicos o abstractos para facilitar su futura aplicación.

Descripción de esta primera actividad: Desarrolle un diario de clase en el cual escriba en formato libre la respuesta a alguna, ninguna o todas las siguientes cuestiones. (Puede dejar de responder estas cuestiones ya que se le invita al lector a desarrollar un formato libre, si lo desea, de su percepción e inquietudes relacionadas con el desarrollo de los temas descritos en toda la unidad temática).

¿Entendí que es un sistema formal? Si o No, que es lo que no comprendí.

¿Cómo me siento con respecto a la lectura de estos conceptos?

¿Cuál es mi percepción del desarrollo de los temas?

En el diario de clase se puede escribir sobre: Mi opinión personal, hacer una relatoría con comentarios personales, crítica constructiva del material y de los eventos ocurridos en mi actividad de lectura.

Ejemplos de frases (con argumentación) : Me pareció aburrido el tema por...

Me pareció interesante por...

No entendí el concepto de... etc.